Week 1: Big Data — The Big Picture

Study Guide | Ultimate Big Data Masters Program

---

1. Introduction to Big Data & Fundamentals

What is Big Data?

Big Data refers to data that is too large, complex, or fast-moving to be efficiently stored and processed using a single conventional computing system.

It is commonly described using the **5 V's of Big Data**:

V	Meaning	Example
Volume	Massive quantities of data	Petabytes of social media or sensor data
Variety	Different types and formats of data	Structured, semi-structured, and unstructured data
Velocity	Speed at which data is generated and processed	Real-time transactions during an online sale
Veracity	Quality, accuracy, and reliability of data	Removing inaccurate or inconsistent records
Value	Business value extracted from data	Using customer data to improve business decisions

Types of Data

1. Structured Data

- Has a fixed schema.
- Organized into rows and columns.
- Example: MySQL relational tables.

2. Semi-Structured Data

- Has some organizational structure but does not follow a rigid tabular schema.
- Examples: CSV, JSON, XML.

3. Unstructured Data

- Does not follow a predefined schema.
- Examples: images, videos, raw logs, documents.

---

2. Monolithic vs. Distributed Systems

Traditional monolithic systems struggle to handle Big Data because of limitations in scalability and hardware capacity.

Feature	Monolithic System	Distributed System
Architecture	Single large system containing the required hardware resources	Cluster of interconnected machines (nodes) working together
Scaling	Vertical scaling	Horizontal scaling
Approach	Add CPU/RAM to an existing machine	Add more machines/nodes to the cluster
Scalability	Limited by the capacity of a single machine	Can scale by adding additional nodes

Vertical Scaling

Increasing the capacity of a single machine by adding resources such as:

- CPU
- RAM
- Storage

Horizontal Scaling

Increasing capacity by adding more machines to the cluster.

Core Requirements of Big Data Systems

1. **Distributed Storage** — Store data across multiple physical machines.
2. **Distributed Processing** — Process data concurrently across multiple nodes.
3. **Scalability** — Allow the system to expand horizontally as data volume increases.

---

3. Hadoop Ecosystem

Hadoop is an open-source framework designed primarily for distributed storage and distributed processing of large datasets.

Core Components

HDFS — Hadoop Distributed File System

Provides distributed storage by spreading data across multiple nodes.

MapReduce

A distributed processing framework used to process large datasets.

> **Note:** MapReduce is now less commonly used for modern Big Data processing because of its relatively slower execution model and complexity compared with Spark.

YARN — Yet Another Resource Negotiator

Manages and allocates computing resources across the Hadoop cluster.

Hadoop Ecosystem Tools

Tool	Purpose	Modern / Cloud Alternative
Sqoop	Transfer data between RDBMS and HDFS	Azure Data Factory / AWS Glue
Pig	Data processing and scripting	Modern Spark-based processing
Hive	SQL-like querying of data stored in Hadoop	Spark SQL / Cloud data warehouses
Oozie	Workflow scheduling and management	Azure Data Factory / Airflow
HBase	NoSQL database for low-latency random read/write operations	Cosmos DB / DynamoDB

---

4. Introduction to Apache Spark

Apache Spark was developed to address several limitations of MapReduce, particularly its execution speed and programming complexity.

Key Characteristics

- General-purpose distributed compute engine
- In-memory processing capabilities
- High performance
- Supports batch and streaming workloads
- Supports multiple programming languages

Spark vs. MapReduce

A key concept to remember:

> **Spark replaces MapReduce as a processing engine; it does not replace Hadoop's entire ecosystem.**

Spark can work with independent storage systems and resource managers.

Supported Languages

- **Python** → PySpark
- **Scala**
- **Java**
- **R**

Spark Stack

Storage + Compute + Resource Management

Storage
→ HDFS / Amazon S3 / ADLS

Compute
→ Apache Spark

Resource Manager
→ YARN / Kubernetes

---

5. Database vs. Data Warehouse vs. Data Lake

High-Level Comparison

Feature	Database	Data Warehouse	Data Lake
Primary Use	Transactions	Analytics & BI	Big Data storage & analytics
Typical Workload	OLTP	OLAP	Analytics / Data Science
Data	Structured	Primarily structured	Raw data in any format
Schema	Schema on Write	Schema on Write	Schema on Read
Typical Flow	Transactions	ETL	ELT
Examples	MySQL, Oracle	Teradata, Redshift	HDFS, Amazon S3, ADLS Gen2

Database — OLTP

Optimized for **real-time transactional operations**.

Characteristics:

- Structured data
- Schema on Write
- Fast transactions
- High consistency requirements

Examples: MySQL, Oracle

Data Warehouse — OLAP

Designed for:

- Historical analysis
- Reporting
- Business intelligence
- Analytical queries

Typically uses an **ETL (Extract → Transform → Load)** process.

Examples: Teradata, Amazon Redshift

Data Lake

A Data Lake stores large volumes of raw data in multiple formats.

Characteristics:

- Structured, semi-structured, and unstructured data
- Schema on Read
- Generally follows ELT
- Cost-effective large-scale storage

Examples: HDFS, Amazon S3, ADLS Gen2

---

6. Data Engineering Workflow

A **Data Engineer** acts as a bridge between **Data Producers / Owners** and **Data Consumers**.

Typical Data Engineering Flow

Sources → Ingestion → Data Lake → Processing → Serving Layer → BI / Consumers

Architecture Comparison

Layer	On-Premise	Azure	AWS
Ingestion	Sqoop	Azure Data Factory	AWS Glue
Data Lake	HDFS	ADLS Gen2	Amazon S3
Processing	MapReduce / Spark	Azure Databricks / Synapse	Databricks / Athena / Redshift
Serving	Hive / HBase	Azure SQL / Cosmos DB	RDS / DynamoDB

Serverless vs. Serverful Computing

Feature	Serverless	Serverful
Infrastructure	Shared resource pool	Dedicated compute resources
Pricing	Often based on usage/query/data scanned	Often based on provisioned resources
Management	Minimal infrastructure management	More infrastructure management
Best For	Ad-hoc queries	Predictable production workloads

Examples	AWS Athena, Synapse Serverless	Amazon Redshift, Synapse Dedicated

Key Idea

Serverless:

Pay for what you use; infrastructure is managed for you.

Serverful:

Dedicated compute resources provide greater control and predictable performance.

---

7. HDFS Architecture Deep Dive

HDFS (Hadoop Distributed File System) is Hadoop's distributed storage system.

It follows a **Master–Worker architecture**.

Key Components

NameNode — Master

The **NameNode** manages filesystem metadata.

It maintains:

- Directory structure
- File metadata
- File-to-block mapping
- Block location information

When a client wants to access a file, it first communicates with the NameNode to determine where the required blocks are located.

DataNode — Worker

DataNodes store the **actual physical data blocks**.

They are responsible for:

- Storing data blocks

- Serving data to clients
- Replicating blocks
- Reporting their status to the NameNode

HDFS Defaults

Property	Default
Block Size	128 MB
Replication Factor	3

Heartbeat Mechanism

DataNodes periodically send **heartbeat signals** to the NameNode to indicate that they are operational.

If a DataNode stops responding, the NameNode can identify it as unavailable and initiate recovery/re-replication of the affected blocks.

Data Locality

Data Locality means moving computation closer to where the data physically resides.

Instead of moving large volumes of data across the network:

> **Move computation to the data.**

This reduces:

- Network traffic
- Data transfer overhead
- Processing latency

and improves overall cluster performance.

Quick Revision

Big Data → 5 V's

Volume + Variety + Velocity + Veracity + Value

Big Data Architecture

Distributed Storage + Distributed Processing + Scalability

Hadoop

HDFS → Storage
MapReduce → Processing
YARN → Resource Management

Spark

Modern distributed compute engine that can replace MapReduce

Data Architecture

Database → Transactions → OLTP

Data Warehouse → Analytics → OLAP

Data Lake → Raw Big Data → Schema on Read

Data Engineering

Source → Ingestion → Data Lake → Processing → Serving → BI

HDFS

NameNode → Metadata

DataNode → Actual Data

Data Locality → Compute near the data

Important Concepts to Remember

1. Big Data is not simply "a lot of data."
It involves challenges related to volume, variety, velocity, veracity, and value.

2. Horizontal scaling is fundamental to Big Data.
Instead of continuously buying a bigger machine, distributed systems add more machines.

3. Spark is a compute engine.
It does not inherently replace distributed storage such as HDFS or S3.

4. Data Lake ≠ Data Warehouse.
A Data Lake is designed to store raw data at scale, while a Data Warehouse is optimized for structured analytical workloads.

5. HDFS separates metadata from actual data.
NameNode manages metadata; DataNodes store the actual blocks.

Week 2: Distributed Storage & Data Lake

Study Guide | Ultimate Big Data Masters Program

---

1. HDFS Fundamentals & Architecture Deep Dive

HDFS Architecture

HDFS (Hadoop Distributed File System) follows a **Master–Worker architecture**.

Component	Role	Responsibility
NameNode	Master	Stores filesystem metadata, directory structure, file-to-block mappings, and block locations
DataNode	Worker	Stores the actual physical data blocks

Simple Analogy

Think of the **NameNode as the index of a book**.

- The **NameNode** tells you where the information is located.
- The **DataNodes** contain the actual information.

HDFS Read Architecture

Client → NameNode → DataNode(s)

The NameNode is involved in **metadata lookup**, but the actual data is read directly from the DataNodes.

HDFS Read Process

Step 1 — Client requests file information

The client sends a request to the **NameNode** asking where the required file blocks are located.

Step 2 — NameNode provides block locations

The NameNode checks its metadata and returns the locations/IP addresses of the relevant DataNodes.

Step 3 — Client reads data directly

The client connects directly to the respective **DataNodes** and streams the required blocks.

Key Point

> **NameNode provides metadata; DataNodes provide data.**

The actual file data does **not** pass through the NameNode.

---

2. HDFS Block Size

The default HDFS block size in modern Hadoop is typically **128 MB**.

Block size has a direct impact on:

- Number of blocks
- NameNode metadata
- Parallelism
- Processing performance

Smaller Block Size

Example: **32 MB or 64 MB**

Advantages

- Creates more blocks.
- Provides more potential units of parallel processing.

Disadvantages

- Increases the total number of blocks.
- Increases metadata maintained by the NameNode.
- Can increase NameNode memory requirements.

Larger Block Size

Example: **256 MB or 512 MB**

Advantages

- Reduces the total number of blocks.
- Reduces NameNode metadata overhead.

Disadvantages

- Fewer blocks mean fewer parallel processing units.
- May reduce processing parallelism for certain workloads.

Quick Comparison

Block Size	Main Advantage	Main Disadvantage
Smaller	More parallelism	More metadata
Larger	Less metadata	Less parallelism
128 MB	Common default	Balance between the two

> **Remember:** Block size is a trade-off between **parallelism and metadata overhead**.

---

3. HDFS High Availability, Scalability & Fault Tolerance

NameNode Federation

NameNode Federation allows multiple independent NameNodes/namespaces to operate within the same HDFS cluster.

It helps:

- Scale metadata management
- Reduce the memory limitations of a single NameNode
- Improve namespace scalability

Fault Tolerance

HDFS achieves fault tolerance primarily through **data replication**.

The typical default replication factor is:

Replication Factor = 3

This means each block can have multiple copies distributed across DataNodes.

If one DataNode fails, another replica can serve the data.

Secondary NameNode

The **Secondary NameNode** periodically combines:

- **FsImage**
- **Edit Logs**

This helps keep metadata checkpoints manageable and makes recovery faster.

> **Important:** The Secondary NameNode is **not a backup NameNode** and does not automatically take over when the primary NameNode fails.

Rack Awareness

HDFS uses **Rack Awareness** to distribute block replicas across different physical racks.

This protects against failures such as:

- Rack failure
- Network switch failure
- Power failure affecting an entire rack

Key Concept

Replication protects data from node failures, while rack awareness protects data from larger infrastructure failures.

---

4. Linux Command Reference

Data Engineers frequently work with Linux environments when interacting with Hadoop and distributed storage systems.

Common Linux Commands

Command	Category	Purpose	Example
`pwd`	Directory	Display current working directory	`pwd`
`cd`	Directory	Change directory	`cd /data`
`ls -ltr`	File System	List files ordered by modification time	`ls -ltr`
`ls -laR`	File System	Recursively list all files, including hidden files	`ls -laR`
`touch`	File Operations	Create an empty file / update timestamp	`touch file.txt`
`chmod`	Security	Change file permissions	`chmod 777 file1`
`mkdir -p`	Directory	Create nested directories	`mkdir -p /data/raw/input`
`rm -R`	Cleanup	Recursively remove directories/files	`rm -R folder`
`cp -R`	File Operations	Recursively copy files/directories	`cp -R source destination`
`mv`	File Operations	Move or rename files/directories	`mv old.txt new.txt`
`cat`	Analysis	Display file contents	`cat file.txt`
`grep`	Analysis	Search/filter text patterns	`grep "error" logfile.txt`

Linux Permissions

The `chmod` command changes file permissions.

For example:

chmod 777 file1

This grants:

- **Owner:** Read + Write + Execute
- **Group:** Read + Write + Execute
- **Others:** Read + Write + Execute

> **Caution:** `chmod 777` grants very broad permissions and should generally be avoided in production unless specifically required.

---

5. Essential HDFS Commands

HDFS commands commonly use either:

hadoop fs

or:

hdfs dfs

List Directory Contents

hadoop fs -ls /user/itv005857

Create Directory Hierarchy

hadoop fs -mkdir -p /user/itv005857/dir1/dir2

Upload Data: Local → HDFS

hadoop fs -put /local/path/data.csv /hdfs/target/path/

Alternative:

hadoop fs -copyFromLocal /local/path/data.csv /hdfs/target/path/

Download Data: HDFS → Local

hadoop fs -get /hdfs/target/path/data.csv /local/path/

Alternative:

hadoop fs -copyToLocal /hdfs/target/path/data.csv /local/path/

Copy Within HDFS

hadoop fs -cp /hdfs/source /hdfs/destination

Move / Rename Within HDFS

hadoop fs -mv /hdfs/source /hdfs/destination

HDFS Command Cheat Sheet

Operation	Command
List	`hadoop fs -ls`
Create directory	`hadoop fs -mkdir -p`
Upload	`hadoop fs -put`
Download	`hadoop fs -get`
Copy	`hadoop fs -cp`
Move / Rename	`hadoop fs -mv`

---

6. HDFS vs. Cloud Data Lakes

One of the most important changes introduced by cloud data platforms is the **separation of compute and storage**.

Traditional HDFS

Compute + Storage → Same Cluster

HDFS typically operates as part of a Hadoop cluster where compute and storage infrastructure are closely coupled.

Cloud Data Lake

Compute ↔ Independent Cloud Storage

Cloud object storage such as **Amazon S3** and **Azure ADLS Gen2** allows storage to exist independently from compute clusters.

This enables multiple compute engines or clusters to access the same underlying data.

Storage Architecture Comparison

Feature	On-Premise HDFS	Cloud Data Lake
Storage Type	Distributed file system	Object storage
Examples	HDFS	Amazon S3, Azure ADLS Gen2
Storage Model	Blocks / files	Objects
Compute & Storage	Tightly coupled	Decoupled
Persistence	Closely tied to cluster infrastructure	Independent of compute resources
Multi-Cluster Access	More tightly tied to Hadoop cluster	Multiple compute clusters can access same storage
Scaling	Scale cluster infrastructure	Scale storage and compute independently

Object Storage

Cloud data lakes use **object storage**, where data is stored as objects along with metadata and identifiers.

Examples:

- **Amazon S3**
- **Azure ADLS Gen2**

Key Advantage of Cloud Data Lakes

> **Compute and storage can scale independently.**

For example, you can have:

Cloud Storage

↓

Spark Cluster 1

Spark Cluster 2

Databricks Cluster

Athena

all accessing the same underlying data.

---

7. Distributed Processing: MapReduce

MapReduce is a distributed processing programming model designed to process large datasets across multiple machines.

The basic flow is:

Input → Map → Shuffle & Sort → Reduce → Output

MapReduce Data Flow

Input Data
↓

Record Reader
↓
Mapper
↓
Shuffle & Sort
↓
Reducer
↓
Output

---

8. MapReduce Execution Phases

Phase 1 — Record Reader & Mapper

The **Record Reader** reads the input data and converts it into key-value pairs.

The **Mapper** processes the input and produces intermediate key-value pairs.

Conceptually:

Input (K_1, V_1)

↓

Mapper

↓

Intermediate (K_2, V_2)

Data Locality

Map tasks generally run as close as possible to the DataNode containing the required data.

This follows the principle:

> **Move computation to the data rather than moving large amounts of data across the network.**

Phase 2 — Shuffle

The Shuffle phase transfers intermediate Mapper output across the network.

Its primary purpose is to ensure that values belonging to the same key are sent to the same Reducer.

For example:

Mapper 1 → (Apple, 2)
Mapper 2 → (Apple, 4)
Mapper 3 → (Apple, 3)

After Shuffle:

Reducer → Apple → [2, 4, 3]

Phase 3 — Sort

The intermediate data is organized and grouped by key before being passed to the Reducer.

Conceptually:

Apple → [2, 4, 3]
Banana → [5, 1]
Orange → [7, 2]

Phase 4 — Reducer

The Reducer processes grouped values and performs the required aggregation or business logic.

For example:

Apple → [2, 4, 3]

Reducer:
2 + 4 + 3 = 9

Output:
Apple → 9

---

9. MapReduce Architecture

Overall Flow

Input File Blocks

↓

Multiple Mappers

↓

Shuffle & Sort

↓

Reducers

↓

Final Output

Key Characteristics

- Multiple Mappers can execute **in parallel**.
- Shuffle involves **network transfer**.
- Sort groups intermediate data by key.
- Reducers process grouped values.
- Data locality minimizes unnecessary data movement during the Map phase.

Quick Revision

HDFS

NameNode → Metadata

DataNode → Actual Data

Replication → Fault Tolerance

Rack Awareness → Protection against rack-level failures

Federation → Namespace scalability

HDFS Read

Client → NameNode → Block Locations → DataNodes → Data

> NameNode does **not** serve the actual file data.

HDFS Block Size

Smaller blocks → More parallelism + More metadata

Larger blocks → Less metadata + Less parallelism

Cloud Data Lake

HDFS → Tightly coupled compute + storage

Cloud Data Lake → Decoupled compute + storage

MapReduce

Input → Map → Shuffle → Sort → Reduce → Output

Important Concepts to Remember

1. **NameNode stores metadata, not actual file data.**

2. **DataNodes store the actual HDFS blocks.**

3. **The client reads data directly from DataNodes after obtaining block locations from the NameNode.**

4. **Replication provides fault tolerance; Rack Awareness provides additional protection against rack-level failures.**

5. Secondary NameNode is not a standby NameNode.

6. Cloud object storage separates storage from compute, allowing both to scale independently.

7. Shuffle is one of the most network-intensive stages of MapReduce.

8. Data locality reduces network traffic by bringing computation closer to the data.

Week 3: Distributed Processing Fundamentals & Apache Spark

Study Guide | Ultimate Big Data Masters Program

---

1. MapReduce Deep-Dive & Optimizations

In a MapReduce job, processing large amounts of distributed data relies primarily on two phases:

Mapper

The **Mapper** provides parallelism.

It:

- Reads input blocks.
- Processes the input records.
- Generates intermediate key-value pairs.

Reducer

The **Reducer** provides aggregation.

It:

- Receives Mapper output.
- Groups data by key.
- Performs aggregation or other reduction logic.

- Produces the final result.

Optimization Rule of Thumb

> **Push as much computation as possible toward the Mapper and minimize the work performed by the Reducer.**

Doing more work earlier in the pipeline can reduce the amount of data that needs to be transferred and processed later.

Configuring the Number of Reducers

By default, a MapReduce job runs with **1 Reducer**.

The number of Reducers can be configured based on workload requirements.

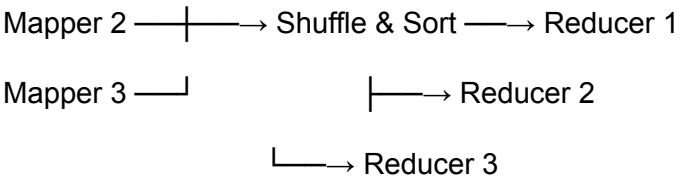
Number of Reducers	When to Use	Effect
1 Reducer	Small or simple workloads	Simple execution but may become a bottleneck
Multiple Reducers (>1)	Large aggregations	Allows parallel reduction and avoids a single Reducer bottleneck
0 Reducers	Jobs requiring no aggregation	Mapper output becomes the final output; Shuffle & Sort can be skipped

When to Increase Reducers

Use multiple Reducers when a single Reducer takes too long to process a large aggregation.

Example:

Mapper 1 ──┐



This allows the reduction workload to be distributed across multiple machines.

When to Set Reducers to 0

If the job does not require aggregation—for example, a simple filtering operation—the number of Reducers can be set to **0**.

In this case:

Mapper → Final Output

The expensive **Shuffle and Sort** phases can be skipped.

---

2. Shuffle, Sort & Partition

When multiple Reducers are used, the MapReduce framework needs a mechanism to determine which Reducer should receive each intermediate key-value pair.

This is where **Partitioning, Shuffle, and Sort** come into play.

Partitioning

Partitioning determines which key-value pair is sent to which Reducer.

Important Rule

> **Number of partitions = Number of Reducers**

For example:

3 Reducers → 3 Partitions

Default Partitioner

The default system partitioner generally uses a **hash-based partitioning mechanism**.

A simplified example is:

Partition = hash(key) % number_of_reducers

This ensures that the same key is consistently sent to the same Reducer.

Custom Partitioner

Developers can implement custom logic to control how data is distributed.

For example:

Word length < 4 → Reducer 1

Word length ≥ 4 → Reducer 2

Shuffle

Shuffle is the physical transfer of intermediate Mapper output across the network to the appropriate Reducer.

Key Idea

> **Shuffle = Network movement of Mapper output to Reducers.**

Because Shuffle involves network communication, it can become an expensive stage of a distributed job.

Sort

The **Sort** phase groups data by key on the Reducer side so that all values belonging to the same key can be processed together.

Example:

Before Sort:

Apple → 1

Orange → 1

Apple → 1

Banana → 1

Orange → 1

After Sort:

Apple → [1, 1]

Banana → [1]

Orange → [1, 1]

This grouped structure is then provided to the Reducer.

---

3. Combiners — Local Aggregation

A **Combiner** is essentially a **mini-reducer** that performs local aggregation on the Mapper output **before the data is transferred across the network**.

Without a Combiner

Mapper → Shuffle → Reducer

Large amounts of intermediate data may need to travel across the network.

With a Combiner

Mapper → Combiner → Shuffle → Reducer

The Combiner reduces the amount of data that needs to be transferred.

Advantages

- Reduces network traffic.
- Improves performance.
- Performs local aggregation.
- Reduces the amount of intermediate data sent to Reducers.

Example

Suppose a Mapper produces:

Apple → 1

Apple → 1

Apple → 1

Apple → 1

A Combiner can locally convert this to:

Apple → 4

Instead of sending four records over the network, only one aggregated record needs to be transferred.

Important Caution

Combiners are safe for operations such as:

- **Sum**
- **Min**
- **Max**

However, they **cannot be directly used for calculating an Average**.

Why?

Because averaging already-averaged subsets can produce an incorrect final average unless the computation is designed to retain the required count and sum information.

> **Remember:** A Combiner should only be used when the operation is safe for local partial aggregation.

---

4. The Transition to Apache Spark

Challenges of MapReduce

MapReduce was revolutionary for distributed processing, but it has several limitations.

1. Poor Performance

MapReduce relies heavily on **disk I/O**.

When multiple MapReduce jobs are chained together, intermediate results are repeatedly:

Written to Disk → Read from Disk → Processed → Written to Disk

This creates significant overhead.

2. High Code Complexity

Simple data-processing operations can require large amounts of complex Java code.

This makes development:

- More time-consuming
- Harder to maintain
- More difficult to debug

3. Batch Only

Traditional MapReduce is primarily designed for **batch processing**.

It is not designed for:

- Real-time streaming
- Interactive querying
- Low-latency processing

---

5. What is Apache Spark?

Apache Spark is an open-source, multi-language, general-purpose distributed **compute engine**.

One of its major advantages is its ability to keep intermediate data in **memory (RAM)** when appropriate, reducing repeated disk I/O.

Key Characteristics

- Open-source
- Distributed
- General-purpose
- Multi-language
- In-memory processing capabilities
- High performance
- Supports batch and streaming workloads

Why is Spark Faster?

A simplified comparison:

MapReduce

Process

↓

Write to Disk

↓

Read from Disk

↓

Process

↓

Write to Disk

Spark

Process

↓

Keep Intermediate Data in Memory

↓

Continue Processing

This reduction in disk I/O can significantly improve performance for workloads involving multiple processing stages.

Plug-and-Play Architecture

Spark does **not** provide its own distributed storage system.

It can work with external storage systems such as:

- HDFS
- Amazon S3
- Azure ADLS Gen2

Spark can also run with external resource managers such as:

- YARN
- Kubernetes
- Mesos

Spark Stack

Storage

HDFS / S3 / ADLS

↓

Compute

Apache Spark

↓

Resource Management

YARN / Kubernetes / Mesos

Key Concept

Spark is primarily a compute engine, not a storage system.

---

6. Apache Spark vs. Databricks

These two terms are often confused, but they are not the same thing.

Feature	Apache Spark	Databricks
Type	Open-source distributed processing framework	Managed cloud data and AI platform built around Spark and other technologies
Cost	Open-source software	Commercial managed platform

Compute Management	Requires infrastructure configuration	Managed cloud infrastructure
Notebooks	Available through various environments	Integrated collaborative notebooks
Optimization	Core Spark capabilities	Additional platform-level optimizations and managed capabilities
Security & Governance	Depends on deployment	Integrated platform capabilities

Simple Way to Remember

> **Apache Spark = Distributed Compute Framework**
>
> **Databricks = Managed Data & AI Platform that provides Spark and many additional capabilities**

---

7. Spark Architecture & Core Concepts

Spark APIs

Spark provides different levels of APIs for processing data.

Layer 1 — Core APIs

The foundational API is based on **RDDs (Resilient Distributed Datasets)**.

RDDs provide:

- Low-level control
- Flexibility
- Fine-grained transformations

However, they generally require more code and are more difficult to work with than higher-level APIs.

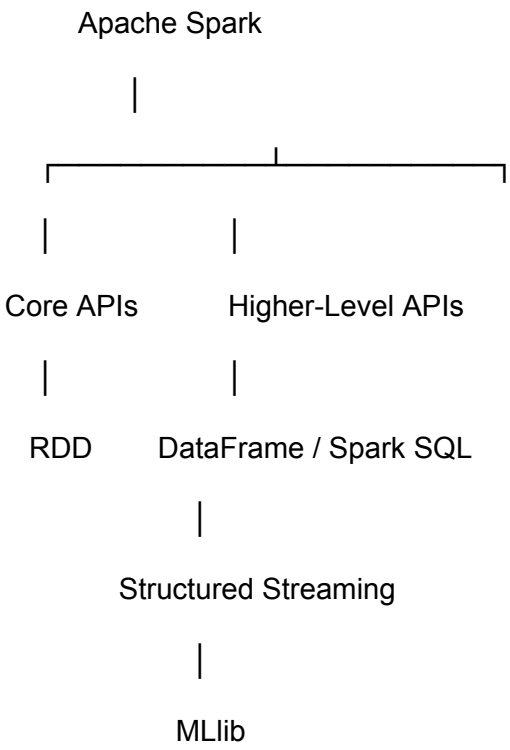
Layer 2 — Higher-Level APIs

Spark provides higher-level abstractions such as:

- **DataFrames**
- **Spark SQL**
- **Structured Streaming**
- **MLlib**

These APIs provide simpler syntax and allow Spark to apply additional optimizations.

API Hierarchy



---

8. Resilient Distributed Dataset — RDD

An **RDD (Resilient Distributed Dataset)** is a fundamental distributed data abstraction in Apache Spark.

RDD Characteristics

1. Resilient

RDDs are **fault tolerant**.

If a partition is lost because a worker node fails, Spark can use **Lineage** to recompute the lost partition.

2. Distributed

RDD data is divided into **partitions** and distributed across worker nodes.

3. Dataset

An RDD represents a distributed collection of data.

RDD Immutability

RDDs are **immutable**.

Once an RDD has been created, it cannot be modified.

Instead, transformations create a **new RDD** from an existing RDD.

For example:

RDD 1

↓ map()

RDD 2

↓ filter()

RDD 3

The original RDDs remain unchanged.

---

9. RDD Fault Tolerance & Lineage

One of Spark's key strengths is its ability to recover lost data without necessarily requiring a complete copy of every intermediate dataset.

Suppose:

RDD 1 → RDD 2 → RDD 3 → RDD 4

If a partition of **RDD 4** is lost because a worker fails, Spark can use the transformation history—called **Lineage**—to recompute that partition.

Key Concept

> **Lineage = The history of transformations used to build an RDD.**

This is one of the reasons RDDs are considered **resilient**.

---

10. Transformations vs. Actions

Spark operations are broadly divided into two categories:

Type	Description	Evaluation	Examples
Transformation	Creates a new RDD/DataFrame from an existing dataset	**Lazy**	`map()`, `filter()`
Action	Triggers execution and returns or writes a result	**Triggers execution**	`collect()`, `saveAsTextFile()`

Transformations

A transformation creates a new dataset based on an existing dataset.

Examples:

map()

filter()

flatMap()

reduceByKey()

Transformations are **lazy**.

When a transformation is called, Spark does not immediately execute it.

Instead, Spark records the required operation.

Actions

An Action triggers actual execution of the accumulated transformations.

Examples:

collect()

saveAsTextFile()

count()

When an Action is encountered, Spark evaluates the required execution plan.

Simple Rule

> **Transformation = What should Spark do?**
>
> **Action = Now actually do it.**

---

11. Lazy Evaluation & DAG

What is Lazy Evaluation?

Spark uses **Lazy Evaluation** for transformations.

For example:

rdd1 = ...

rdd2 = rdd1.map(...)

rdd3 = rdd2.filter(...)

Spark does not immediately execute `map()` or `filter()`.

Instead, it builds an execution plan.

Only when an Action is called does Spark execute the required operations.

Directed Acyclic Graph — DAG

The execution plan created by Spark is represented as a **DAG (Directed Acyclic Graph)**.

Example:

Load Data

↓

map()

↓

filter()

↓

reduceByKey()

↓

collect()

↓

Execution

Why is Lazy Evaluation Important?

Because Spark waits until it sees the complete execution plan, it can optimize the processing.

For example:

1 Billion Records

↓

Filter

↓

5 Records

Instead of unnecessarily processing all 1 billion records through every possible operation, Spark can optimize the execution plan so that filtering happens as early as possible.

> **Lazy Evaluation allows Spark to understand the complete DAG before execution and optimize the execution plan.**

---

12. Spark Execution: Word Count Example

To write code using PySpark, an entry point to the Spark environment is required.

SparkSession

SparkSession is the modern entry point for Spark applications.

It provides access to Spark functionality and consolidates functionality that historically existed across:

- SparkContext
- SQLContext
- HiveContext

When working directly with RDDs, the **SparkContext** is still accessible through the SparkSession.

PySpark RDD Word Count Workflow

Step 1 — Load Data

```
rdd1 = spark.sparkContext.textFile(
    "/user/itv005857/data/inputfile.txt"
)
```

This loads the input file into an RDD.

Step 2 — Split Lines into Words

```
rdd2 = rdd1.flatMap(
    lambda x: x.split(" ")
)
```

`flatMap()` splits each line into individual words.

Example:

Input:

"hello world"

Output:

hello

world

Step 3 — Assign Count of 1

```
rdd3 = rdd2.map(
    lambda word: (word, 1)
)
```

Each word is converted into a key-value pair:

hello → (hello, 1)

world → (world, 1)

hello → (hello, 1)

Step 4 — Aggregate by Key

```
rdd4 = rdd3.reduceByKey(
    lambda x, y: x + y
)
```

`reduceByKey()` combines values belonging to the same key.

Example:

hello → [1, 1, 1]

becomes:

hello → 3

Step 5 — Trigger Execution & Save Output

```
rdd4.saveAsTextFile(
    "/user/itv005857/data/newoutput"
)
```

This is an **Action**, so Spark executes the accumulated DAG.

Alternatively:

```
rdd4.collect()
```

can be used to bring the result back to the driver for display.

> **Caution:** Avoid using `collect()` on very large datasets because it brings the entire result into the driver's memory and can cause an **Out Of Memory (OOM)** error.

---

13. Complete PySpark Word Count Example

```
# 1. Load data into an RDD
rdd1 = spark.sparkContext.textFile(
    "/user/itv005857/data/inputfile.txt"
)
```

```
# 2. Transformations (Lazy)
rdd2 = rdd1.flatMap(
    lambda x: x.split(" ")
)

rdd3 = rdd2.map(
    lambda word: (word, 1)
)

rdd4 = rdd3.reduceByKey(
    lambda x, y: x + y
)

# 3. Action (Triggers execution of the DAG)
rdd4.saveAsTextFile(
    "/user/itv005857/data/newoutput"
)

# Alternative: use collect() to print results
# Use with caution on large datasets
# rdd4.collect()
```

Quick Revision

Concept	Remember
Mapper	Provides parallel processing and generates intermediate key-value pairs
Reducer	Aggregates values by key
Partitioner	Determines which Reducer receives each key
Shuffle	Transfers Mapper output across the network
Sort	Groups data by key before reduction
Combiner	Performs local aggregation before Shuffle
Spark	Distributed compute engine designed to overcome several MapReduce limitations
RDD	Immutable, distributed, fault-tolerant data abstraction
Lineage	Transformation history used to recompute lost RDD partitions
Transformation	Lazy operation that creates a new dataset
Action	Triggers execution

DAG	Logical execution plan representing dependencies between operations
Lazy Evaluation	Allows Spark to optimize the execution plan before running it
SparkSession	Modern entry point for Spark applications
`collect()`	Brings results to the driver; risky for large datasets
`saveAsTextFile()`	Writes RDD output to storage

Important Concepts to Remember

1. **Mapper = Parallelism; Reducer = Aggregation.**
2. **Try to perform as much computation as possible before the Shuffle stage.**
3. **Number of partitions generally equals the number of Reducers.**
4. **Shuffle is expensive because it involves network data transfer.**
5. **A Combiner reduces the amount of data transferred during Shuffle by performing local aggregation.**
6. **Spark is a compute engine, not a storage system.**
7. **RDDs are immutable and fault tolerant through Lineage.**
8. **Transformations are lazy; Actions trigger execution.**
9. **Lazy Evaluation allows Spark to build and optimize the DAG before execution.**

10. `collect()` should be used carefully because it brings the complete result to the Driver.

One-line mental model for Week 3

MapReduce:
Map → Partition → Shuffle → Sort → Reduce

Spark:
Load → Transform → Build DAG → Action → Execute

RDD:
Immutable + Distributed + Fault Tolerant + Lineage

Week 4: Apache Spark Core APIs (RDD)

Study Guide | Ultimate Big Data Masters Program

---

1. Python Foundations for PySpark

Normal Functions vs. Lambda Functions

Normal Function

A **normal function** is defined using `def`, has an explicit name, and can be reused throughout a program.

```
def my_sum(x, y):
    return x + y
```

Lambda Function

A **lambda function** is an anonymous, inline function generally used for short, single-use logic.

lambda x, y: x + y

Comparison

Feature	Normal Function	Lambda Function
Definition	Uses `def`	Uses `lambda`
Name	Explicitly named	Usually anonymous
Use Case	Reusable / complex logic	Short, inline operations
Example	`def my_sum(x, y):`	`lambda x, y: x + y`

Python `map()` / `reduce()` vs. Spark `map()` / `reduce()`

The names are similar, but the execution model is different.

Python

Python `map()` and `reduce()` are higher-order functions that generally operate locally on the machine running the Python program.

Spark

Spark `map()` and `reduce()` are distributed operations that can execute across multiple cluster worker nodes.

> **Key Difference:** Python operations are primarily local, while Spark operations are designed for distributed processing.

---

2. End-to-End PySpark Practice Use Cases

Dataset Schema

order_id, date, customer_id, order_status

Sample Input

1,2013-07-25 00:00:00.0,11599,CLOSED

Spark Session Initialization

```
from pyspark.sql import SparkSession

import getpass


username = getpass.getuser()


spark = SparkSession.builder \

    .config('spark.ui.port', '0') \

    .config("spark.sql.warehouse.dir", f"/user/{username}/warehouse") \

    .enableHiveSupport() \

    .master('yarn') \

    .getOrCreate()
```

Load Data into an RDD

```
orders_rdd = spark.sparkContext.textFile(
    "/public/trendytech/retail_db/orders/*"
)
```

Use Case 1 — Count Orders by Category

Requirement: Count orders in each category and sort the result by count in descending order.

```
mapped_orders = orders_rdd.map(
    lambda x: (x.split(",")[3], 1)
)

category_counts = mapped_orders.reduceByKey(
    lambda x, y: x + y
)

sorted_category_counts = category_counts.sortBy(
    lambda x: x[1],
    False
)
```

Execution Logic

Orders

↓

Extract order_status

↓

(status, 1)

↓

reduceByKey()

↓

Count by category

↓

sortBy(count, descending)

Use Case 2 — Top 10 Premium Customers

Requirement: Find the top 10 customers based on the number of orders placed.

```
customer_orders = orders_rdd.map(
    lambda x: (x.split(",")[2], 1)
)


top_10_customers = customer_orders \
    .reduceByKey(lambda x, y: x + y) \
    .sortBy(lambda x: x[1], False) \
    .take(10)
```

Execution Logic

Orders

↓

Extract customer_id

↓

(customer_id, 1)

↓

reduceByKey()

↓

Count orders per customer

↓

sortBy(count, descending)

↓

take(10)

Use Case 3 — Count Distinct Customers

Requirement: Count the number of unique customers who placed at least one order.

```
distinct_customers_count = orders_rdd \
    .map(lambda x: x.split(",")[2]) \
    .distinct() \
    .count()
```

Execution Logic

Orders

↓

Extract customer_id

↓

distinct()

↓

count()

Use Case 4 — Customers with Maximum CLOSED Orders

Requirement: Find customers with the highest number of `CLOSED` orders.

```
closed_orders = orders_rdd.filter(
    lambda x: x.split(",")[3] == 'CLOSED'
)


top_closed_customers = closed_orders \
    .map(lambda x: (x.split(",")[2], 1)) \
    .reduceByKey(lambda x, y: x + y) \
    .sortBy(lambda x: x[1], False)
```

Execution Logic

Orders

↓

filter(status == CLOSED)

↓

(customer_id, 1)

↓

reduceByKey()

↓

Count CLOSED orders

↓

sortBy(count, descending)

---

3. RDD Transformations & Actions Reference

Operation	Type	Behavior / Output Shape
`map(func)`	Transformation	**Narrow:** One input record produces one output record.
`filter(func)`	Transformation	**Narrow:** Filters records based on a condition; output size ≤ input size.
`flatMap(func)`	Transformation	**Narrow:** One input can produce multiple outputs; collections are flattened.
`reduceByKey(func)`	Transformation	**Wide:** Performs aggregation by key and involves Shuffle.

`sortBy(func)` / `sortByKey()`	Transformation	**Wide:** Requires data redistribution for global sorting.
`distinct()`	Transformation	**Wide:** Removes duplicate records across partitions.
`join()`	Transformation	**Wide:** Performs a key-based join across distributed partitions.
`reduce(func)`	Action	Aggregates all elements into a single final value.
`count()`	Action	Returns the total number of records.
`countByValue()`	Action	Returns the frequency of each unique value.

---

4. Partitioning & Development Workflow

Developing Locally with `parallelize()`

When developing Spark logic, loading a multi-terabyte dataset just to test a transformation is inefficient.

Instead, developers can use **`parallelize()`** to create an RDD from a small local collection.

Example

```
words = [
    "big", "Data", "Is", "SUPER", "Interesting",
```

```
    "BIG", "data", "IS", "A", "Trending", "technology"
]

result = spark.sparkContext.parallelize(words) \
    .map(lambda x: x.lower()) \
    .map(lambda x: (x, 1)) \
    .reduceByKey(lambda x, y: x + y) \
    .collect()
```

Development Flow

Small Local Collection

↓

parallelize()

↓

RDD

↓

map()

↓

map()

↓

reduceByKey()

↓

collect()

> **Best Practice:** Use small sample datasets during development and testing before running logic against large production datasets.

---

5. RDD Partitioning

Partitioning determines how data is divided into smaller logical pieces that can be processed in parallel.

Key Rule

A commonly taught relationship is:

> **Number of RDD partitions is influenced by the number of underlying input splits/blocks and the configuration of the Spark application.**

For example, a **1 GB file** with **128 MB HDFS blocks** may result in approximately:

1 GB ÷ 128 MB = 8 input blocks

which can provide approximately **8 input partitions**, subject to Spark's input-splitting behavior and configuration.

`getNumPartitions()`

Returns the number of partitions in an RDD.

rdd.getNumPartitions()

`defaultMinPartitions`

Defines a minimum baseline for the number of partitions used by certain Spark operations.

The default is commonly **2**, depending on Spark configuration.

`defaultParallelism`

Defines the default level of parallelism used by certain Spark operations.

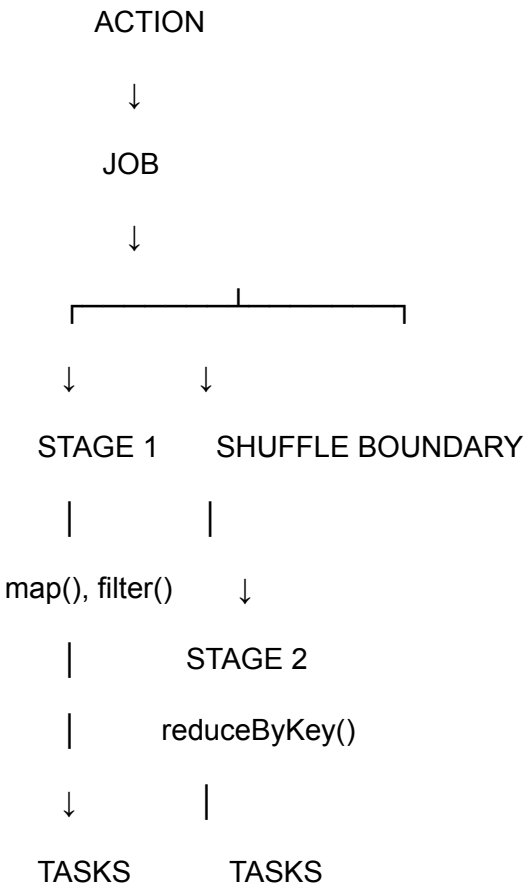
It represents the number of tasks that can generally execute concurrently based on the available cluster configuration.

---

6. DAG Execution Engine: Jobs, Stages & Tasks

Spark breaks execution into **Jobs → Stages → Tasks**.

Overall Flow



Job

A **Job** is created when an **Action** is invoked on an RDD.

Examples:

count()

collect()

saveAsTextFile()

Each Action can trigger a separate Spark Job.

Stage

A **Stage** is a subdivision of a Job.

Stages are separated by **Wide Transformations / Shuffle boundaries**.

For example:

map()

↓

filter()

↓

reduceByKey()

↓

sortBy()

↓

collect()

The Shuffle caused by `reduceByKey()` creates a stage boundary.

Simplified Formula

For a straightforward linear DAG:

Number of Stages ≈ Number of Shuffle Boundaries + 1

Task

A **Task** is the smallest unit of execution sent to a worker for processing a partition.

Key Relationship

For a given stage:

Number of Tasks ≈ Number of Partitions processed by that stage

Example:

Stage 1

├── Task 1 → Partition 1

├── Task 2 → Partition 2

├── Task 3 → Partition 3

└── Task 4 → Partition 4

Therefore:

> **One task generally processes one partition for a stage.**

---

7. Narrow vs. Wide Transformations

This is one of the **most important concepts in Spark**.

Narrow Transformation

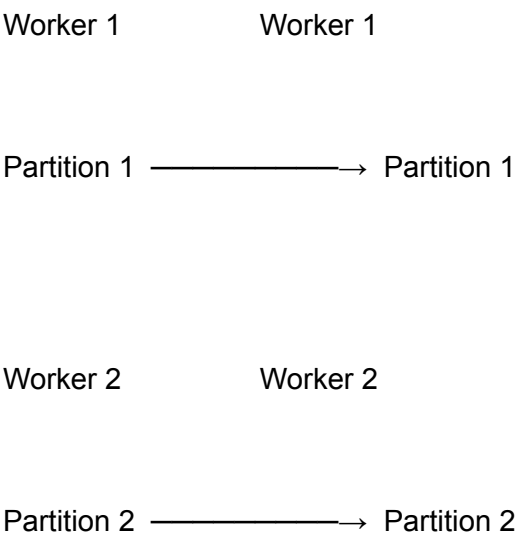
In a **Narrow Transformation**, each output partition depends on a small number of input partitions, typically without requiring a Shuffle across the cluster.

Examples:

- `map()`

- `filter()`
- `flatMap()`

Example



Data can generally remain on the same worker.

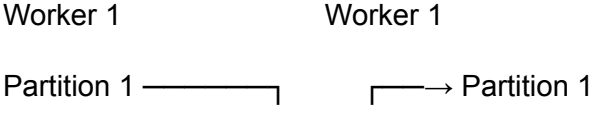
Wide Transformation

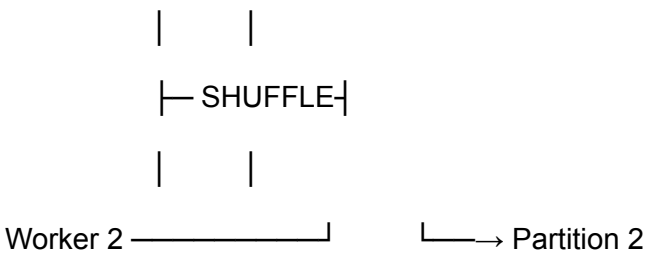
A **Wide Transformation** requires data to be redistributed across partitions, generally involving a **Shuffle**.

Examples:

- `reduceByKey()`
- `groupByKey()`
- `join()`
- `distinct()`
- `sortBy()`

Example





Comparison

Feature	Narrow Transformation	Wide Transformation
Shuffle	No	Yes
Network Transfer	Minimal / none	Significant
Performance	Generally faster	Generally more expensive
Stage Boundary	No	Yes
Examples	`map()`, `filter()`	`reduceByKey()`, `groupByKey()`

Remember: Wide Transformation = Shuffle = Stage Boundary

---

8. `reduceByKey()` vs. `groupByKey()`

Both operations work with key-value data, but they behave very differently.

`reduceByKey()`

`reduceByKey()` performs local aggregation before the Shuffle whenever possible.

Example:

Worker 1:

Apple → 1

Apple → 1

Apple → 1

Local aggregation:

Apple → 3

Only the aggregated result needs to be transferred during Shuffle.

Advantages

- Reduces network traffic.
- Uses local aggregation.
- More efficient for aggregations.
- Reduces memory pressure compared with sending all raw values.

`groupByKey()`

`groupByKey()` transfers all values associated with a key across the network before grouping them.

Example:

Apple → 1

Apple → 1

Apple → 1

Apple → 1

All values may be transferred before grouping:

Apple → [1, 1, 1, 1]

This can generate significant network and memory overhead.

Comparison

Feature	`reduceByKey()`	`groupByKey()`
Local Aggregation	Yes	No
Shuffle Data	Reduced	Potentially much larger
Network Traffic	Lower	Higher
Memory Pressure	Lower	Higher
Preferred for Aggregation	✅ Yes	❌ Usually avoid when aggregation can be performed directly

> **Rule of Thumb:** When performing an aggregation by key, prefer `reduceByKey()` over `groupByKey()` whenever the operation allows it.

---

9. Advanced RDD Optimization Techniques

Broadcast Join

A standard join between two large RDDs can require significant network Shuffle.

However, if one dataset is very small compared with the other, a **Broadcast Join** can be much more efficient.

Example

Suppose:

- `orders` = **1 GB**
- `customers` = **1 MB**

Instead of shuffling the large `orders` dataset, Spark can broadcast the small customer dataset to the worker nodes.

Broadcast Join Process

Step 1 — Broadcast Small Dataset

The Driver sends the small dataset to the worker nodes.

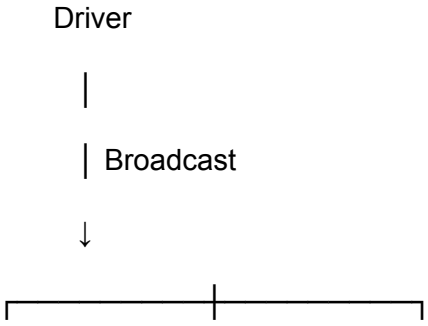
Step 2 — Local Lookup

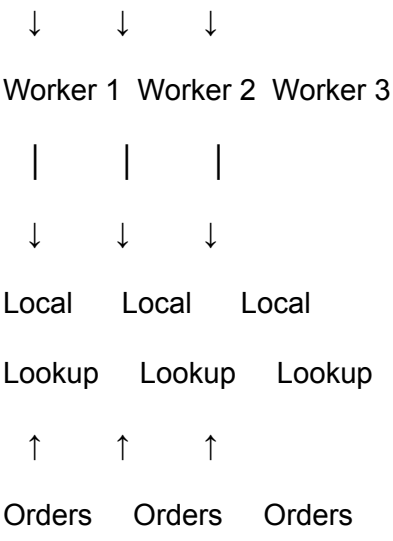
Each worker stores the broadcast dataset and performs local lookups.

Step 3 — Avoid Large Shuffle

The large RDD does not need to be redistributed simply to access the small lookup dataset.

Conceptual Architecture





Example

```
# Broadcast Small Dataset
customers_broadcast = spark.sparkContext.broadcast(
    customers_rdd.collectAsMap()
)

# Map-side local join on Large RDD
joined_rdd = orders_rdd.map(
    lambda x: (
        x,
        customers_broadcast.value.get(x.customer_id)
    )
)
```

Key Idea

Broadcast the small dataset; avoid shuffling the large dataset.

---

10. Partition Management: `repartition()` vs. `coalesce()`

Both functions modify the number of partitions, but their behavior is different.

Property	`repartition()`	`coalesce()`
Direction	Increase or decrease partitions	Primarily decreases partitions
Shuffle	Full Shuffle	Avoids full Shuffle when reducing partitions
Distribution	Generally produces more evenly distributed partitions	May result in uneven partition sizes
Best Use Case	Increasing parallelism or redistributing data	Reducing partitions after filtering
Cost	More expensive due to Shuffle	Generally cheaper when reducing partitions

`repartition()`

Use when you need to:

- Increase the number of partitions.
- Decrease partitions while accepting a full Shuffle.
- Redistribute data more evenly.

Example:

rdd.repartition(10)

`coalesce()`

Use primarily when reducing the number of partitions.

Example:

rdd.coalesce(5)

A common use case is after a heavy `filter()` operation has significantly reduced the amount of data.

> **Quick Rule:**
> **Increase partitions** → **`repartition()`**
> **Reduce partitions cheaply** → **`coalesce()`**

---

11. In-Memory Management: `cache()` vs. `persist()`

Spark transformations are lazy.

If an RDD is used repeatedly across multiple Actions, Spark may recompute its lineage each time unless the RDD is persisted.

Without Caching

RDD

↓

Transformation

↓

Action 1 → Recompute

↓

Action 2 → Recompute

↓

Action 3 → Recompute

With Caching

RDD

↓

Transformation

↓

Cache / Persist

↓

Memory

↓

Action 1

Action 2

Action 3

---

cache()

`cache()` is a convenient shortcut for persisting an RDD using Spark's default storage level.

For RDDs, this is commonly:

MEMORY_ONLY

rdd.cache()

---

persist()

`persist()` provides greater control over the storage level.

Common options include:

Storage Level	Meaning
MEMORY_ONLY	Store data in memory
MEMORY_AND_DISK	Store in memory; spill to disk if required
MEMORY_ONLY_SER	Store serialized objects in memory
DISK_ONLY	Store data only on disk

Example:

rdd.persist()

or:

rdd.persist(StorageLevel.MEMORY_AND_DISK)

`cache()` vs. `persist()`

- **`cache()` = Simple default persistence**
- **`persist()` = Configurable persistence**

Quick Revision

Concept	Remember
Lambda	Anonymous inline function for short logic
Spark `map()`	Distributed transformation
`parallelize()`	Creates an RDD from a local collection
Partition	Logical unit of distributed data processing
Narrow Transformation	No Shuffle; dependencies remain local
Wide Transformation	Requires Shuffle across partitions
Job	Triggered by an Action
Stage	Separated by Shuffle boundaries

Task	Processes a partition within a stage
`reduceByKey()`	Local aggregation + Shuffle; preferred for aggregation
`groupByKey()`	Shuffles raw values; generally more expensive
Broadcast Join	Sends small dataset to workers to avoid large Shuffle
`repartition()`	Full Shuffle; can increase or decrease partitions
`coalesce()`	Primarily reduces partitions without full Shuffle
`cache()`	Uses default persistence level
`persist()`	Allows explicit storage-level selection

Important Concepts to Remember

1. Narrow transformations do not require a Shuffle; Wide transformations do.

2. Wide transformations create Shuffle boundaries and therefore separate Spark Stages.

3. A Job is triggered by an Action.

4. A Task generally processes one partition for a particular Stage.

5. `reduceByKey()` is generally preferred over `groupByKey()` for aggregation because it performs local aggregation before Shuffle.

6. Broadcast Join is useful when one dataset is small enough to distribute to all worker nodes.

7. `repartition()` involves a full Shuffle, while `coalesce()` is primarily used to reduce partitions more efficiently.

8. Cache/persist reused RDDs to avoid repeatedly recomputing their lineage.

9. `cache()` provides convenient default persistence; `persist()` gives explicit control over the storage level.

Week 4 Mental Model

PySpark Basics

Python Function → Lambda → Spark Transformation

RDD Processing

RDD → Partition → Transformation → Shuffle (if Wide) → Action

Spark Execution

Action → Job → Stages → Tasks → Partitions

Performance

Narrow → No Shuffle → Faster

Wide → Shuffle → Network I/O → More Expensive

Aggregation

`reduceByKey()` → Local Aggregation → Less Shuffle

`groupByKey()` → Shuffle Raw Values → More Expensive

Partition Management

Increase / Redistribute → `repartition()`

Reduce → `coalesce()`

Reuse

Repeated RDD → `cache()` / `persist()` → **Avoid Recomputing Lineage**

WEEK 5

Spark APIs — DataFrames & Spark SQL

---

1. Introduction to Higher-Level APIs

Apache Spark provides multiple APIs for processing distributed data.

At a high level, Spark APIs can be divided into:

- **Low-Level APIs:** RDDs
- **Higher-Level APIs:** DataFrames and Spark SQL

RDDs contain distributed data without explicit schema information, whereas DataFrames and Spark SQL work with structured data containing **columns, data types, and schema metadata**.

Low-Level vs. High-Level Spark APIs

RDDs — Low-Level APIs	DataFrames & Spark SQL — High-Level APIs
Raw, unstructured data	Structured data
Data distributed across partitions	Data organized into rows and columns

No explicit schema context	Explicit schema and metadata
More manual optimization	Automatic optimization through Catalyst
More developer control	Easier and more optimized query processing

Key Concept

RDD → Data + Partitions

DataFrame → Data + Partitions + Schema

Spark SQL → DataFrame/Table + SQL Interface + Optimization

Persistent vs. Temporary Data

One important distinction is between **Spark SQL tables** and **DataFrames**.

Spark SQL / Spark Tables

Spark SQL tables are **persistent**.

- Physical data is stored on persistent storage such as:
  - HDFS
  - Amazon S3
  - Azure ADLS Gen2
- Table metadata/schema is maintained in a central **Metastore**.
- Tables continue to exist after the current Spark session ends.
- They can be accessed by different Spark applications or sessions.

DataFrames

DataFrames are effectively **RDDs enriched with schema**.

- DataFrames are session-scoped.
- Their temporary metadata is maintained in the Spark session.

- They disappear when the Spark session terminates unless persisted as a table or written to storage.

Why Are Higher-Level APIs Faster?

DataFrames and Spark SQL provide Spark with **structural information** about the data.

This allows Spark's **Catalyst Optimizer** to:

1. Understand the structure of the data.
2. Analyze the user's query.
3. Create an optimized logical execution plan.
4. Push filters closer to the data source.
5. Eliminate unnecessary columns.
6. Optimize the physical execution plan.
7. Reduce unnecessary resource consumption.

Key Takeaway

> **RDDs tell Spark HOW to process data.**
> **DataFrames and Spark SQL allow Spark to understand WHAT you want to achieve and optimize the execution.**

---

2. DataFrame Reader Framework

The **DataFrame Reader** provides a unified way to load external data into a DataFrame.

Standard Syntax

```
orders_df = spark.read \
    .format("csv") \
    .option("header", "true") \
    .option("inferSchema", "false") \
    .load("/public/trendytech/orders_wh/*")
```

Important Options

Option	Purpose
`format()`	Specifies the input file format
`option("header", "true")`	Indicates that the first row contains column names
`option("inferSchema", "false")`	Prevents Spark from automatically inferring data types
`load()`	Reads the actual data

`inferSchema` Performance Warning

Avoid using:

.option("inferSchema", "true")

in production pipelines whenever possible.

Schema inference requires Spark to examine the data to determine appropriate data types. This can result in additional scanning and execution overhead.

Production Recommendation

Define the schema explicitly instead of relying on inference.

---

3. File Format Reader Shortcuts

Spark provides shortcuts for common file formats.

CSV

```
df_csv = spark.read.csv(
    "/path/to/data.csv",
    header=True,
    inferSchema=False
)
```

JSON

```
df_json = spark.read.json("/path/to/data.json")
```

Parquet

```
df_parquet = spark.read.parquet("/path/to/data.parquet")
```

Parquet is a **columnar file format** and contains schema information within the file.

ORC

```
df_orc = spark.read.orc("/path/to/data.orc")
```

Spark Table

```
df_table = spark.read.table("database_name.table_name")
```

---

4. DataFrames vs. Spark SQL Tables

DataFrames and Spark SQL views can be used together.

The general flow is:

DataFrame → Temporary View → Spark SQL

and the resulting table/view can then be queried back through Spark.

DataFrame → SQL View

orders_df.createOrReplaceTempView("orders")

Query the View

result_df = spark.sql("""

    SELECT *

    FROM orders

""")

---

5. Temporary Views

Spark provides several methods for creating temporary views.

`createTempView()`

Creates a session-scoped temporary view.

orders_df.createTempView("orders")

If a view with the same name already exists, Spark throws an error.

`createOrReplaceTempView()`

Creates a session-scoped temporary view and replaces an existing view with the same name.

orders_df.createOrReplaceTempView("orders")

This is commonly used during development.

`createGlobalTempView()`

Creates a global temporary view that can be accessed across multiple Spark sessions within the same Spark application.

orders_df.createGlobalTempView("orders")

Global temporary views are accessed through the `global_temp` database.

spark.sql("""

SELECT *

FROM global_temp.orders

""")

`createOrReplaceGlobalTempView()`

Creates or replaces a global temporary view.

orders_df.createOrReplaceGlobalTempView("orders")

Quick Comparison

Method	Scope	Replaces Existing View?
`createTempView()`	Current Spark session	No
`createOrReplaceTempView()`	Current Spark session	Yes
`createGlobalTempView()`	All sessions in the Spark application	No
`createOrReplaceGlobalTempView()`	All sessions in the Spark application	Yes

---

6. Managed vs. External Tables

When creating persistent tables through Spark SQL / Hive Metastore, tables can be broadly classified as:

1. **Managed Tables**
2. **External Tables**

Managed Table

A managed table is one where Spark manages both:

- Metadata
- Physical data

Typical structure:

Managed Table

|

+---- Metadata → Hive Metastore

|

+---- Data → Spark Warehouse

DROP TABLE Behavior

When a managed table is dropped:

Metadata → Deleted

Physical Data → Deleted

External Table

An external table stores metadata in the Metastore while the actual data remains in an externally specified location.

External Table

|

+---- Metadata → Hive Metastore

|

+---- Data → External Storage Location

DROP TABLE Behavior

When an external table is dropped:

Metadata → Deleted

Physical Data → Preserved

Managed vs. External Tables

Feature	Managed Table	External Table
Data ownership	Spark manages metadata and data	Spark manages metadata only
Storage location	Default warehouse directory	Explicit `LOCATION`
Metadata	Hive Metastore	Hive Metastore
`DROP TABLE`	Deletes metadata + data	Deletes metadata only
Data preserved after DROP?	❌ No	✅ Yes
Typical use	Dedicated pipelines	Shared enterprise data

Example: Managed Table

```
CREATE TABLE orders (
    order_id BIGINT,
    customer_id BIGINT,
    order_status STRING
);
```

Example: External Table

```
CREATE TABLE orders (
    order_id BIGINT,
    customer_id BIGINT,
    order_status STRING
)
LOCATION '/data/orders';
```

---

7. DML Operations: Apache Spark vs. Databricks / Delta Lake

Standard file formats such as CSV and Parquet do not natively provide the same transactional capabilities as Delta Lake.

Open-Source Apache Spark

Generally supports:

- `SELECT`
- `INSERT INTO`

Standard CSV/Parquet tables do not natively support:

- `UPDATE`
- `DELETE`

Databricks / Delta Lake

Delta Lake provides support for:

- `INSERT`
- `UPDATE`

- `DELETE`
- `MERGE`

This enables ACID-style transactional operations on data.

---

8. DataFrame API vs. Spark SQL API

The same analytical requirement can generally be implemented using either:

- **DataFrame API**
- **Spark SQL**

Example 1: Top 15 Customers by Order Count

DataFrame API

```
result_df = orders_df.groupBy("customer_id") \
                .count() \
                .sort("count", ascending=False) \
                .limit(15)
```

Spark SQL API

```
result_sql = spark.sql("""
    SELECT customer_id, COUNT(order_id) AS count
    FROM orders
    GROUP BY customer_id
    ORDER BY count DESC
    LIMIT 15
```

""")

Example 2: Active Customer Count

DataFrame API

```
active_customers = orders_df \
    .select("customer_id") \
    .distinct() \
    .count()
```

Spark SQL API

```
active_customers_sql = spark.sql("""
    SELECT COUNT(DISTINCT customer_id) AS active_customers
    FROM orders
""")
```

---

9. Spark Method Categorization

Method	Type	Description
`filter()`	Transformation	Filters rows

`where()`	Transformation	Filters rows using a condition
`select()`	Transformation	Selects columns
`groupBy().count()`	Transformation	Groups and aggregates data
`orderBy()`	Transformation	Sorts DataFrame
`show()`	Action	Displays rows
`take()`	Action	Returns specified number of rows
`head()`	Action	Returns first row(s)
`collect()`	Action	Brings all rows to the driver
`count()`	Action	Counts total rows
`printSchema()`	Utility	Displays schema
`cache()`	Utility	Marks DataFrame for caching

Important

`collect()` should be used carefully with large datasets.

It brings the entire result to the **Driver**, which can cause an **Out Of Memory (OOM)** error.

---

10. Cluster-Level Resource Optimization

Spark performance depends heavily on appropriate executor configuration.

Example Worker Node

Assume:

- **10 Worker Nodes**
- **16 CPU Cores per Worker**
- **64 GB RAM per Worker**

The goal is to divide each worker's resources into balanced Spark executors.

---

11. Extreme Executor Strategies

There are two extreme approaches:

Strategy 1: Thin Executors

One executor gets:

- 1 CPU core
- 4 GB RAM

Therefore:

16 executors per worker

Problems

1. Multithreading within each JVM is eliminated.
2. Broadcast variables may need separate copies in many executors.
3. Memory can be duplicated unnecessarily.

Strategy 2: Fat Executors

One executor gets:

- 16 CPU cores
- 64 GB RAM

Therefore:

1 executor per worker

Problems

1. HDFS I/O throughput can suffer with too many concurrent threads.
2. Large JVM memory pools can lead to long Garbage Collection pauses.
3. A single large executor can become a bottleneck.

---

12. Balanced Executor Strategy

A balanced approach is preferred.

Worker Node

Worker Node

16 CPU Cores

64 GB RAM

|

|-- Reserve 1 Core + 1 GB RAM

|

+-- Available to Spark

15 Cores

63 GB RAM

Use:

5 CPU cores per executor

Therefore:

15 available cores ÷ 5 cores per executor

= 3 executors per worker

---

13. Executor Memory Calculation

Step 1: Calculate Available Resources

Total worker resources:

16 CPU cores + 64 GB RAM

Reserve:

1 CPU core + 1 GB RAM

Therefore:

Available CPU = 16 - 1

= 15 cores

Available RAM = 64 - 1

= 63 GB

Step 2: Calculate Executors per Worker

Using 5 cores per executor:

Executors per worker

= 15 / 5

= 3 executors

Step 3: Calculate Raw RAM per Executor

Raw RAM per executor

= 63 GB / 3

= 21 GB

Step 4: Calculate Memory Overhead

The notes use:

Overhead = max(384 MB, 7% × Executor Memory)

For a 21 GB executor:

7% × 21 GB

≈ 1.5 GB

Therefore:

Usable Executor RAM

= 21 GB - 1.5 GB

≈ 19 GB

14. Final Cluster Configuration

For **10 worker nodes**:

10 Workers

×

3 Executors per Worker

=

30 Executors

One executor is reserved for the **YARN ApplicationMaster** according to the configuration described in the notes.

Therefore:

30 - 1

=

29 Usable Executors

Final Configuration

Resource	Configuration
Worker Nodes	10
CPU per Worker	16 cores

RAM per Worker	64 GB
Executors per Worker	3
Total Executors	30
ApplicationMaster Allocation	1
Usable Executors	**29**
CPU per Executor	**5 cores**
Usable Heap RAM per Executor	**~19 GB**

---

15. Quick Revision Sheet

DataFrames & Spark SQL

RDD

→ Low-level API
→ Raw distributed data
→ No explicit schema

DataFrame

→ RDD + Schema
→ Structured rows and columns
→ Catalyst optimization

Spark SQL

→ SQL interface for structured data
→ Uses Catalyst Optimizer

Temporary Views

createTempView()

↓

Session Scoped

createOrReplaceTempView()

↓

Session Scoped + Replace

createGlobalTempView()

↓

Application Scoped

createOrReplaceGlobalTempView()

↓

Application Scoped + Replace

Managed vs External

MANAGED TABLE

DROP TABLE

↓

Metadata deleted

+

Data deleted

EXTERNAL TABLE

DROP TABLE

↓

Metadata deleted

+

Data preserved

Transformations vs Actions

Transformations

↓

Lazy

↓

Build Execution Plan

Actions

↓

Trigger Execution

↓

Return / Display Result

Executor Sizing

Worker

16 Cores + 64 GB

↓

Reserve

1 Core + 1 GB

↓

15 Cores + 63 GB

↓

3 Executors

↓

5 Cores / Executor

↓

21 GB Raw RAM / Executor

↓

~1.5 GB Overhead

↓

~19 GB Usable Heap / Executor

---

16. Key Interview Points

1. Why are DataFrames generally preferred over RDDs?

Because DataFrames provide schema information that allows Spark's **Catalyst Optimizer** to optimize the execution plan.

2. What happens when a managed table is dropped?

Both **metadata and physical data** are deleted.

3. What happens when an external table is dropped?

The **metadata is deleted**, but the underlying physical data remains.

4. What is the danger of `collect()`?

It brings the complete dataset to the Driver and can cause **Out Of Memory (OOM)** errors.

5. What is the difference between a temporary and global temporary view?

A normal temporary view is available within the current Spark session, while a global temporary view can be accessed across sessions within the same Spark application.

6. Why avoid `inferSchema` in production?

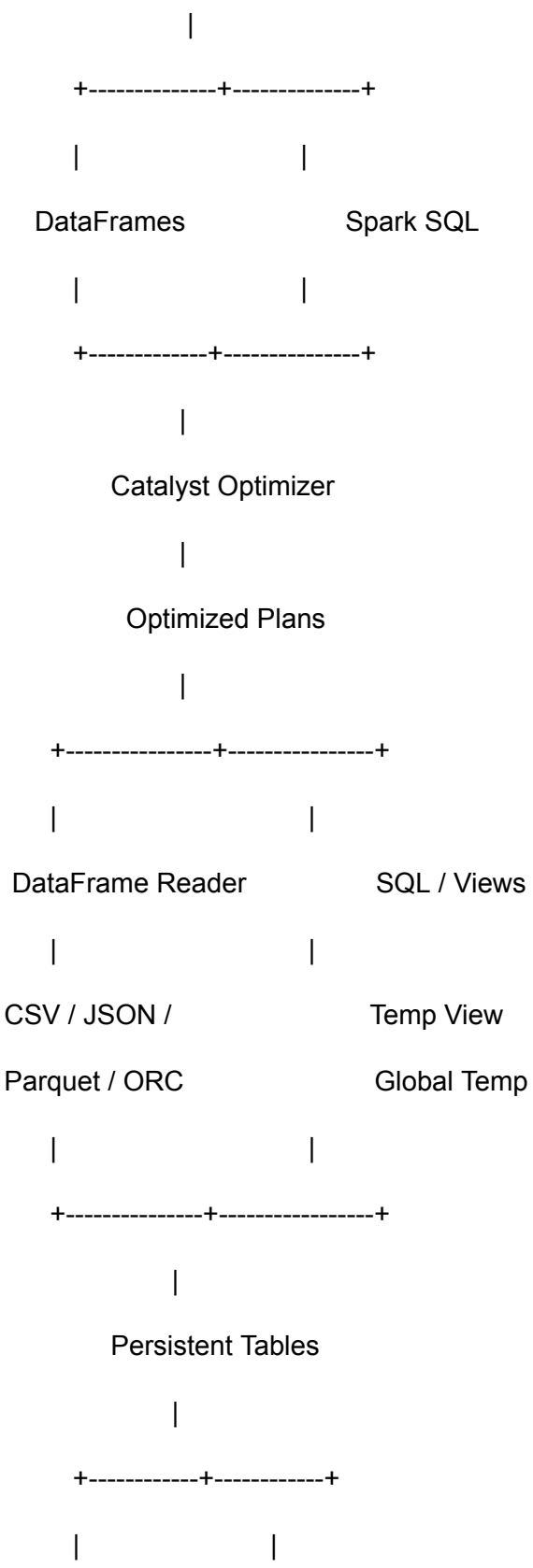
It introduces additional data scanning and can increase execution time; explicit schema definition is preferred.

7. What is the purpose of executor sizing?

To balance CPU, memory, I/O throughput, parallelism, and garbage-collection overhead for efficient Spark execution.

WEEK 5 — ONE-PAGE MEMORY MAP

SPARK HIGH-LEVEL APIs


DataFrames
Spark SQL
Catalyst Optimizer
Optimized Plans
DataFrame Reader
SQL / Views
CSV / JSON /
Parquet / ORC
Temp View
Global Temp
Persistent Tables

Managed External

| |

DROP = Data + DROP = Metadata

Metadata deleted only

Week 6: Spark DataFrame Transformations & Deep Dive

---

1. Schema Management: Inference vs. Enforcement

Schema inference allows Spark to automatically guess data types by scanning input data. However, it is **not recommended for production workloads**.

Disadvantages of Schema Inference

1. **Inaccurate Data Types:** Dates or timestamps may be incorrectly inferred as strings, or integers as longs.
2. **Performance Overhead:** Spark must scan the entire dataset (or a large sample) twice to infer column types, adding significant execution time.

> **Sampling Ratio Alternative:** If inference is mandatory during initial exploratory testing, use `.option("samplingRatio", 0.1)` to force Spark to scan only a **10% subset** of records.

Development Example

```
df = spark.read \
    .format("csv") \
    .option("header", "true") \
    .option("inferSchema", "true") \
```

```
    .option("samplingRatio", 0.1) \
    .load("/public/yelp-dataset/yelp_user.csv")
```

Enforcing Schema — Production Best Practice

Schema enforcement explicitly defines data types upfront, preventing extra data scans and ensuring data consistency.

If a data value fails type casting during read, Spark sets that specific cell value to `null` without crashing the query.

Schema Enforcement Approaches

Approach	Description
DDL String Schema	Compact SQL-like string defining column names and data types
Programmatic StructType	Programmatic schema definition using `StructType` and `StructField`

Approach 1: DDL String Schema

A compact DDL string defines column names and data types.

```
orders_schema = "order_id long, order_date string, cust_id long, order_status string"


df = spark.read \
    .format("csv") \
    .schema(orders_schema) \
```

```
    .load("/public/trendytech/datasets/orders_sample1.csv")
```

Approach 2: Programmatic `StructType` & `StructField`

This approach is useful for nested structures and programmatic data pipelines.

```
from pyspark.sql.types import StructType, StructField, LongType, StringType, IntegerType, DateType


orders_schema_struct = StructType([
    StructField("orderid", LongType(), nullable=True),
    StructField("orderdate", DateType(), nullable=True),
    StructField("customerid", IntegerType(), nullable=True),
    StructField("orderstatus", StringType(), nullable=True)
])


df = spark.read \
    .format("csv") \
    .schema(orders_schema_struct) \
    .load("/public/trendytech/datasets/orders_sample1.csv")
```

---

2. Handling Date & Timestamp Formats

The default date pattern expected by Apache Spark is:

`yyyy-MM-dd`

If incoming raw date values do not match this pattern, reading directly into `DateType` can result in `null` values or parsing errors.

Strategies to Handle Custom Date Patterns

Strategy 1: Explicit Date Format Option

Specify the exact custom pattern during the read operation.

```
df = spark.read \
    .format("csv") \
    .schema(orders_schema) \
    .option("dateFormat", "MM-dd-yyyy") \
    .load("/public/trendytech/datasets/orders_sample1.csv")
```

> **Important:** Month formatting is case-sensitive.
>
> `MM` = Month
> `mm` = Minutes

Using lowercase `mm` when you mean month can result in incorrect date parsing.

Strategy 2: Read as String and Cast

A recommended approach is to load raw date columns as `StringType` first and then explicitly convert them using `to_date()`.

```
from pyspark.sql.functions import to_date


# Load raw date as StringType
raw_df = spark.read \
    .schema("order_id long, order_date string, cust_id long, order_status string") \
```

```
    .csv("/public/trendytech/datasets/orders_sample1.csv")


# Convert string to DateType
transformed_df = raw_df.withColumn(
    "order_date_parsed",
    to_date("order_date", "MM-dd-yyyy")
)
```

---

3. DataFrame Read Modes

When reading structured data, corrupt or malformed records are processed according to the configured **Read Mode**.

Read Mode	Default	Behaviour on Malformed Records
`permissive`	**Yes**	Sets corrupted/mismatched fields to `null` while preserving valid fields
`dropmalformed`	No	Drops the entire corrupted row
`failfast`	No	Immediately throws an exception and stops Spark job execution

Example

```
df = spark.read \
```

```
    .format("csv") \
    .schema(orders_schema) \
    .option("mode", "dropmalformed") \
    .load("/public/trendytech/datasets/orders_sample1.csv")
```

Quick Memory Trick

PERMISSIVE → Preserve

DROPMALFORMED → Drop

FAILFAST → Fail

---

4. DataFrame Creation Patterns & RDD Conversion

Methods to Instantiate DataFrames

1. Reading External Files

```
df1 = spark.read.csv("path/to/file.csv")
```

2. Querying Spark SQL Tables / Views

```
df2 = spark.sql("SELECT * FROM default.orders")
```

```
df3 = spark.table("default.orders")
```

3. Generating Numeric Ranges

```
df_range = spark.range(0, 10, 2)
```

Produces an `id` column containing:

0, 2, 4, 6, 8

4. Converting In-Memory Python Lists

```
orders_list = [
    (1, '2013-07-25 00:00:00.0', 11599, 'CLOSED'),
    (2, '2013-07-25 00:00:00.0', 256, 'PENDING_PAYMENT')
]
```

Two-Step Name Assignment

```
df_list = spark.createDataFrame(orders_list) \
    .toDF("order_id", "order_date", "customer_id", "order_status")
```

One-Step Full Schema Enforcement

```
schema_str = "order_id long, order_date string, cust_id int, order_status string"

df_list_enforced = spark.createDataFrame(
    orders_list,
    schema=schema_str
)
```

Converting RDDs to DataFrames

DataFrames are conceptually RDDs bound to an explicit schema.

RDDs can be converted to DataFrames using two approaches:

Approach	Syntax
Approach 1 — SparkSession	`spark.createDataFrame(rdd, schema)`
Approach 2 — RDD Method	`rdd.toDF(schema)`

Sample RDD

```
raw_rdd = spark.sparkContext.parallelize([
    (1, '2013-07-25 00:00:00.0', 11599, 'CLOSED'),
    (2, '2013-07-25 00:00:00.0', 256, 'PENDING_PAYMENT')
])
```

Approach 1: Using SparkSession

```
df_from_rdd1 = spark.createDataFrame(
    raw_rdd,
    schema=schema_str
)
```

Approach 2: Using RDD's `toDF()` Method

```
df_from_rdd2 = raw_rdd.toDF([
```

```
    "order_id",
    "order_date",
    "customer_id",
    "order_status"
])
```

---

5. Working with Complex & Nested Schemas

JSON records and complex data formats often contain nested structures such as **structs** and **maps**.

Nested JSON Example

{"customer_id": 1, "fullname": {"firstname": "sumit", "lastname": "mittal"}, "city": "bangalore"}

{"customer_id": 2, "fullname": {"firstname": "ram", "lastname": "kumar"}, "city": "hyderabad"}

Approach 1: Nested DDL String

```
nested_ddl = "customer_id long, fullname struct<firstname:string, lastname:string>, city string"


df_nested_ddl = spark.read \
    .format("json") \
    .schema(nested_ddl) \
    .load("path/to/nested_customers.json")
```

Approach 2: Nested Programmatic `StructType`

```
nested_struct = StructType([
    StructField("customer_id", LongType()),
    StructField("fullname", StructType([
        StructField("firstname", StringType()),
        StructField("lastname", StringType())
    ])),
    StructField("city", StringType())
])


df_nested_struct = spark.read \
    .format("json") \
    .schema(nested_struct) \
    .load("path/to/nested_customers.json")
```

---

6. Key DataFrame Column Transformations

Transformation	Functional Purpose	Syntax Example
`withColumn()`	Adds a new column or overwrites an existing column	`df.withColumn("new_col", expr("subtotal * 0.1"))`

`withColumnRenamed()`	Renames an existing column without changing its data type or value	`df.withColumnRenamed("old_name", "new_name")`
`drop()`	Removes one or more columns	`df.drop("cust_id", "order_date")`

`select()` vs. `selectExpr()`

`select()`

Used for selecting existing column names or column objects.

SQL expressions must be wrapped inside the `expr()` function.

`selectExpr()`

Accepts raw SQL expression strings directly.

Example

```
from pyspark.sql.functions import expr


df_select = df.select(
    "order_item_id",
    expr("CASE WHEN product_name LIKE '%Nike%' THEN product_price * 1.2 ELSE product_price END AS adjusted_price")
)
```

Using `selectExpr()`:

```
df_select_expr = df.selectExpr(
    "order_item_id",
    "CASE WHEN product_name LIKE '%Nike%' THEN product_price * 1.2 ELSE product_price END AS adjusted_price"
)
```

Quick Comparison

Method	SQL Expression Handling
`select()`	Requires `expr()`
`selectExpr()`	Accepts SQL expressions directly

---

7. Deduplication: `distinct()` vs. `dropDuplicates()`

Sample Duplicate Data

```
data = [
    (1, "Kapil", 34),
    (1, "Kapil", 34),
    (1, "Satish", 26),
```

```
    (2, "Satish", 26)
]

df_dup = spark.createDataFrame(
    data,
    ["id", "name", "age"]
)
```

---

distinct()

Deduplicates records by evaluating **all columns simultaneously**.

```
df_unique_all = df_dup.distinct()
```

Example

The following two rows are duplicates:

1, Kapil, 34

1, Kapil, 34

`distinct()` removes one of them.

---

dropDuplicates()

Deduplicates records based on a **specific subset of key columns**.

```
df_unique_subset = df_dup.dropDuplicates(["name", "age"])
```

Quick Comparison

Method	Deduplication Criteria
`distinct()`	All columns
`dropDuplicates()`	Selected columns

---

8. Deep Dive into `SparkSession` & Deployment Modes

Role of `SparkSession`

Prior to Spark 2.0, applications managed separate contexts:

- **SparkContext** → RDD APIs
- **SQLContext** → DataFrames
- **HiveContext** → Hive Metastore access

Modern Spark unifies these under a single entry point:

`SparkSession`

SparkSession Encapsulation



/ | \

v v v

SparkContext SQLContext HiveContext

(RDD APIs) (DataFrames) (Metastore)

Shared Contexts & Multiple Sessions

A single Spark Application can run **only one `SparkContext`**.

However, multiple isolated `SparkSession` objects can be instantiated within the same application using:

spark.newSession()

These sessions:

- Run independently.
- Have separate temporary views.
- Can have separate settings.
- Share the underlying `SparkContext`.

Primary SparkSession

from pyspark.sql import SparkSession

spark = SparkSession.builder \

.appName("Production Pipeline") \

.config("spark.sql.warehouse.dir", "/user/itv005857/warehouse") \

.enableHiveSupport() \

```
    .master("yarn") \
    .getOrCreate()
```

Creating an Isolated Secondary Session

```
isolated_session = spark.newSession()
```

---

9. Deployment Modes: Client vs. Cluster

Spark applications can run in two common deployment modes:

1. **Client Mode**
2. **Cluster Mode**

Client Deployment Mode

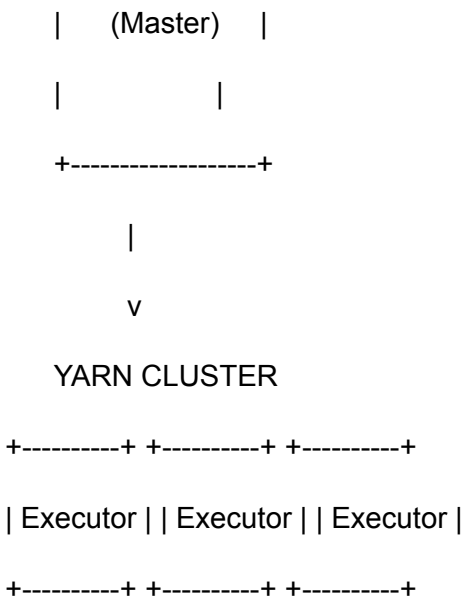
Typical Use: Interactive development

Examples:

- Jupyter Notebooks
- REPL
- Interactive development environments

Architecture

```
        CLIENT / GATEWAY NODE
        +-------------------+
        |                   |
        |  Driver Process   |
```



Characteristics

- The **Driver process runs locally** on the client/submission/gateway machine.
- Executors run on the YARN cluster.
- Useful for interactive development.

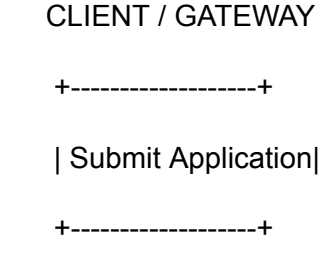
Risk

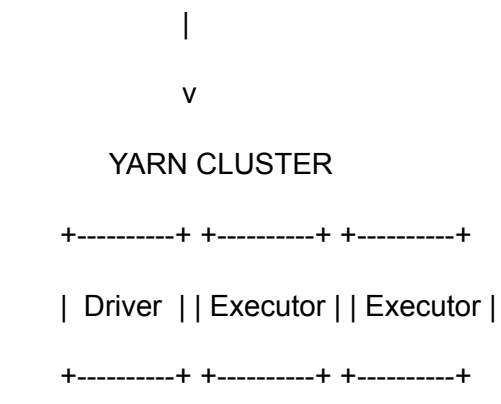
If the client machine disconnects or shuts down, the driver fails and the cluster job terminates.

Cluster Deployment Mode

Typical Use: Production environments

Architecture





Characteristics

- The **Driver process runs inside the cluster**.
- The driver runs within an **ApplicationMaster container** on one of the cluster worker nodes.
- The client only submits the application.

Benefits

- Better isolation from the local workstation.
- The application continues running even if the submitting machine disconnects.
- Driver–executor communication remains within the cluster.

Week 6 — Quick Revision

Schema Management

Schema Inference

→ Spark automatically determines data types
→ Convenient for exploration
→ Can be slower and inaccurate
→ Avoid for production pipelines

Schema Enforcement

→ Developer explicitly defines data types
→ More predictable
→ Better suited for production

Date Handling

Default Spark format:

yyyy-MM-dd

Custom format:

MM-dd-yyyy

Two approaches:

Read with `dateFormat`

OR

Read as String → Convert using `to_date()`

Read Modes

Mode	What Happens?
`permissive`	Invalid fields → `null`
`dropmalformed`	Invalid row → dropped

`failfast`	Invalid row → job fails

Memory Trick

PERMISSIVE → PRESERVE

DROPMALFORMED → DROP

FAILFAST → FAIL

DataFrame Creation

DataFrames can be created from:

Files → SQL → Tables → Ranges → Python Lists → RDDs

Column Transformations

Function	Purpose
`withColumn()`	Add / replace column
`withColumnRenamed()`	Rename column
`drop()`	Remove column
`select()`	Select columns / expressions

`selectExpr()`	Select using SQL expressions

Deduplication

distinct()

↓

Considers ALL columns

dropDuplicates()

↓

Considers SELECTED columns

Spark Context vs. Spark Session

One SparkContext per Spark Application

Multiple SparkSessions can exist within the same application

Spark Application

|

+---- SparkContext (ONE)

|

+---- SparkSession 1

|

+---- SparkSession 2

|

+---- SparkSession 3

Deployment Modes

Feature	Client Mode	Cluster Mode
Driver Location	Client / Gateway	Cluster
Typical Usage	Development / Interactive	Production
Client Disconnect	Job can terminate	Job can continue
Driver–Executor Communication	Driver outside cluster	Driver inside cluster

Remember

CLIENT MODE → Driver on Client → Development

CLUSTER MODE → Driver in Cluster → Production

Week 6 — Key Interview Points

1. Why is schema enforcement preferred over schema inference?
2. What is the difference between `StructType` and DDL schema?
3. How do you handle custom date formats in Spark?
4. What are the three Spark read modes?
5. How can you create a DataFrame from an RDD?
6. What is the difference between `select()` and `selectExpr()`?

7. What is the difference between `distinct()` and `dropDuplicates()`?
8. Why does Spark use `SparkSession`?
9. How many `SparkContext` objects can exist in one Spark application?
10. What is the difference between client mode and cluster mode?
11. Why is cluster mode generally preferred for production jobs?
12. What happens to a Spark job in client mode if the client machine disconnects?

WEEK 7: APACHE SPARK CACHING IN-DEPTH

---

1. Monitoring & Debugging: Spark UI and History Server

Monitoring job execution and resource consumption in Apache Spark relies mainly on two interfaces:

Spark UI vs. Spark History Server

Feature	Spark UI	Spark History Server
Availability	During job execution	After job completion
Purpose	Real-time monitoring	Post-mortem analysis
Data Source	Active SparkContext	Event logs
Main Use	Debug active jobs	Analyze completed jobs

Important Tabs	Jobs, Stages, Storage, Executors, SQL	Same historical execution information

Spark History Server

The **Spark History Server** reads event logs generated by completed Spark applications.

It is useful for:

- Analyzing completed jobs
- Post-mortem debugging
- Performance auditing
- Reviewing historical Spark applications

It becomes particularly useful after the application's `SparkContext` has terminated.

Spark UI

The **Spark UI** runs while the Spark application is active.

By default, it is commonly available on:

http://<driver-host>:4040

The Spark UI provides real-time information about:

- Jobs
- Stages
- Tasks
- Executors
- Storage
- DAG execution
- Cached datasets
- Resource utilization

Key Difference

Spark UI

↓

Real-Time Monitoring

↓

Application Currently Running

Spark History Server

↓

Historical Monitoring

↓

Application Already Completed

---

2. Fundamentals of Caching and Persistence

Spark uses **lazy evaluation**.

When transformations are applied to an RDD or DataFrame, Spark does not immediately execute them.

Instead, Spark builds a logical execution plan and executes it only when an action is triggered.

Without Caching

Suppose we have:

Source Data

↓

Transformation 1

↓

Transformation 2

↓

Action 1

After Action 1 finishes, Spark does not automatically keep the transformed dataset in memory.

If another action is executed:

Source Data

↓

Transformation 1

↓

Transformation 2

↓

Action 2

Spark may need to recompute the transformations again.

Therefore:

Action 1

↓

Full computation

Action 2

↓

Full computation AGAIN

With Caching

If the intermediate dataset is cached:

Source Data

↓

Transformation 1

↓

Transformation 2

↓

Cache

↓

Action 1

The resulting dataset is stored according to the selected storage level.

A subsequent action can reuse the cached data:

```
                ┌──→ Action 1
                │
Source → Transformations → Cache
                │
                └──→ Action 2
```

This avoids repeating expensive transformations.

When Should You Cache?

Cache when:

- The dataset is reused multiple times.
- The transformation logic is expensive.
- The dataset is reasonably sized.
- Multiple downstream actions depend on the same dataset.

Avoid caching when:

- The dataset is used only once.
- The dataset is extremely large.
- The dataset cannot fit effectively into available cluster storage.
- Recomputing the dataset is cheaper than maintaining the cache.

Lazy Evaluation of `cache()`

Calling:

df.cache()

does **not** immediately load the DataFrame into memory.

Caching is lazy.

The data is actually materialized when an action is executed.

Example:

df.cache()

df.count()

Here:

df.cache()

↓

Marks DataFrame for caching

df.count()

↓

Action

↓

Data is computed and cached

---

3. Data Serialization and Storage Formats

Cached data can be represented in two major forms:

1. **Deserialized**
2. **Serialized**

Deserialized Data

Data is stored as actual Java/Python objects in memory.

Advantages

- Faster CPU access
- No deserialization required
- Convenient for direct processing

Disadvantages

- Requires more memory
- Object headers create additional memory overhead

Serialized Data

Data is converted into a compact binary representation.

Advantages

- Requires less memory
- More space efficient

Disadvantages

- Requires CPU time to serialize and deserialize data
- Slightly slower access compared with directly available objects

---

Comparison

Property	Deserialized	Serialized
Representation	Java/Python objects	Binary byte stream
Memory usage	Higher	Lower
CPU usage	Lower	Higher
Access speed	Faster	Slower
Space efficiency	Lower	Higher

Simple Rule

Deserialized

↓

More Memory

Less CPU

Serialized

↓

Less Memory

More CPU

---

4. `cache()` vs. `persist()`

Caching and persistence allow Spark to store intermediate datasets for reuse.

`cache()`

`cache()` is a convenient shortcut for using the default storage level.

For an RDD:

rdd.cache()

The default storage level is:

MEMORY_ONLY

For a DataFrame:

df.cache()

The default storage level is:

MEMORY_AND_DISK

RDD Default Caching

rdd.cache()

Equivalent conceptually to:

rdd.persist(StorageLevel.MEMORY_ONLY)

The RDD attempts to remain in memory.

If a partition cannot be retained in memory, Spark can recompute that partition from lineage when required.

DataFrame Default Caching

df.cache()

Uses:

MEMORY_AND_DISK

Data is stored in memory where possible.

If memory is insufficient, partitions can spill to local disk.

`persist()`

`persist()` provides more control over the storage level.

Example:

from pyspark.storagelevel import StorageLevel

df.persist(StorageLevel.DISK_ONLY)

Other storage levels include:

StorageLevel.MEMORY_ONLY

StorageLevel.MEMORY_AND_DISK

StorageLevel.DISK_ONLY

StorageLevel.MEMORY_ONLY_2

Storage Level Concept

A storage level can be represented conceptually as:

```
StorageLevel(
    useDisk,
    useMemory,
    useOffHeap,
    deserialized,
    replication
)
```

For example:

df.persist(StorageLevel.DISK_ONLY)

means the cached data is stored only on disk.

Another example:

df.persist(StorageLevel.MEMORY_AND_DISK)

allows Spark to use both memory and disk.

Replication

Spark can maintain multiple copies of cached partitions.

For example:

df.persist(StorageLevel.MEMORY_ONLY_2)

The `_2` indicates a replication factor of 2.

This means cached partitions can have two copies across the cluster.

---

5. Locality Levels

Spark attempts to execute tasks as close as possible to the data.

This minimizes unnecessary network communication.

Two important locality levels are:

- `PROCESS_LOCAL`

- NODE_LOCAL

PROCESS_LOCAL

The data is already available inside the executor's process/JVM.

For example:

Executor JVM

↓

Cached Data in Memory

↓

Task

No disk or network transfer is required.

Therefore:

PROCESS_LOCAL

↓

Fastest

NODE_LOCAL

The data is located on the same physical worker node but is not already available inside the executor's memory.

For example:

Worker Node

↓

Local Disk / HDFS Block

↓

Executor

↓

Task

The task needs local disk access to retrieve the data.

Comparison

Locality	Data Location	Relative Speed
PROCESS_LOCAL	Executor memory	Fastest
NODE_LOCAL	Same worker node	Slower than process local

General Principle

Closer to Data

↓

Less Data Movement

↓

Better Performance

---

6. Spark SQL Caching

Spark SQL provides commands for caching tables and views.

CACHE TABLE

CACHE TABLE caching_demo_db.orders1;

This caches the table.

CACHE LAZY TABLE

CACHE LAZY TABLE caching_demo_db.orders1;

This marks the table for caching but delays materialization until the data is actually accessed.

Removing a Cached Table

UNCACHE TABLE caching_demo_db.orders1;

This removes the specified table from the cache.

Clearing All Cached Tables

CLEAR CACHE;

This clears all cached tables/views in the active session.

Important Difference

CACHE TABLE

↓

Eager caching

CACHE LAZY TABLE

↓

Lazy caching

---

7. Cache Invalidation and `REFRESH TABLE`

Spark interacts with multiple layers of data and metadata.

A simplified architecture is:

1. Backend Storage

HDFS / S3 / ADLS

↓

2. Spark Catalog / Metadata

↓

3. Memory Cache

Suppose a new file is manually added to HDFS.

The backend storage changes:

HDFS

↓

New File Added

But Spark's existing metadata and cache may not immediately know about the change.

Therefore, Spark may need to refresh the table.

REFRESH TABLE table_name;

This forces Spark to refresh its knowledge of the table's underlying data.

Internal vs. External Data Changes

Changes through Spark

If data is modified through Spark APIs, Spark can manage its metadata and cache state.

Changes outside Spark

If files are manually added, deleted, or modified in HDFS/S3, Spark may need an explicit refresh.

Example:

spark.sql("REFRESH TABLE table_name")

---

8. When Caching Can Hurt Performance

Caching is not always beneficial.

For example, Parquet files already contain useful metadata and columnar structures.

Parquet can support:

- Column pruning
- Predicate filtering
- Metadata-based optimizations
- Row-group-level optimizations
- Dictionary encoding

If such data is cached in a less efficient representation, Spark may consume significantly more memory.

Therefore:

Caching ≠ Always Faster

The correct rule is:

Cache when:

Expensive computation

+

Multiple reuse

+

Reasonable dataset size

Otherwise, caching can consume valuable cluster memory without providing meaningful performance benefits.

WEEK 8: APACHE SPARK ARCHITECTURE & ADVANCED AGGREGATIONS

---

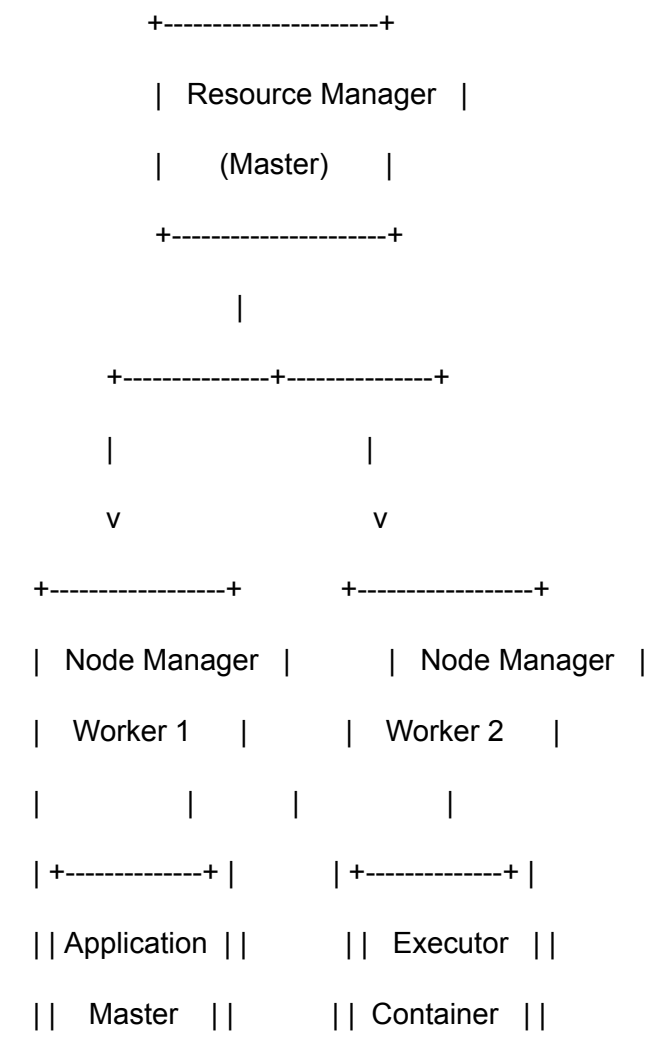
1. Spark on YARN Execution Architecture

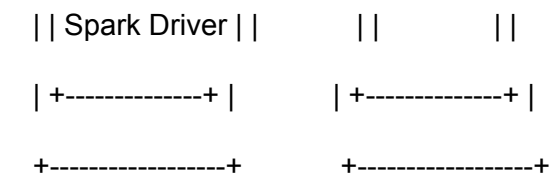
Apache Spark can run on top of Hadoop YARN.

The primary YARN components involved are:

- Resource Manager
- Node Manager
- ApplicationMaster
- Spark Driver
- Spark Executors

YARN Architecture





Execution Flow

Step 1: Application Submission

The client submits a Spark application.

For example:

spark-submit my_application.py

Step 2: Driver Allocation

The YARN Resource Manager coordinates with a Node Manager to launch the ApplicationMaster.

In Spark cluster mode:

ApplicationMaster

↓

Spark Driver

Step 3: Resource Negotiation

The ApplicationMaster communicates with the Resource Manager to obtain resources for Spark executors.

Step 4: Executor Launch

Executors are launched inside YARN containers.

The overall flow becomes:

Client

↓

Resource Manager

↓

ApplicationMaster

↓

Spark Driver

↓

Executors

↓

Tasks

---

2. Client Mode vs. Cluster Mode

Spark applications can commonly be deployed in:

1. Client Mode
2. Cluster Mode

Client Mode

In client mode, the Spark Driver runs on the machine from which the application is submitted.

Client / Gateway Machine

|

+-- Spark Driver

|

v

YARN Cluster

|

+-- Executor 1

+-- Executor 2

+-- Executor 3

Advantages

- Useful for interactive development
- Convenient for notebooks
- Useful for REPL/shell-based development

Disadvantage

If the client machine disconnects or shuts down, the Driver may fail and the Spark application can terminate.

Cluster Mode

In cluster mode, the Driver runs inside the cluster.

Client Machine

|

| Submit Application

v

YARN Cluster

|

+-- ApplicationMaster / Driver

|

+-- Executor 1

+-- Executor 2

+-- Executor 3

Advantages

- Driver is independent of the submitting workstation
- Suitable for production workloads
- Better isolation from client-side failures

Comparison

Feature	Client Mode	Cluster Mode
Driver Location	Client/Gateway machine	Inside cluster
Typical Usage	Development / Interactive	Production
Client Disconnect	Can terminate application	Application can continue
Driver Isolation	Lower	Higher

Easy Interview Answer

> **Client mode is mainly useful for interactive development, while cluster mode is preferred for production because the Driver runs inside the cluster rather than on the submitting machine.**

---

3. Five Ways of Referencing Columns in PySpark

PySpark provides multiple ways to reference DataFrame columns.

---

1. String Notation

Useful for straightforward column references.

df.select("order_id", "order_status")

---

2. DataFrame Prefix Notation

Useful when working with multiple DataFrames, especially joins.

df.select(orders_df.order_id, customers_df.cust_id)

This can help resolve ambiguity between columns having the same name.

---

3. Bracket / Dictionary Notation

df.select(df["order_date"])

---

4. Column Object Notation

Use `col()` or `column()` to work with column objects and their methods.

```
from pyspark.sql.functions import col, column


df.select(
    col("cust_id"),
    column("order_status")
)
```

Example:

```
df.where(
    col("order_status").like("PENDING%")
)
```

---

5. Expression Notation

The `expr()` function allows SQL expressions to be written directly.

```
from pyspark.sql.functions import expr


df.select(
    expr("cust_id + 1 AS new_cust_id")
)
```

Quick Reference

Method	Example	Common Use
String	`"order_id"`	Simple column selection
DataFrame prefix	`df.order_id`	Joins / ambiguity
Bracket	`df["order_id"]`	Column reference
`col()` / `column()`	`col("order_id")`	Programmatic operations
`expr()`	`expr("cust_id + 1")`	SQL expressions

---

4. Types of Aggregations

Aggregations can be broadly categorized into:

1. Simple Aggregations
2. Grouping Aggregations
3. Windowing Aggregations

Simple Aggregations

Simple aggregation processes multiple rows and produces a single result.

Examples:

- `count()`
- `sum()`
- `avg()`
- `countDistinct()`

Example:

```
from pyspark.sql.functions import count, countDistinct, sum, avg


df.select(
    count("*").alias("total_records"),
    countDistinct("invoiceno").alias("unique_invoices"),
    sum("quantity").alias("total_qty"),
    avg("unitprice").alias("avg_price")
).show()
```

Conceptually:

Multiple Rows

↓

Aggregation

↓

Single Result

---

5. Grouping Aggregations

Grouping aggregation first divides records into groups and then performs calculations for every group.

Example:

```
from pyspark.sql.functions import sum, expr


df.groupBy("country", "invoiceno") \
    .agg(
        sum("quantity").alias("total_qty"),
        expr("sum(quantity * unitprice) AS total_invoice_val")
    ) \
    .sort("invoiceno") \
    .show()
```

Conceptually:

All Rows

↓

Group By Key

↓

Aggregate Each Group

↓

One Result Row Per Group

---

6. Windowing Aggregations

Window functions perform calculations across related rows **without collapsing the rows into one row per group**.

This is the major difference between:

GROUP BY

and:

WINDOW FUNCTION

GROUP BY

Multiple rows

↓

Aggregation

↓

One row per group

Window Function

Multiple rows

↓

Calculation across related rows

↓

Original rows remain

---

7. Window Specification

A Window Specification commonly consists of:

1. `partitionBy()`

Defines the logical groups within which the calculation is performed.

2. `orderBy()`

Defines the ordering of rows inside each partition.

3. Frame Boundaries

Defines which rows are included in the calculation.

Common functions include:

rowsBetween()

rangeBetween()

Running Total Example

```
from pyspark.sql.window import Window

from pyspark.sql.functions import sum


running_window = Window \

    .partitionBy("country") \

    .orderBy("weeknum") \

    .rowsBetween(

        Window.unboundedPreceding,

        Window.currentRow

    )
```

```
df_with_running = df.withColumn(
    "running_total",
    sum("invoicevalue").over(running_window)
)
```

Conceptually:

Country

↓

Sort by Week

↓

Week 1 → Week 1 Total

Week 2 → Week 1 + Week 2

Week 3 → Week 1 + Week 2 + Week 3

...

---

8. `rank()` vs. `dense_rank()` vs. `row_number()`

These functions are commonly used for ranking records within a window.

Consider:

Student	Marks

Ankur	100
Satish	100
Kapil	100
Kaushik	99
Ram	99
Rohit	98

The ranking results are:

Student	Marks	`rank()`	`dense_rank()`	`row_number()`
Ankur	100	1	1	1
Satish	100	1	1	2
Kapil	100	1	1	3
Kaushik	99	4	2	4
Ram	99	4	2	5

Rohit	98	6	3		6

---

rank()

Rows with the same value receive the same rank.

However, the next rank skips numbers.

Example:

1, 1, 1, 4

---

dense_rank()

Rows with the same value receive the same rank, but ranks do not contain gaps.

Example:

1, 1, 1, 2

---

row_number()

Every row receives a unique sequential number.

Example:

1, 2, 3, 4, 5, 6

Code Example

```
from pyspark.sql.window import Window

from pyspark.sql.functions import rank, dense_rank, row_number, desc


rank_window = Window \

    .partitionBy("country") \

    .orderBy(desc("invoicevalue"))


df_ranked = df \

    .withColumn("rank", rank().over(rank_window)) \

    .withColumn("dense_rank", dense_rank().over(rank_window)) \

    .withColumn("row_num", row_number().over(rank_window))
```

---

9. `lead()` and `lag()`

Window functions can also be used to compare the current row with previous or future rows.

`lag()`

`lag()` retrieves a value from a previous row.

```
lag("invoicevalue", 1)
```

means:

Current Row

↓

Look 1 row backward

`lead()`

`lead()` retrieves a value from a subsequent row.

lead("invoicevalue", 1)

means:

Current Row

↓

Look 1 row forward

Example

```
from pyspark.sql.window import Window

from pyspark.sql.functions import lag, lead


window_lead_lag = Window \

    .partitionBy("country") \

    .orderBy("weeknum")
```

```
df_comparison = df \
    .withColumn(
        "prev_week_invoice",
        lag("invoicevalue", 1).over(window_lead_lag)
    ) \
    .withColumn(
        "next_week_invoice",
        lead("invoicevalue", 1).over(window_lead_lag)
    )
```

Conceptually:

Previous Row ← Current Row → Next Row

```
    lag()         lead()
      ↓             ↓
Previous Value    Next Value
```

---

10. Pivoting Log Data

Pivoting converts row-oriented data into a column-oriented summary.

For example, suppose server logs contain:

loglevel month

INFO January

ERROR January

INFO February

WARN February

A pivot can transform the data into:

	January	February
INFO	1	1
ERROR	1	0
WARN	0	1

Standard Pivot

```
pivoted_df = log_df \
    .groupBy("loglevel") \
    .pivot("month") \
    .count()
```

---

11. Pivot Performance Optimization

When the pivot column contains unknown values, Spark may need to scan the dataset to discover all distinct pivot values.

For example:

```
log_df \
    .groupBy("loglevel") \
```

```
    .pivot("month") \
    .count()
```

Spark may first need to discover all unique months.

This introduces an additional scan.

Optimized Pivot

If the possible values are already known, provide them explicitly.

```
months = [
    "January",
    "February",
    "March",
    "April",
    "May",
    "June",
    "July",
    "August",
    "September",
    "October",
    "November",
    "December"
]

optimized_pivot_df = log_df \
```

```
    .groupBy("loglevel") \
    .pivot("month", months) \
    .count()
```

Why is this better?

Without Explicit Values

Dataset

↓

Discover DISTINCT months

↓

Perform Pivot

With explicit values:

Known Month List

↓

Perform Pivot Directly

Therefore, providing the pivot values can eliminate the extra distinct-value discovery scan.

WEEK 7 + WEEK 8 QUICK REVISION

Week 7 — Caching

Remember:

cache()

↓

Reuse expensive computations

Key concepts:

- Spark UI → Real-time monitoring
- History Server → Completed application analysis
- `cache()` → Default storage level
- `persist()` → Custom storage level
- RDD default → `MEMORY_ONLY`
- DataFrame default → `MEMORY_AND_DISK`
- `PROCESS_LOCAL` → Fastest locality
- `REFRESH TABLE` → Refresh table metadata/data visibility
- Don't cache blindly

Week 8 — Architecture & Aggregations

Remember:

Client

↓

Resource Manager

↓

ApplicationMaster / Driver

↓

Executors

↓

Tasks

And:

Simple Aggregation

↓

Whole Dataset → Result

Grouping Aggregation

↓

Groups → One Result Per Group

Window Aggregation

↓

Related Rows → Calculation

↓

Original Rows Preserved

Ranking Cheat Sheet

rank()

→ Same rank for ties

→ Skips ranks

dense_rank()

→ Same rank for ties

→ Does NOT skip ranks

row_number()

→ Always unique

→ Sequential numbering

Window Cheat Sheet

partitionBy()

→ Defines groups

orderBy()

→ Defines order

rowsBetween()

rangeBetween()

→ Defines frame

Row Navigation

lag()

→ Previous row

lead()

→ Next row

Pivot Optimization

pivot(column)

↓

Spark discovers distinct values

↓

Extra scan

pivot(column, known_values)

↓

Skip discovery scan

↓

Better performance

Week 9: Apache Spark Internals & DataFrame Writer API

---

1. DataFrame Writer API & Write Modes

The **DataFrame Writer API (`df.write`)** is used to write transformed DataFrame results to internal data lakes such as HDFS, ADLS Gen2, and AWS S3, as well as external data targets such as relational databases, NoSQL systems, and data warehouses.

Basic DataFrame Writer Syntax

```
orders_df.write \
    .format("parquet") \
```

```
.mode("overwrite") \
.option("path", "/user/itv006763/sparkwritedemo") \
.save()
```

Supported Write Modes

Mode	Behavior
`overwrite`	Replaces existing files/directories at the target destination.
`append`	Adds new output files alongside existing data.
`ignore`	Silently skips the write if the target already exists.
`errorIfExists`	Default mode. Throws an `AnalysisException` if the target already exists.

Key Point

Write Mode = What should Spark do if the target already exists?

- `overwrite` → Replace it
- `append` → Add to it
- `ignore` → Do nothing
- `errorIfExists` → Throw an error

---

2. Storage Techniques: Partitioning vs. Bucketing

Spark provides **partitioning** and **bucketing** to organize data physically and reduce unnecessary data scanning during queries.

Partitioning vs. Bucketing

PARTITIONING (partitionBy)	BUCKETING (bucketBy)
• Low-cardinality columns	• High-cardinality columns
Example: country, status	Example: customer_id, id
• Creates folders	• Creates bucket files
• Enables partition pruning	• Optimizes joins and scans

2.1 Partitioning — `partitionBy()`

Partitioning is best suited for **low-cardinality columns** that are frequently used in filtering conditions.

Examples:

- `order_status`
- `country`
- `state`

How Partitioning Works

Spark creates separate directories for each unique partition value.

For example:

orders/

├── order_status=CLOSED/

├── order_status=COMPLETE/

├── order_status=PENDING/

└── order_status=CANCELLED/

If a query asks for:

WHERE order_status = 'CLOSED'

Spark can scan only the `CLOSED` directory instead of the entire dataset.

This is called **Partition Pruning**.

Single-Level Partitioning

```
orders_df.write \
    .format("parquet") \
    .mode("overwrite") \
    .partitionBy("order_status") \
    .save("/path/to/partitioned_orders")
```

Multi-Level Partitioning

Multiple columns can be used to create hierarchical directories.

```
customer_df.write \
    .format("parquet") \
    .mode("overwrite") \
    .partitionBy("customer_state", "customer_city") \
    .save("/path/to/nested_partitions")
```

This produces a structure similar to:

customer_state=Karnataka/

customer_city=Bangalore/

customer_state=Telangana/

customer_city=Hyderabad/

Best Practice

Use partitioning when the column has **low cardinality**.

Avoid partitioning on extremely high-cardinality columns such as:

customer_id

order_id

transaction_id

because this can create a very large number of directories and small files.

2.2 Bucketing — `bucketBy()`

Bucketing is designed for **high-cardinality columns**.

Examples:

- `customer_id`
- `order_id`
- `user_id`

Instead of creating one folder for every unique value, Spark distributes records across a **fixed number of bucket files**.

Bucketing Formula

Bucket Number = hash(column) % number_of_buckets

Example

```
customer_final_df.write \
    .format("parquet") \
    .mode("overwrite") \
    .bucketBy(8, "customer_id") \
    .sortBy("customer_id") \
    .saveAsTable("itv006753_bucketing_db.customersnew")
```

Here:

- Number of buckets = `8`
- Bucket column = `customer_id`
- Data is sorted by `customer_id`
- Output is saved as a Spark SQL table

Important

`bucketBy()` is used with:

.saveAsTable()

rather than a simple file-path `.save()` workflow.

2.3 Partitioning vs. Bucketing — Comparison

Feature	Partitioning	Bucketing
API	`partitionBy()`	`bucketBy()`
Ideal column	Low cardinality	High cardinality
Example	`country`, `status`	`customer_id`, `order_id`
Physical structure	Folders/directories	Bucket files
Main optimization	Partition pruning	Bucket pruning / SMB joins
Typical write	`.save()` or table	`.saveAsTable()`
Number of partitions/buckets	Based on unique values	Fixed number specified by developer

Remember

> **Low Cardinality → Partitioning**
>
> **High Cardinality → Bucketing**

---

3. Initial Partition Calculation Mechanics

Initial partitions determine the number of parallel tasks Spark can use when reading files into DataFrames.

The source material gives the following formula:

{Partition Size}

$$
\min
\left(
\text{maxPartitionBytes},
\frac{\text{File Size}}{\text{defaultParallelism}}
\right)
$$

The default value of:

spark.sql.files.maxPartitionBytes

is **128 MB**.

Example 1: 1.1 GB File, 2 Cores

File Size = 1.1 GB

Default Parallelism = 2

Partition Size

= min(128 MB, 1.1 GB / 2)

= 128 MB

Partitions

= 1.1 GB / 128 MB

≈ 9 partitions

Example 2: 1.1 GB File, 16 Cores

File Size = 1.1 GB

Default Parallelism = 16

Partition Size

= min(128 MB, 1.1 GB / 16)

= 75 MB

Partitions

= 1.1 GB / 75 MB

≈ 16 partitions

Key Learning

The number of initial partitions depends on:

1. File size
2. `spark.sql.files.maxPartitionBytes`
3. `defaultParallelism`

---

4. Splittable vs. Non-Splittable Files

Not every file can be divided into multiple input partitions.

Splittable Formats

Examples given in the source:

- Uncompressed CSV/Text
- Parquet
- ORC
- Avro

These can generally be split into multiple partitions.

Non-Splittable Formats

Examples given in the source:

- Gzip-compressed CSV
- Gzip-compressed JSON
- Snappy-compressed CSV/JSON

These can result in **one partition per file**, creating a potential single-threaded bottleneck.

Important Concept

Large Splittable File

↓

Multiple Input Partitions

↓

Multiple Tasks

↓

Parallel Processing

Whereas:

Large Non-Splittable File

↓

One Input Partition

↓

One Task

↓

Potential Bottleneck

---

5. Submitting Production Jobs Using `spark-submit`

Interactive notebooks typically use **Client Mode**, while production applications are commonly submitted using `spark-submit` in **Cluster Mode**.

Production Submission Example

```
spark3-submit \
  --deploy-mode cluster \
  --master yarn \
  --num-executors 4 \
  --executor-cores 5 \
  --executor-memory 19G \
  --driver-cores 2 \
  --driver-memory 4G \
  --conf spark.dynamicAllocation.enabled=false \
  /path/to/production_pipeline.py
```

Important Parameters

Parameter	Purpose
`--deploy-mode cluster`	Runs driver inside the cluster

`--master yarn`	Uses YARN as resource manager
`--num-executors 4`	Requests 4 executors
`--executor-cores 5`	Gives each executor 5 cores
`--executor-memory 19G`	Gives each executor 19 GB memory
`--driver-cores 2`	Gives driver 2 CPU cores
`--driver-memory 4G`	Gives driver 4 GB memory
`--conf`	Sets Spark configuration properties

Configuration Precedence

From **highest priority to lowest priority**:

1. SparkSession.builder configuration

↓

2. spark-submit arguments

↓

3. spark-defaults.conf

Therefore:

Application code > spark-submit > spark-defaults.conf

Week 9 — Quick Revision

DataFrame Writer

df.write

Write Modes

overwrite

append

ignore

errorIfExists

Storage Optimization

Low Cardinality → partitionBy()

High Cardinality → bucketBy()

Partition Pruning

Filter on partition column

↓

Spark skips irrelevant directories

↓

Less data scanned

Production Submission

spark-submit

↓

Cluster Mode

↓

YARN

↓

Driver + Executors

Week 10: Apache Spark Optimizations — Part 1

---

1. Internal Working of `groupBy()` & Empty Partition Problem

A `groupBy()` operation is a **wide transformation** because data needs to be shuffled across the cluster.

By default, Spark SQL uses:

spark.sql.shuffle.partitions = 200

This means Spark initially creates **200 shuffle partitions**.

Example

Suppose a dataset contains **9 distinct order statuses**.

Raw Dataset

↓

Local Partial Aggregation

↓

Shuffle

↓

200 Shuffle Partitions

Only the partitions corresponding to the actual keys may contain data.

Partitions 1–9

→ Active

Partitions 10–200

→ Empty

Therefore, Spark may end up creating many unnecessary tasks.

Problem

If:

Distinct Keys = 9

Shuffle Partitions = 200

then potentially:

9 active partitions

191 empty partitions

The empty tasks still create:

- Scheduler overhead
- Task-launch overhead
- Unnecessary execution planning

---

2. Adaptive Query Execution — AQE

Adaptive Query Execution (AQE) dynamically changes Spark's execution plan using runtime statistics.

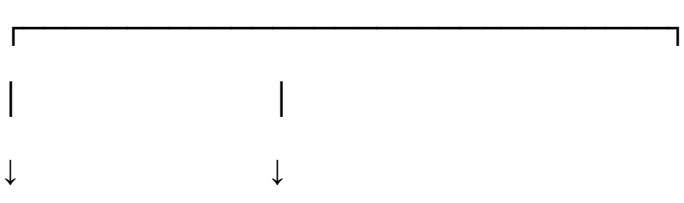
According to the source:

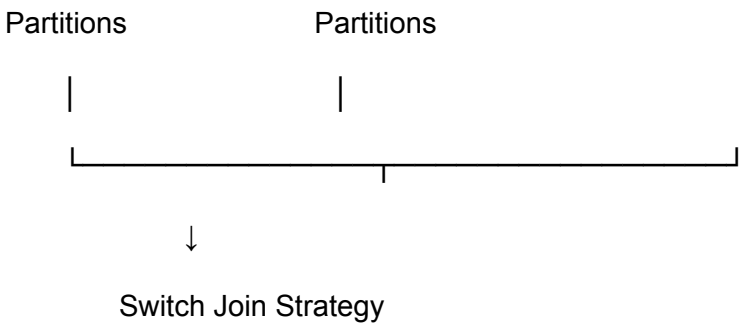
- Introduced in **Spark 3.0**
- Enabled by default in **Spark 3.2+**

Enable AQE

spark.conf.set("spark.sql.adaptive.enabled", True)

AQE has three major optimization capabilities:





2.1 AQE — Coalescing Shuffle Partitions

AQE can combine small or empty shuffle partitions after observing the actual runtime data.

Without AQE

200 shuffle partitions

↓

Only 9 contain useful data

↓

191 unnecessary tasks

With AQE

200 shuffle partitions

↓

Runtime statistics

↓

Small/empty partitions combined

↓

Fewer useful partitions

↓

Less task overhead

Key Benefit

> AQE dynamically adjusts partitioning based on actual runtime data rather than relying solely on static configuration.

2.2 AQE — Dynamic Partition Skew Handling

A **data skew** occurs when one partition contains significantly more data than others.

Example

Suppose:

COMPLETE → 80% of all records

PENDING → 10%

CANCELLED → 5%

OTHERS → 5%

The partition containing `COMPLETE` can become much larger than other partitions.

This creates a **straggler task**.

AQE can detect the skewed partition and split it into smaller sub-partitions during execution.

Result

Large Skewed Partition

↓

AQE detects skew

↓

Split into smaller partitions

↓

More balanced execution

2.3 AQE — Dynamic Join Strategy Switching

AQE can also change the join strategy at runtime.

For example:

Initial plan:

Sort-Merge Join

↓

Runtime filtering reduces dataset size

↓

Dataset now fits broadcast threshold

↓

AQE switches to:

Broadcast Hash Join

This can significantly improve join performance.

---

3. Join Types in PySpark

Join Type	PySpark Keyword	Behavior
Inner	`"inner"`	Keeps only matching records from both DataFrames.
Left Outer	`"left"` / `"left_outer"`	Keeps all left records; unmatched right columns become `NULL`.
Right Outer	`"right"` / `"right_outer"`	Keeps all right records; unmatched left columns become `NULL`.
Full Outer	`"full"` / `"full_outer"`	Keeps all records from both DataFrames.
Left Semi	`"left_semi"`	Returns left records that have a match in the right DataFrame. Right-side columns are not included.
Left Anti	`"left_anti"`	Returns left records that have no match in the right DataFrame.

---

4. Spark Join Execution Strategies

Spark can use different physical strategies depending on dataset sizes and execution conditions.

Three Important Strategies

1. Broadcast Hash Join

2. Shuffle Hash Join

3. Sort-Merge Join

4.1 Broadcast Hash Join — BHJ

Broadcast Hash Join is generally used when one DataFrame is sufficiently small.

The source specifies the default broadcast threshold as:

spark.sql.autoBroadcastJoinThreshold = 10 MB

Working

Suppose:

Orders = 1 GB

Customers = 5 MB

Spark can:

Small Customers DF

↓

Driver

↓

Broadcast

↓

All Executors

Large Orders DF

↓

No large-table shuffle

↓

Local Hash Lookup

Advantage

The large DataFrame does not need to be shuffled across the network.

Key Point

Small table + large table → Broadcast Hash Join

4.2 Shuffle Hash Join — SHJ

Shuffle Hash Join is applicable when datasets require shuffling but one side can still be hashed efficiently within each partition.

Working

DataFrame A

↓

Shuffle

↓

Partition by Join Key

↓

Hash

DataFrame B

↓

Shuffle

↓

Partition by Join Key

↓

Hash

↓

Join matching partitions

The smaller partition is used to build an in-memory hash table.

4.3 Sort-Merge Join — SMJ

Sort-Merge Join is commonly used when joining **two large DataFrames** that cannot be broadcast.

It involves three major steps:

1. Shuffle

↓

2. Sort

↓

3. Merge

Step 1 — Shuffle

Records are distributed according to the join key.

Step 2 — Sort

Each partition sorts records by the join key.

Step 3 — Merge

The sorted datasets are merged to find matching keys.

Key Point

Two large DataFrames → Shuffle + Sort + Merge

---

5. Sort-Merge-Bucket Join — SMB Join

When two very large tables are joined using a standard Sort-Merge Join, Spark may have to perform:

Shuffle

↓

Sort

↓

Merge

For very large datasets, the shuffle and sorting operations can be extremely expensive.

Bucketing can eliminate this repeated work.

SMB Join Requirements

For Spark to take advantage of a Sort-Merge-Bucket Join, the tables should be:

1. Bucketed on the same join key
2. Using the same number of buckets
3. Sorted by the join key

Step 1 — Bucket Orders Table

```
orders_df.write \
    .mode("overwrite") \
    .bucketBy(8, "cust_id") \
    .sortBy("cust_id") \
    .saveAsTable("bucketed_db.orders_table")
```

Step 2 — Bucket Customers Table

```
customers_df.write \
    .mode("overwrite") \
    .bucketBy(8, "customer_id") \
    .sortBy("customer_id") \
    .saveAsTable("bucketed_db.customers_table")
```

Both tables use:

Number of buckets = 8

Orders join key = cust_id

Customers join key = customer_id

The columns have different names but represent the corresponding join keys.

Step 3 — Perform the Join

```
joined_df = spark.sql("""
    SELECT *
    FROM bucketed_db.orders_table o
    JOIN bucketed_db.customers_table c
      ON o.cust_id = c.customer_id
""")
```

Why SMB Join Is Faster

Without bucketing:

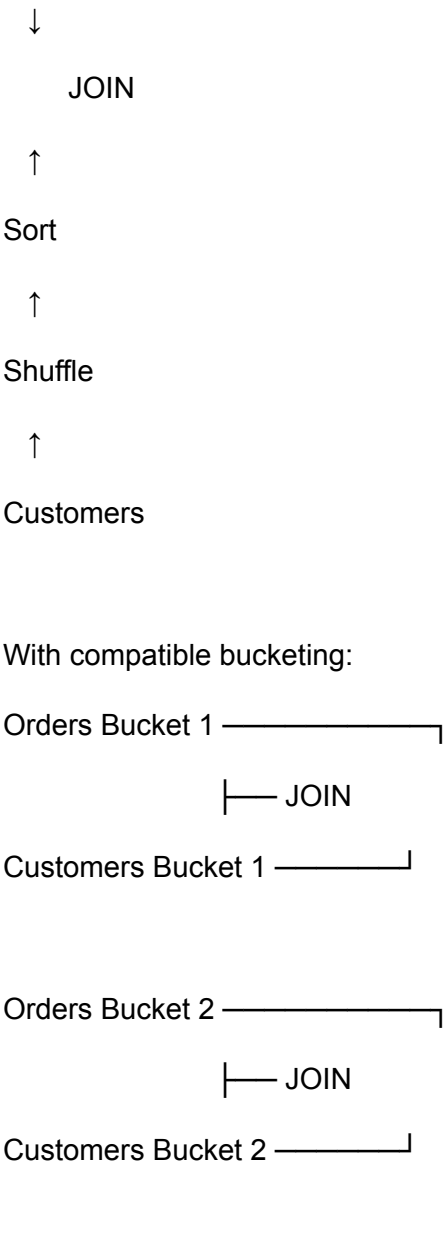
Orders

↓

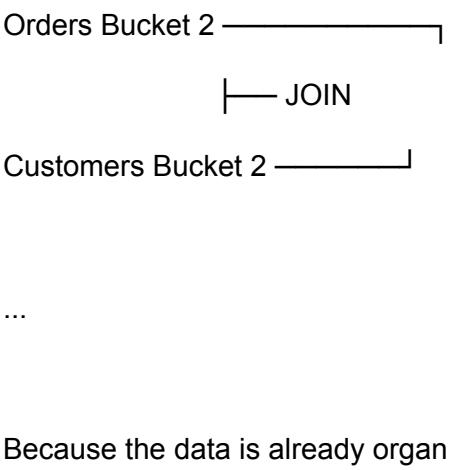
Shuffle

↓

Sort



With compatible bucketing:



...

Because the data is already organized into matching buckets and sorted by the join key, Spark can avoid the expensive shuffle and sort phases.

Core Idea

> **Pre-bucketing prepares the data for future large-scale joins.**

Week 10 — Quick Revision

`groupBy()`

groupBy()

↓

Wide Transformation

↓

Shuffle

↓

Default: 200 shuffle partitions

AQE

AQE

├── Coalesces small/empty shuffle partitions

├── Handles partition skew

└── Can switch join strategies at runtime

Join Strategy Cheat Sheet

Situation	Preferred Strategy
Small + Large DataFrame	Broadcast Hash Join

Large + Medium DataFrame	Shuffle Hash Join
Large + Large DataFrame	Sort-Merge Join
Large pre-bucketed tables	Sort-Merge-Bucket Join

Most Important Optimization Concepts

Partitioning

↓

Partition Pruning

Bucketing

↓

Large-table Join Optimization

AQE

↓

Runtime Optimization

Broadcast Join

↓

Avoid Large Table Shuffle

SMB Join

↓

Avoid Repeated Shuffle + Sort

⭐ Interview Memory Line

> **Broadcast when one side is small, AQE when runtime data changes the plan, and bucketing when repeatedly joining very large tables on the same key.**

WEEK 11: APACHE SPARK MEMORY MANAGEMENT & FILE FORMATS

1. Executor Memory Architecture

When requesting memory for an executor container, such as `--executor-memory 2G`, Spark divides the memory into several functional regions.

Executor Memory Breakdown

Container Memory

- **Off-Heap / Memory Overhead**
  - Memory overhead
  - Optional PySpark memory
  - Optional off-heap allocations
- **JVM Heap Memory**
  - Reserved Memory
  - Unified Memory
    - Storage Memory
    - Execution Memory
  - User Memory

1.1 Reserved Memory

- Fixed size: **300 MB**
- Used internally by Spark.
- This memory is reserved for Spark's internal engine and runtime processes.

1.2 User Memory

User Memory is approximately **40% of usable JVM heap memory**.

It is used for:

- User-defined data structures
- Custom UDF variables
- Metadata required for raw RDD transformations

Formula:

User Memory = (Java Heap − 300 MB) × (1 − spark.memory.fraction)

1.3 Unified Memory

Unified Memory is approximately **60% of usable JVM heap memory**.

It is shared between:

Storage Memory

- Cached DataFrames
- Persistent tables
- Broadcast variables

Execution Memory

- Shuffles
- Joins
- Sorts
- Aggregations

1.4 Memory Overhead

Memory overhead is outside the JVM heap and helps prevent memory-related failures caused by:

- OS processes
- PySpark processes
- Non-JVM tasks

Formula:

Memory Overhead = max(384 MB, 10% × spark.executor.memory)

---

2. Dynamic Memory Eviction Rules

Spark allows Storage and Execution Memory to dynamically borrow unused space from one another.

Scenario A: Execution Needs More Memory

Storage → Execution

Execution can evict cached blocks from Storage Memory when additional memory is required for operations such as:

- Sorts
- Joins
- Aggregations

Scenario B: Storage Needs More Memory

Execution → Storage: NOT allowed

Storage cannot evict memory actively being used by execution tasks.

If there is insufficient memory:

- Cached data may spill to disk.
- The requested data may fail to cache.

Key Rule to Remember

> **Execution can evict Storage, but Storage cannot evict active Execution.**

---

3. Aggregation Engines: Hash Aggregate vs. Sort Aggregate

Spark SQL can use different physical operators for grouping aggregations.

3.1 Hash Aggregate

Hash aggregation is generally faster and operates approximately in:

O(N)

Characteristics:

- Uses hash tables.
- Updates aggregate values directly.
- Works efficiently with suitable mutable data types.
- Generally avoids the additional sorting overhead of SortAggregate.

3.2 Sort Aggregate

Sort aggregation generally has:

O(N log N)

Characteristics:

- Requires data to be sorted by grouping keys.
- Introduces additional CPU overhead.
- Can be used when HashAggregate is not suitable.

Comparison

Feature	Hash Aggregate	Sort Aggregate
Time Complexity	O(N)	O(N log N)
Main Mechanism	Hash table	Sorting
Performance	Generally faster	Generally slower
Memory	Uses hash-based aggregation	Requires sorting

Use	Suitable data types / aggregation plans	Fallback or suitable cases

Key Takeaway

The source material emphasizes that data types can influence the aggregation strategy. A grouping key represented as a `String` may lead Spark toward SortAggregate, while suitable numeric or timestamp types may allow HashAggregate.

---

4. Spark Optimizer Architecture: Catalyst

The **Catalyst Optimizer** transforms high-level DataFrame or SQL operations into optimized execution plans.

Catalyst Optimization Pipeline

DataFrame / SQL Query

↓

1. Unresolved Logical Plan

↓

2. Analyzed Logical Plan

↓

3. Optimized Logical Plan

↓

4. Physical Plan

↓

5. Code Generation

4.1 Unresolved Logical Plan

At this stage, Spark checks the query structure and syntax.

It has not yet fully resolved:

- Tables
- Columns
- Data types

4.2 Analyzed Logical Plan

Spark uses the Catalog to resolve:

- Table names
- Column names
- Data types
- Functions

The result is a fully resolved logical plan.

4.3 Optimized Logical Plan

Spark applies logical optimization rules such as:

- **Predicate Pushdown** — filters data as early as possible.
- **Projection / Column Pruning** — reads only required columns.

4.4 Physical Plan

Spark selects actual execution strategies.

Examples:

- Broadcast Hash Join
- Sort-Merge Join
- Hash Aggregate
- Sort Aggregate

4.5 Code Generation

Spark can generate optimized Java bytecode through **Whole-Stage Code Generation** for efficient execution.

---

5. File Formats: Row-Based vs. Column-Based

File format selection has a direct impact on:

- Storage requirements
- Network bandwidth
- Query performance
- Compression
- Read/write performance

5.1 Row-Based Formats

Examples:

- CSV
- Avro

Data is stored row by row.

Advantages

- Efficient for writing complete records.
- Useful for ingestion and landing-zone workloads.

Disadvantages

- Less efficient when queries require only a subset of columns.
- Can result in higher I/O during analytical queries.

5.2 Column-Based Formats

Examples:

- Parquet
- ORC

Data is stored column by column.

Advantages

- Efficient analytical reads.

- Better compression.
- Supports column pruning.
- Supports predicate pushdown.

---

6. Text-Based File Formats

Common examples:

- CSV
- JSON
- XML

Main Drawbacks

Values are generally represented as text, meaning Spark may need to parse strings into appropriate data types.

This can introduce:

- Additional CPU overhead
- Larger storage requirements
- Higher I/O

JSON and XML can also be bulky compared with specialized Big Data file formats.

---

7. Avro vs. Parquet vs. ORC

Feature	Apache Avro	Apache Parquet	Apache ORC
Layout	Row-based	Columnar	Columnar
Primary Use Case	Data ingestion / landing zones	Analytical querying with Spark	Analytical querying with Hive

Write Performance	Fast	Slower	Slower
Read Performance	Slower	Fast	Fast
Compression	Moderate	High	Very High
Schema Evolution	Supported	Supported	Supported

Simple Rule

Avro → Good for ingestion

Parquet → Excellent for Spark analytics

ORC → Commonly associated with Hive analytics

---

8. Parquet Internal Architecture

A Parquet file is organized into:

1. **Row Groups**
2. **Column Chunks**
3. **Pages**

Conceptually:

Parquet File

→ Row Group 1

→ Column Chunk 1

→ Page 1, Page 2, Page 3

→ Column Chunk 2

→ Page 1, Page 2, Page 3

→ Row Group 2

→ ...

The file also contains metadata describing the structure and statistics.

8.1 Row Groups

A Row Group contains a collection of rows.

Each Row Group contains separate Column Chunks for each column.

Example:

Row Group 1

- Column Chunk: `order_id`
- Column Chunk: `order_date`
- Column Chunk: `customer_id`

8.2 Column Chunks

A Column Chunk contains the data for one column within a Row Group.

This column-oriented organization enables Spark to avoid reading columns that are not required.

8.3 Pages

Column Chunks are further divided into Pages.

Pages can contain statistics that help Spark determine whether particular data needs to be read.

---

9. Parquet Query Optimizations

9.1 Column Pruning

Suppose a table contains:

- `order_id`
- `order_date`
- `customer_id`
- `order_amount`
- `order_status`

If the query only requires:

order_id

order_amount

Spark can avoid reading the other columns.

This reduces:

- Disk I/O
- Network I/O
- Memory consumption

9.2 Predicate Pushdown

Parquet stores statistics such as:

- Minimum value
- Maximum value

for relevant data regions.

Suppose a Row Group contains:

order_id:

min = 1

max = 20,000

and the query asks for:

order_id = 50,000

Spark can determine that this Row Group cannot contain the requested value and skip reading it.

Key Idea

Column Pruning → Skip unnecessary columns

Predicate Pushdown → Skip unnecessary data

---

10. Parquet Encoding Techniques

Parquet uses lightweight encoding techniques before applying general-purpose compression.

10.1 Dictionary Encoding

Repeated values are stored using dictionary IDs instead of repeating the full value.

Example:

Bangalore

Bangalore

Bangalore

Hyderabad

Hyderabad

can be represented conceptually using:

1

1

1

2

2

with a dictionary:

1 → Bangalore

2 → Hyderabad

10.2 Bit Packing

Bit packing reduces the number of bits required to store integer values.

Instead of allocating unnecessary bits for every value, only the required number of bits is used.

10.3 Delta Encoding

Instead of storing every value independently, Parquet can store the difference between consecutive values.

Example:

100

105

110

115

can conceptually become:

100

+5

+5

+5

10.4 Run-Length Encoding (RLE)

Repeated consecutive values can be represented using a value and its count.

Example:

A A A A A B B B

can conceptually become:

A × 5

B × 3

---

11. Compression Codecs

Compression codecs are used with container formats such as Parquet and ORC.

Codec	Compression Speed	Compression Ratio	Splittable	Typical Use
Snappy	Very Fast	Moderate	No by itself; works efficiently within Parquet/ORC	Default-style choice for Parquet/ORC
LZO	Very Fast	Moderate	Yes, with indexing	High-speed processing
Gzip	Moderate / Slow	High	No by itself	Storage-constrained data
Bzip2	Very Slow	Very High	Yes	Cold data / archival

General Trade-Off

Faster compression → Usually lower compression ratio

Higher compression ratio → Usually higher CPU cost

---

12. Schema Evolution in Parquet

Parquet supports schema evolution.

When reading Parquet datasets with compatible schema changes, Spark can merge schemas using:

```
df = spark.read \
    .option("mergeSchema", "true") \
    .parquet("/public/trendytech/datasets/parquet-schema-evol-demo1")
```

The important option is:

mergeSchema = true

This allows Spark to combine compatible schemas across Parquet files.

---

13. Quick Revision Sheet

Executor Memory

JVM Heap

- Reserved Memory
- Unified Memory
    - Storage
    - Execution
- User Memory

Outside JVM

- Memory Overhead
- PySpark memory, when configured
- Optional off-heap memory

Memory Rule

> **Execution can evict Storage, but Storage cannot evict active Execution.**

Aggregations

HashAggregate
→ Hash table
→ Generally faster
→ O(N)

SortAggregate
→ Sorting

→ Generally slower
→ O(N log N)

Catalyst

Unresolved Logical Plan

↓

Analyzed Logical Plan

↓

Optimized Logical Plan

↓

Physical Plan

↓

Code Generation

File Formats

CSV / JSON / XML
→ Text-based

Avro
→ Row-based

Parquet / ORC
→ Column-based

Parquet

Row Group → Column Chunk → Page

Parquet Optimizations

Column Pruning
→ Skip unnecessary columns

Predicate Pushdown
→ Skip unnecessary data

Encoding

Dictionary Encoding
→ Repeated values → IDs

Bit Packing
→ Reduce bits used

Delta Encoding
→ Store differences

RLE
→ Store repeated values + count

Compression

Snappy
→ Fast

Gzip
→ Higher compression

Bzip2
→ Very high compression, slower

LZO
→ Fast and splittable with indexing

WEEKS 12 & 13

End-to-End Enterprise Big Data & PySpark Project

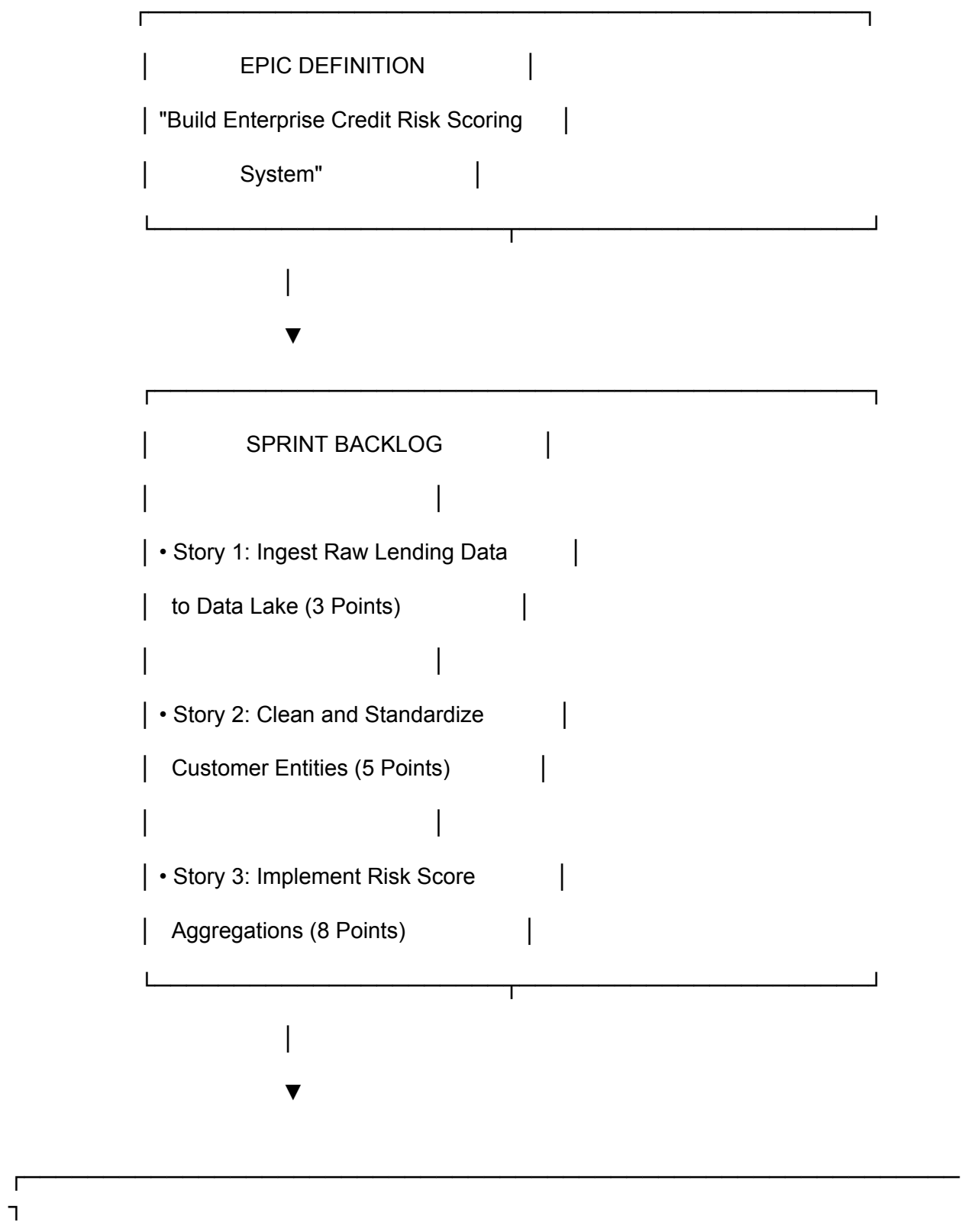
---

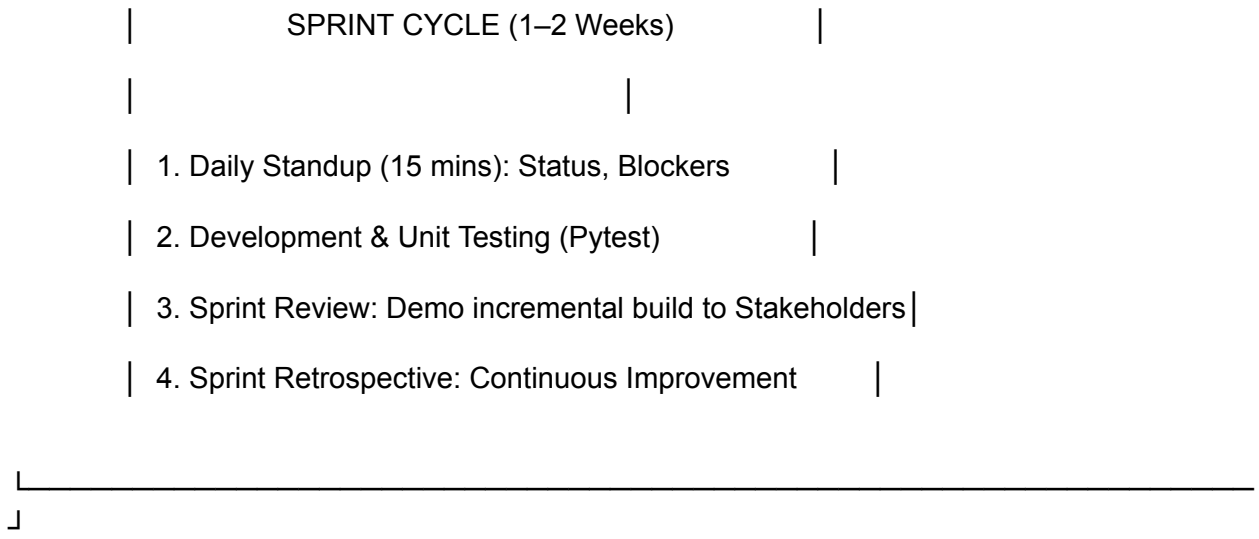
1. Project Lifecycles & Agile Project Delivery

Building an enterprise-grade Big Data pipeline requires balancing **technical design** with **business domain context**.

Projects are delivered iteratively using **Agile Methodology** and **Scrum Frameworks**.

Agile Iterative Sprint Lifecycle





Agile vs. Waterfall Model

Waterfall Model

- Linear and sequential.
- Requirements are frozen upfront.
- Late architectural modifications can be costly.

Agile Framework (Scrum)

- Iterative and adaptive.
- Work is broken down into **1–2 week Sprints**.
- Produces incremental and testable pipeline features.

---

2. Enterprise Problem Statement: Credit Risk Analysis

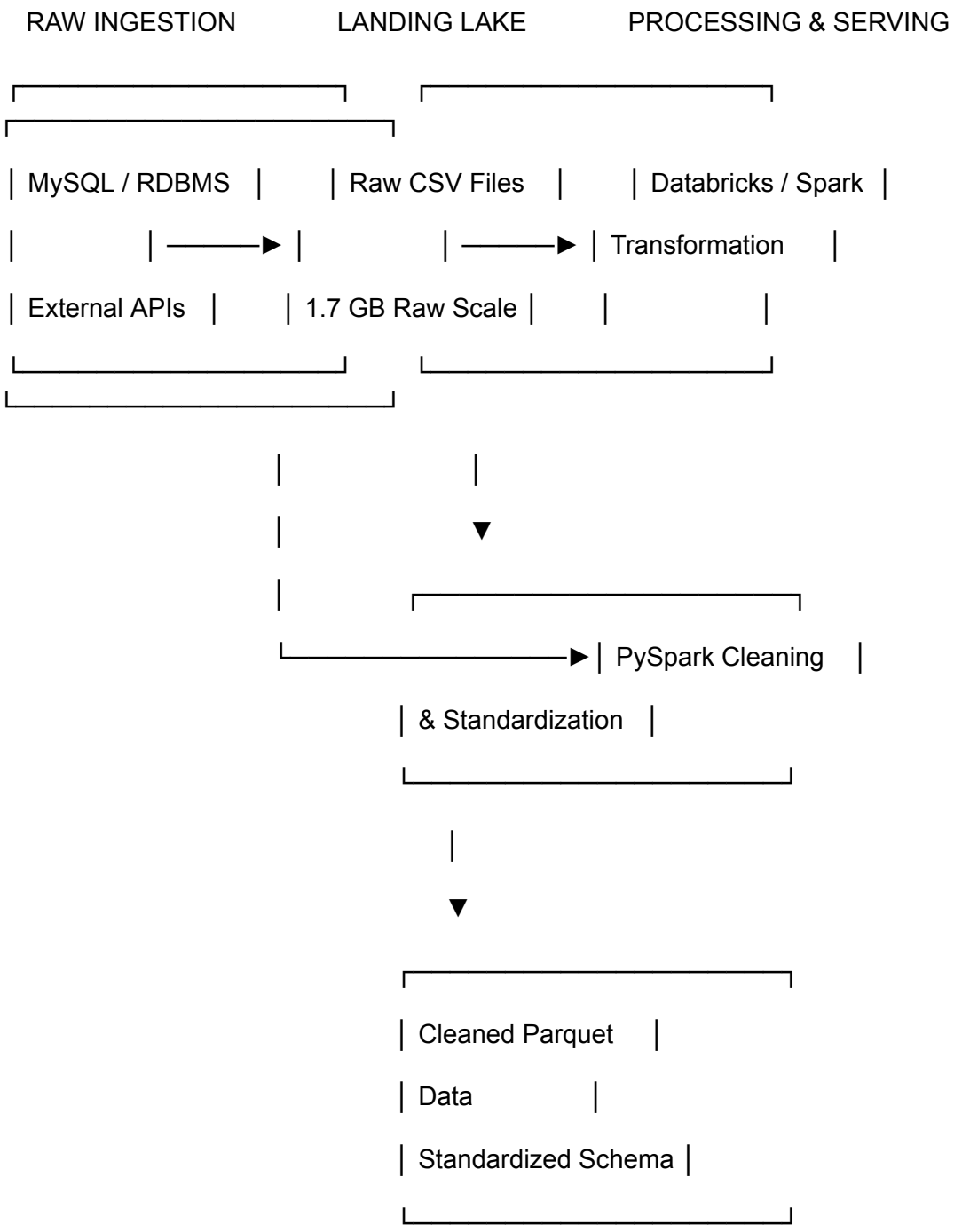
To demonstrate a production architecture, consider a **FinTech Lending Institution Case Study**.

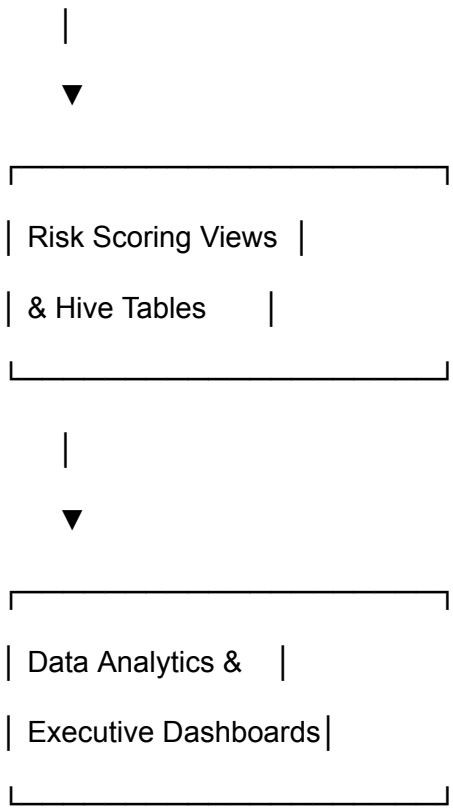
Business Goal

The business goal is to analyze historical borrower data to compute an automated **Credit Risk Score** that determines:

- Loan approvals
- Interest rates

High-Level Enterprise Data Pipeline





---

3. Data Ingestion & Hash Surrogate Keys

The raw dataset consists of a **1.7 GB monolithic CSV file** containing:

- **118 attributes**
- **2.2+ million records**

The ingestion pipeline splits this raw data into distinct functional domain datasets:

1. `customers_data`

Contains borrower profiles.

2. `loans_data`

Contains:

- Applied loan amounts
- Loan terms
- Interest rates

3. `loan_repayments`

Contains:

- Historical principal
- Interest
- Payment dates

4. `loan_defaulters`

Contains:

- Historical delinquencies
- Public records
- Bankruptcies

Generating Deterministic Hash Keys (`emp_id`)

Because raw borrower records lack a unique primary key (`member_id` contains nulls), a surrogate key called **`emp_id`** is generated.

A **SHA-256 algorithm** is applied across 9 composite key attributes.

```
from pyspark.sql.functions import sha2, concat_ws


# Generating a deterministic composite primary key

composite_cols = [

    'emp_title',

    'emp_length',

    'home_ownership',

    'annual_inc',
```

```
    'zip_code',
    'addr_state',
    'grade',
    'sub_grade',
    'verification_status'
]

# SHA-256 produces a 256-bit (64 hex character) unique hash code
customers_raw_df = raw_df.withColumn(
    "emp_id",
    sha2(concat_ws("||", *composite_cols), 256)
)
```

---

4. Multi-Stage Data Cleaning & Standardization

Data Cleaning Pipeline

Raw Ingestion Data

Schema Enforcement & Renaming

Regex Text Parsing

|

▼

Outlier & Null Imputation

|

▼

Cleaned Output Storage

Stage 1: Cleaning `customers_data`

1. Explicit Schema Assignment

Loads raw data using explicit:

- Float
- Integer
- String

data types.

2. Column Renaming

Standardizes ambiguous attribute names.

Examples:

annual_inc → annual_income

addr_state → address_state

3. Audit Columns

Appends ingestion timestamps using:

withColumn("ingest_date", current_timestamp())

4. Regex Cleaning (`emp_length`)

Strips non-digit characters using regex `\D`.

Example:

"10+ years" → 10

```
from pyspark.sql.functions import regexp_replace, col

# Replace all non-digits (\D) with empty strings and cast to Integer
cleaned_emp_df = df.withColumn(
    "emp_length",
    regexp_replace(col("emp_length"), r"(\D)", "").cast("int")
)
```

5. Imputing Nulls

Fills missing `emp_length` values with the column average:

```
.na.fill(
    avg_emp_duration,
    subset=["emp_length"]
)
```

6. State Code Normalization

Enforces 2-character state codes and replaces invalid text with:

"NA"

Stage 2: Cleaning `loans_data`

1. Term Conversion

Converts raw string terms such as:

" 36 months"

into numeric years.

```
(
    (
        regexp_replace(
            "loan_term_months",
            " months",
            ""
        ).cast("float") / 12
    ).cast("int")
)
```

2. Categorical Lookup Mapping

Maps low-frequency loan purposes into an `"other"` category.

```
when(
    col("loan_purpose").isin(lookup_list),
    col("loan_purpose")
).otherwise("other")
```

Stage 3: Cleaning `loan_repayments` & `loan_defaulters`

1. Total Payment Correction

Reconstructs zero total payments when principal was received.

```
from pyspark.sql.functions import when, col


# Rebuilding total payment if total_payment_received is incorrectly 0.0
repayments_df = df.withColumn(
    "total_payment_received",
    when(
        (col("total_payment_received") == 0.0) &
        (col("total_principal_received") != 0.0),
        col("total_principal_received") +
        col("total_interest_received") +
        col("total_late_fee_received")
```

```
    ).otherwise(col("total_payment_received"))
)
```

2. Null Replacement

Replaces `0.0` sentinel values in date fields with `None` / `null`.

3. Defaulter Categorization

Splits defaulter data into two separate Parquet storage targets:

Delinquencies

delinq_2yrs > 0

Public Records / Inquiries

pub_rec > 0

OR

inq_last_6mths > 0

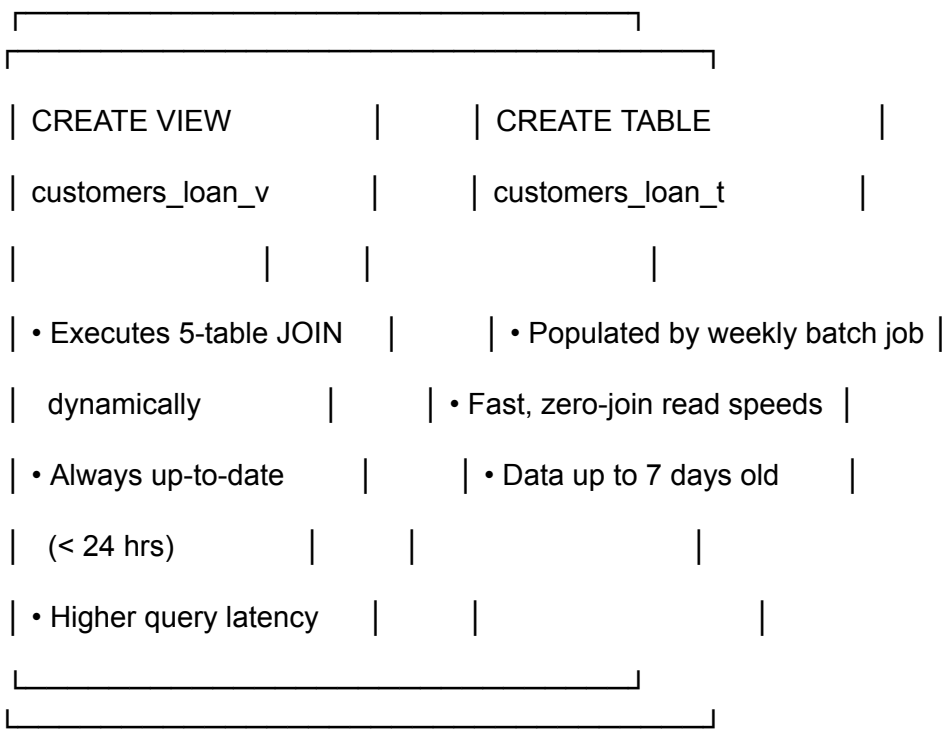
---

5. Enterprise Data Serving Layer: Views vs. Managed & External Tables

Downstream teams such as **BI Analysts** and **ML Engineers** require different access patterns to cleaned datasets.

Serving Layer Access Architecture

CONSOLIDATED REAL-TIME VIEW PRE-CALCULATED MATERIALIZED TABLE



External Tables for Multi-Team Access

For shared access across teams, datasets are exposed through **External Tables** registered in the Hive Metastore.

External tables preserve the physical Parquet data on disk even if the table metadata is dropped.

-- Creating an External Table on Top of Cleaned Parquet Data

```
CREATE EXTERNAL TABLE lending_club.customers (
    member_id STRING,
    emp_title STRING,
    emp_length INT,
```

```
    home_ownership STRING,
    annual_income FLOAT,
    address_state STRING,
    total_high_credit_limit FLOAT,
    ingest_date TIMESTAMP
)
STORED AS PARQUET
LOCATION '/public/trendytech/lendingclubproject/cleaned/customers_parquet';
```

---

6. Business Logic: Credit Risk Scoring Engine

The automated **Credit Risk Score** evaluates three weighted financial pillars.

Formula

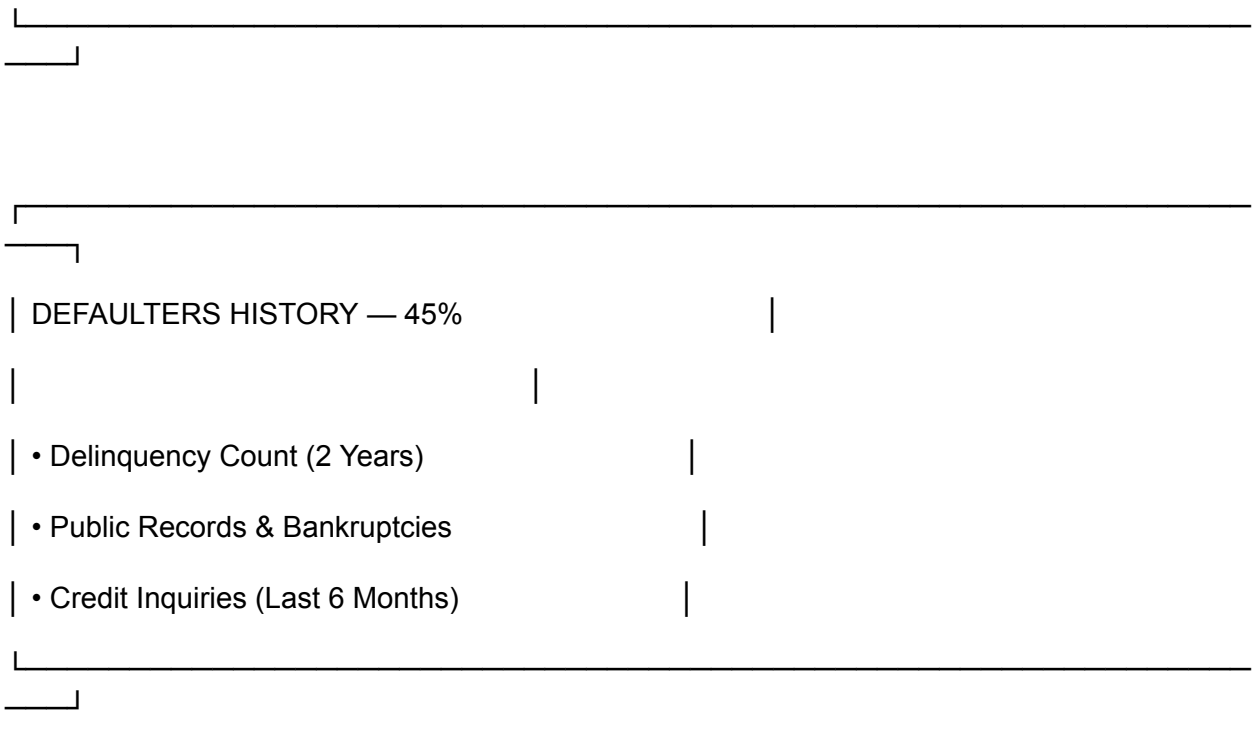
Credit Risk Score

= (0.20 × Payment History Points)

- (0.45 × Defaulters History Points)
- (0.35 × Financial Health Points)

Credit Risk Score Weighting Distribution

```
┌──────────────────────────────────────────────────────────────────────┐
│ PAYMENT HISTORY — 20%                                                │
│                                                                      │
│ • Last Payment vs. Monthly Installment Ratio                         │
│ • Total Payment Received vs. Principal Funded Ratio                  │
```



Dynamic Point Parameterization

Hardcoding point boundaries directly into queries is discouraged in production.

Dynamic limits are passed through configuration parameters.

Registering dynamic score thresholds in SparkConf

```
spark.conf.set("spark.sql.unacceptable_rated_pts", 0)

spark.conf.set("spark.sql.very_bad_rated_pts", 100)

spark.conf.set("spark.sql.bad_rated_pts", 250)

spark.conf.set("spark.sql.good_rated_pts", 500)

spark.conf.set("spark.sql.very_good_rated_pts", 650)

spark.conf.set("spark.sql.excellent_rated_pts", 800)
```

Risk Grade Categorization Matrix

Calculated Risk Score	Assigned Final Loan Grade	Risk Status
> 650	**Grade A**	Lowest Risk / Instant Approval
500 – 650	**Grade B**	Low Risk / Standard Rates
350 – 500	**Grade C**	Moderate Risk / Increased Interest
200 – 350	**Grade D**	High Risk / Strict Review

100 – 200	**Grade E**	Very High Risk / Manual Approval Required
≤ 100	**Grade F**	Unacceptable Risk / Rejected

---

7. Modular Architecture & Project Structure

Production PySpark applications are organized into **modular packages** rather than monolithic notebook scripts.

retail_analysis/

│

├── configs/

│ ├── application.conf

│ │ # Environment file paths (LOCAL, TEST, PROD)

│ │

│ └── pyspark.conf

│ # Spark submit configs (cores, memory, master)

│

├── data/

│ # Local sample data for unit testing

│

├── lib/

│ ├── ConfigReader.py

│ │ # ConfigParser wrapper for app/spark configs

```
│   │
│   ├── DataReader.py
│   │   # Encapsulated DataFrame reader methods
│   │
│   ├── DataManipulation.py
│   │   # Pure transformation methods
│   │
│   ├── logger.py
│   │   # Log4j wrapper class
│   │
│   └── Utils.py
│       # SparkSession creation utility
│
├── application_main.py
│   # Application driver entry point
│
├── tests/
│   ├── conftest.py
│   │   # Pytest fixtures (SparkSession setup/teardown)
│   │
│   └── test_retail.py
│       # Automated Pytest unit test suites
│
├── Pipfile
```

│ # Dependency definitions

│

└── Pipfile.lock

Locked package versions and hashes

---

8. Virtual Environments & Isolation (`pipenv` & `pyenv`)

To prevent dependency conflicts across different client projects on a shared machine, tools such as **`pyenv`** and **`pipenv`** can isolate Python runtimes and libraries.

Environment Isolation

Global Machine Environment

Python 3.10

│

├── Project 1 VirtualEnv (pipenv)

│ ├── Python 3.8

│ ├── PySpark 3.2.1

│ └── Pytest

│

└── Project 2 VirtualEnv (pipenv)

├── Python 3.10

├── PySpark 3.5.0

└── Delta Lake

Core `pipenv` CLI Reference Commands

Activate Virtual Environment

pipenv shell

Install a Specific Package Version

pipenv install pyspark==3.2.1

This installs the specified package version and locks hashes in `Pipfile.lock`.

Execute a Command Inside the Environment

pipenv run python application_main.py LOCAL

Delete the Current Virtual Environment

pipenv --rm

---

9. Unit Testing Framework with `pytest`

Testing pipeline functions requires isolated test execution using **`pytest`**.

`tests/conftest.py`

Centralized fixture file:

```
import pytest
from lib.Utils import get_spark_session

@pytest.fixture
def spark():
    """Sets up a local SparkSession for testing and tears it down after tests run."""
    spark_session = get_spark_session("LOCAL")

    yield spark_session

    spark_session.stop()
```

---

tests/test_retail.py

```
import pytest

from lib.DataReader import read_customers, read_orders
from lib.DataManipulation import filter_closed_orders


def test_customer_count(spark):
    """Verifies customer DataFrame read row count."""
    customer_df = read_customers(spark, "LOCAL")
```

```
    assert customer_df.count() == 12435



@pytest.mark.transformation
def test_filter_closed_orders(spark):
    """Verifies status filtering transformation logic."""
    orders_df = read_orders(spark, "LOCAL")

    closed_orders_count = filter_closed_orders(orders_df).count()

    assert closed_orders_count == 7556
```

Executing Pytest Suites

Run All Tests

```
python -m pytest
```

Discovers and executes all test functions beginning with `test_`.

Run Tests in Verbose Mode

```
python -m pytest -v
```

Run Only Transformation Tests

```
python -m pytest -m transformation
```

Runs test functions tagged with:

@pytest.mark.transformation

---

10. Production Logging via Log4j

Using plain `print()` statements in production code:

- Slows down execution.
- Cannot write to centralized log files.
- Lacks configurable severity levels.

Spark includes **Log4j** to support structured, performance-friendly logging.

Log4j Priority Hierarchy

DEBUG < INFO < WARN < ERROR < FATAL

Lowest Severity Highest Severity

Log4j Wrapper Class

```
# lib/logger.py


class Log4j:


    def __init__(self, spark):
```

```
        # Obtain the JVM LogManager instance from the active SparkContext
        log4j = spark._jvm.org.apache.log4j

        self.logger = log4j.LogManager.getLogger(
            "com.trendytech.retail.application"
        )

    def info(self, message):
        self.logger.info(message)

    def warn(self, message):
        self.logger.warn(message)

    def error(self, message):
        self.logger.error(message)
```

Configuring Custom Log4j Properties

To instruct Spark to use custom Log4j parameters, add the Log4j configuration file option during `SparkSession` creation.

```
SparkSession.builder \
    .config(
        "spark.driver.extraJavaOptions",
        "-Dlog4j.configuration=file:log4j.properties"
```

```
) \
.getOrCreate()
```

WEEKS 12 & 13 — QUICK REVISION CHECKLIST

Project & Agile

- Agile Methodology
- Scrum Framework
- Sprint Backlog
- Sprint Cycle
- Daily Standup
- Sprint Review
- Sprint Retrospective
- Agile vs. Waterfall

Enterprise Data Pipeline

- Raw ingestion
- HDFS / S3 Landing Lake
- PySpark processing
- Parquet storage
- Hive Tables
- Data Analytics & Dashboards

Data Cleaning

- Explicit schema
- Column renaming
- Audit columns
- Regex cleaning
- Null imputation
- State normalization
- Term conversion

- Categorical mapping
- Data correction
- Defaulter categorization

Data Engineering

- SHA-256
- Surrogate keys
- Composite keys
- External Tables
- Views
- Managed Tables
- Parquet

Credit Risk

- Payment History
- Defaulters History
- Financial Health
- Weighted scoring
- Dynamic configuration
- Risk Grade A–F

Production PySpark

- Modular architecture
- Configuration management
- `pyenv`
- `pipenv`
- Unit testing
- `pytest`
- Fixtures
- Log4j
- Production logging

Important Commands

pipenv shell

pipenv install pyspark==3.2.1

pipenv run python application_main.py LOCAL

pipenv --rm

python -m pytest

python -m pytest -v

python -m pytest -m transformation

WEEKS 1–13 — OVERALL STUDY COVERAGE

By Week 13, the study guide covers:

1. **Big Data, Hadoop & HDFS Storage Foundations**
2. **Spark Core Architecture**
    - RDDs
    - Lineage
    - DAGs
    - Partitioning
3. **Higher-Level APIs**
    - DataFrames
    - Spark SQL
    - Catalyst Optimizer
4. **Resource Allocation & Executor Sizing**
5. **Advanced Transformations**
    - Windowing
    - Pivoting
    - Aggregations

6. **Spark Internals & Optimization**
    - AQE
    - Join Strategies
    - Sort-Merge Join
    - Sort-Merge-Bucket Join
    - Broadcast Join
7. **Memory Management & File Formats**
    - Heap / Off-Heap
    - Memory Eviction
    - Parquet
    - Encoding
    - Compression
8. **End-to-End Enterprise PySpark Project**
    - Data Ingestion
    - Data Cleaning
    - Risk Scoring
    - Modular Design
    - Pytest
    - Log4j

WEEKS 14 & 15

Git, GitHub, CI/CD, Jenkins & Apache Hive

WEEK 14: VERSION CONTROL (GIT/GITHUB) & CI/CD PIPELINES WITH JENKINS

---

1. Code Versioning & Collaboration: Git vs. GitHub

Modern data engineering teams use version control systems to:

- Collaborate on application code
- Track code changes

- Maintain commit history
- Manage branches
- Review changes through Pull Requests
- Maintain clean release cycles
- Automate CI/CD workflows

Git vs. GitHub

Git — Local Version Control Engine

Git runs on the developer's local machine and is responsible for:

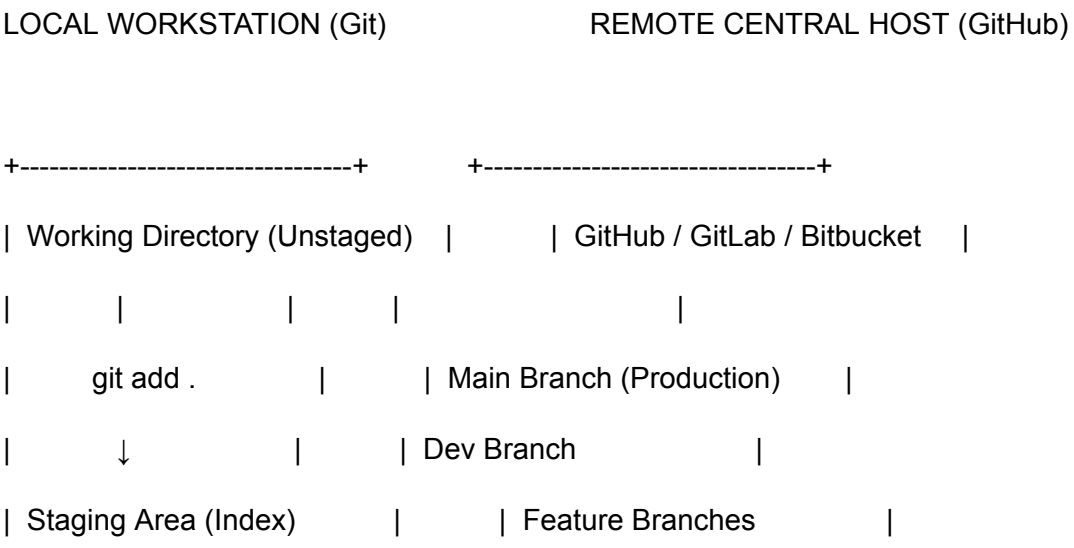
- Tracking changes
- Managing local branches
- Creating commits
- Maintaining local commit history
- Managing the local `.git` repository

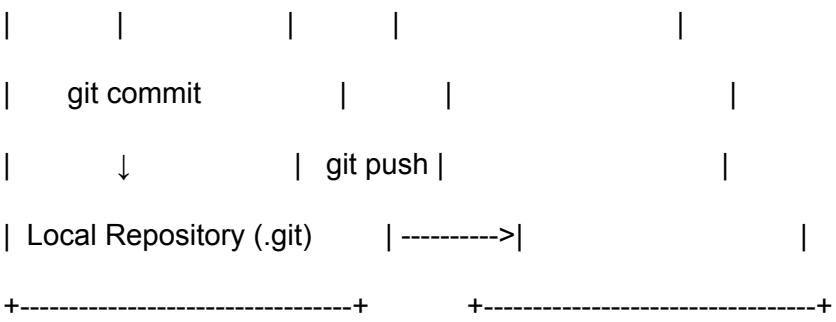
GitHub — Remote Repository Host

GitHub hosts repositories remotely and enables:

- Team collaboration
- Remote repository management
- Pull Requests (PRs)
- Code reviews
- Branch management
- CI/CD integration

Local vs. Remote Repository Architecture





Key Difference

> **Git manages version control locally, while GitHub provides a remote platform for collaboration, code review, and CI/CD integration.**

---

2. Core Git Workflow & Essential Commands

2.1 Three-State Git Architecture

Git manages code through three primary states.

1. Working Directory — Unstaged

Contains:

- Modified files
- Newly created files
- Untracked files

These changes have not yet been selected for the next commit.

2. Staging Area — Index

Files are moved into the staging area using:

git add .

The staging area defines which changes will be included in the next commit.

3. Local Repository — Committed

The staged changes are permanently recorded in the local Git history using:

git commit -m "commit message"

Three-State Code Lifecycle

Working Directory

|

git add

↓

Staging Area

|

git commit

↓

Local Repository

Easy Way to Remember

Working Directory → Staging Area → Repository

`git add` → `git commit`

---

3. Essential Git Commands

3.1 Initialization & Synchronization

```
git init
```

Initializes a brand-new Git repository in the current directory.

git init

`git clone`

Downloads an existing remote repository to the local machine.

git clone <repo-url>

`git remote add origin`

Connects a local repository to a remote GitHub repository.

git remote add origin <repo-url>

`git pull`

Fetches and merges the latest commits from a remote branch.

git pull origin <branch>

`git push`

Uploads local committed changes to the remote repository.

git push origin <branch>

3.2 Staging & Committing

`git status`

Displays:

- Modified files
- Untracked files
- Staged files
- Current repository status

git status

`git add .`

Stages all modified and newly created files.

git add .

`git commit`

Creates a commit containing the staged changes.

git commit -m "commit message"

3.3 Branch Management

Display local branches

git branch

Rename current branch to `main`

git branch -M main

Create and switch to a new branch

git checkout -b <new-branch>

Delete a local branch

git branch -d <branch-name>

---

4. Advanced Git Operations

Advanced Git workflows include:

- Undoing changes
- Forking repositories
- Stashing uncommitted work
- Managing upstream repositories

4.1 Undoing Changes

Different Git commands are used depending on the state of the change.

State of Changes	Recovery Command
Unstaged modifications	`git restore <file>`
Staged changes	`git restore --staged <file>`
Committed changes — keep code	`git reset HEAD~1`

Committed changes — wipe code	`git reset --hard <commit-hash>`

Unstaged Modifications

Discard local modifications:

git restore <file-name>

Staged Changes

Move a staged file back to the unstaged state:

git restore --staged <file-name>

Soft Reset / Commit Reset

Undo the most recent commit while retaining the changes in the working directory:

git reset HEAD~1

Hard Reset

Move the repository back to a particular commit and discard subsequent changes:

git reset --hard <commit-hash>

> **Important:** `git reset --hard` can permanently remove local changes.

---

5. Forking & Open-Source / Organizational Contributions

A **Fork** creates a copy of an upstream repository under your own GitHub account.

Typical Fork Workflow

Original Repository

|

Fork

↓

Personal GitHub Repository

|

Clone

↓

Local Repository

|

Feature Branch

|

Commit

|

Push

↓

Pull Request

|

↓

Original Repository

Steps

1. Fork

Create a copy of the upstream repository in your personal GitHub account.

2. Clone & Branch

Clone the fork locally and create a feature branch.

3. Push & Pull Request

Commit the changes, push them to your fork, and open a Pull Request against the original repository.

4. Sync with Upstream

Add the original repository as an upstream remote:

git remote add upstream <original-repo-url>

Pull the latest changes:

git pull upstream main

---

6. Git Stashing

Git Stash temporarily stores uncommitted changes so that you can switch tasks without creating a commit.

`git stash`

Stores current modifications.

git stash

`git stash -u`

Stores modifications along with untracked files.

git stash -u

`git stash list`

Displays all available stash entries.

git stash list

`git stash pop`

Restores the most recent stash and removes it from the stash stack.

git stash pop

`git stash clear`

Removes all stash entries.

git stash clear

---

7. Enterprise Branching & Deployment Strategy

Enterprise projects commonly isolate development, testing, and production environments using dedicated branches.

Enterprise Branching Flow

Feature Branch

↓

DEV

↓

TEST

↓

UAT

↓

MAIN / PROD

7.1 Feature Branch

Example:

feature-RA-XXXX

Used for:

- Individual developer work
- JIRA-ticket-specific changes
- Isolated development

Feature branches are generally deleted after merging.

7.2 DEV Branch

Used to:

- Integrate changes from multiple developers
- Perform initial functional checks

7.3 TEST Branch

Used for:

- Automated integration testing
- Quality Assurance (QA)
- Controlled test execution

7.4 UAT Branch

UAT = User Acceptance Testing

Business stakeholders validate whether the application meets the required business expectations.

7.5 MAIN / PROD

Contains production-ready code.

Production deployments are performed from the stable production branch.

---

8. CI/CD Pipelines & Jenkins

8.1 Continuous Integration

Continuous Integration (CI) automatically builds and tests code when changes are pushed to the repository.

Typical CI activities include:

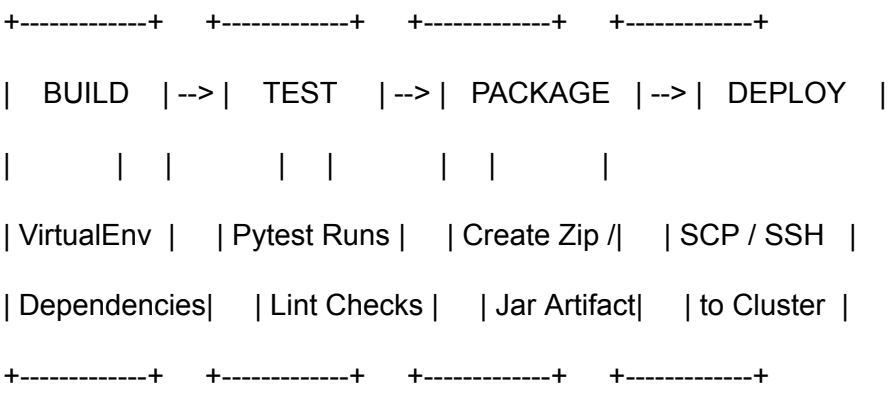
- Installing dependencies
- Building the application
- Running unit tests
- Running lint checks

- Validating the code

8.2 Continuous Deployment

Continuous Deployment (CD) automates the packaging and deployment of application artifacts to target environments.

CI/CD Pipeline



Pipeline Stages

1. Build

- Create environment
- Install dependencies

2. Test

- Execute Pytest
- Run lint checks

3. Package

- Create ZIP/JAR/application artifact

4. Deploy

- Transfer the artifact
- Deploy to the target cluster/environment

9. Jenkinsfile

Jenkins pipelines can be defined as code using a root-level **`Jenkinsfile`**.

```
pipeline {
    agent any


    environment {
        LABS = credentials('labcreds')
    }


    stages {
        stage('Build') {
            steps {
                echo "Building environment and installing dependencies..."
                sh 'pip3 install --user pipenv'
                sh '~/.local/bin/pipenv --rm || exit 0'
                sh '~/.local/bin/pipenv install'
            }
        }


        stage('Test') {
            steps {
                echo "Executing automated unit testing suite..."
```

```
            sh '~/.local/bin/pipenv run pytest -v'
        }
    }

    stage('Package') {
        steps {
            echo "Packaging PySpark application artifacts..."
            sh 'zip -r retailproject.zip .'
        }
    }

    stage('Deploy') {
        steps {
            echo "Deploying artifact to gateway edge node..."
            sh '''
                sshpass -p $LABS_PSW scp -o StrictHostKeyChecking=no -r . \
                $LABS_USR@g02.itversity.com:/home/itv005857/retailproject
            '''
        }
    }
  }
}
```

Important Jenkins Concepts

Concept	Meaning
`pipeline`	Defines the Jenkins pipeline
`agent any`	Allows execution on any available Jenkins agent
`environment`	Defines environment variables
`credentials()`	Retrieves credentials stored securely in Jenkins
`stages`	Groups the pipeline's major phases
`stage()`	Defines an individual pipeline phase
`steps`	Contains commands executed within a stage
`sh`	Executes shell commands

WEEK 15: APACHE HIVE & METASTORE INTEGRATION

---

1. OLTP vs. OLAP

OLTP — Online Transaction Processing

OLTP systems support day-to-day operational workloads.

Typical characteristics:

- Inserts
- Updates
- Deletes
- Individual record reads
- Low latency
- High concurrency
- Relational databases
- Schema-on-write

Examples include:

- MySQL
- Oracle
- Other relational databases

OLAP — Online Analytical Processing

OLAP systems support large-scale analytical workloads.

Typical characteristics:

- Historical analysis
- Large scans
- Aggregations
- High throughput
- Distributed processing
- Schema-on-read

Apache Hive is commonly used as a SQL abstraction layer over distributed storage.

OLTP vs. OLAP Comparison

Dimension	OLTP	OLAP / Hive
Primary Workload	High-concurrency operations	Large-scale historical analysis
Operations	Inserts, Updates, Deletes, Single Reads	Large reads, scans and aggregations
Underlying Systems	MySQL, Oracle, RDBMS	Hive / Distributed Warehouses
Schema	Schema-on-Write	Schema-on-Read
Latency	Low	Higher
Throughput	Lower	Extremely High
Data Scale	Relatively smaller operational datasets	Massive analytical datasets

Key Difference

> **OLTP is optimized for operational transactions, while OLAP is optimized for large-scale analytical processing.**

---

2. Apache Hive Architecture

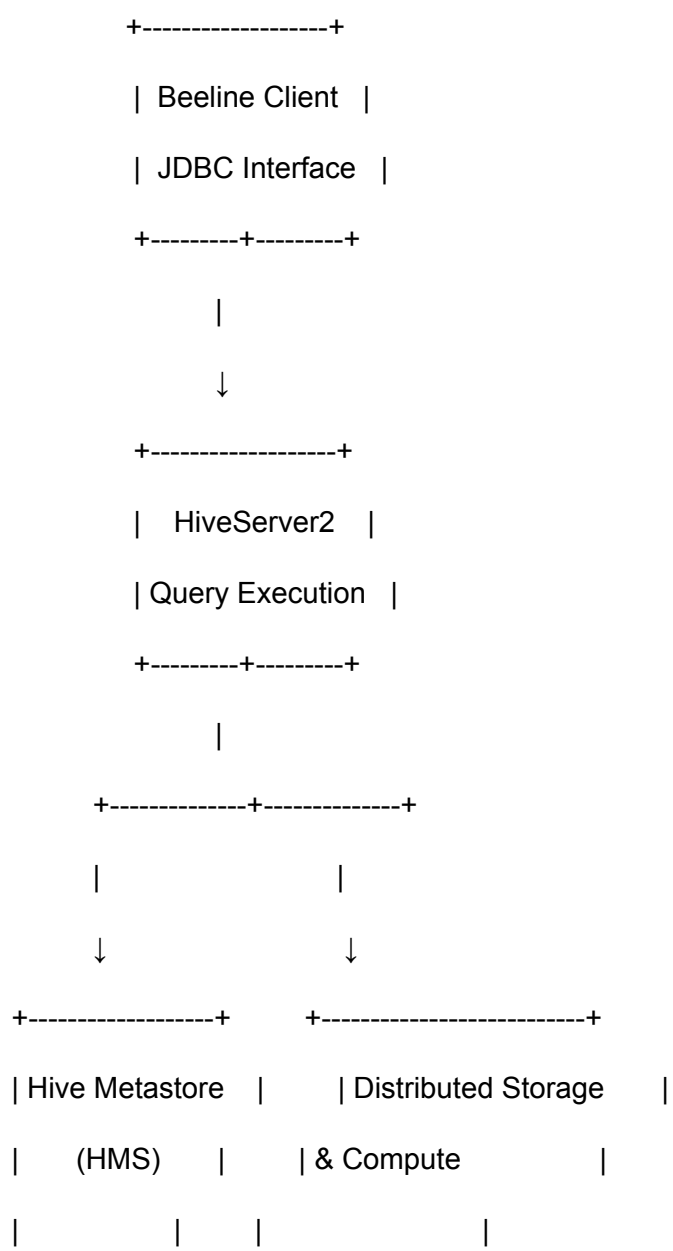
Apache Hive provides an SQL-like abstraction layer called **HQL (Hive Query Language)** over distributed storage such as:

- HDFS
- Amazon S3
- ADLS

Hive can work with distributed compute engines such as:

- MapReduce
- Tez
- Spark

Hive Architecture



| Schemas | | Storage: HDFS/S3/ADLS |

| Column Types | | Compute: MR/Tez/Spark |

| File Locations | | |

+------------------+ +-------------------------+

2.1 Hive Metastore (HMS)

The **Hive Metastore** stores metadata about tables, including:

- Table schemas
- Column names
- Column data types
- File locations

The metadata itself is stored in a relational database such as MySQL.

Why use an RDBMS for the Metastore?

A relational database provides efficient and low-latency metadata management and schema updates.

2.2 HiveServer2

HiveServer2:

- Receives client queries
- Accepts JDBC/ODBC requests
- Processes HQL
- Compiles queries into distributed jobs
- Returns query results

2.3 Beeline

Beeline is a JDBC-based command-line client used to connect to HiveServer2.

Example:

beeline -u "jdbc:hive2://m02.itversity.com:10000/;auth=noSasl"

Architecture Flow

Beeline

↓

HiveServer2

↓

Hive Metastore + Compute Engine

↓

HDFS / S3 / ADLS

---

3. Hive Table Types

Hive primarily supports two important table types:

1. Managed Tables
2. External Tables

3.1 Managed Table

A **Managed Table** is controlled by Hive.

Hive manages:

- Table metadata
- Physical data

The default warehouse location is controlled by:

hive.metastore.warehouse.dir

DROP Behavior

When a managed table is dropped:

DROP TABLE table_name;

Both:

- Metadata
- Physical data

are deleted.

3.2 External Table

An **External Table** is useful when the underlying data is shared with other systems or teams.

Hive manages:

- Table metadata

The actual data remains in the custom storage location.

DROP Behavior

When an external table is dropped:

- Metastore metadata is removed
- Physical files remain intact

Managed vs. External Tables

Property	Managed Table	External Table

Hive owns metadata	Yes	Yes
Hive owns data	Yes	No
Storage location	Default warehouse	Custom `LOCATION`
`DROP TABLE`	Deletes metadata + data	Deletes metadata only
Best Use Case	Hive-managed lifecycle	Shared datasets / external systems

---

3.3 Creating an External Hive Table

```
CREATE EXTERNAL TABLE IF NOT EXISTS orders_external (
    order_id INT,
    order_date STRING,
    customer_id INT,
    order_status STRING
)
ROW FORMAT DELIMITED
FIELDS TERMINATED BY ','
STORED AS TEXTFILE
LOCATION '/user/itv005857/hive_datasets/orders';
```

Key Point

> **External tables are useful when Hive should manage the metadata without owning or deleting the underlying data files.**

---

4. Hive Performance Optimizations

Two major storage optimization techniques are:

- Partitioning
- Bucketing

4.1 Partitioning

Partitioning divides data into **directories/folders** based on a column.

Example:

/warehouse/orders/

│

├── order_status=CLOSED/

│ └── part-0000

│

└── order_status=COMPLETE/

└── part-0000

Best Use Case

Partitioning works well with **low-cardinality columns**, such as:

- `order_status`
- `country`

Main Benefit

Partition Pruning

If a query filters on a partition column, Hive can skip directories that do not match the filter.

---

5. Bucketing

Bucketing divides data into a fixed number of **files/buckets** based on a hash function.

Example:

/warehouse/orders/

├── 000000_0 → Bucket 1

├── 000001_0 → Bucket 2

├── 000002_0 → Bucket 3

└── 000003_0 → Bucket 4

Best Use Case

Bucketing is useful for **high-cardinality columns**, such as:

- `order_id`
- Customer identifiers

Example:

CLUSTERED BY (order_id) INTO 4 BUCKETS

Main Benefits

- Optimizes joins
- Helps point lookups
- Supports bucket-based join strategies

---

6. Partitioning vs. Bucketing

Feature	Partitioning	Bucketing
Physical Structure	Folders / directories	Files
Best Column Type	Low cardinality	High cardinality
Example	`country`, `order_status`	`order_id`, `customer_id`
Main Optimization	Partition Pruning	Bucket-based joins / lookups
Number of Values	Relatively few	Potentially many
Primary Benefit	Avoids scanning unnecessary directories	Organizes records into fixed bucket files

Easy Way to Remember

Partitioning → Folders

Bucketing → Files

---

7. Hive Join Optimization Strategies

Hive supports multiple join optimization techniques.

7.1 Map-Side Join / Broadcast Join

Trigger

Used when one table is small enough to fit into memory.

The source material specifies a default threshold of approximately:

25 MB

Mechanism

1. Small table is loaded into memory.
2. An in-memory Hash Table is created.
3. The small table is distributed to nodes.
4. The Reduce phase is bypassed.

Benefit

Avoids the expensive Reduce phase and improves join performance.

---

8. Bucket Map Join

Bucket Map Join is useful when the datasets are too large to broadcast completely.

Requirements

Both tables must:

- Be bucketed on the join key
- Have compatible bucket counts

The number of buckets in one table should be an **integer multiple** of the other.

Example:

Table A → 4 buckets

Table B → 8 buckets

8 is an integer multiple of 4.

Mechanism

Instead of loading an entire table into memory, Hive processes matching buckets individually.

---

9. Sort-Merge-Bucket (SMB) Join

SMB Join is optimized for large datasets where both tables are already bucketed and sorted.

Requirements

Both tables must:

1. Be bucketed on the join key
2. Have the **same number of buckets**
3. Be sorted by the join key

Example:

Table A → 8 buckets

Table B → 8 buckets

Join Key → customer_id

Sorted By → customer_id

Mechanism

Hive sequentially merges corresponding bucket files.

This avoids:

- Network shuffling
- In-memory sorting

Key Difference

Bucket Map Join

→ Bucket counts can be integer multiples

SMB Join

→ Exact same number of buckets

→ Same join key

→ Pre-sorted on join key

---

10. Hive SMB Join Configuration

The following settings are used to enable/optimize SMB joins:

SET hive.input.format=org.apache.hadoop.hive.ql.io.BucketizedHiveInputFormat;

SET hive.auto.convert.sortmerge.join=true;

SET hive.optimize.bucketmapjoin=true;

SET hive.optimize.bucketmapjoin.sortedmerge=true;

SET hive.enforce.bucketing=true;

SET hive.enforce.sorting=true;

---

11. Transactional ACID Tables in Hive

Hive supports **ACID transactions** for workloads that require operations such as:

- INSERT
- UPDATE
- DELETE

ACID stands for:

A — Atomicity

C — Consistency

I — Isolation

D — Durability

These capabilities can be useful for workloads such as **Slowly Changing Dimensions (SCD)** and streaming updates.

11.1 Hive ACID Storage Structure

Hive stores transactional changes using base and delta files.

HDFS Directory/

├── base_0000001/

│ └── Initial snapshot

│

├── delta_0000002_0000002/

```
│   └── New inserts / updates
│
├── delta_0000003_0000003/
│   └── New inserts / updates
│
└── delete_delta_0000004_0000004/
    └── Recorded deletions
```

Read-Time Processing

At read time, Hive combines:

Base

+

Delta

+

Delete Delta

↓

Final Logical Dataset

---

12. Requirements for Full ACID Tables

According to the provided course material, full Hive ACID tables must:

1. Be **Managed Tables**
2. Use the **ORC** file format
3. Be **Bucketed**
4. Have:

```
TBLPROPERTIES ('transactional'='true')
```

12.1 Enabling Hive ACID Support

```
SET hive.support.concurrency=true;
```

```
SET hive.txn.manager=org.apache.hadoop.hive.ql.lockmgr.DbTxnManager;
```

12.2 Creating a Full ACID Table

```
CREATE TABLE orders_trx (
    order_id INT,
    order_date STRING,
    customer_id INT,
    order_status STRING
)
CLUSTERED BY (order_id) INTO 4 BUCKETS
STORED AS ORC
TBLPROPERTIES ('transactional'='true');
```

Remember

Full ACID Hive Table

Managed

+

ORC

+

Bucketed

+

transactional=true

---

13. Insert-Only Transactional Tables

Hive also supports **insert-only transactional tables**.

These tables are useful when the workload only requires:

INSERT

and does not require:

UPDATE

DELETE

Key Difference

According to the provided material:

> Insert-only transactional tables support all file formats, including CSV/Text.

They are created using:

TBLPROPERTIES (

```
    'transactional'='true',
    'transactional_properties'='insert_only'
)
```

---

14. MSCK REPAIR TABLE

The Problem

Hive's Metastore maintains information about table partitions.

Suppose a partition directory is manually added to HDFS:

hadoop fs -cp ...

Hive may not automatically know that the new partition exists.

Example

```
Hive Metastore
      |
      | Does not know
      ↓
HDFS
├── order_status=CLOSED
├── order_status=COMPLETE
└── order_status=PENDING  ← Manually added
```

The Metastore needs to be synchronized with the actual partition directories.

`MSCK REPAIR TABLE`

Use:

MSCK REPAIR TABLE database_name.table_name;

This scans the relevant HDFS directory structure and updates the Hive Metastore with partitions that were added outside Hive/Spark.

Key Takeaway

> **`MSCK REPAIR TABLE` synchronizes partition metadata in the Hive Metastore with partition directories physically present in storage.**

WEEK 14–15 QUICK REVISION SHEET

Git

Working Directory

↓ git add

Staging Area

↓ git commit

Local Repository

↓ git push

Remote Repository

Must-Know Commands

git init

git clone

git status

git add .

git commit

git push

git pull

git branch

git checkout -b

git restore

git reset

git stash

Enterprise Branching

Feature

↓

DEV

↓

TEST

↓

UAT

↓

PROD

CI/CD

BUILD

↓

TEST

↓

PACKAGE

↓

DEPLOY

Hive

OLTP vs OLAP

OLTP

→ Transactions

→ Low latency

→ Inserts / Updates / Deletes

→ Schema-on-Write

OLAP

→ Analytics

→ Large scans

→ Aggregations

→ Schema-on-Read

Hive Architecture

Beeline

↓

HiveServer2

↓

Hive Metastore

+

Compute Engine

↓

HDFS / S3 / ADLS

Managed vs External

MANAGED TABLE

→ Hive owns metadata + data

→ DROP = metadata + data deleted

EXTERNAL TABLE

→ Hive owns metadata only

→ DROP = metadata deleted

→ Data remains

Partitioning vs Bucketing

PARTITIONING

→ Folders

→ Low cardinality

→ Partition pruning

BUCKETING

→ Files

→ High cardinality

→ Join optimization

Hive Join Strategies

Map-Side Join

→ Small table

→ Broadcast

→ Avoids Reduce

Bucket Map Join

→ Bucketed tables

→ Bucket count can be an integer multiple

SMB Join

→ Same bucket count

→ Same join key

→ Sorted

→ Avoids shuffle + sorting

Hive ACID

Full ACID

→ Managed

→ ORC

→ Bucketed

→ transactional=true

MSCK

MSCK REPAIR TABLE database_name.table_name;

Purpose: Synchronize partition metadata between HDFS and the Hive Metastore.

⭐ Week 14–15 Interview Focus

If you're revising these weeks for a **Big Data / PySpark / Databricks interview**, prioritize these questions:

1. **Git vs GitHub — what's the difference?**
2. **Explain Git's three-stage architecture.**
3. **`git restore` vs `git reset` vs `git revert`**
4. **What is Git Stash and when would you use it?**
5. **What is a feature branch?**
6. **Explain DEV → TEST → UAT → PROD branching.**
7. **What is CI/CD?**
8. **Explain the stages of a Jenkins pipeline.**
9. **What is a Jenkinsfile?**
10. **What is Apache Hive?**
11. **What is the Hive Metastore?**
12. **HiveServer2 vs Hive Metastore**
13. **Managed vs External tables**
14. **Partitioning vs Bucketing**
15. **Map-Side Join vs Bucket Map Join vs SMB Join**
16. **What is an SMB Join?**
17. **What are Hive ACID tables?**
18. **Requirements for a full ACID table**
19. **What are Base, Delta and Delete Delta files?**
20. **What is `MSCK REPAIR TABLE`?**

One-Line Memory Map

Git → Version Control → Branching → Jenkins → CI/CD → Hive → Metastore → Tables → Partitioning/Bucketing → Joins → ACID → MSCK

Week 16: Resource Estimation, Production Debugging & Interview Preparation

1. Resource Estimation Framework for PySpark Jobs

Accurately sizing cluster resources for production PySpark jobs requires balancing **data volume, hardware configuration, and SLA requirements**. Simply allocating more resources does not always make a job faster.

Core Estimation Factors

1. Data Volume & Memory Expansion

Data stored on disk is usually compressed and serialized. Once loaded into memory, its representation can expand significantly.

A practical estimation rule is:

$$
\text{Memory Requirement} \approx \text{Disk Data Size} \times 4\text{–}6
$$

For example:

- 100 GB of input data
- Approximate in-memory requirement = **400–600 GB**

This expansion factor is particularly important when estimating executor memory for operations such as:

- Aggregations
- Joins
- Sorting
- Shuffles
- Caching

---

2. SLA Requirements

Resource allocation should also depend on **how quickly the job needs to finish**.

For example:

- If a 100 GB job can run for 2 hours → fewer resources may be acceptable.
- If the same job must complete in 15 minutes → significantly more parallelism may be required.

Therefore:

> **Resource estimation is not only about data size; it is also about the required completion time.**

---

3. Hardware Baseline

A commonly used baseline in the course is:

- **5 CPU cores per executor**
- Approximately **1 GB execution memory per CPU core**

Using 5 cores per executor helps balance:

- Parallelism
- HDFS/network I/O
- Garbage Collection
- Executor overhead

---

2. Example: 100-Node Cluster

Assume:

100 Nodes

Each Node:

16 CPU Cores

64 GB RAM

Reserve resources for the operating system:

OS Reservation:

1 CPU Core

1 GB RAM

Therefore:

Available per Node:

15 CPU Cores

63 GB RAM

With 5 cores per executor:

Executors per Node = 15 / 5

= 3 Executors

Each executor gets approximately:

5 CPU Cores

19 GB RAM

Therefore, the entire cluster can support approximately:

100 Nodes × 3 Executors

= 300 Executors

300 Executors × 5 Cores

= 1,500 CPU Cores

Cluster Summary

Resource	Value
Nodes	100
Cores per Node	16
RAM per Node	64 GB

Usable Cores per Node	15
Usable RAM per Node	63 GB
Executors per Node	3
Cores per Executor	5
RAM per Executor	~19 GB
Total Executors	300
Total Available Cores	1,500

---

3. Scenario A — 100 GB Aggregation Job

Suppose we have:

Input Data = 100 GB

Partition Size = 128 MB

Approximate number of input partitions:

[
100GB / 128MB \approx 800
]

So we have approximately:

800 input partitions

Stage 1 — Reading Input

If we want to process all 800 partitions simultaneously:

Required CPU Cores = 800

With 5 cores per executor:

[
800 / 5 = 160
]

Therefore:

160 executors

are required.

Stage 1

800 Input Partitions

↓

800 CPU Cores

↓

160 Executors

↓

5 Cores / Executor

Stage 2 — Shuffle

The default value of:

spark.sql.shuffle.partitions = 200

would produce approximately:

[
100GB / 200 = 500MB
]

per shuffle partition.

That is considerably larger than the 128 MB input-partition target.

Considering memory expansion, a 500 MB partition could require:

[
500MB \times 4 = 2GB
]

or even more depending on the workload.

This can cause:

- Memory pressure
- Disk spilling
- Garbage Collection
- Executor OOM

Solution

Increase:

spark.conf.set("spark.sql.shuffle.partitions", 800)

Now:

[
100GB / 800 = 125MB
]

per shuffle partition, which is close to the 128 MB baseline.

Key Idea

> **Shuffle partition sizing is important because the default 200 partitions may be too few for large datasets.**

Option 1 — Maximum Parallelism

Executors = 160

Cores = 800

Stage 1:

800 partitions → 1 wave

Stage 2:

800 shuffle partitions → 1 wave

This gives maximum parallelism but uses more resources.

Option 2 — Moderate Resource Footprint

Suppose we use:

80 Executors

5 cores each

= 400 cores

Then:

Stage 1:

800 partitions / 400 cores

= 2 waves

For Stage 2, using 400 shuffle partitions:

[
100GB / 400 = 250MB
]

per shuffle partition.

So:

Executors = 80

CPU Cores = 400

Stage 1 → 2 waves

Stage 2 → 400 shuffle partitions

Important Trade-off

Strategy	Resources	Parallelism	Cost
Maximum	160 executors	Very High	Higher
Moderate	80 executors	Medium	Lower

The goal is not always maximum parallelism.

The goal is an appropriate balance between execution time and resource consumption.

---

4. Scenario B — Large-Scale Join

Consider:

Orders = 100 GB

Customers = 50 GB

Total:

[
100GB + 50GB = 150GB
]

With 128 MB partitions:

[
150GB / 128MB \approx 1,200
]

So we have approximately:

1,200 input partitions

Maximum Parallelism

To process all 1,200 partitions simultaneously:

Required Cores = 1,200

With 5 cores per executor:

[
1200 / 5 = 240
]

Therefore:

240 executors

would be required.

Shuffle partitions can similarly be set around:

spark.sql.shuffle.partitions = 1200

Optimized Resource Allocation

Instead of 240 executors, suppose we use:

120 Executors

× 5 Cores

= 600 Cores

Stage 1

1,200 partitions

600 available cores

Therefore:

1,200 / 600

= 2 waves

Stage 2

Set:

spark.sql.shuffle.partitions = 600

Then:

[
150GB / 600 = 250MB
]

per shuffle partition.

So the job uses:

120 Executors

600 Cores

Stage 1 → 2 waves

Stage 2 → 600 shuffle partitions

---

5. Dynamic Resource Allocation

Static resource allocation can result in idle executors.

For example:

Stage 1 → Heavy workload

Stage 2 → Medium workload

Stage 3 → Small workload

If the cluster always maintains the same number of executors, resources can remain unused during lighter stages.

Dynamic Resource Allocation allows Spark to:

- Add executors when there is a backlog of tasks
- Remove idle executors
- Adapt resource consumption to workload requirements

Configuration

```
spark.conf.set("spark.dynamicAllocation.enabled", True)



spark.conf.set(

    "spark.dynamicAllocation.minExecutors",

    10

)



spark.conf.set(

    "spark.dynamicAllocation.maxExecutors",

    200

)



spark.conf.set(

    "spark.dynamicAllocation.initialExecutors",

    20

)



spark.conf.set(

    "spark.dynamicAllocation.schedulerBacklogTimeout",
```

```
    "1s"
)

spark.conf.set(
    "spark.dynamicAllocation.executorIdleTimeout",
    "60s"
)
```

Important Parameters

Parameter	Meaning
`enabled`	Turns Dynamic Allocation on/off
`minExecutors`	Minimum number of executors
`initialExecutors`	Starting number
`maxExecutors`	Maximum allowed
`schedulerBacklogTimeout`	Time before Spark requests additional executors
`executorIdleTimeout`	Time after which idle executors can be removed

---

6. Practical PySpark Coding Problems

Problem 1 — Remove Duplicate Records from Parquet

```
from pyspark.sql import SparkSession


spark = SparkSession.builder \
    .appName("Deduplication") \
    .getOrCreate()


source_path = "path/to/input/data.parquet"
target_path = "path/to/output/results"


df = spark.read.parquet(source_path)


df_clean = df.dropDuplicates()


df_clean.write \
    .mode("overwrite") \
    .parquet(target_path)
```

Key Concept

dropDuplicates()

removes duplicate rows across all columns.

---

7. Problem 2 — `collect_list`

Suppose the input is:

("a", "aa", 1)

("a", "aa", 2)

("b", "bb", 5)

("b", "bb", 3)

("b", "bb", 4)

Create:

```
from pyspark.sql.functions import collect_list


data = [
    ("a", "aa", 1),
    ("a", "aa", 2),
    ("b", "bb", 5),
    ("b", "bb", 3),
    ("b", "bb", 4)
]


df = spark.createDataFrame(
    data,
    ["col1", "col2", "col3"]
```

```
)

result_df = df.groupBy(
    "col1",
    "col2"
).agg(
    collect_list("col3").alias("col3")
)

result_df.show()
```

The same can be done using Spark SQL:

```
df.createOrReplaceTempView("my_table")

result_sql = spark.sql("""
    SELECT
        col1,
        col2,
        collect_list(col3) AS col3
    FROM my_table
    GROUP BY col1, col2
""")
```

Key Concept

`collect_list()` converts multiple values belonging to a group into an **array**.

---

8. Problem 3 — Exploding Nested Arrays

Suppose a JSON record contains:

```
{
  "dept_id": 101,
  "e_id": [10101, 10102, 10103]
}
```

Read it:

```
json_df = spark.read.json(
    "path/to/nested_file.json"
)
```

Use `explode()`:

```
from pyspark.sql.functions import explode

exploded_df = json_df.select(
    "dept_id",
    explode("e_id").alias("e_id")
)

exploded_df.show()
```

Conceptually:

Before:

dept_id e_id

101 [10101, 10102, 10103]

After:

dept_id e_id

101 10101

101 10102

101 10103

Key Concept

`explode()` converts an array element into separate rows.

---

9. Problem 4 — Handling Multi-Character Delimiters

Standard CSV readers may not directly handle unusual delimiters such as:

~|

Example:

Name~/Age

Brayan, Gomez~|25

John, Cleark~|30

Read as raw text:

```
input_df = spark.read.text(
    "/path/to/messedupfile.txt"
)
```

Extract the header:

```
header_row = input_df.first()[0]

schema_cols = header_row.split("~/")
```

Remove the header:

```
data_df = input_df.filter(
    input_df.value != header_row
)
```

Parse using an RDD:

```
parsed_df = data_df.rdd \
    .map(lambda row: row[0].split("~|")) \
    .toDF(schema_cols)
```

```
parsed_df.show()
```

Key Concept

When a standard reader cannot handle the file structure, one approach is:

```
Text File
   ↓
DataFrame
   ↓
RDD
   ↓
Custom Parsing
   ↓
DataFrame
```

---

10. Problem 5 — Calculate Team Size

Given:

```
data = [
    (1, 8),
    (2, 8),
    (3, 8),
    (4, 7),
```

```
    (5, 9),
    (6, 9)
]

employee_df = spark.createDataFrame(
    data,
    ["employee_id", "team_id"]
)

First group employees:
from pyspark.sql.functions import (
    collect_list,
    size,
    explode
)

result_df = employee_df \
    .groupBy("team_id") \
    .agg(
        collect_list("employee_id")
        .alias("employee_list")
    ) \
    .withColumn(
        "team_size",
```

```
        size("employee_list")
    ) \
    .selectExpr(
        "explode(employee_list) as employee_id",
        "team_size"
    )

result_df.show()
```

Processing Flow

Employee Records

↓

groupBy(team_id)

↓

collect_list(employee_id)

↓

size(employee_list)

↓

explode(employee_list)

↓

Employee + Team Size

This combines three important Spark functions:

collect_list()

size()

explode()

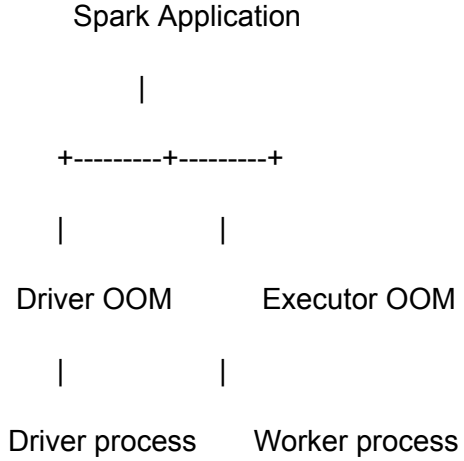
---

11. Production Debugging & Root Cause Analysis

A production Spark job can fail because of:

- Driver memory issues
- Executor memory issues
- Data skew
- Excessive shuffling
- Insufficient partitions
- Excessive caching
- Disk spilling
- Inefficient file formats
- Poor aggregation strategies

A useful first distinction is:



---

12. Driver OOM

A common cause is:

df.collect()

If the DataFrame is very large, `collect()` attempts to bring the entire dataset into the **driver's memory**.

Common Causes

- Calling `.collect()` on large DataFrames
- Broadcasting excessively large tables
- Excessive driver-side logs/buffers

Example

Avoid:

data = df.collect()

for large datasets.

Instead, keep processing distributed whenever possible.

---

13. Executor OOM

Executor OOM happens when an individual executor cannot handle its workload.

Common Causes

- Too few shuffle partitions
- Data skew
- Excessive caching
- Large partitions
- Insufficient executor memory

Possible Solutions

Increase:

spark.sql.shuffle.partitions

Filter data earlier:

df = df.filter(...)

or increase executor memory when appropriate:

--executor-memory

---

14. Diagnosing Data Skew Using Spark UI

A classic skew pattern is:

Task 1 → 10 seconds

Task 2 → 11 seconds

Task 3 → 9 seconds

...

Task 199 → 10 seconds

Task 200 → 2 hours

This indicates that one partition contains substantially more data than others.

Why?

Suppose:

Customer A → 10 records

Customer B → 15 records

Customer C → 20 records

BIG_CUSTOMER → 50 million records

When grouped or joined by customer ID, the `BIG_CUSTOMER` records can end up concentrated in a single partition.

This is **data skew**.

---

15. Fixing Data Skew

Two approaches highlighted in the material are:

1. AQE Skew Join

Enable:

```
spark.conf.set(
    "spark.sql.adaptive.skewJoin.enabled",
    True
)
```

Spark's Adaptive Query Execution can detect and mitigate skewed partitions during execution.

2. Salting

For heavily skewed keys, manually modify the join key by adding a salt value.

Conceptually:

Original Key:

customer_123

Salted:

customer_123_1

customer_123_2

customer_123_3

...

This distributes a heavily concentrated key across multiple partitions.

---

16. Disk Spill

When Spark does not have enough execution memory, it can spill intermediate data to disk.

Spark UI may show:

Spill (Memory)

Spill (Disk)

Large spill values can indicate memory pressure.

Potential Solutions

- Increase the number of shuffle partitions
- Reduce partition size
- Filter data earlier
- Review memory allocation

- Consider off-heap memory where appropriate

The course material also highlights:

spark.memory.fraction

spark.memory.offHeap.size

as memory-related configuration areas.

---

17. Optimizing Aggregations

From Week 11, remember the distinction:

HashAggregate

↓

Fast

↓

Works efficiently with suitable primitive/mutable types

SortAggregate

↓

Sorting overhead

↓

Can be slower

When appropriate, using primitive types such as:

Integer

Long

Timestamp

can allow Spark to use more efficient aggregation strategies.

---

18. Optimizing Storage Reads

Use columnar formats such as:

Parquet

ORC

and compression such as:

Snappy

Advantages include:

- Reduced storage footprint
- Reduced I/O
- Column pruning
- Predicate pushdown
- Better query performance

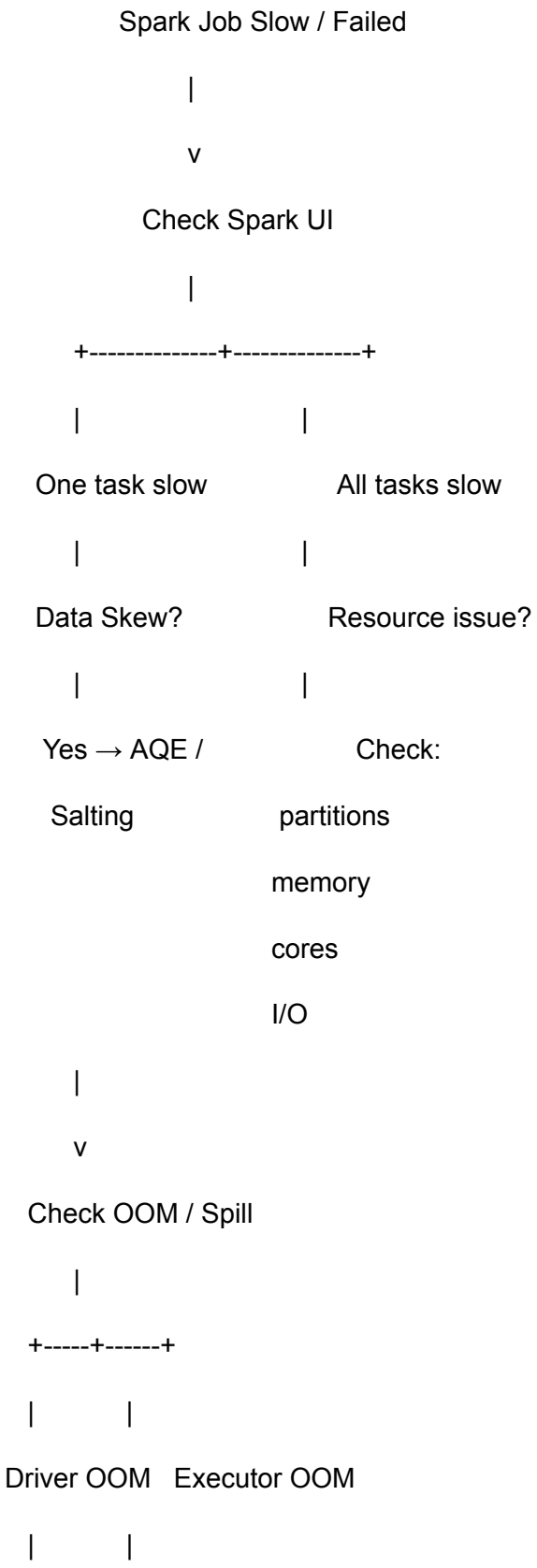
Partitioning is useful when queries frequently filter on **low-cardinality columns**.

For recurring large-table joins, pre-bucketing may also be considered.

---

19. Production Debugging Decision Tree

A useful way to approach a failing Spark job:


Spark Job Slow / Failed
Check Spark UI
One task slow
All tasks slow
Data Skew?
Resource issue?
Yes → AQE / Salting
Check: partitions memory cores I/O
Check OOM / Spill
Driver OOM
Executor OOM

collect() skew / partitions

broadcast cache / memory

---

20. Interview Preparation — High-Value Questions

Resource Estimation

Q1. Why can't we simply allocate maximum resources to every Spark job?

Because more resources do not automatically produce proportional performance improvements. Excessive resources can increase cost while resources remain underutilized.

Q2. Why is data size multiplied by 4–6 during memory estimation?

Disk data is typically compressed/serialized, while in-memory representations can be significantly larger. The course uses **4×–6×** as a practical estimation factor.

Q3. Why are 5 cores per executor commonly used in the baseline?

The course uses 5 cores as a balanced baseline to maintain parallelism while avoiding excessive GC and I/O pressure.

Shuffle

Q4. What happens if `spark.sql.shuffle.partitions` is too low?

Each shuffle partition becomes larger, increasing memory pressure and potentially causing:

- Spill
- Slow tasks
- Executor OOM

Q5. How do you choose the number of shuffle partitions?

Consider:

Total shuffle data

↓

Desired partition size

↓

Number of shuffle partitions

↓

Available CPU parallelism

Dynamic Allocation

Q6. What is Dynamic Resource Allocation?

It allows Spark to dynamically add and remove executors based on workload demand.

Debugging

Q7. How can you identify data skew from Spark UI?

Look for a stage where most tasks finish quickly but one or a few tasks take significantly longer.

Q8. How can you fix data skew?

Common approaches from the course:

- AQE skew join
- Salting
- Better partitioning strategies

Driver vs Executor

Q9. What is a common cause of Driver OOM?

Calling:

df.collect()

on a large dataset.

Q10. What causes Executor OOM?

Potential causes include:

- Large partitions
- Too few shuffle partitions
- Data skew
- Excessive caching
- Insufficient executor memory

---

21. Week 16 Quick Revision Sheet

WEEK 16 — MUST REMEMBER

RESOURCE ESTIMATION

│

├── Disk → Memory expansion ≈ 4–6×

├── 5 cores / executor

├── ~1 GB execution memory / CPU core

├── SLA determines required parallelism

└── Balance speed vs. resource cost

SHUFFLE

│

├── Too few partitions → Large partitions

├── Large partitions → Memory pressure

├── Memory pressure → Spill / OOM

└── Tune spark.sql.shuffle.partitions

DYNAMIC ALLOCATION

│

├── minExecutors

├── initialExecutors

├── maxExecutors

├── schedulerBacklogTimeout

└── executorIdleTimeout

PYSPARK

│

├── dropDuplicates()

├── collect_list()

├── explode()

├── groupBy()

└── size()

PRODUCTION DEBUGGING

│

├── Driver OOM

│ └── collect(), large driver-side operations

│

├── Executor OOM

│ └── skew, large partitions, insufficient memory

│

├── Data Skew

│ ├── AQE skew join

│ └── Salting

│

├── Disk Spill

│ └── Insufficient execution memory

│

└── Storage Optimization

├── Parquet / ORC

├── Snappy

├── Partitioning

└── Bucketing

The 10 things I'd memorize from Week 16

1. **Disk data can expand 4–6× in memory.**
2. **5 cores/executor** is the baseline used in the course.
3. More resources ≠ automatically faster jobs.
4. **Too few shuffle partitions → large partitions → OOM/spill.**
5. `spark.sql.shuffle.partitions` controls shuffle parallelism.
6. **Dynamic Allocation** adds/removes executors based on demand.
7. `.collect()` on huge data can cause **Driver OOM**.
8. One abnormally slow task → suspect **data skew**.
9. Fix skew using **AQE skew join or salting**.
10. Use **Parquet/ORC + compression + partitioning** for efficient data access.

WEEK 17

Cloud Computing Foundations & Azure Global Infrastructure

---

1. Cloud Computing Foundations & Economic Models

Cloud computing provides **on-demand availability of computing resources** such as servers, storage, databases, and networking over the internet.

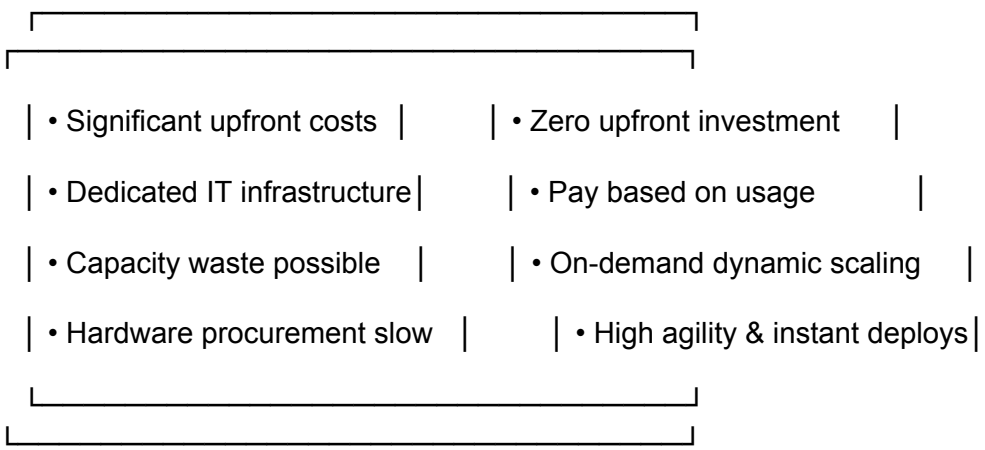
Cloud services generally follow a **pay-as-you-go** delivery model, allowing organizations to consume resources based on their requirements rather than purchasing and maintaining physical infrastructure.

On-Premise vs. Cloud

ON-PREMISE vs. CLOUD

ON-PREMISE (CapEx Dominant)

CLOUD COMPUTING (OpEx Dominant)



Capital Expenditure (CapEx)

CapEx refers to the large upfront investment required to purchase physical infrastructure.

Examples include:

- Physical servers
- Networking hardware
- Cooling systems
- Data-center infrastructure
- Real estate

The purchased equipment depreciates over time. Scaling generally requires purchasing additional hardware.

Operational Expenditure (OpEx)

OpEx refers to variable operational costs where organizations pay for resources according to their consumption.

Cloud environments allow organizations to:

- Pay based on usage
- Scale resources up or down
- Deploy infrastructure quickly
- Avoid large upfront hardware investments

CapEx vs. OpEx

Feature	On-Premise — CapEx	Cloud — OpEx
Upfront Cost	Significant initial investment	No upfront capital investment
Ongoing Costs	Relatively static maintenance costs	Dynamic, usage-based
Maintenance	Managed by in-house teams	Primarily handled by cloud provider
Scaling	Slow and limited by physical capacity	Fast and elastic
Deployment	Requires hardware procurement	Resources can be provisioned quickly

---

2. Core Cloud Characteristics

Cloud computing provides several important characteristics that distinguish it from traditional infrastructure.

2.1 Agility

Agility is the ability to provision and deploy IT infrastructure rapidly.

Traditional infrastructure may require months for procurement and installation, whereas cloud infrastructure can often be provisioned within minutes.

2.2 Scalability

Scalability is the ability to increase or decrease resources according to workload requirements.

There are two primary types.

Vertical Scaling — Scale Up / Scale Down

Increasing or decreasing the CPU and RAM of an existing machine.

Before:

VM → 4 CPU + 16 GB RAM

Scale Up:

VM → 8 CPU + 32 GB RAM

Horizontal Scaling — Scale Out / Scale In

Adding or removing multiple compute nodes.

Before:

[VM 1]

Scale Out:

[VM 1] [VM 2] [VM 3] [VM 4]

Scale In:

[VM 1] [VM 2]

2.3 Elasticity

Elasticity is the ability of a cloud environment to dynamically and automatically adjust resources based on real-time demand.

Example:

Low Traffic

↓

2 VMs

Traffic Increases

↓

5 VMs

Traffic Decreases

↓

2 VMs

Scalability vs. Elasticity

- **Scalability:** Ability to increase or decrease capacity.
- **Elasticity:** Ability to dynamically and automatically adjust capacity according to demand.

2.4 Fault Tolerance

Fault tolerance is the ability of a system to remain operational even when individual components fail.

It is achieved using redundancy and failure-isolation mechanisms.

2.5 Disaster Recovery (DR)

Disaster Recovery refers to architectural strategies that allow systems to recover from major outages or disasters.

A common approach is maintaining failover infrastructure in another region.

Primary Region

Secondary Region



Service Recovery

2.6 High Availability (HA)

High Availability focuses on keeping systems operational with minimal unexpected downtime.

The availability formula is:

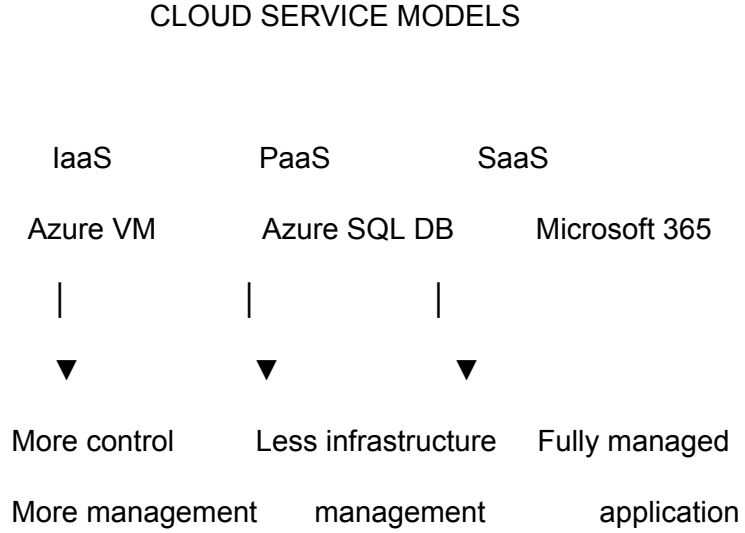
[
Availability =
\frac{Total\ Uptime}
{Total\ Uptime + Downtime}
]

Higher availability means less downtime.

---

3. Cloud Service Models & Deployment Strategies

Cloud services are commonly divided into three major service models:



3.1 Infrastructure as a Service — IaaS

IaaS provides fundamental infrastructure such as:

- Virtual machines
- Compute
- Networking
- Storage

Example:

Azure Virtual Machines

The customer manages the operating system, runtime, applications, and data.

The cloud provider manages the underlying physical infrastructure.

3.2 Platform as a Service — PaaS

PaaS provides a managed platform where users can focus primarily on their application and data.

Examples:

- Azure SQL Database
- Databricks

The cloud provider manages infrastructure and much of the underlying platform.

3.3 Software as a Service — SaaS

SaaS provides complete applications that users access directly.

Examples:

- Microsoft 365
- Google Drive

The vendor manages the underlying infrastructure, platform, and application.

IaaS vs. PaaS vs. SaaS

Layer	IaaS	PaaS	SaaS
Application Data	Customer	Customer	Vendor-managed
Application Code	Customer	Customer	Vendor-managed
Runtime	Customer	Provider	Vendor
Middleware	Customer	Provider	Vendor

Operating System	Customer	Provider	Vendor
Virtualization	Provider	Provider	Vendor
Hardware	Provider	Provider	Vendor
Storage & Network	Provider	Provider	Vendor

Memory Trick

IaaS → Infrastructure

PaaS → Platform

SaaS → Software

The further you move from IaaS toward SaaS, the more infrastructure management is handled by the provider.

---

4. Cloud Deployment Models

4.1 Public Cloud

Infrastructure is provided by a cloud provider and made available to customers over the internet.

Examples:

- Microsoft Azure
- Amazon Web Services
- Google Cloud Platform

Advantages:

- High elasticity

- Low initial investment
- Rapid provisioning

4.2 Private Cloud

Infrastructure is dedicated exclusively to one organization.

Advantages:

- Greater control
- Strong security and compliance capabilities

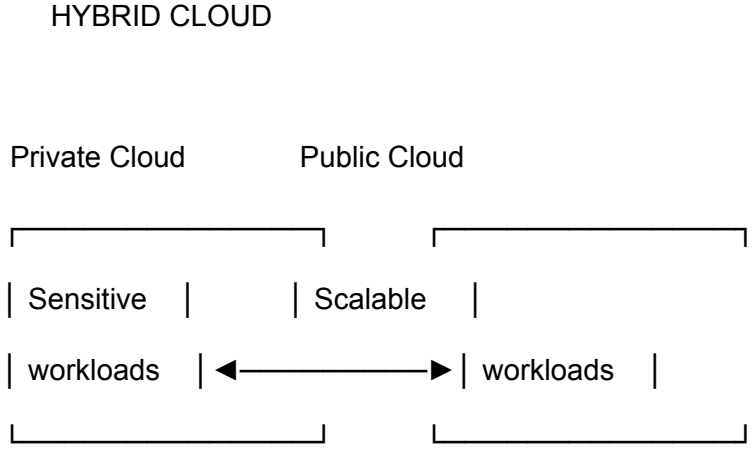
Disadvantage:

- Higher infrastructure and operational requirements

4.3 Hybrid Cloud

A hybrid cloud combines public and private cloud infrastructure.

For example:

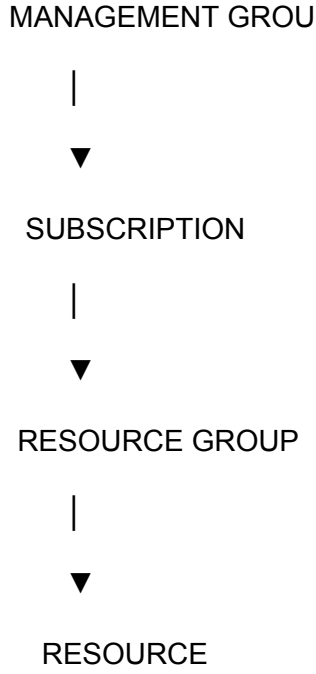


Sensitive workloads can remain in private infrastructure while scalable workloads can use public-cloud resources.

---

5. Azure Resource Management & Account Hierarchy

Azure organizes resources using a hierarchical structure:



5.1 Management Groups

Management Groups are containers used to manage:

- Access
- Policies
- Governance

across multiple Azure subscriptions.

5.2 Subscription

A **Subscription** is an important boundary for:

- Billing

- Access control
- Resource management

Examples may include different billing or organizational subscriptions.

5.3 Resource Group

A **Resource Group (RG)** is a logical container for resources that share a common lifecycle.

Examples of resources inside a Resource Group:

Resource Group

├── Virtual Machine

├── Storage Account

├── Database

└── Networking Resources

Important Resource Group Rules

1. Every resource must belong to **exactly one Resource Group**.
2. Resource Groups **cannot be nested**.
3. Resources can be moved between Resource Groups.
4. Resources in the same Resource Group can exist in **different physical regions**.
5. The Resource Group itself stores metadata in its assigned region.

5.4 Resources

Resources are individual Azure services.

Examples:

- Virtual Machines
- Storage Accounts
- Databases
- Networking components

5.5 Azure Resource Manager — ARM

Azure Resource Manager (ARM) is the management layer through which Azure resources are controlled.

Requests can originate from:

- Azure Portal
- Azure CLI
- PowerShell
- SDKs
- REST APIs

Azure Portal

Azure CLI

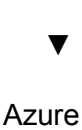
PowerShell

SDK

REST API



Azure Resource Manager

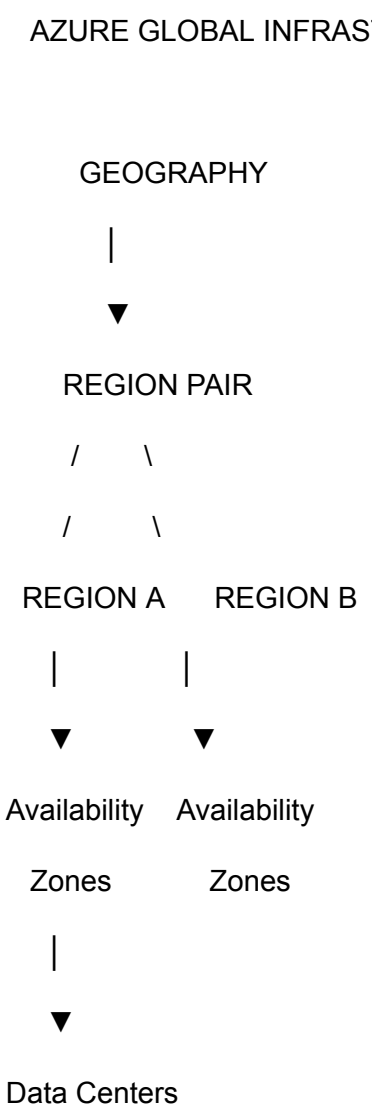


Azure Resources

---

6. Azure Global Infrastructure

Azure infrastructure can be understood using the following hierarchy:



---

7. Data Centers

A **Data Center** is a physical facility containing interconnected servers and infrastructure.

It contains:

- Servers
- Networking

- Power systems
- Cooling systems
- Physical infrastructure

A data center represents the physical foundation of cloud infrastructure.

---

8. Azure Regions

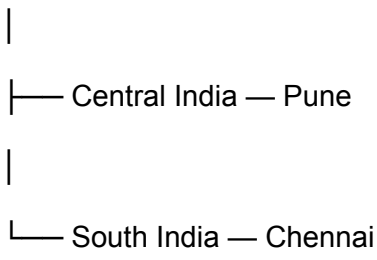
An **Azure Region** is a geographical area containing one or more data centers connected through a low-latency network.

Region selection can depend on:

- Proximity to users
- Service availability
- Pricing
- Availability Zone support
- Regulatory requirements

Example from the course material:

India Geography



---

9. Availability Zones

An **Availability Zone** is a physically separate data-center location within an Azure region.

Availability Zones have independent:

- Power

- Cooling
- Networking

They protect workloads from failures affecting an individual data-center location.

Zone-Enabled Region

A region is considered **Zone Enabled** when it contains three or more independent Availability Zones, according to the course material.

Zonal vs. Zone-Redundant Services

Zonal Service

A resource is pinned to a specific Availability Zone.

Examples:

- Azure Virtual Machines
- Managed Disks

Zone-Redundant Service

Azure automatically replicates the service across multiple Availability Zones.

Examples:

- Azure SQL
- Storage Accounts

---

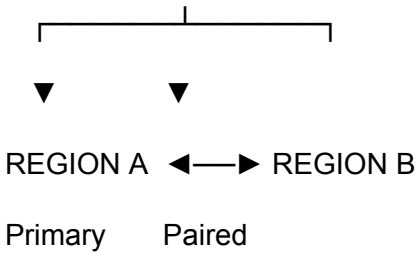
10. Azure Region Pairs

Azure regions are paired with another region within the same geography.

The purpose is to provide resilience against region-wide disasters.

GEOGRAPHY

|



Examples from the course material:

- Central India ↔ South India
- East US ↔ West US

Region pairs help organizations design disaster recovery strategies.

---

11. Azure Geographies

A **Geography** represents a broad geographical boundary containing one or more Azure regions.

Examples:

- India
- United States
- Europe

Geographies are important for:

- Data residency
- Regulatory compliance
- Data sovereignty

---

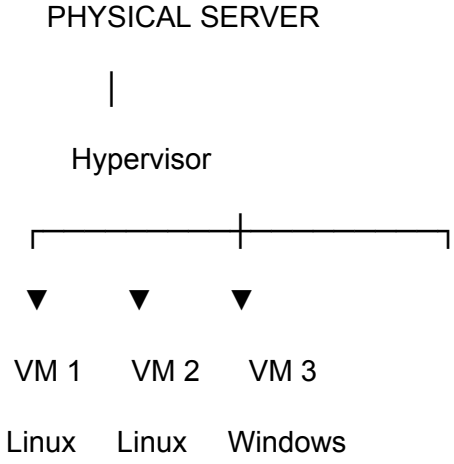
12. Azure Compute — Virtual Machines

An **Azure Virtual Machine (VM)** is an IaaS compute service.

A VM provides:

- CPU
- Memory
- Storage
- Guest operating system
- Network connectivity

Multiple VMs can run on a physical host through a **Hypervisor**.



The hypervisor provides isolation between virtual machines while allowing hardware resources to be shared efficiently.

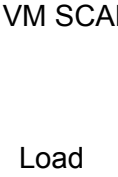
---

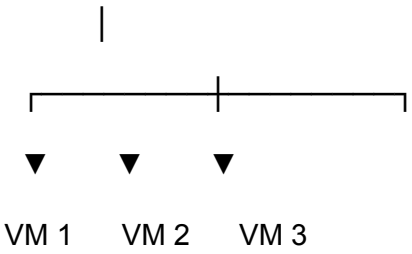
13. Virtual Machine Scale Sets — VMSS

Virtual Machine Scale Sets (VMSS) allow Azure to automatically deploy and manage groups of identical VMs.

VMSS can:

- Deploy identical VM instances
- Scale out during increased demand
- Scale in during reduced demand
- Balance workloads across instances





Traffic increases



VM 4

VM 5

---

14. Fault Domains & Update Domains

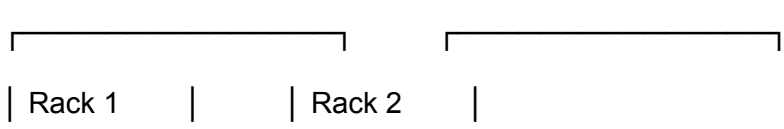
Azure provides two important concepts for availability sets.

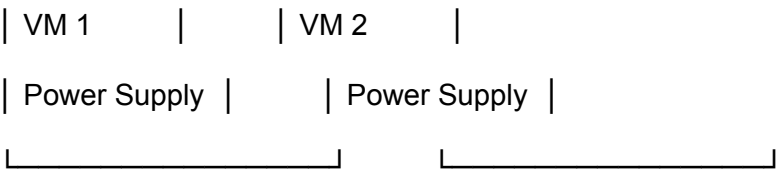
14.1 Fault Domain — FD

A **Fault Domain** represents a physical grouping of hardware that shares common infrastructure such as:

- Power source
- Network switch
- Physical rack

Fault Domains protect against localized physical hardware failures.





14.2 Update Domain — UD

An **Update Domain** is a logical grouping of infrastructure that undergoes maintenance or software patching together.

The objective is to ensure that not all VMs are unavailable at the same time.

Update Domain 0 → Maintenance

Update Domain 1 → Running

Update Domain 2 → Running

↓

Update Domain 0 → Running

Update Domain 1 → Maintenance

Update Domain 2 → Running

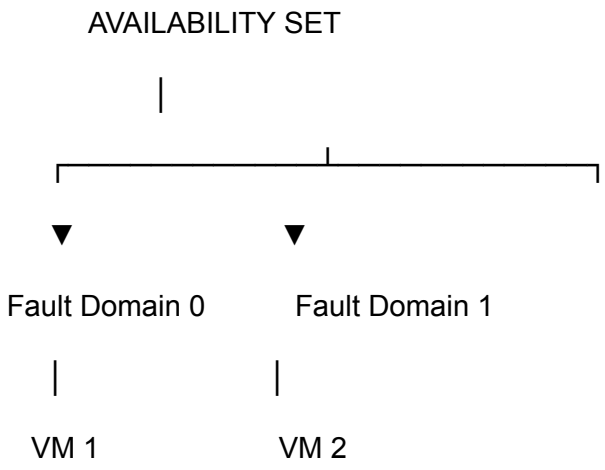
---

15. Availability Sets

An **Availability Set** groups VMs across multiple Fault Domains and Update Domains **within a single data center**.

It protects against:

- Rack failures
- Hardware failures
- Planned maintenance events



The key limitation is that Availability Sets do **not** protect against a complete data-center outage.

---

16. Availability Zones vs. Availability Sets

Feature	Availability Set	Availability Zone
Scope	Within one data center	Across separate data centers
Protection	Rack/hardware failures	Data-center failures
Fault Domain	Yes	Physical separation
Update Domain	Yes	Not the primary concept

Resilience Level	Lower	Higher
Typical Use	Intra-datacenter availability	Datacenter-level resilience

Easy Way to Remember

Availability Set → Separate VMs inside a data center

Availability Zone → Separate VMs across data centers

---

17. Azure Availability SLA Comparison

The course material presents the following availability targets:

Configuration	Failure Scenario	Azure Guaranteed SLA	Approx. Annual Maximum Downtime
Single VM	Basic instance failure	**99.9%**	~8 hours 45 minutes
Availability Set — ≥2 VMs	Intra-datacenter rack failure	**99.95%**	~4 hours 22 minutes
Availability Zones — ≥2 zones	Data-center-level outage	**99.99%**	~52 minutes

Availability Calculation

[
Availability =

\frac{Uptime}{Uptime + Downtime}
]

As the number of **9s** increases, the permitted downtime decreases.

99.9% → ~8h 45m downtime/year

99.95% → ~4h 22m downtime/year

99.99% → ~52m downtime/year

---

18. Week 17 — Key Takeaways

Cloud Fundamentals

- Cloud computing provides on-demand resources.
- **CapEx** involves upfront infrastructure investment.
- **OpEx** is usage-based operational expenditure.
- Cloud provides agility, scalability and elasticity.
- Vertical scaling changes the size of a machine.
- Horizontal scaling changes the number of machines.
- HA minimizes downtime.
- DR focuses on recovery from major failures.

Cloud Service Models

IaaS → Infrastructure

PaaS → Platform

SaaS → Software

Azure Resource Hierarchy

Management Group

↓

Subscription

↓

Resource Group

↓

Resource

Azure Physical Hierarchy

Geography

↓

Region Pair

↓

Region

↓

Availability Zone

↓

Data Center

Availability

Fault Domain

↓

Protects against physical hardware/rack failures

Update Domain

↓

Protects against planned maintenance

Availability Set

↓

Protects within a data center

Availability Zone

↓

Protects across separate data centers

---

19. Interview Questions — Week 17

Basic

1. What is cloud computing?
2. What is the difference between CapEx and OpEx?
3. What is vertical scaling?
4. What is horizontal scaling?
5. What is elasticity?
6. What is fault tolerance?
7. What is High Availability?
8. What is Disaster Recovery?
9. What are IaaS, PaaS and SaaS?
10. What are public, private and hybrid clouds?

Azure

11. Explain the Azure resource hierarchy.
12. What is a Subscription?
13. What is a Resource Group?
14. Can a Resource Group contain resources from different regions?
15. Can Resource Groups be nested?

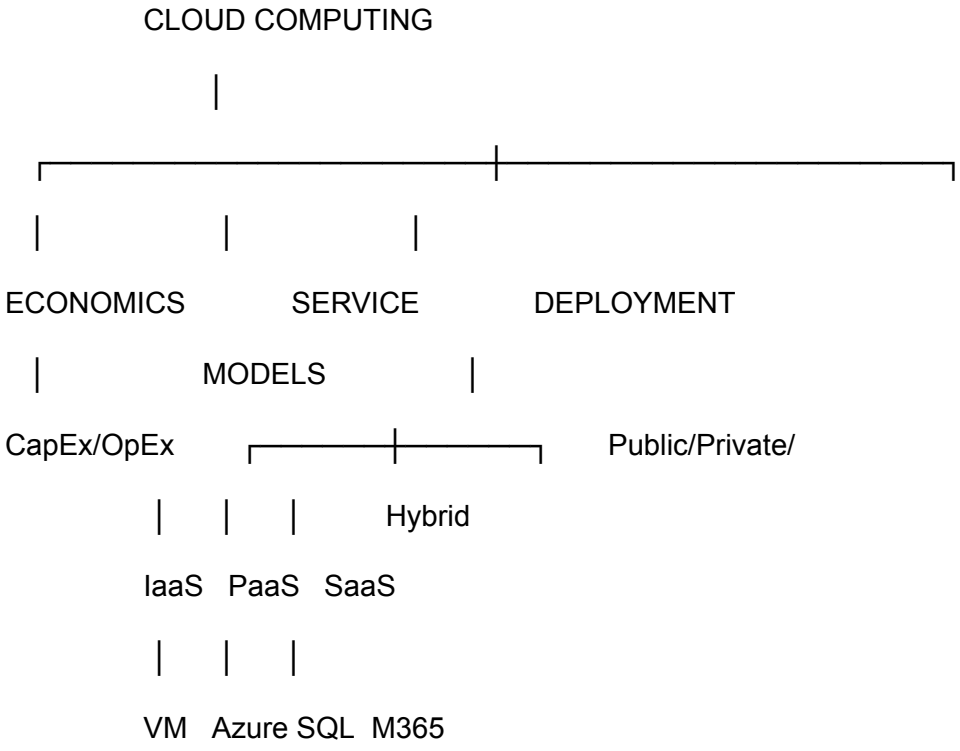
16. What is Azure Resource Manager?
17. What is an Azure Region?
18. What is an Availability Zone?
19. What is a Region Pair?
20. What is an Azure Geography?

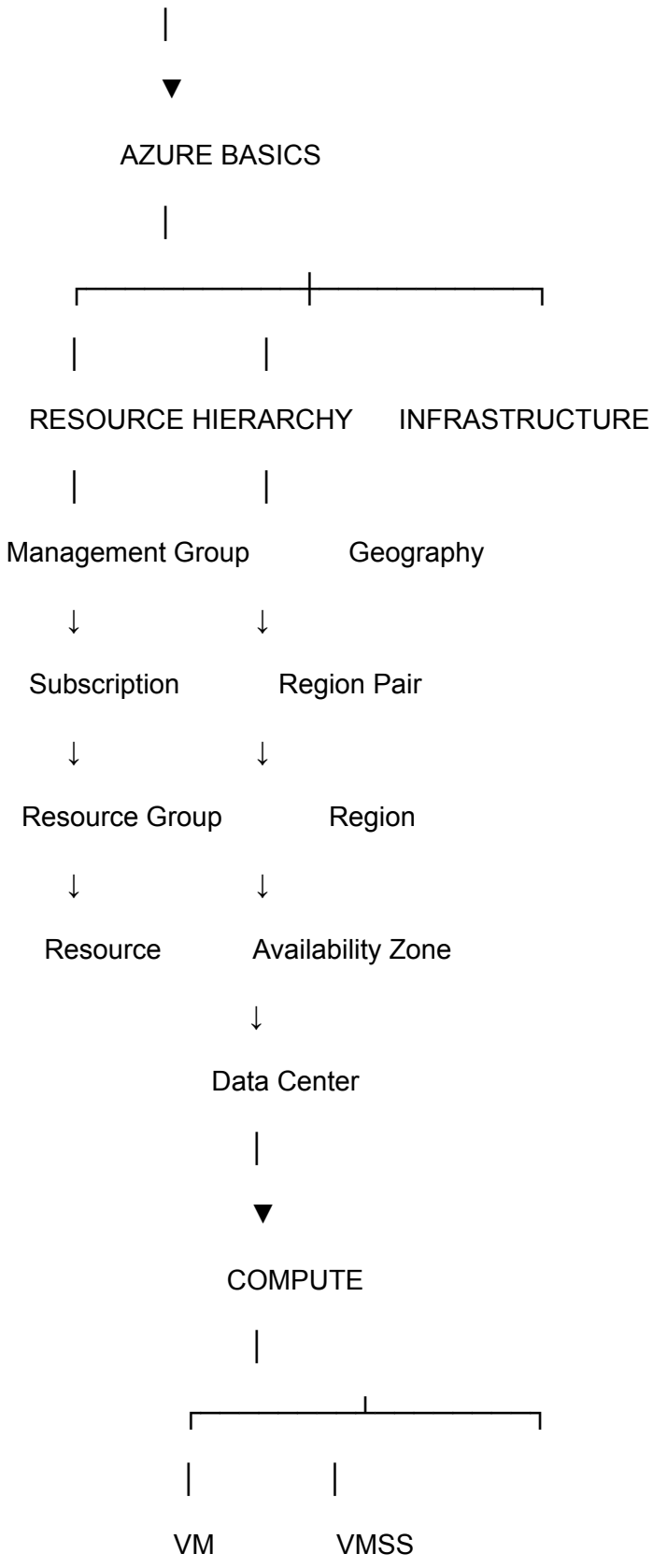
Azure Compute & Availability

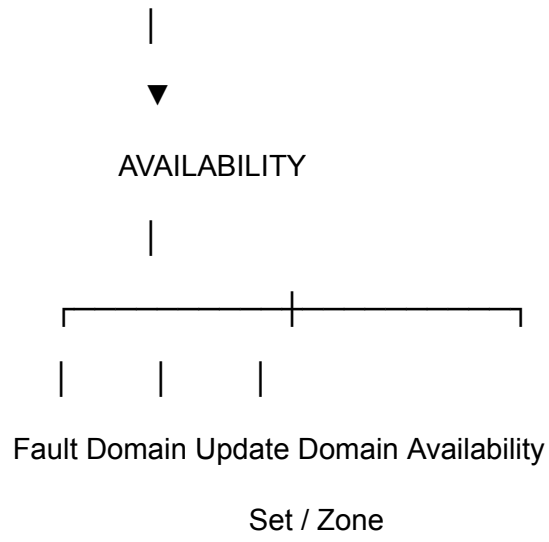
21. What is an Azure Virtual Machine?
22. What is VMSS?
23. What is a Fault Domain?
24. What is an Update Domain?
25. What is an Availability Set?
26. What is an Availability Zone?
27. What is the difference between Availability Set and Availability Zone?
28. How does Azure protect against rack failure?
29. How does Azure protect against data-center failure?
30. What is the difference between zonal and zone-redundant services?

---

20. One-Page Revision Map




AZURE BASICS
RESOURCE HIERARCHY
INFRASTRUCTURE
Management Group
Geography
Subscription
Region Pair
Resource Group
Region
Resource
Availability Zone
Data Center
COMPUTE
VM
VMSS



Final Mental Model

> **Cloud gives you elastic infrastructure. Azure organizes that infrastructure through subscriptions and resource groups, physically distributes it across regions and availability zones, and provides compute services such as VMs and VM Scale Sets with multiple mechanisms for high availability and fault tolerance.**

Week 18: Azure Storage Services & Azure Data Lake Storage Gen2 (ADLS Gen2)

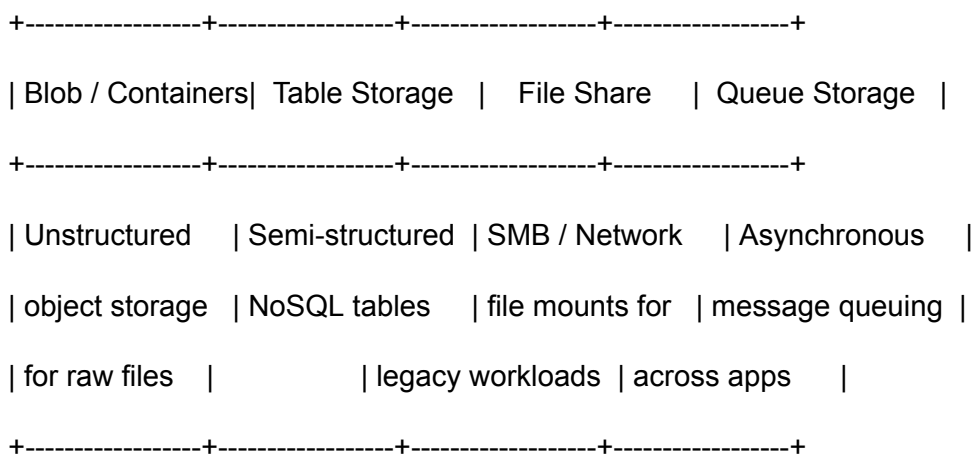
---

1. Azure Storage Account

An **Azure Storage Account** is a foundational cloud storage service designed to provide highly durable, scalable, and secure storage.

It supports multiple storage services:

[Azure Storage Account]



Core Storage Services

Service	Purpose
Blob Storage	Stores unstructured objects such as files, images, CSV, JSON, Parquet, etc.
Table Storage	NoSQL key-value storage for semi-structured data
File Storage	Managed file shares accessible using SMB
Queue Storage	Asynchronous message queuing between applications

---

2. Azure Storage Access Tiers

Azure Blob Storage provides different access tiers based on how frequently data is accessed.

Hot Tier

- Designed for **frequently accessed data**
- Higher storage cost
- Lower access/retrieval cost
- Suitable for active datasets

Cool Tier

- Designed for **infrequently accessed data**
- Lower storage cost than Hot
- Higher retrieval cost
- Intended for data stored for at least **30 days**

Archive Tier

- Designed for **rarely accessed historical data**
- Lowest storage cost
- Highest retrieval cost
- Intended for data stored for at least **180 days**

Rehydration

Data stored in Archive cannot be accessed immediately like Hot or Cool data.

It must first undergo **rehydration**, which moves it back into an online access tier.

This process can take several hours.

Early Deletion

Deleting or moving data before the minimum retention period can result in early deletion charges:

- Cool → minimum **30 days**
- Archive → minimum **180 days**

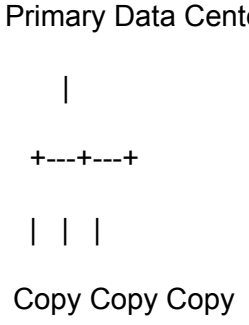
---

3. Azure Storage Redundancy

Azure provides multiple replication strategies to protect against hardware, data-center, and regional failures.

Redundancy	Replication	Protection
LRS	3 copies within one data center	Server, disk, or rack failure
ZRS	3 synchronous copies across Availability Zones	Data-center failure
GRS	LRS in primary + LRS in secondary region	Regional disaster
RA-GRS	GRS + read access to secondary	Regional outage with read availability
GZRS	ZRS in primary + LRS in secondary	Zone + regional failures

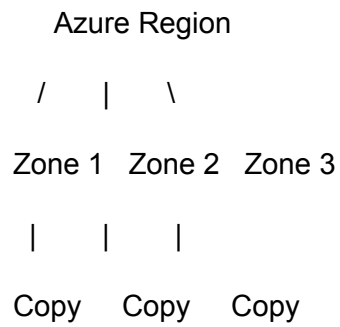
LRS — Locally Redundant Storage



Three copies are maintained within the same physical data center.

Protects primarily against localized hardware failures.

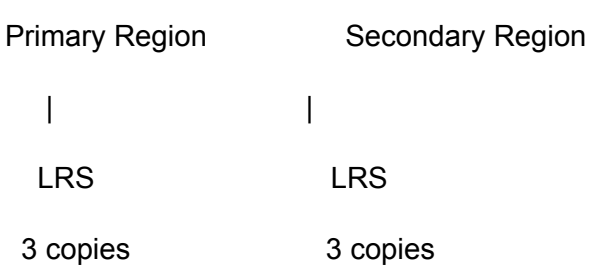
ZRS — Zone-Redundant Storage



Data is synchronously replicated across Availability Zones.

Protects against an entire data center becoming unavailable.

GRS — Geo-Redundant Storage



Three copies exist in the primary region and three in the paired secondary region.

RA-GRS

Provides the same geo-redundancy as GRS while also allowing **read access to the secondary region**.

GZRS — Geo-Zone-Redundant Storage

Combines zone-level redundancy in the primary region with geo-redundancy to a secondary region.

Primary Region

Zone 1

Zone 2 --> ZRS

Zone 3

|

| Geo Replication

v

Secondary Region

LRS

---

4. Standard Blob Storage vs. ADLS Gen2

Azure Data Lake Storage Gen2 (ADLS Gen2) is built on Azure Blob Storage and optimized for large-scale analytics workloads.

The major distinction is the **Hierarchical Namespace (HNS)**.

Standard Blob Storage ADLS Gen2

HNS Disabled HNS Enabled

| |

Flat namespace Hierarchical namespace

| |

Blob objects Directories + files

| |

Limited ACL model POSIX-style ACLs

Standard Blob Storage

- Flat namespace
- Objects are stored within containers
- No true directory hierarchy
- Directory operations can require object-level operations

ADLS Gen2

- Hierarchical Namespace enabled
- Supports directories and nested folders
- Supports granular file and directory permissions
- Optimized for Big Data analytics

---

5. Hierarchical Namespace (HNS)

HNS transforms the flat Blob namespace into a hierarchical directory structure.

For example:

/raw

  /sales

    /2026

      /01

        orders.parquet

      /02

        orders.parquet

/processed

/sales

orders.parquet

One important benefit is that directory operations such as rename or move can be handled as **metadata operations** rather than requiring every underlying file to be copied and deleted individually.

---

6. POSIX Access Control Lists (ACLs)

ADLS Gen2 supports granular permissions at directory and file levels.

Example:

Storage Account

|

+-- raw/

| |

| +-- sales/

| +-- finance/

|

+-- processed/

Different users or applications can be granted different permissions for different paths.

This makes ADLS Gen2 particularly suitable for enterprise data lakes where multiple teams require controlled access to shared data.

---

7. Key Takeaways — Week 18

Remember these concepts for interviews:

1. **Azure Storage Account** is the foundation for Azure storage services.
2. **Hot, Cool, Archive** tiers optimize storage cost according to access frequency.
3. **LRS** protects against localized hardware failures.
4. **ZRS** protects against Availability Zone failures.
5. **GRS** protects against regional disasters.
6. **RA-GRS** provides read access to the secondary region.
7. **GZRS** combines zone and geographic redundancy.
8. **ADLS Gen2 = Blob Storage + Hierarchical Namespace + Data Lake capabilities.**
9. **HNS** enables hierarchical directories and efficient directory operations.
10. **ACLs** provide granular file- and directory-level access control.

Week 19: Azure Databricks Architecture & Delta Lake Deep Dive

---

1. Azure Databricks

Azure Databricks is an Apache Spark-based analytics platform optimized for Azure.

It simplifies Spark-based data engineering by providing managed infrastructure, notebooks, cluster management, security integrations, and collaborative development capabilities.

Instead of manually configuring Spark infrastructure, users can provision and manage Spark environments through Databricks.

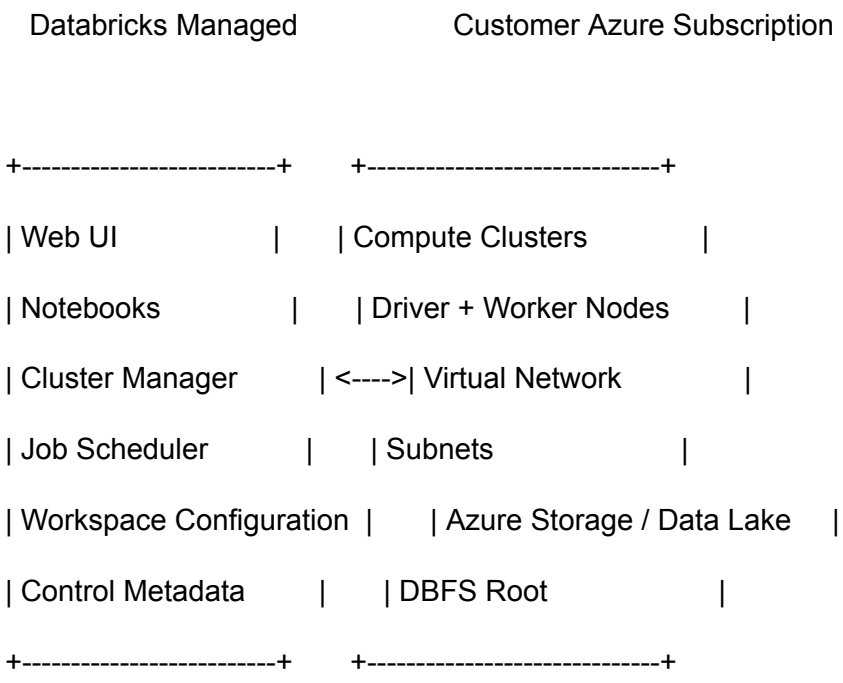
---

2. Azure Databricks Architecture

Azure Databricks follows a **Control Plane / Data Plane** architecture.

Azure Databricks Architecture

CONTROL PLANE DATA PLANE



Control Plane

Managed by Databricks.

Includes:

- Web application
- Notebook interface
- Cluster management
- Job scheduling
- Workspace configuration
- Control metadata

Data Plane

Contains the actual compute and data resources.

Includes:

- Driver nodes
- Worker nodes
- Virtual networks
- Subnets
- Azure Storage

- Data Lake resources

---

3. Databricks Pricing Model

Azure Databricks involves two primary cost components:

1. Azure Infrastructure Cost

Includes infrastructure such as:

- Virtual machines
- Disks
- Networking

2. Databricks Software Cost

Databricks charges based on **DBUs — Databricks Units**.

Therefore:

Total Databricks Cost

=

Azure Infrastructure Cost

+

Databricks DBU Cost

---

4. Databricks Cluster Types

All-Purpose Cluster

Used mainly for interactive development.

Characteristics:

- Manually started

- Used for notebooks and development
- Can remain active until manually terminated
- Higher cost if left running unnecessarily

Job Cluster

Designed for automated production workloads.

Characteristics:

- Created when a job starts
- Runs the workload
- Automatically terminates after completion
- Better suited for production pipelines

All-Purpose Cluster

Developer

|

v

Notebook

|

v

Persistent Cluster

Job Cluster

Workflow

|

v

Create Cluster

|

v

Execute Job

|

v

Terminate Cluster

---

5. Cluster Modes

Single Node Cluster

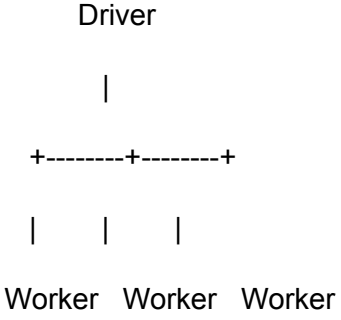
One VM acts as both:

- Driver
- Worker

Suitable for small workloads and single-user processing.

Standard Cluster

Contains:



Suitable for distributed Spark processing.

High Concurrency Cluster

Designed for multiple users sharing compute resources.

Provides workload isolation and resource sharing.

---

6. Databricks Magic Commands

Databricks notebooks support multiple programming languages.

`%python`

Runs Python code.

%python

```
df = spark.read.parquet("/data/orders")
```

`%sql`

Runs SQL.

%sql

```
SELECT * FROM orders;
```

`%scala`

Runs Scala.

`%r`

Runs R.

`%md`

Creates Markdown documentation.

`%fs`

Provides a shortcut for file-system operations.

---

7. Databricks Utilities — `dbutils`

`dbutils` provides utility functions for common Databricks operations.

File System Utilities

List Files

dbutils.fs.ls("dbfs:/FileStore")

Read File Header

dbutils.fs.head("path", 65536)

Create Directory

dbutils.fs.mkdirs("path")

Move Files

dbutils.fs.mv("src", "dest", recurse=True)

Delete Files

```
dbutils.fs.rm("path", recurse=True)
```

---

8. Notebook Orchestration

Databricks notebooks can invoke other notebooks.

Run Another Notebook

```
dbutils.notebook.run(
    "path/to/notebook",
    timeout_seconds,
    arguments
)
```

This is useful for creating modular workflows.

Return a Value

```
dbutils.notebook.exit("SUCCESS")
```

The parent notebook can receive the returned value.

---

9. Databricks Widgets

Widgets allow users to provide parameters dynamically.

Examples include:

- Dropdown

- Combobox
- Multiselect
- Text input

```
dbutils.widgets.text(
    "environment",
    "dev",
    "Environment"
)
```

Retrieve the selected value:

```
environment = dbutils.widgets.get("environment")
```

This allows the same notebook to operate against different environments or datasets.

---

10. Azure Storage Authentication

Databricks can access Azure Storage using different authentication mechanisms.

The three approaches covered are:

Account Access Key

|

v

SAS Token

|

v

Service Principal / OAuth

---

11. Account Access Key

An account access key provides broad access to the storage account.

Example:

```
dbutils.fs.mount(
    source="wasbs://retaildb@storageaccount.blob.core.windows.net",
    mount_point="/mnt/retaildb",
    extra_configs={
        "fs.azure.account.key.storageaccount.blob.core.windows.net":
        "ACCESS_KEY"
    }
)
```

Limitation

Access keys can provide very broad permissions and therefore have weaker granularity from a security perspective.

---

12. SAS Token

A **Shared Access Signature (SAS)** provides more restricted access.

It can be scoped by:

- Resource
- Permissions
- Validity period

Example:

```
spark.conf.set(
    "fs.azure.account.auth.type.storage.dfs.core.windows.net",
    "SAS"
)
```

SAS is more granular than an account key because access can be restricted and time-bound.

---

13. Service Principal / OAuth

A **Service Principal** represents an application identity in Azure.

It can be used with OAuth authentication and Azure RBAC/ACLs.

Typical credentials include:

Tenant ID

Client ID

Client Secret

Conceptually:

```
Databricks
    |
    | OAuth
    v
Azure AD / Entra ID
    |
    v
Service Principal
```

|

v

ADLS Gen2

This approach is suitable for enterprise workloads requiring fine-grained identity-based access.

---

14. Databricks Secret Scopes

Never hardcode sensitive credentials directly inside notebooks.

Instead of:

client_secret = "actual-secret-value"

use a secret scope:

client_secret = dbutils.secrets.get(

"my_scope",

"client_secret"

)

The notebook references the secret without exposing the raw credential.

---

15. Secret Scope Types

Azure Key Vault-Backed Secret Scope

Secrets are maintained in **Azure Key Vault**.

Advantages:

- Centralized secret management
- Can be shared with other Azure applications
- Enterprise-oriented secret management

Databricks-Backed Secret Scope

Secrets are stored in Databricks' internal encrypted storage.

Example:

databricks secrets create-scope \

--scope demoscopedatabricksbacked

Store a secret:

databricks secrets put \

--scope demoscopedatabricksbacked \

--key storageKey

Retrieve it:

```
secret_key = dbutils.secrets.get(
    "demoscopedatabricksbacked",
    "storageKey"
)
```

---

16. Traditional Data Lake vs. Delta Lake

Traditional data lakes store files such as:

CSV

JSON

Parquet

However, simply storing files does not provide full database-like transaction capabilities.

Common challenges include:

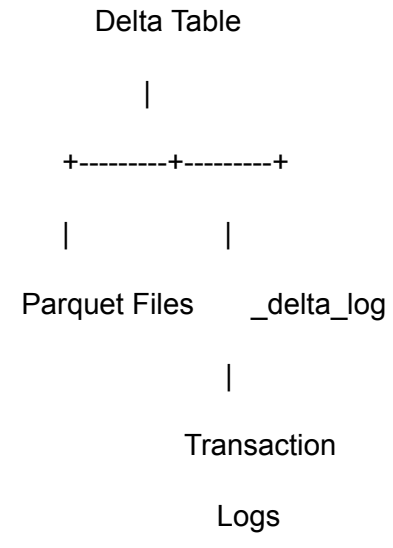
- Failed writes leaving inconsistent state
- Concurrent reads and writes
- Lack of schema enforcement
- Limited UPDATE/DELETE functionality
- No native table version history

Delta Lake addresses these limitations.

---

17. Delta Lake Architecture

The most important concept:



In simplified form:

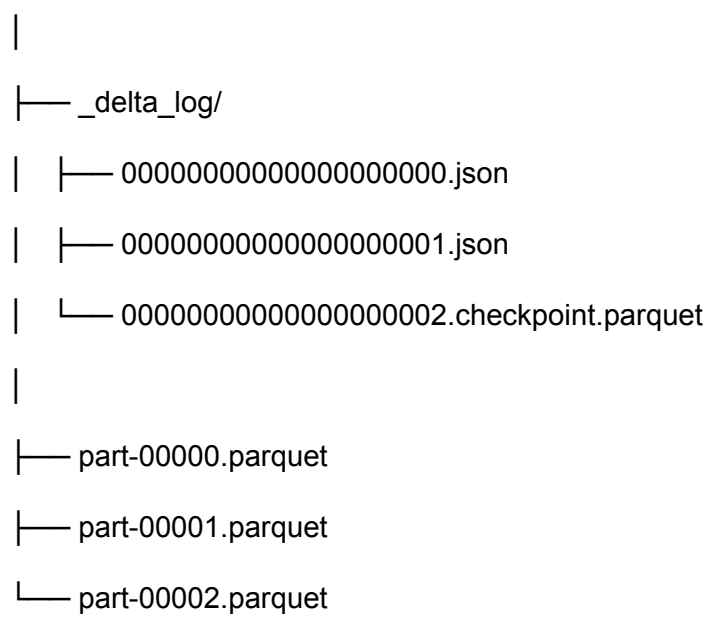
$$
\boxed{\text{Delta Table} = \text{Parquet Data Files} + \text{Transaction Log}}
$$

The transaction log is stored inside:

_delta_log/

Example:

orders/



---

18. Delta Lake Transaction Log

The transaction log records changes made to the table.

It can contain operations such as:

ADD file

REMOVE file

UPDATE metadata

CHANGE schema

The transaction log allows Spark to determine which Parquet files currently belong to the table.

---

19. Delta Write Protocol

A simplified Delta write works like this:

1. Write new Parquet files

   |

   v

2. Verify successful write

   |

   v

3. Commit transaction log

   |

   v

4. New table version becomes visible

If the job fails before the transaction is committed, the incomplete files are not considered part of the valid table state.

This is a key mechanism behind Delta Lake's transactional guarantees.

---

20. Delta Read Protocol

When Spark reads a Delta table:

Delta Table

  |

v

_delta_log

|

v

Determine valid files

|

v

Read relevant Parquet files

|

v

Return current table state

The transaction log tells the reader which files are active.

---

21. How UPDATE Works in Delta Lake

Delta Lake does not directly modify a row inside an existing Parquet file.

Suppose:

part-001.parquet

contains a row that needs modification.

Conceptually:

Existing file

part-001.parquet

|

| Read + modify

v

New file

part-005.parquet

The transaction log then records:

REMOVE part-001.parquet

ADD part-005.parquet

The old file is no longer part of the current table snapshot.

---

22. How DELETE Works

Similarly, a delete can involve rewriting the relevant data into a new file.

part-001.parquet

|

v

Remove deleted rows

|

v

part-006.parquet

Transaction log:

REMOVE part-001.parquet

ADD part-006.parquet

---

23. `COPY INTO`

`COPY INTO` provides an idempotent mechanism for loading external files into Delta tables.

Example:

```
COPY INTO retaildb.ordersdelta
FROM '/mnt/retaildb/raw_incoming/'
FILEFORMAT = CSV
FORMAT_OPTIONS (
   'header' = 'true',
   'inferSchema' = 'true'
);
```

It is useful when files arrive incrementally in an external location.

---

24. Schema Enforcement

Schema enforcement prevents incompatible data from being written into a Delta table.

For example, if the table expects:

customer_id : INT

order_status : STRING

and incoming data has incompatible structure, the write can be rejected.

This helps prevent accidental schema corruption.

---

25. Schema Evolution

Schema evolution allows the target schema to expand when new columns appear.

Example:

```
df_new_cols.write \
    .mode("append") \
    .format("delta") \
    .option("mergeSchema", "true") \
    .save("/mnt/retaildb/delta/orders.delta")
```

Difference

Schema Enforcement

↓

Reject incompatible schema

Schema Evolution

↓

Allow compatible schema changes

---

26. Delta Table Constraints

Delta tables can enforce data quality rules.

NOT NULL

```
ALTER TABLE retaildb.ordersdelta

ALTER COLUMN customer_id SET NOT NULL;
```

CHECK Constraint

```
ALTER TABLE retaildb.ordersdelta

ADD CONSTRAINT status_ck

CHECK (

   order_status IN

   ('CLOSED',

    'COMPLETE',

    'PROCESSING',

    'PENDING',

    'ON_HOLD',

    'CANCELED')

);
```

---

27. Delta Lake Time Travel

Delta Lake maintains table history through transaction versions.

View History

```
DESCRIBE HISTORY retaildb.ordersdelta;
```

Query an Older Version

```
SELECT *
FROM retaildb.ordersdelta
VERSION AS OF 2;
```

Query by Timestamp

```
SELECT *
FROM retaildb.ordersdelta
TIMESTAMP AS OF '2023-09-23T14:01:01.000+0000';
```

This makes historical analysis and auditing possible.

---

28. Restore a Delta Table

If an accidental modification occurs, a previous version can be restored.

```
RESTORE TABLE retaildb.ordersdelta
TO VERSION AS OF 0;
```

Conceptually:

```
Version 0
  |
Version 1
  |
```

Version 2

  |

Version 3 ← Current

  |

  | Restore

  v

Version 0 ← Current state

---

29. Week 19 — Key Interview Concepts

Focus particularly on these:

Azure Databricks

- Control Plane vs Data Plane
- DBU
- All-Purpose vs Job Cluster
- Driver vs Worker
- Single Node vs Standard Cluster
- Databricks Magic Commands
- `dbutils`
- Notebook orchestration
- Widgets

Azure Storage Security

- Access Key
- SAS
- Service Principal
- OAuth
- Secret Scopes
- Azure Key Vault

Delta Lake

- Parquet + `_delta_log`
- ACID transactions
- Transaction log
- UPDATE/DELETE internals
- Schema enforcement
- Schema evolution
- `COPY INTO`
- Constraints
- Time Travel
- `DESCRIBE HISTORY`
- `VERSION AS OF`
- `TIMESTAMP AS OF`
- `RESTORE TABLE`

Week 18 vs Week 19 — Quick Separation

Week	Main Focus
Week 18	Azure Storage, Blob Storage, ADLS Gen2, access tiers, redundancy, HNS, ACLs
Week 19	Azure Databricks, clusters, `dbutils`, authentication, secrets, Delta Lake, ACID, schema evolution, time travel

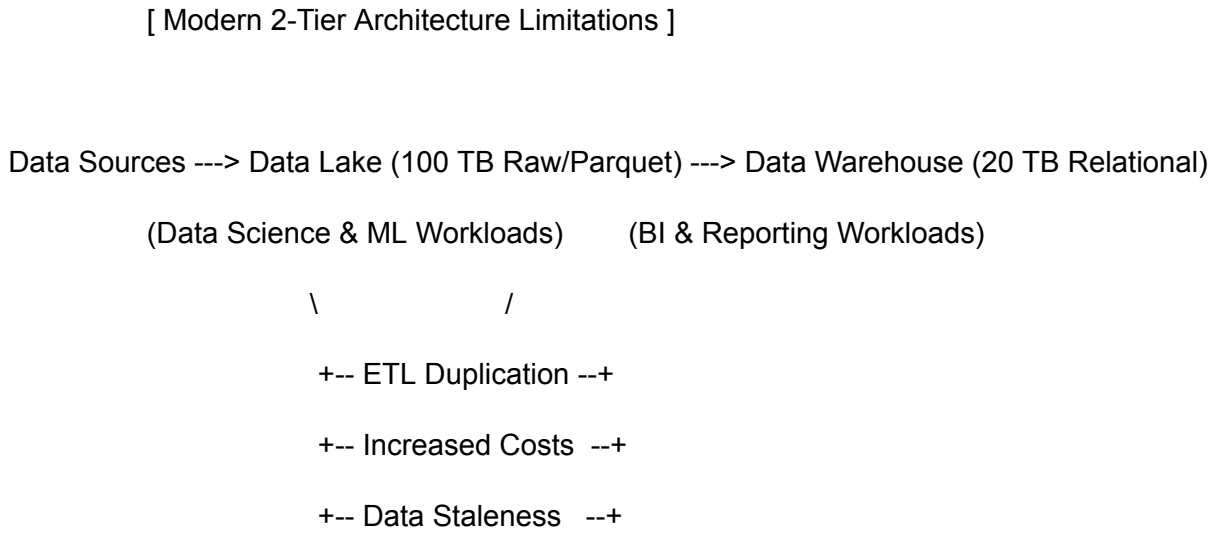
Week 20: Databricks Lakehouse Architecture, Delta Engine & Medallion Architecture

---

1. Databricks Lakehouse Architecture Overview

The **Lakehouse Architecture** combines the scalability and cost efficiency of Data Lakes with the reliability, ACID transactions, and performance features traditionally associated with Data Warehouses.

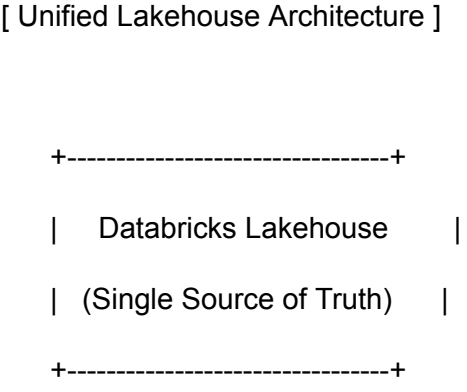
Traditional 2-Tier Architecture

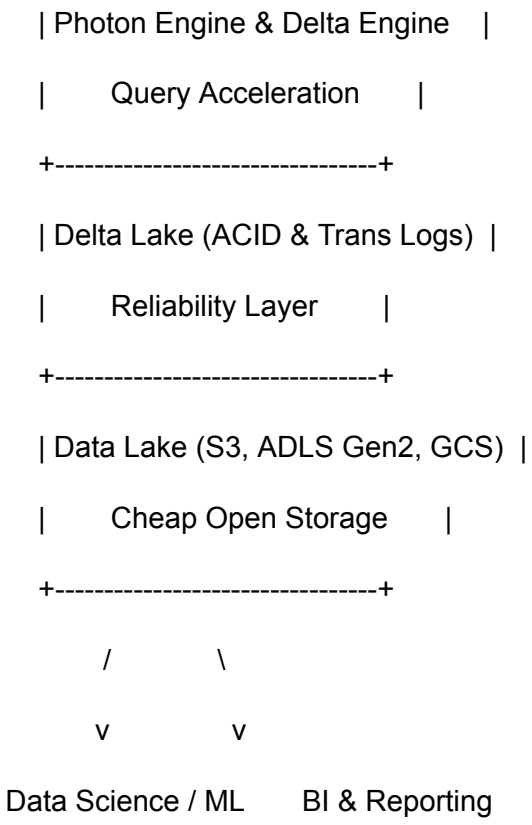


The traditional architecture often requires maintaining both a Data Lake and a Data Warehouse, resulting in:

- Duplicate data storage
- Additional ETL pipelines
- Increased infrastructure and operational costs
- Data synchronization challenges
- Potential data staleness

Unified Lakehouse Architecture





The Lakehouse combines:

1. **Data Lake Storage** — S3, ADLS Gen2, or GCS for scalable and inexpensive storage.
2. **Delta Lake** — Provides ACID transactions, schema enforcement, and transaction history.
3. **Delta/Photon Engine** — Provides optimized query execution and performance improvements.
4. **Unified Access** — Supports Data Science, Machine Learning, Streaming, BI, and reporting workloads.

Key Architectural Advantages

1. Eliminates Data Duplication

A Lakehouse reduces the need to maintain separate storage and reporting copies of the same data.

2. Unified Workloads

The same data platform can support:

- Machine Learning
- Data Science
- Streaming
- Business Intelligence
- Reporting
- Analytical workloads

3. Open Data Formats

Lakehouse storage continues to use open formats such as **Parquet**, while Delta Lake adds transaction-management capabilities through the `_delta_log`.

Key Concept

Data Lake

+

Delta Lake

+

Query Optimization

=

Lakehouse

---

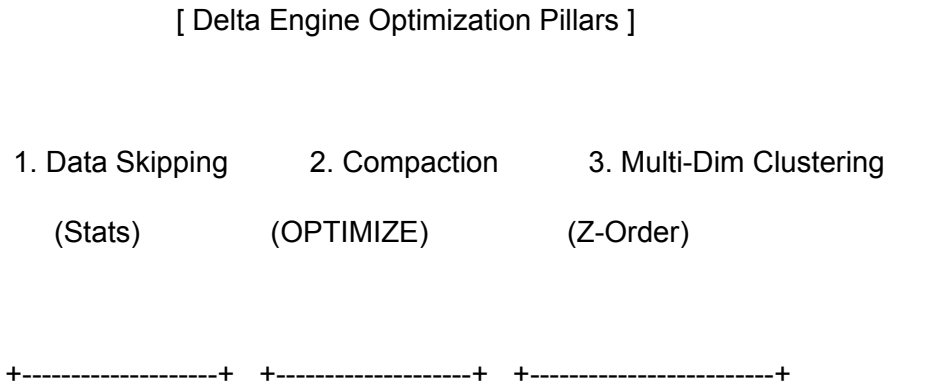
2. Delta Engine Performance & Optimization Techniques

The **Delta Engine** is a high-performance execution layer designed to improve the performance of reads, writes, joins, and aggregations over Delta Lake data.

Major optimization techniques include:

1. Data Skipping
2. File Compaction
3. Z-Ordering
4. Delta Cache

5. Auto-Optimization
6. Photon Execution Engine



2.1 Data Skipping via Statistics

Delta Lake maintains file-level statistics such as:

- Minimum values
- Maximum values
- Other file-level metadata

These statistics are recorded in the Delta transaction log.

Example

Suppose a query executes:

```
SELECT *
FROM orders
WHERE total_amt = 234;
```

Delta can inspect the file statistics before reading the actual Parquet files.

If a file contains:

File A → min = 1, max = 100

File B → min = 101, max = 200

File C → min = 201, max = 300

Then the query can skip Files A and B because `234` cannot exist in those files.

Query: total_amt = 234

File A: 1 ———————— 100 SKIP

File B: 101 ———————— 200 SKIP

File C: 201 ———————— 300 READ

Benefit

Data skipping reduces unnecessary physical file scans and improves query performance.

Delta vs. Standard Parquet

The source material highlights that Delta maintains centralized transaction-log metadata, allowing the query engine to use statistics for file skipping before reading the underlying data files.

2.2 Delta Cache vs. Spark Cache

Delta Cache and Spark Cache operate at different levels.

Delta Cache

Delta Cache stores remote Data Lake files on the local SSD storage of worker nodes in an optimized intermediate format.

Characteristics:

- Works with Parquet/Delta formats.
- Uses local SSD storage.
- Reduces repeated remote storage reads.
- Automatically handles stale cached blocks when source files change or clusters restart.

Example configuration:

```
spark.conf.set(

   "spark.databricks.io.cache.enabled",

   "true"

)
```

Spark Cache

Spark `.cache()` stores DataFrame/RDD representations in memory, using JVM heap or off-heap memory depending on the configuration.

Comparison

Feature	Delta Cache	Spark Cache
Storage	Local SSD	Memory / JVM heap / off-heap
Primary Purpose	Cache remote data files	Cache processed DataFrame/RDD data
Format	Parquet / Delta	DataFrame / RDD representation

Managed By	Databricks I/O caching	Spark caching mechanism
Main Benefit	Reduces remote storage reads	Avoids repeated computation

---

2.3 File Compaction — Bin Packing & `OPTIMIZE`

Continuous operations such as:

- `INSERT`
- `UPDATE`
- `DELETE`
- Streaming writes

can generate many small files.

A large number of small files causes:

- More file-open operations
- Increased metadata overhead
- More task scheduling overhead
- Slower query execution

Solution: Bin Packing

The `OPTIMIZE` command combines smaller files into larger, query-friendly files.

OPTIMIZE taxidb.trips_delta;

Conceptually:

Before OPTIMIZE

File 1 → 10 MB

File 2 → 15 MB

File 3 → 20 MB

File 4 → 12 MB

File 5 → 18 MB

↓

OPTIMIZE

↓

File A → Large consolidated file

File B → Large consolidated file

The source material describes optimized physical file blocks as being up to approximately **1 GB**.

Important

`OPTIMIZE` performs file compaction, but by itself it does **not** provide multidimensional clustering.

For multidimensional data organization, use **Z-Ordering**.

2.4 Z-Ordering — Multidimensional Clustering

Z-Ordering is a data co-location technique that organizes related rows into the same physical file groups.

It is particularly useful for columns frequently used in:

- `WHERE` filters
- Joins
- `GROUP BY` operations

Example

Suppose a table contains employee IDs.

Without Z-Ordering:

File 1 → emp_id: 1, 5, 8

File 2 → emp_id: 2, 4, 9

File 3 → emp_id: 3, 6, 10

Query:

WHERE emp_id = 4

Potentially all three files need to be examined.

With Z-Ordering:

File 1 → emp_id: 1, 2, 3

File 2 → emp_id: 4, 5, 6

File 3 → emp_id: 8, 9, 10

The query can narrow the search to File 2.

Z-Order Example

```
OPTIMIZE taxidb.trips_delta_new

ZORDER BY (passenger_count);
```

Key Difference

OPTIMIZE

↓

Compacts small files

ZORDER

↓

Improves data co-location and data skipping

Z-Ordering is therefore typically used together with `OPTIMIZE`.

2.5 Vacuuming Old Files — `VACUUM`

Delta Lake maintains historical table versions to support:

- Time Travel
- Snapshot isolation
- Historical queries

As a result, old Parquet files may remain on storage even after they are no longer part of the current active table state.

`VACUUM`

The `VACUUM` command permanently removes physical data files that:

1. Are no longer referenced by the active Delta transaction log.
2. Are older than the configured retention period.

The source material specifies the default retention period as **7 days**.

Current Delta Table

|

+---- Active files → KEEP

|

+---- Old unreferenced files → VACUUM

Important Impact

Once historical files are permanently removed using `VACUUM`, Time Travel may no longer be able to access table versions that depend on those files.

Dry Run

VACUUM taxidb.trips_delta

RETAIN 1 HOURS

DRY RUN;

Permanent Deletion

VACUUM taxidb.trips_delta

RETAIN 1 HOURS;

Retention Safety Check

For retention below 7 days:

SET spark.databricks.delta.retentionDurationCheck.enabled = false;

Important Warning

Reducing retention can remove files required for historical reads. Therefore, short retention periods should be used carefully.

2.6 Auto-Optimization Features

Delta Lake can automatically optimize file sizes during and after writes.

There are two major capabilities:

1. Optimized Writes

Optimized Writes coalesce smaller partitions before writing data to storage.

The goal is to create larger and more efficient data files.

2. Auto Compaction

Auto Compaction performs lightweight file consolidation after writes when a partition accumulates a significant number of small files.

Enabling Auto-Optimization

```
ALTER TABLE taxidb.trips_delta

SET TBLPROPERTIES (

   'delta.autoOptimize.optimizeWrite' = 'true',

   'delta.autoOptimize.autoCompact' = 'true'

);
```

Conceptual Flow

Small Input Partitions

|

v

Optimized Writes

|

v

Better-Sized Files

|

v

Auto Compaction

|

v

Reduced Small-File Problem

2.7 Photon Execution Engine

Photon is a native vectorized execution engine implemented in C++ and designed to accelerate Spark SQL and DataFrame workloads in Databricks.

It is particularly useful for:

- Large joins
- Complex aggregations
- Compute-intensive SQL
- Large-scale analytical workloads

Compatibility

Photon works with:

- Parquet
- Delta Lake

Key Concept

Spark SQL / DataFrame Workload

|

v

Photon Engine

|

v

Vectorized Native Execution

|

v

Improved Query Performance

Photon can be enabled through the **Use Photon Acceleration** option when configuring appropriate Databricks compute.

---

3. Delta Architecture — Medallion Design Pattern

The **Medallion Architecture** organizes Lakehouse data into progressively refined layers.

The three primary layers are:

BRONZE → SILVER → GOLD

Each layer improves data quality and usability.

[Medallion Architecture Workflow]

Raw Sources BRONZE SILVER GOLD

+----------+ +-------------+ +-------------+ +---------------+

| Streaming | | Raw | | Cleaned & | | Aggregated |

| Batch CSV |--->| Ingestion |--->| Validated |--->| Business |

| RDBMS DBs | | Append-Only | | Delta Tables | | Data / BI |

+----------+ +----------+ +----------+ +----------+

3.1 Bronze Layer

The Bronze layer contains raw ingested data.

Characteristics:

- Raw data
- Minimal transformation
- Often append-only
- Preserves source information
- Can contain ingestion metadata

Examples of metadata:

filename

createdon

ingestion timestamp

Purpose

The Bronze layer acts as a raw audit archive and provides a source-of-truth representation of ingested data.

3.2 Silver Layer

The Silver layer contains cleaned and structured data.

Typical operations include:

- Data validation
- Filtering invalid records
- Deduplication
- Type conversion
- Data cleansing
- Schema standardization
- Business-rule application

Purpose

Silver data is suitable for:

- Machine Learning
- Data Science
- Operational analytics
- Further business transformations

3.3 Gold Layer

The Gold layer contains business-ready data.

Typical operations include:

- Aggregations
- Business calculations
- Domain-specific transformations
- Reporting-oriented structures

Purpose

Gold data is primarily used for:

- BI dashboards
- Executive reporting
- Business analytics
- Analytical applications

Medallion Layer Comparison

Layer	Data State	Main Purpose	Typical Consumers
Bronze	Raw / minimally processed	Source archive & audit	Engineering
Silver	Cleaned / validated / structured	Analytics & ML preparation	Data Scientists / Engineers
Gold	Aggregated / business-ready	Reporting & BI	Business Users / Analysts

Easy Way to Remember

BRONZE = Raw

SILVER = Refined

GOLD = Business Ready

---

4. Change Data Feed (CDF) Mechanics

Change Data Feed (CDF) tracks row-level changes occurring between versions of a Delta table.

It can capture:

- `INSERT`
- `UPDATE`
- `DELETE`

CDF provides metadata describing the type and version of each change.

[Change Data Feed Metadata]

```
+------------+--------------------+------------------+---------------------+
| Operation  | _change_type       | _commit_version  | _commit_timestamp   |
+------------+--------------------+------------------+---------------------+
| INSERT     | insert             | 1                | 2026-07-25...       |
| UPDATE     | update_preimage    | 2                | 2026-07-25...       |
| UPDATE     | update_postimage   | 2                | 2026-07-25...       |
| DELETE     | delete             | 3                | 2026-07-25...       |
+------------+--------------------+------------------+---------------------+
```

CDF Change Types

Change	`_change_type`
Insert	`insert`
Update — old value	`update_preimage`
Update — new value	`update_postimage`
Delete	`delete`

4.1 Enabling CDF During Table Creation

```
CREATE TABLE retaildb.orders (
```

```
    order_id INT,
    order_date STRING,
    customer_id INT,
    order_status STRING
)
USING DELTA
TBLPROPERTIES (
    delta.enableChangeDataFeed = true
);
```

4.2 Enabling CDF on an Existing Table

```
ALTER TABLE retaildb.orders
SET TBLPROPERTIES (
    delta.enableChangeDataFeed = true
);
```

4.3 Enabling CDF as a Session Default

```
SET spark.databricks.delta.properties.defaults.enableChangeDataFeed = true;
```

---

5. Querying Change Data Feed

The `table_changes()` function can retrieve changes beginning from a specified Delta table version.

From a Specific Version

SELECT *

FROM table_changes('retaildb.orders', 1);

This reads changes beginning from version `1` through the latest available version.

Between Two Versions

SELECT *

FROM table_changes('retaildb.orders', 1, 3);

This retrieves changes across the specified inclusive version range.

CDF Storage

The source material specifies that non-insert DML operations such as:

- `UPDATE`
- `DELETE`
- `MERGE`

can record change events in the `_change_data` directory within the Delta table path.

---

6. End-to-End Medallion Pipeline Implementation

The following example demonstrates an enterprise-style Medallion pipeline using:

- `COPY INTO`
- Change Data Feed

- `table_changes()`
- `MERGE INTO`
- Delta tables
- Generated columns
- Aggregations

Step 1: Create Database

CREATE DATABASE IF NOT EXISTS retaildb;

Step 2: Create Bronze Layer

The Bronze layer stores raw order data along with ingestion metadata.

```
CREATE TABLE retaildb.orders_bronze (
    order_id INT,
    order_date STRING,
    customer_id INT,
    order_status STRING,
    filename STRING,
    createdon TIMESTAMP
)
USING DELTA
LOCATION "dbfs:/FileStore/data/orders_bronze.delta"
PARTITIONED BY (order_status)
TBLPROPERTIES (
    delta.enableChangeDataFeed = true
```

);

Step 3: Ingest Data Using `COPY INTO`

`COPY INTO` is used for idempotent ingestion of new external files.

```
COPY INTO retaildb.orders_bronze
FROM (
   SELECT
      order_id::INT,
      order_date::STRING,
      customer_id::INT,
      order_status::STRING,
      INPUT_FILE_NAME() AS filename,
      CURRENT_TIMESTAMP() AS createdon
   FROM 'dbfs:/FileStore/raw'
)
FILEFORMAT = CSV
FORMAT_OPTIONS ('header' = 'true');
```

Step 4: Create Silver Layer

The Silver table contains cleaned and structured data.

```
CREATE TABLE retaildb.orders_silver (
```

```
    order_id INT,
    order_date DATE,
    customer_id INT,
    order_status STRING,
    order_year INT GENERATED ALWAYS AS (YEAR(order_date)),
    order_month INT GENERATED ALWAYS AS (MONTH(order_date)),
    createdon TIMESTAMP,
    modifiedon TIMESTAMP
)
USING DELTA
LOCATION "dbfs:/FileStore/data/orders_silver.delta"
PARTITIONED BY (order_status)
TBLPROPERTIES (
    delta.enableChangeDataFeed = true
);
```

Step 5: Read Bronze Changes and Merge into Silver

Create a temporary view over the Bronze Change Data Feed:

```
CREATE OR REPLACE TEMPORARY VIEW orders_bronze_changes AS
SELECT *
FROM table_changes('retaildb.orders_bronze', 1)
WHERE order_id > 0
  AND customer_id > 0
```

```
  AND order_status IN (
      'PAYMENT_REVIEW',
      'PROCESSING',
      'CLOSED',
      'SUSPECTED_FRAUD',
      'COMPLETE',
      'PENDING',
      'CANCELLED',
      'PENDING_PAYMENT'
  );
```

Then merge the changes into Silver:

```
MERGE INTO retaildb.orders_silver tgt
USING orders_bronze_changes src
   ON tgt.order_id = src.order_id

WHEN MATCHED THEN
  UPDATE SET
    tgt.order_status = src.order_status,
    tgt.customer_id = src.customer_id,
    tgt.modifiedon = CURRENT_TIMESTAMP()

WHEN NOT MATCHED THEN
  INSERT (
```

```
    order_id,
    order_date,
    customer_id,
    order_status,
    createdon,
    modifiedon
  )
  VALUES (
    src.order_id,
    TO_DATE(src.order_date),
    src.customer_id,
    src.order_status,
    CURRENT_TIMESTAMP(),
    CURRENT_TIMESTAMP()
  );
```

Logical Flow

```
Bronze CDF
   |
   v
Filter Valid Records
   |
   v
table_changes()
```

```
    |
    v
MERGE INTO
    |
    +------ MATCHED ------> UPDATE Silver
    |
    +--- NOT MATCHED -----> INSERT Silver
```

---

7. Create the Gold Layer

The Gold layer contains aggregated business-level information.

```
CREATE TABLE retaildb.orders_gold (
    customer_id INT,
    order_status STRING,
    order_year INT,
    num_orders INT
)
USING DELTA
LOCATION "dbfs:/FileStore/data/orders_gold.delta"
TBLPROPERTIES (
    delta.enableChangeDataFeed = true
);
```

Step 8: Populate Gold Layer

The Silver layer is aggregated to create the business-ready Gold dataset.

```
INSERT OVERWRITE TABLE retaildb.orders_gold
SELECT
    customer_id,
    order_status,
    order_year,
    COUNT(order_id) AS num_orders
FROM retaildb.orders_silver
GROUP BY
    customer_id,
    order_status,
    order_year;
```

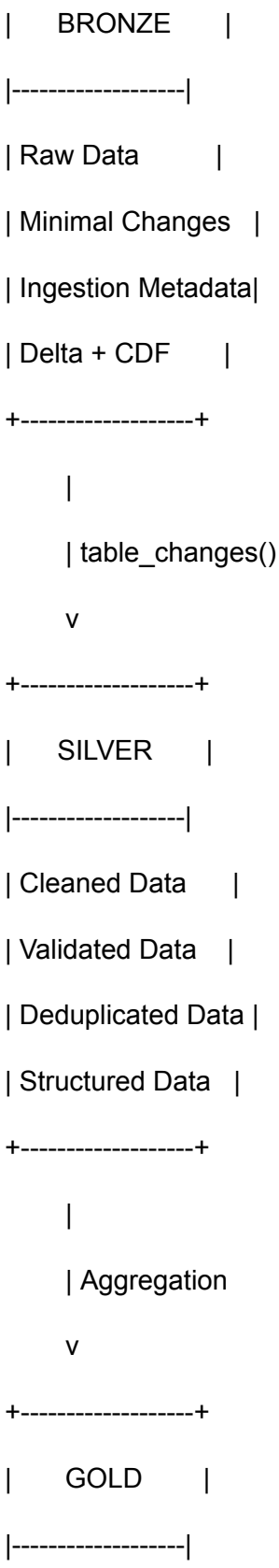
---

8. Complete Medallion Architecture Flow

The entire pipeline can be summarized as:

```
                END-TO-END LAKEHOUSE PIPELINE


 Raw Files / Sources
        |
        v
+------------------+
```


BRONZE
Raw Data
Minimal Changes
Ingestion Metadata
Delta + CDF
table_changes()
SILVER
Cleaned Data
Validated Data
Deduplicated Data
Structured Data
Aggregation
GOLD

| Business Metrics |

| Aggregations |

| BI / Reporting |

+------------------+

|

v

BI / Analytics / Business Applications

---

9. Week 20 — Interview Revision Points

Lakehouse

Q: What is a Lakehouse?

A Lakehouse combines the scalability and low-cost storage of a Data Lake with the reliability and analytical capabilities traditionally associated with Data Warehouses.

Delta Lake

Q: Why is Delta Lake important?

Delta Lake adds ACID transactions, schema management, DML capabilities, transaction history, and Time Travel to Data Lake storage.

Data Skipping

Q: What is Data Skipping?

Data Skipping uses file-level statistics such as minimum and maximum values to avoid scanning files that cannot satisfy a query predicate.

OPTIMIZE

Q: Why do we use `OPTIMIZE`?

`OPTIMIZE` compacts small files into larger files, reducing file-management and query overhead.

Z-Ordering

Q: What is Z-Ordering?

Z-Ordering co-locates related data based on selected columns to improve data skipping for frequently filtered or queried columns.

VACUUM

Q: Why is `VACUUM` used?

`VACUUM` permanently removes old, unreferenced physical files that are beyond the configured retention period.

Photon

Q: What is Photon?

Photon is a native vectorized execution engine designed to accelerate Spark SQL and DataFrame workloads.

Medallion Architecture

Q: Explain Bronze, Silver, and Gold.

BRONZE → Raw ingestion

SILVER → Cleaned and validated data

GOLD → Business-ready aggregated data

Change Data Feed

Q: What does CDF provide?

CDF provides row-level change information for operations such as:

INSERT

UPDATE

DELETE

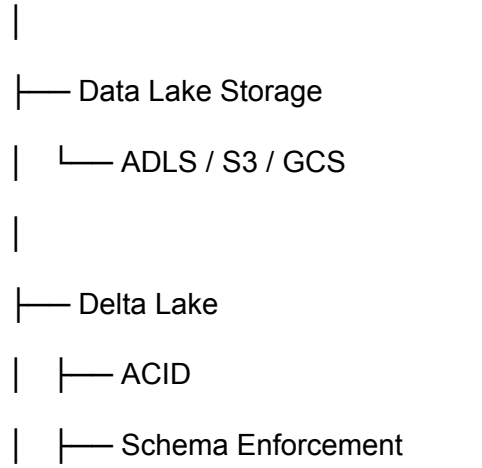
It can be queried using:

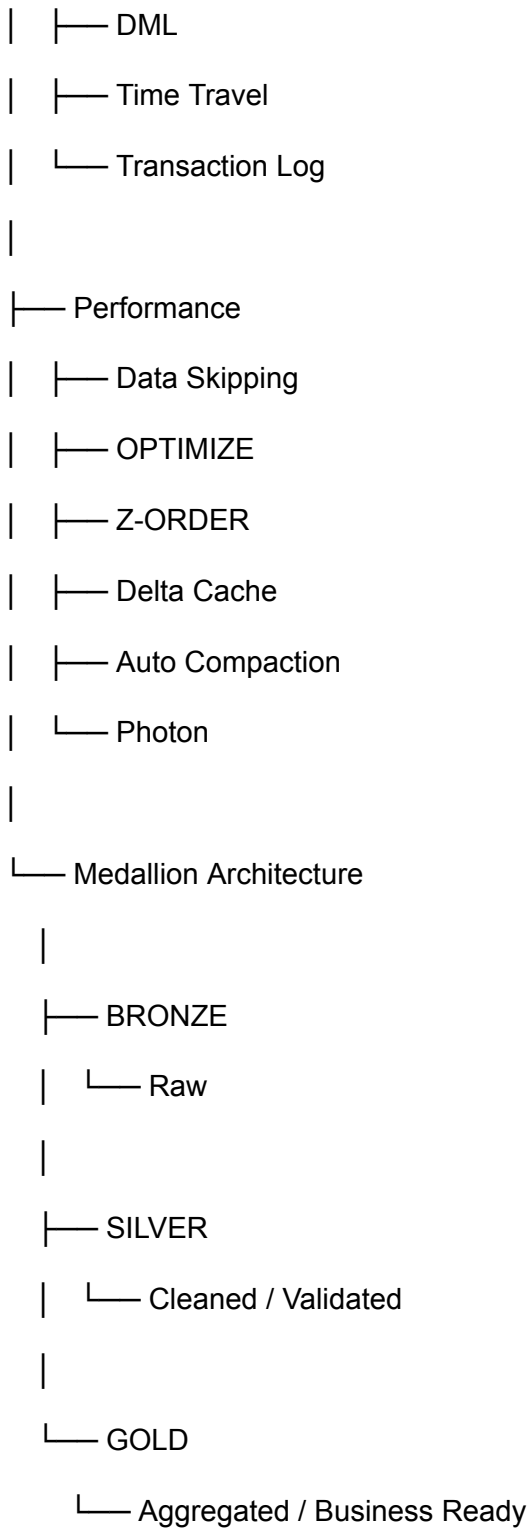
table_changes()

---

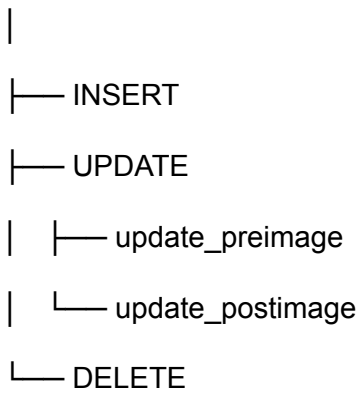
10. Week 20 — Quick Revision Sheet

LAKEHOUSE

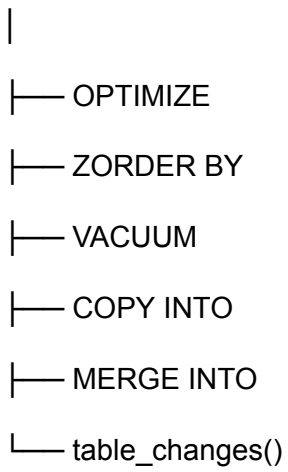



DML
Time Travel
Transaction Log
Performance
Data Skipping
OPTIMIZE
Z-ORDER
Delta Cache
Auto Compaction
Photon
Medallion Architecture
BRONZE
Raw
SILVER
Cleaned / Validated
GOLD
Aggregated / Business Ready

CDF



Core Commands



Week 20 Key Takeaways

1. **Lakehouse** unifies Data Lake and Data Warehouse capabilities.
2. **Delta Lake** provides reliability and transactional capabilities on Data Lake storage.
3. **Data Skipping** avoids unnecessary file scans using file-level statistics.
4. **OPTIMIZE** addresses small-file problems through compaction.
5. **Z-Ordering** improves data locality and data skipping.
6. **VACUUM** removes old unreferenced physical files.
7. **Delta Cache** caches remote data on local SSD storage.
8. **Photon** accelerates compute-intensive SQL and DataFrame workloads.
9. **Bronze** stores raw data.
10. **Silver** stores cleaned and validated data.
11. **Gold** stores business-ready aggregated data.
12. **CDF** tracks row-level changes between Delta table versions.

13. `COPY INTO` supports incremental/idempotent file ingestion.
14. `MERGE INTO` is used to synchronize changes between datasets.
15. A common enterprise pattern is:

Source

↓

Bronze

↓

CDF

↓

Silver

↓

Aggregation

↓

Gold

↓

BI / Analytics

Week 21: Databricks Unity Catalog & Enterprise Data Governance

1. Introduction to Unity Catalog

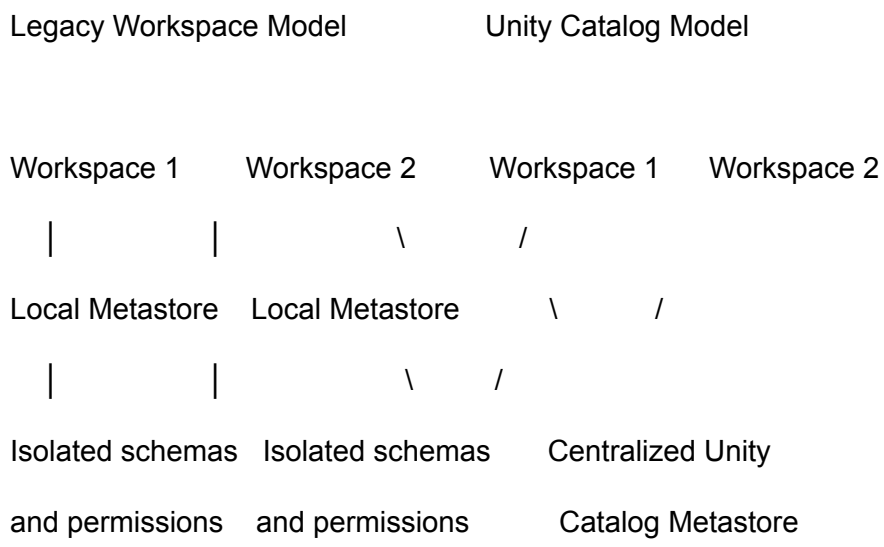
Unity Catalog is a unified, fine-grained data governance solution for data assets such as tables, views, volumes, and files within the Databricks Lakehouse Platform.

Workspace-Level Governance vs. Unity Catalog

WORKSPACE-LEVEL GOVERNANCE

vs.

CENTRALIZED UNITY CATALOG



Limitations of Workspace-Level Governance

Without Unity Catalog:

- Metastores are isolated within individual Databricks workspaces.
- Access control policies are workspace-specific.
- User directories are siloed.
- Tables and views registered in one workspace may not be directly available in another.
- Organizations may need duplicate data registration and fragmented security policies.

Key Pillars of Unity Catalog

1. **Centralized Metadata Management**
   A single metastore can be associated with multiple Databricks workspaces.
2. **Standard ANSI SQL Security Model**
   Access to database objects can be controlled using SQL commands such as:

GRANT SELECT ON TABLE ...

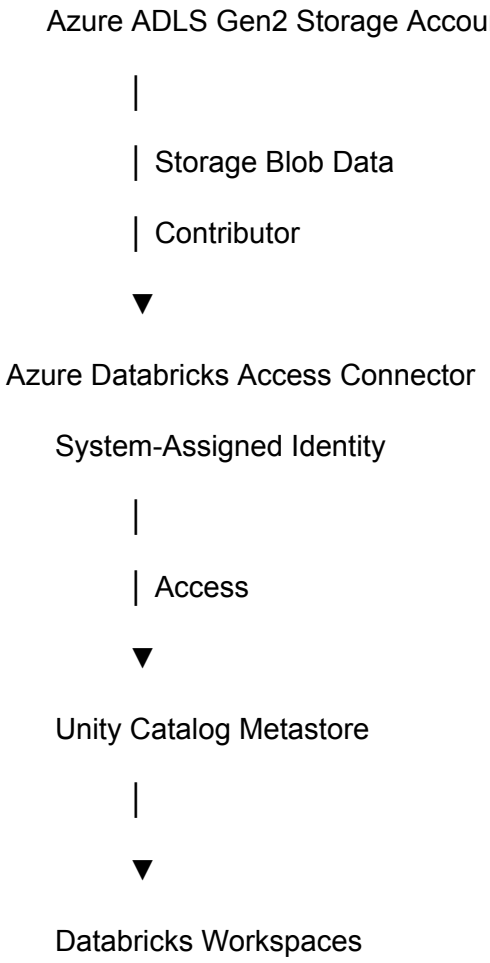
3. **Fine-Grained Auditing**
   User-level query activity, table access, and administrative actions can be audited.

4. **Data Lineage**
   Tracks table- and column-level relationships to understand how data moves through pipelines.
5. **Data Discovery**
   Users can search for datasets and other data assets using workspace discovery tools.
6. **Delta Sharing**
   Provides a mechanism for securely sharing datasets with external parties without copying the underlying data.

---

2. Setting Up and Configuring a Unity Catalog Metastore

Unity Catalog on Azure requires integration between **Azure Data Lake Storage Gen2**, an **Azure Databricks Access Connector**, and the Unity Catalog metastore.



Step 1: Create ADLS Gen2 Storage

Create an Azure Storage Account with:

- **Hierarchical Namespace enabled**
- A container for Unity Catalog storage

Example:

Storage Account:

ttunitycatalogsa

Container:

unityroot

Step 2: Create an Access Connector

Create an **Access Connector for Azure Databricks** in Azure.

The Access Connector uses a **system-assigned managed identity**, eliminating the need to manually manage storage access keys or Key Vault secrets for this connection.

Step 3: Assign IAM Permissions

On the ADLS Gen2 storage account:

Access Control (IAM) → Add Role Assignment

Assign:

Storage Blob Data Contributor

to the managed identity of the Access Connector.

Step 4: Create the Unity Catalog Metastore

From the **Databricks Account Console**, an Account Admin can create the metastore.

Configure:

- Metastore name
- Region
- ADLS Gen2 root storage path
- Azure Databricks Access Connector resource

Example storage path:

abfss://unityroot@ttunitycatalogsa.dfs.core.windows.net/

Step 5: Attach the Metastore

Finally, attach the newly created Unity Catalog metastore to the required Databricks workspaces.

---

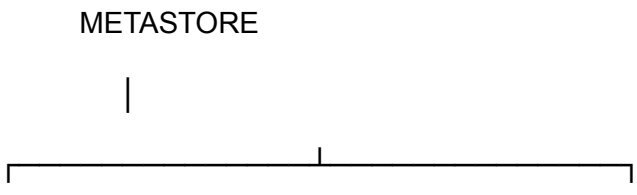
3. Unity Catalog Three-Level Namespace

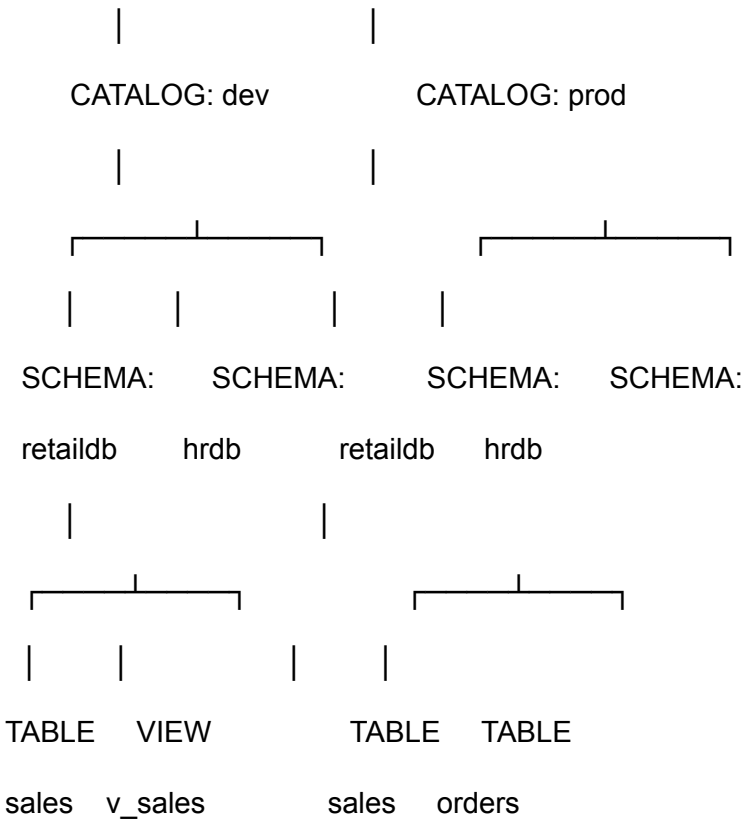
Unity Catalog uses a three-level namespace:

catalog.schema.table

The same structure applies to views and other supported objects.

Hierarchy





Namespace Components

1. Catalog

The top-level logical grouping.

Catalogs can be organized according to:

- Environment
  - `dev`
  - `stage`
  - `prod`
- Business domain
  - `finance`
  - `marketing`
  - `retail`

2. Schema

The second level, traditionally equivalent to a database.

Schemas contain:

- Tables
- Views
- Volumes

3. Tables / Views / Volumes

The lowest level contains the actual data assets or logical abstractions over data.

---

4. Managed Tables vs. External Tables

Feature	Managed Tables	External Tables
Data Location	Unity Catalog-managed storage location	Custom external cloud location
Typical Format	Delta	Delta, Parquet, CSV, ORC
Data Ownership	Managed by Unity Catalog	Data remains in external storage
DROP TABLE	Removes metadata and managed data according to Unity Catalog lifecycle behavior	Removes metadata; underlying files remain
Use Case	Databricks-managed data	Data shared with external systems

Example external location:

abfss://data@storageaccount.dfs.core.windows.net/

When to Use Managed Tables

Useful when:

- Databricks manages the complete data lifecycle.
- Data is primarily consumed through Databricks.
- Centralized governance is desired.

When to Use External Tables

Useful when:

- Data already exists in cloud storage.
- Multiple systems need to access the same files.
- External applications or engines also consume the data.

---

5. Unity Catalog Table Access Permissions

To access:

prod.retaildb.sales

the user needs permissions at each level of the hierarchy.

Catalog



Schema



Table



SELECT

Data

Required privileges:

USAGE on Catalog

USAGE on Schema

SELECT on Table

For example:

```
GRANT USAGE ON CATALOG prod
TO `developer-grp`;

GRANT USAGE ON SCHEMA prod.retaildb
TO `developer-grp`;

GRANT SELECT ON TABLE prod.retaildb.sales
TO `developer-grp`;
```

---

6. Identity Management with SCIM

Enterprise organizations can synchronize users and groups from **Microsoft Entra ID** into Databricks using **SCIM — System for Cross-domain Identity Management**.



Benefits

SCIM enables:

- Centralized user provisioning
- Automatic group synchronization
- Centralized identity management
- Easier onboarding and offboarding
- Reduced manual user administration

SCIM Setup Workflow

Step 1: Generate SCIM Credentials

In the Databricks Account Console:

Settings

↓

User Provisioning

↓

Generate SCIM Token

Obtain:

- SCIM URL
- SCIM Token

Step 2: Configure Entra ID

In the Azure Portal:

Microsoft Entra ID

↓

Enterprise Applications

↓

Azure Databricks SCIM Provisioning Connector

Set:

Provisioning Mode = Automatic

Configure:

- Tenant URL → SCIM URL
- Secret Token → SCIM Token

Then test the connection.

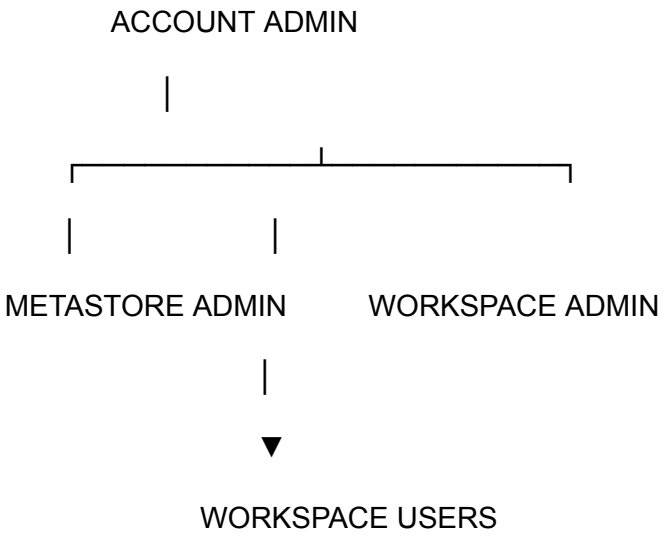
Step 3: Synchronize Groups

Assign Entra ID groups to the provisioning connector so that users and groups are automatically synchronized with Databricks.

---

7. Databricks Administrative Roles

Unity Catalog and Databricks environments can be managed through different administrative personas.



Account Admin

Highest account-level administrative role.

Responsibilities include:

- Managing Databricks workspaces
- Creating Unity Catalog metastores
- Attaching workspaces to metastores
- Managing account-level configurations
- Managing identity provisioning

Metastore Admin

Responsible for governance of Unity Catalog objects.

Responsibilities include:

- Creating catalogs
- Managing external locations
- Managing object ownership
- Granting permissions
- Governing metastore-level resources

Best practice is to assign this responsibility to a dedicated group such as:

metastore-admins

Workspace Admin

Responsible for administration within a particular Databricks workspace.

Responsibilities include:

- Cluster policies
- Workspace access
- Workspace settings
- Notebook policies
- Workspace-level configurations

Workspace User

Regular developer or analyst who consumes the platform.

Typical activities:

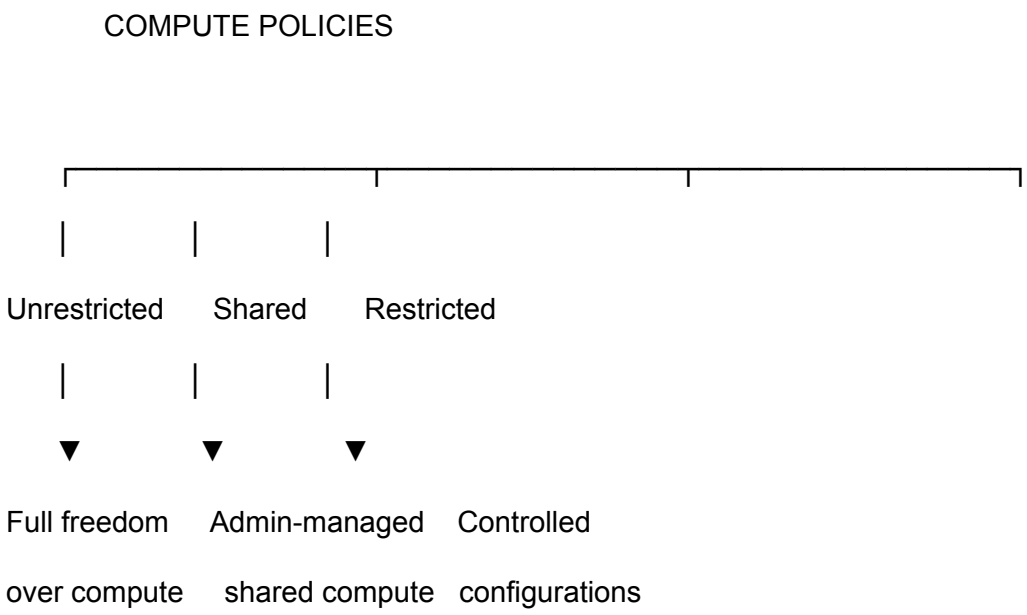
- Creating notebooks
- Writing Spark code
- Running queries
- Executing pipelines
- Attaching notebooks to compute

---

8. Compute Execution & Cluster Policies

Workspace administrators can control how users create and consume compute resources.

There are three major policy approaches:



1. Unrestricted Policy

Developers can create clusters with broad configuration flexibility.

Advantages:

- Maximum flexibility

Disadvantage:

- Higher risk of uncontrolled cloud costs

---

2. Shared Cluster

The Workspace Admin creates shared compute.

Developers attach notebooks to the shared cluster rather than creating their own arbitrary compute.

---

3. Restricted Policy

Developers can create compute but only within predefined limits.

For example:

- Approved VM types
- Allowed Spark versions
- Worker limits
- Auto-termination settings

Example Cluster Policy

```
{
  "node_type_id": {
    "type": "allowlist",
    "values": [
      "Standard_DS3_v2",
      "Standard_DS4_v2"
    ],
    "defaultValue": "Standard_DS4_v2"
  },
  "spark_version": {
    "type": "unlimited",
    "defaultValue": "11.3.x-scala2.12"
  },
  "num_workers": {
    "type": "fixed",
    "value": 0,
```

```
    "hidden": true
  },
  "autotermination_minutes": {
    "type": "unlimited",
    "defaultValue": 60,
    "isOptional": true
  }
}
```

The purpose is to prevent developers from creating unnecessarily expensive or non-compliant infrastructure.

---

9. Unity Catalog Permission Management Using SQL

Unity Catalog permissions can be managed using SQL.

Step 1: Create a Catalog

```
CREATE CATALOG IF NOT EXISTS customer360_dev;
```

Transfer ownership:

```
ALTER CATALOG customer360_dev
OWNER TO `trendytech-metastore-admins`;
```

Step 2: Grant Catalog Access

```
GRANT USAGE

ON CATALOG customer360_dev

TO `trendytech-developer-grp`;
```

Step 3: Create a Schema

```
CREATE SCHEMA IF NOT EXISTS customer360_dev.retaildb;
```

Grant schema permissions:

```
GRANT USAGE, CREATE TABLE, CREATE VIEW

ON SCHEMA customer360_dev.retaildb

TO `trendytech-developer-grp`;
```

Step 4: Grant Table Access

Business analysts can be granted read-only access:

```
GRANT SELECT

ON TABLE customer360_dev.retaildb.sales

TO `business-analysts-grp`;
```

---

10. Week 21 Quick Revision

Unity Catalog

> Centralized governance layer for Databricks data assets.

Three-Level Namespace

catalog.schema.table

Main Governance Capabilities

Metadata

+

Security

+

Auditing

+

Lineage

+

Discovery

+

Data Sharing

Azure Unity Catalog Setup

ADLS Gen2

↓

Access Connector

↓

IAM Role

↓

Unity Catalog Metastore

↓

Databricks Workspace

Identity

Microsoft Entra ID

↓

SCIM

↓

Databricks

Administrative Hierarchy

Account Admin

↓

Metastore Admin

↓

Workspace Admin

↓

Workspace User

Table Access

USAGE → Catalog

USAGE → Schema

SELECT → Table

Most Important Interview Question

Q: Why is Unity Catalog important in an enterprise Databricks environment?

Answer: Unity Catalog provides centralized governance across Databricks workspaces, allowing organizations to manage metadata, access controls, auditing, lineage, discovery, and data sharing from a unified governance layer.

Week 22 — Azure Data Factory (ADF) Foundations & Data Ingestion

1. Azure Data Factory Overview

Azure Data Factory (ADF) is a cloud-based, fully managed, serverless data integration and orchestration service used to automate ETL/ELT workflows across cloud and on-premise data sources.

ADF primarily provides three capabilities:

1. **Data Ingestion**
    - Moves data between source and target systems.
    - Uses high-throughput **Copy Activity**.
    - Supports 100+ native connectors.
    - Example sources/sinks: SQL databases, AWS S3, Azure Blob Storage, ADLS Gen2, Azure SQL Database.
2. **Data Transformation**
    - Uses **Mapping Data Flows** for visual, code-free transformations.
    - Transformation logic is executed using managed Spark-based compute.
3. **Pipeline Orchestration**
    - Controls workflow dependencies and execution.
    - Supports scheduling, event-driven execution, branching, validation, error handling, and monitoring.

When NOT to Use ADF

ADF is not intended to be:

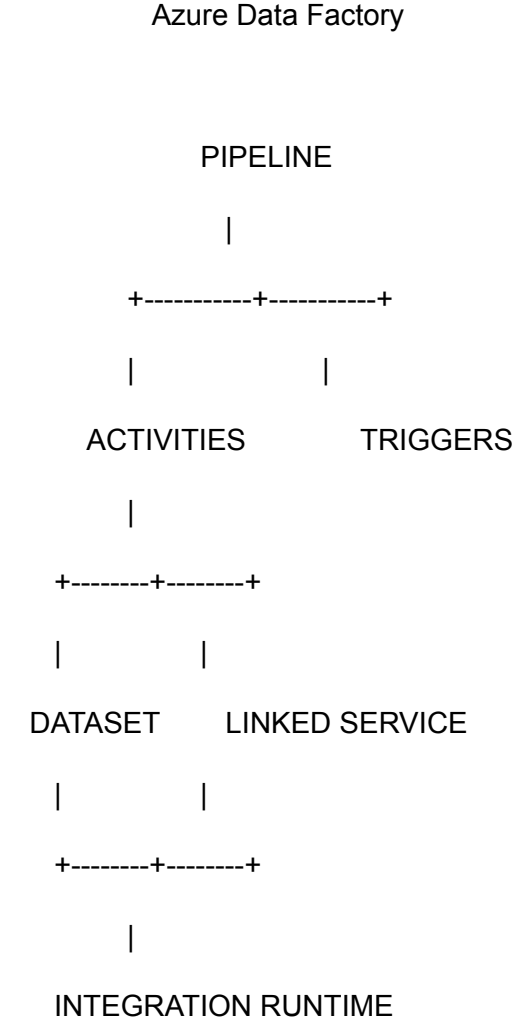
- A data storage system.
- A replacement for complex Spark programming.
- A real-time streaming platform.
- A dedicated database migration/lift-and-shift solution.

For complex Spark workloads, **Azure Databricks** or **Azure Synapse Spark** is generally more appropriate. Real-time workloads are better suited to technologies such as Event Hubs, Stream Analytics, or Spark Structured Streaming.

---

2. Core ADF Components

ADF pipelines are built using several key abstractions:



Linked Service

A **Linked Service** stores connection information required to connect to an external system.

It can contain:

- Connection details
- Endpoint URLs
- Authentication information
- Key Vault references

Analogy: Connection string / credentials.

Dataset

A **Dataset** represents the structure or location of the data used by an activity.

Examples:

- CSV file
- Parquet file
- SQL table
- JSON file

Analogy: File or table pointer.

Activity

An **Activity** represents one processing step inside a pipeline.

Examples:

- Copy Data
- Lookup
- ForEach
- Get Metadata
- Validation
- Execute Pipeline
- Mapping Data Flow

Analogy: Individual function/step.

Pipeline

A **Pipeline** is a logical grouping of activities that together implement a workflow.

Analogy: Workflow routine.

Integration Runtime

The **Integration Runtime (IR)** is the underlying compute infrastructure used by ADF to execute activities and provide connectivity between environments.

Analogy: Compute engine / gateway.

---

3. Copy Activity & Data Ingestion

The **Copy Activity** is ADF's primary data movement mechanism.

Source

|

v

Source Dataset

|

v

Copy Activity

|

v

Sink Dataset

|

v

Target

Example: Azure SQL → ADLS Gen2

Typical workflow:

1. Create an Azure SQL Linked Service.
2. Create an ADLS Gen2 Linked Service.
3. Create a source SQL Dataset.
4. Create a sink Dataset representing the ADLS destination.
5. Create a pipeline.
6. Add a **Copy Data Activity**.
7. Configure source and sink.
8. Map columns where required.
9. Debug/run the pipeline.

Example: HTTP → ADLS Gen2

ADF can also ingest files from HTTP endpoints.

HTTP Endpoint

|

HTTP Linked Service

|

HTTP Dataset

|

Copy Activity

|

ADLS Gen2 Dataset

|

ADLS Gen2

---

4. Parameterization

Creating separate pipelines and datasets for every incoming file leads to **component sprawl**.

For example:

100 Files

↓

100 Pipelines

100 Datasets

100 Copy Activities

A better approach is to create a **metadata-driven reusable pipeline**.

Configuration File

|

v

Lookup Activity

|

v

ForEach Activity

|

v

Parameterized Copy Activity

Parameterization Levels

Linked Service Parameters

Used for dynamic connection information.

Example:

@linkedservice().baseUrl

Dataset Parameters

Used for dynamic file or relative paths.

Example:

@dataset().relativeURL

Pipeline Parameters

High-level parameters exposed at pipeline level.

Examples:

baseURL

relativeURL

filename

---

5. Metadata-Driven Pipeline

A metadata-driven pipeline uses configuration data to determine what should be processed.

Lookup Activity

The **Lookup Activity** reads configuration information such as:

- File names
- URLs
- Relative paths
- Dataset details

The output can then be passed to a ForEach activity.

ForEach Activity

ForEach iterates over the records returned by Lookup.

Example:

@activity('Lookup').output.value

Inside the ForEach loop, parameters can be dynamically supplied:

@item().relativeURL

Key Benefit

Instead of building a pipeline for every file, a **single reusable pipeline** can process many datasets dynamically.

Week 23 — ADF Production Pipelines, Data Flows & Orchestration

1. Production-Grade ADF Pipeline

A production pipeline should not simply copy data. It should validate input, handle failures, securely access credentials, and provide operational control.

Incoming File

|

v

Validation

|

v

Get Metadata

|

v

If Condition

/ \

True False

| |

Copy Fail

|

v

Delete Source

---

2. Validation Activity

The **Validation Activity** checks whether a required condition has been satisfied before the pipeline proceeds.

Typical use case:

Wait for expected file

↓

Check availability

↓

Continue if available

It can be used to verify that an expected file exists before starting ingestion.

---

3. Get Metadata Activity

Get Metadata Activity retrieves information about files or datasets.

Possible metadata includes:

- File existence
- File size
- Child items
- Column count
- Other structural attributes

Example workflow:

Get Metadata

|

v

Check Size

|

v

If Size > 0

---

4. If Condition Activity

The **If Condition Activity** evaluates a Boolean expression and branches the pipeline.

Example:

File Size > 0?

/ \

TRUE FALSE

| |

COPY FAIL

This enables production pipelines to distinguish between valid and invalid inputs.

---

5. Fail Activity

The **Fail Activity** explicitly marks a pipeline execution as failed.

It can be used when:

- Input files are missing.
- Validation rules fail.
- Data quality requirements are not met.
- A required condition isn't satisfied.

Custom error messages can be supplied to make failures easier to troubleshoot.

---

6. Secure Credential Management

Sensitive credentials should not be hardcoded inside ADF pipelines or Linked Services.

Azure Key Vault can securely store credentials such as:

- AWS Access Key ID
- AWS Secret Access Key
- Other third-party authentication secrets

ADF Linked Service

|

v

Azure Key Vault

|

v

Secure Credential

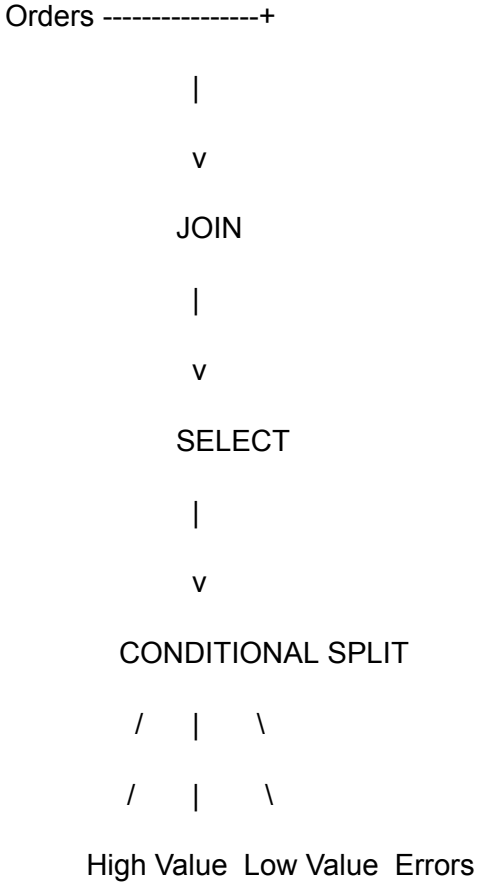
This provides safer enterprise credential management.

---

7. Mapping Data Flows

Mapping Data Flows provide a visual, code-free approach to data transformation within ADF.

They allow users to construct transformation workflows visually instead of manually writing Spark code.



Common Transformations

Join

Combines multiple data streams based on key expressions.

Supported join types include:

- Inner
- Left Outer
- Right Outer

- Full Outer

Select

Used to:

- Rename columns
- Drop columns
- Alias columns

Filter

Filters rows according to a Boolean expression.

Example:

customer_city == 'Caguas'

Conditional Split

Routes records to different output streams based on conditions.

Example:

order_amount > 500

↓

High Value Orders

order_amount <= 500

↓

Low Value Orders

isNull(order_amount)

↓

Erroneous Orders

Aggregate

Groups records and calculates summary metrics.

Example:

SUM(order_item_subtotal)

---

8. ADF Trigger Types

Triggers determine **when a pipeline should execute**.

The primary trigger types covered are:

1. Schedule Trigger
2. Tumbling Window Trigger
3. Storage Event Trigger
4. Custom Event Trigger

Schedule Trigger

Executes a pipeline based on a predefined time schedule.

Example:

Every day at 02:00 UTC

↓

Execute Pipeline

Key characteristics

- Time-based.

- Supports future schedules.
- Supports **many-to-many** relationships between triggers and pipelines.

Tumbling Window Trigger

Executes pipelines over fixed, non-overlapping time windows.

00:00 —— 01:00

01:00 —— 02:00

02:00 —— 03:00

Key capabilities

- Periodic execution.
- Non-overlapping windows.
- Historical backfilling.
- Dependency chaining.
- **1-to-1 relationship** with pipelines.

A major advantage is the ability to process historical windows that were missed earlier.

Storage Event Trigger

Executes a pipeline when a storage event occurs.

Example:

New File Uploaded

↓

Storage Event

↓

ADF Pipeline

↓

Process File

Common events include:

- Blob Created
- Blob Deleted

This is particularly useful for event-driven file ingestion.

Custom Event Trigger

Custom Event Triggers execute pipelines based on events published through custom Event Grid topics.

They are useful when an external application needs to notify ADF that a specific event has occurred.

---

9. Trigger Comparison

Trigger	Trigger Mechanism	Important Capability
Schedule	Time/calendar	Many-to-many pipeline relationship
Tumbling Window	Fixed time windows	Backfilling + dependency chaining
Storage Event	Storage event	Event-driven ingestion
Custom Event	Event Grid custom event	External application-driven execution

Easy Memory Trick

Schedule → "When?"

Tumbling Window → "Which time window?"

Storage Event → "When file arrives?"

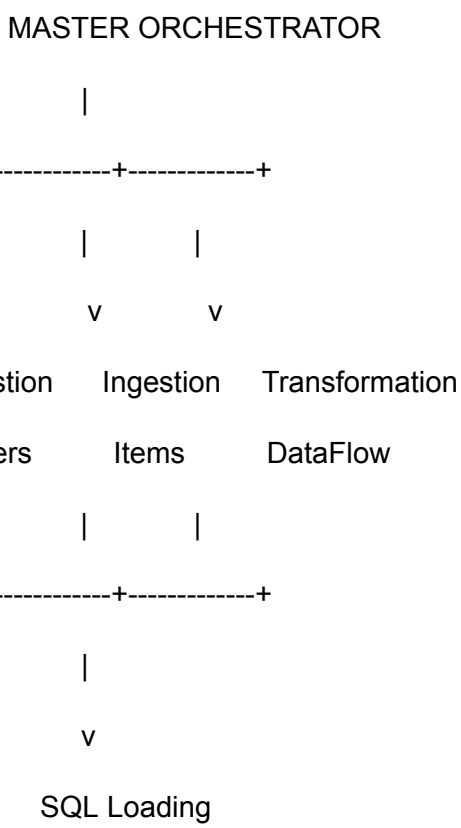
Custom Event → "When application sends event?"

---

10. Execute Pipeline Activity

Large enterprise workflows are often divided into smaller, modular pipelines.

For example:



The **Execute Pipeline Activity** allows one pipeline to invoke another pipeline.

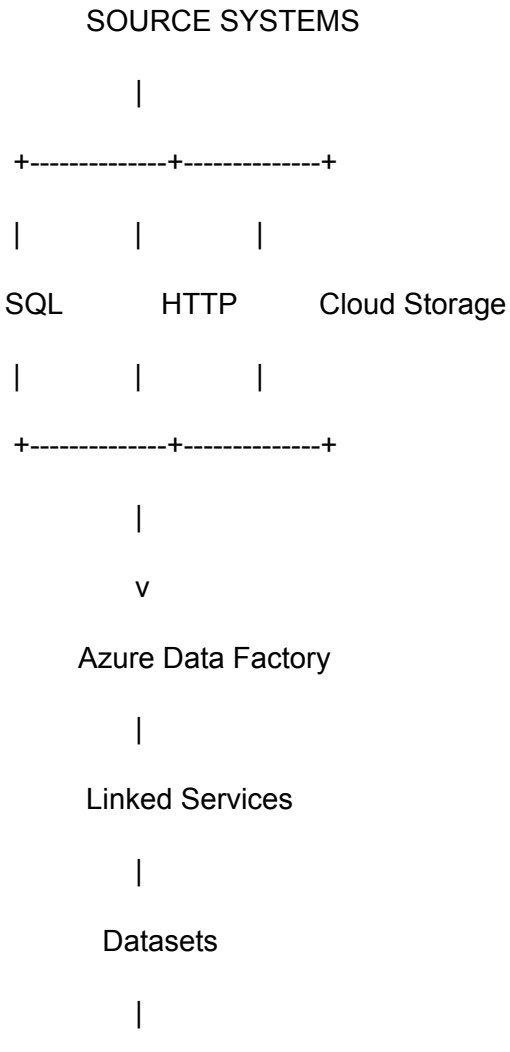
Benefits

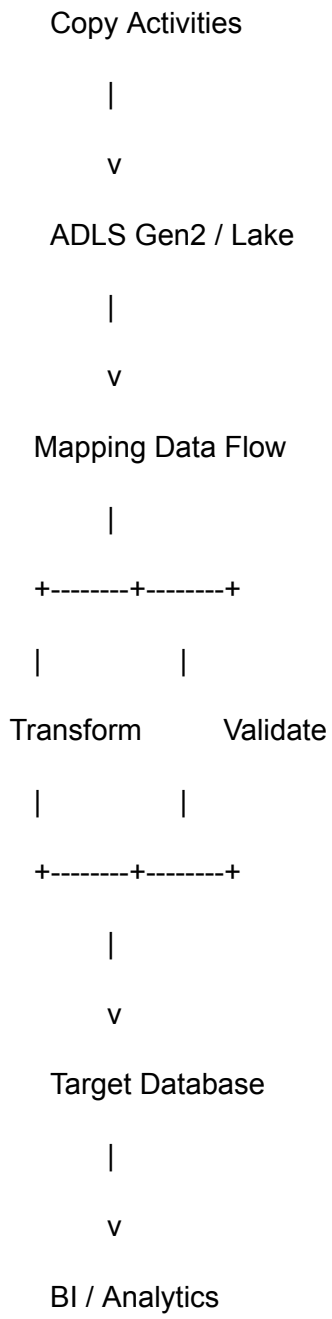
- Modular design
- Reusability
- Easier troubleshooting
- Independent development
- Separation of ingestion and transformation logic
- Better enterprise orchestration

---

11. End-to-End ADF Architecture

Putting the Week 22 and Week 23 concepts together:





Overall ADF Mental Model

Linked Service → HOW to connect

Dataset → WHAT data/location

Activity → WHAT operation

Pipeline → HOW operations are combined

Trigger → WHEN pipeline runs

Integration Runtime → WHERE/how execution occurs

This is the **core framework to remember for ADF interviews and practical implementation**.

Week 24: Azure Synapse Analytics & Enterprise Data Warehousing

---

1. Enterprise Data Warehouse Needs & Architectural Overview

Traditional relational databases (**RDBMS**) such as MySQL and PostgreSQL are primarily designed for **Online Transaction Processing (OLTP)**.

Using OLTP databases for large-scale analytical workloads can create several problems:

Resource Contention

Analytical queries involving large aggregations can consume significant CPU and memory, potentially slowing down operational transactions.

Schema Design

OLTP systems typically use **normalized schemas** optimized for transactional integrity and individual row operations.

OLAP workloads, on the other hand, commonly use **denormalized dimensional models**, such as:

- Star Schema
- Snowflake Schema

These are optimized for large-scale analytical reads.

Historical Data & Scalability

Keeping decades of historical data inside an operational database can be expensive and difficult to scale.

OLTP vs. OLAP

Feature	OLTP	OLAP / Data Warehouse
Primary Purpose	Operational transactions	Analytics & reporting
Data	Current operational data	Historical analytical data
Schema	Highly normalized	Often denormalized
Workload	Small, frequent transactions	Large aggregations
Query Pattern	Single/few records	Millions/billions of records
Scaling	Traditionally vertical	Massively parallel / distributed
Example	MySQL, PostgreSQL	Azure Synapse Dedicated SQL Pool

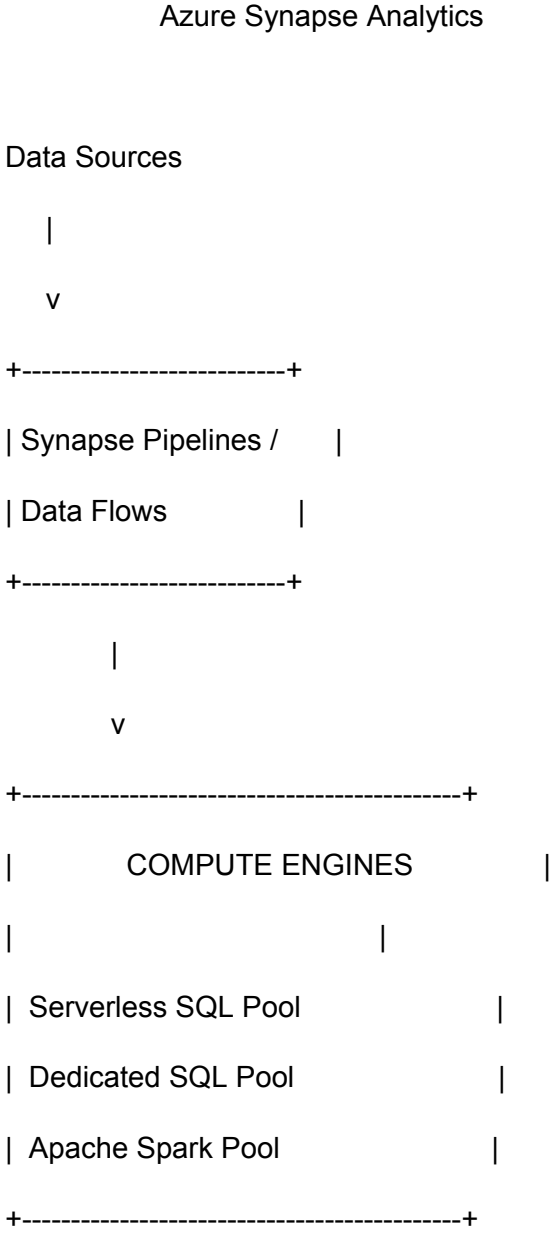
---

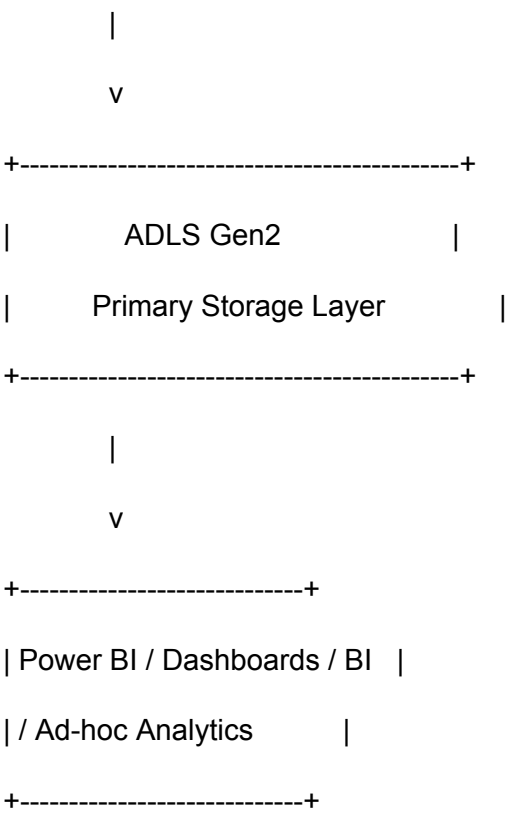
2. Azure Synapse Analytics Architecture

Azure Synapse Analytics is an enterprise analytics platform that combines:

- Data ingestion
- Data warehousing
- Big Data analytics
- Data transformation
- SQL analytics
- Apache Spark

A Synapse workspace uses **Azure Data Lake Storage Gen2 (ADLS Gen2)** as its primary storage layer.





Major Compute Engines

1. Serverless SQL Pool

Used for querying data directly from the Data Lake using T-SQL.

2. Dedicated SQL Pool

Used as an enterprise Data Warehouse using **Massively Parallel Processing (MPP)**.

3. Apache Spark Pool

Used for:

- Big Data processing
- Machine Learning
- Data transformation
- PySpark/Scala workloads

---

3. Serverless SQL Pool

The **Serverless SQL Pool** provides on-demand SQL analytics directly over files stored in ADLS Gen2.

It does not require users to provision or manage dedicated compute infrastructure.

Key Characteristics

- No dedicated cluster provisioning.
- Pay-per-query model.
- Pricing based on the amount of data scanned.
- Supports standard T-SQL.
- Can query files directly from the Data Lake.

Cost Optimization

Columnar formats such as:

- **Parquet**
- **ORC**

are preferred because they reduce the amount of data that must be scanned compared with raw CSV files.

---

4. Querying Data Lake Files with `OPENROWSET`

`OPENROWSET` allows users to query files directly without creating permanent database objects.

Example:

```
SELECT TOP 100

    order_id,

    order_date,

    customer_id,

    order_status

FROM OPENROWSET(
```

```
    BULK 'https://ttsynapsesa.dfs.core.windows.net/raw/orders.parquet',
    FORMAT = 'PARQUET'
) AS [orders_data];
```

Key Benefit

Useful for:

- Ad-hoc exploration
- Data discovery
- Quick querying of Data Lake files

---

5. External Tables in Serverless SQL Pool

External tables provide a reusable logical structure over files stored in ADLS Gen2.

The data remains in external storage.

```
ADLS Gen2
   |
   | Raw files
   v
External Data Source
   |
   v
External File Format
   |
   v
External Table
```

|

v

Serverless SQL Query

Important Point

The supplied material specifies that **managed/internal tables cannot be created inside Serverless SQL Pools**.

Creating an External Table

Step 1: Create Master Key

```
CREATE MASTER KEY ENCRYPTION BY PASSWORD = 'StrongPassword123!';
```

Step 2: Create Database Scoped Credential

```
CREATE DATABASE SCOPED CREDENTIAL SasToken
WITH
   IDENTITY = 'SHARED ACCESS SIGNATURE',
   SECRET = 'sv=2020-08-04&ss=b&srt=sco&sp=rwdlacx&se=2026-12-31...';
```

Step 3: Create External Data Source

```
CREATE EXTERNAL DATA SOURCE ExtDataSrc
WITH (
   LOCATION = 'https://ttsynapsesa.dfs.core.windows.net/raw',
   CREDENTIAL = SasToken
```

```
);
```

Step 4: Create External File Format

```
CREATE EXTERNAL FILE FORMAT TextFileFormat
WITH (
    FORMAT_TYPE = DELIMITEDTEXT,
    FORMAT_OPTIONS (
        FIELD_TERMINATOR = ',',
        FIRST_ROW = 2,
        USE_TYPE_DEFAULT = FALSE
    )
);
```

Step 5: Create External Table

```
CREATE EXTERNAL TABLE dbo.orders_ext (
    order_id BIGINT,
    order_date NVARCHAR(4000),
    customer_id BIGINT,
    order_status NVARCHAR(4000)
)
WITH (
    LOCATION = 'orders.csv',
    DATA_SOURCE = ExtDataSrc,
    FILE_FORMAT = TextFileFormat
```

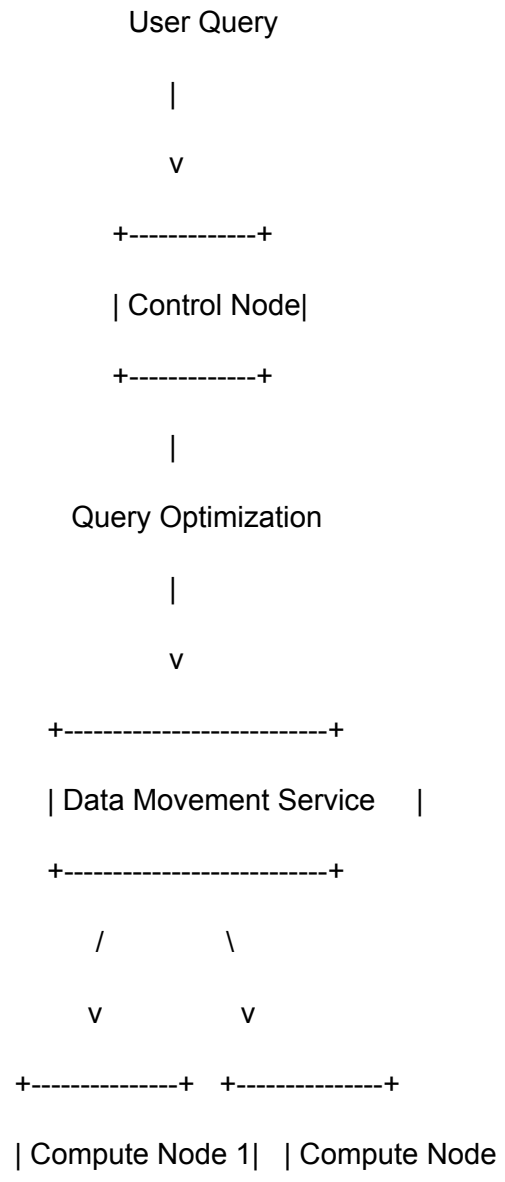
);

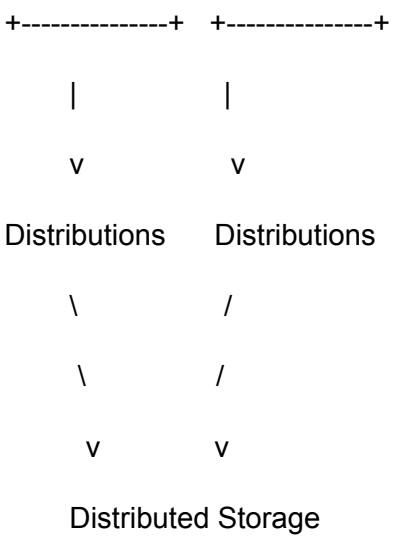
---

6. Dedicated SQL Pool

The **Dedicated SQL Pool**, formerly known as **Azure SQL Data Warehouse**, is designed for enterprise-scale analytical workloads.

It uses **Massively Parallel Processing (MPP)**.





---

7. Dedicated SQL Pool Components

Control Node

The **Control Node** is the frontend component.

It:

- Receives SQL queries.
- Parses queries.
- Optimizes execution plans.
- Coordinates execution across compute nodes.

Compute Nodes

Compute nodes perform the actual processing.

They execute query operations across their assigned data distributions.

Data Movement Service (DMS)

The **Data Movement Service** transfers data between compute nodes when required.

This can happen during:

- Joins

- Aggregations
- Data shuffles

Reducing unnecessary data movement is an important performance consideration.

Data Warehousing Units (DWUs)

DWUs represent the compute capacity allocated to the Dedicated SQL Pool.

The supplied material describes scaling from:

- **DW100c**
- Up to **DW30000c**

60 Distributions

A Dedicated SQL Pool uses **60 fixed logical distributions** for data distribution.

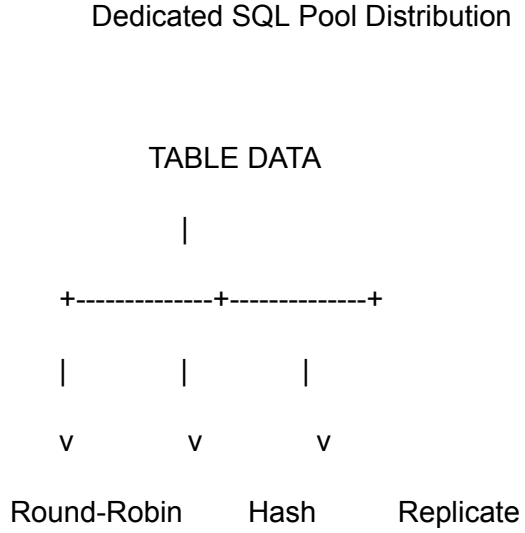
This is a critical concept for understanding Synapse performance.

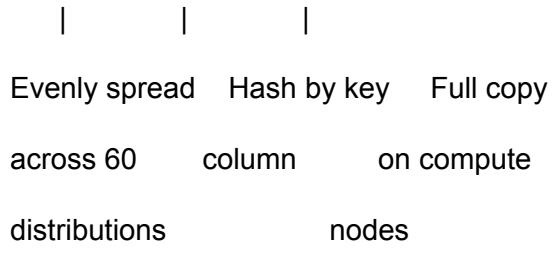
---

8. Table Distribution Strategies

Dedicated SQL Pool provides three primary distribution strategies:

1. Round-Robin
2. Hash
3. Replicate





Round-Robin Distribution

Rows are distributed across the 60 distributions in a round-robin pattern.

Advantages

- Fast loading.
- Simple distribution.
- Generally good for staging.

Disadvantages

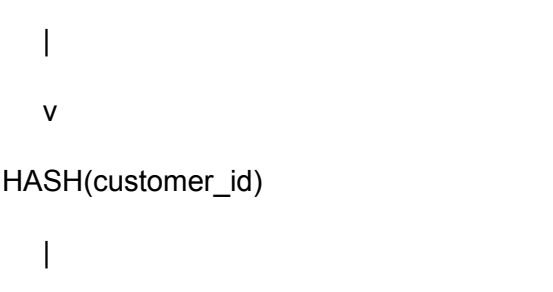
Queries may require data movement for joins and aggregations.

Best Use Case

Staging tables and temporary ingestion tables.

Hash Distribution

A deterministic hash function is applied to a selected column.



v

Distribution Number

Rows with the same distribution key are directed to the same distribution.

Advantage

Can reduce data movement during joins when tables are distributed on compatible keys.

Best Use Case

Large **Fact tables**, particularly very large datasets.

Example:

DISTRIBUTION = HASH(customer_id)

Replicate Distribution

A complete copy of the table is stored on each compute node.

Advantage

Joins with large fact tables can avoid network data movement because the small table is already available locally.

Best Use Case

Small **Dimension tables**.

Distribution Comparison

Distribution	How It Works	Typical Use

Round-Robin	Evenly distributes rows	Staging
Hash	Hashes selected column	Large fact tables
Replicate	Copies entire table	Small dimension tables

Interview Memory Trick

Round-Robin → Load fast

Hash → Join efficiently

Replicate → Small table everywhere

---

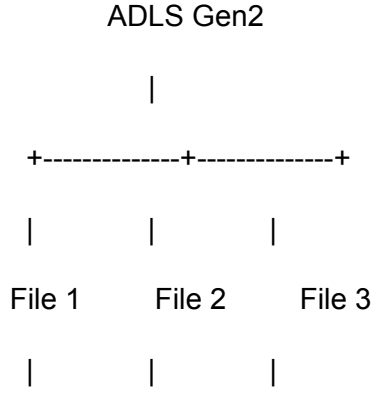
9. High-Speed Data Ingestion

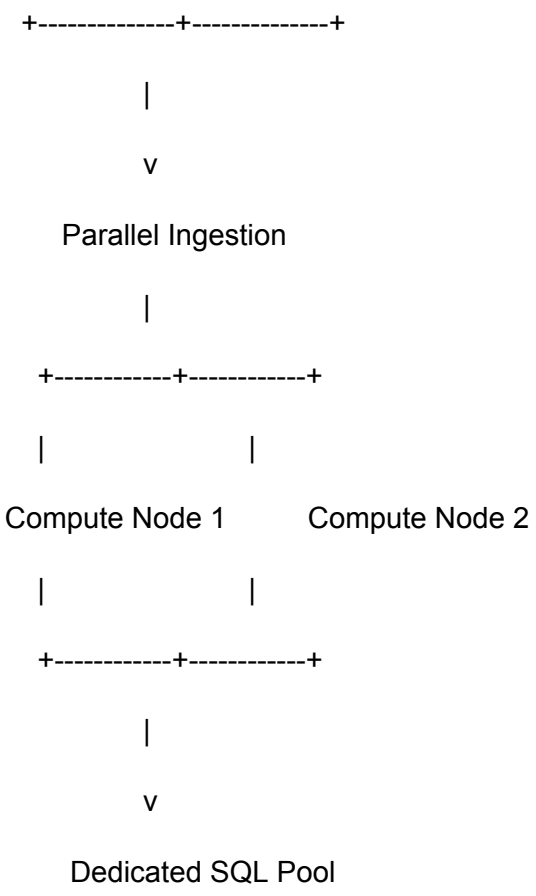
Standard `INSERT` statements are not ideal for high-volume ingestion into Dedicated SQL Pools.

Synapse provides high-performance approaches such as:

- **PolyBase**
- **COPY statement**

Both allow parallel ingestion from ADLS Gen2.





---

10. PolyBase & CTAS

PolyBase allows compute nodes to directly read data from external storage in parallel.

CTAS

CREATE TABLE AS SELECT (CTAS) can be used to create and populate a table from an existing query.

Example:

```
CREATE TABLE dbo.orders_staging
WITH (
    DISTRIBUTION = ROUND_ROBIN,
```

```
    CLUSTERED COLUMNSTORE INDEX
)
AS
SELECT * FROM dbo.orders_ext;
```

A production fact table can use hash distribution:

```
CREATE TABLE dbo.orders_fact
WITH (
    DISTRIBUTION = HASH(customer_id),
    CLUSTERED COLUMNSTORE INDEX
)
AS
SELECT * FROM dbo.orders_ext;
```

Inspecting Distribution

```
DBCC PDW_SHOWSPACEUSED('dbo.orders_fact');
```

This can be used to inspect space usage and row distribution.

---

11. COPY Statement

The **COPY statement** provides a simplified high-throughput ingestion mechanism.

One advantage is that it avoids the need to create external database objects required by the traditional PolyBase approach.

Example:

```
CREATE TABLE dbo.orders_internal (
    order_id BIGINT NOT NULL,
    order_date VARCHAR(45) NOT NULL,
    customer_id BIGINT NOT NULL,
    order_status VARCHAR(45) NOT NULL
)
WITH (
    DISTRIBUTION = HASH(customer_id),
    CLUSTERED COLUMNSTORE INDEX
);
```

Then load data:

```
COPY INTO dbo.orders_internal
FROM 'https://ttsynapsesa.dfs.core.windows.net/raw/orders_*.csv'
WITH (
    FILE_TYPE = 'CSV',
    CREDENTIAL = (
        IDENTITY = 'Shared Access Signature',
        SECRET = 'sv=2020-08-04...'
    ),
    FIRSTROW = 2,
    FIELDTERMINATOR = ','
);
```

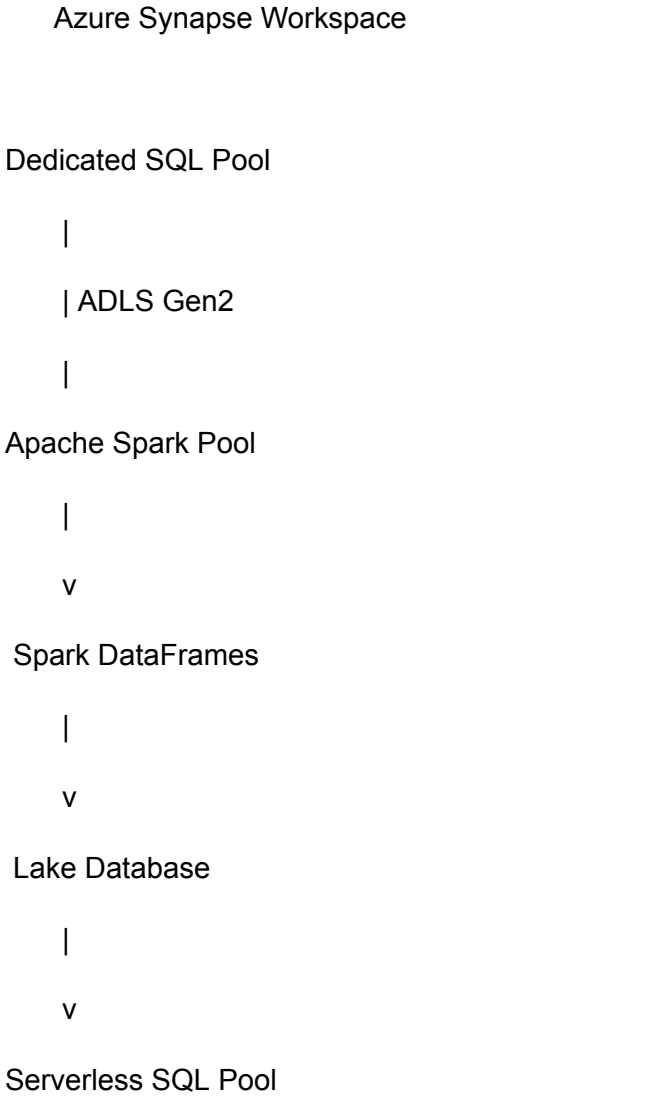
---

12. Apache Spark Pool

The **Apache Spark Pool** provides an in-memory distributed compute environment for:

- Big Data processing
- Machine Learning
- Data transformation
- PySpark
- Scala

It operates alongside the SQL engines within Synapse.



---

13. Dedicated SQL Pool ↔ Spark Interoperability

Spark can read data from a Dedicated SQL Pool using an optimized connector.

Example:

```
val df = spark.read.sqlanalytics("retaildb.dbo.orders_fact")
```

The resulting data can be processed as a Spark DataFrame.

For example:

```
df.write
  .mode("overwrite")
  .saveAsTable("default.spark_orders_summary")
```

---

14. Spark Table Synchronization

When a Spark table is created, Synapse performs background metadata synchronization.

This allows Spark-created tables to become accessible through the **Serverless SQL Pool**.

Conceptually:

```
Spark Table
    |
    v
```

Metadata Synchronization

|

v

Lake Database

|

v

Serverless SQL Pool

|

v

T-SQL Query

This enables different Synapse compute engines to work against a shared analytical environment.

---

15. Complete Synapse Architecture — Interview View

The complete Week 24 architecture can be remembered as:

DATA SOURCES

|

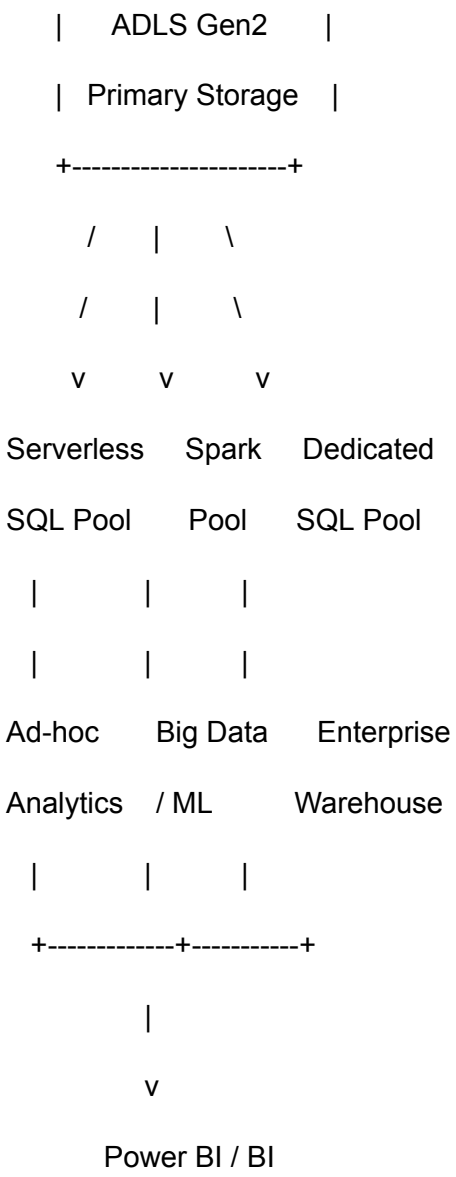
v

Synapse Pipelines / ADF

|

v

+----------------------+



Core Mental Model

Serverless SQL Pool → Query Data Lake without managing dedicated compute

Dedicated SQL Pool → Enterprise Data Warehouse using MPP

Spark Pool → Big Data processing, ML and transformations

ADLS Gen2 → Shared primary storage layer

Distribution → Determines where Dedicated SQL Pool data lives

PolyBase / COPY → High-throughput ingestion

DMS → Moves data between compute nodes when required

DWU → Controls Dedicated SQL Pool compute capacity

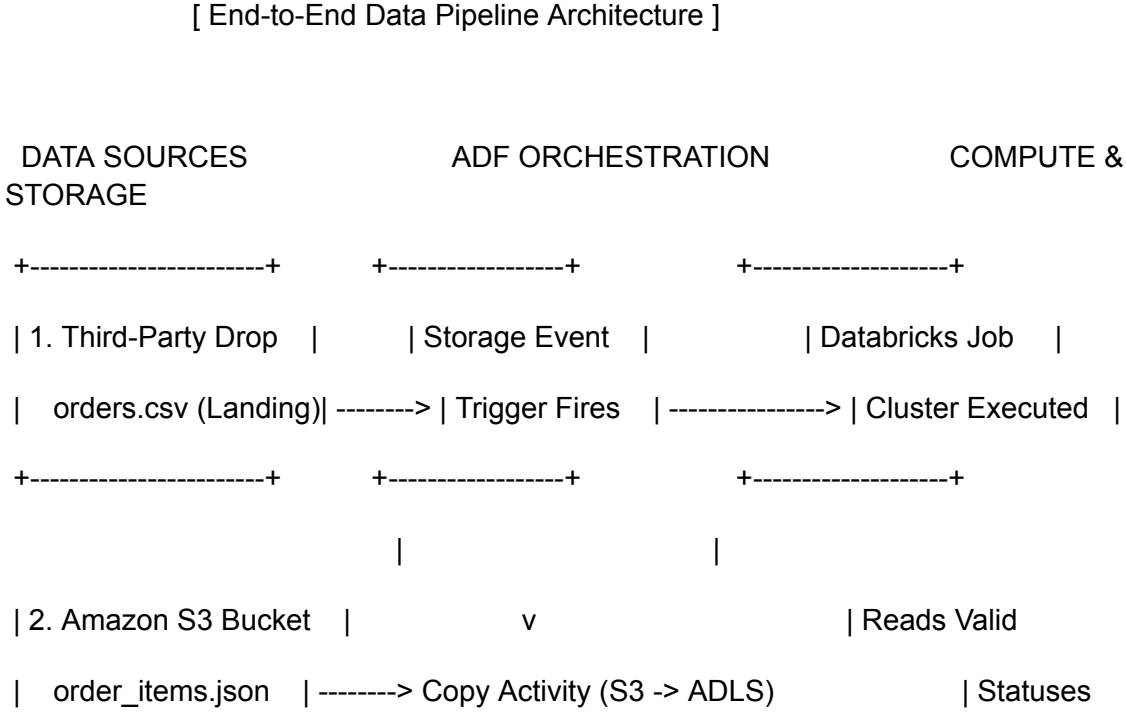
WEEK 25: END-TO-END ENTERPRISE DATA PIPELINE & PRODUCTION INTEGRATION PROJECT

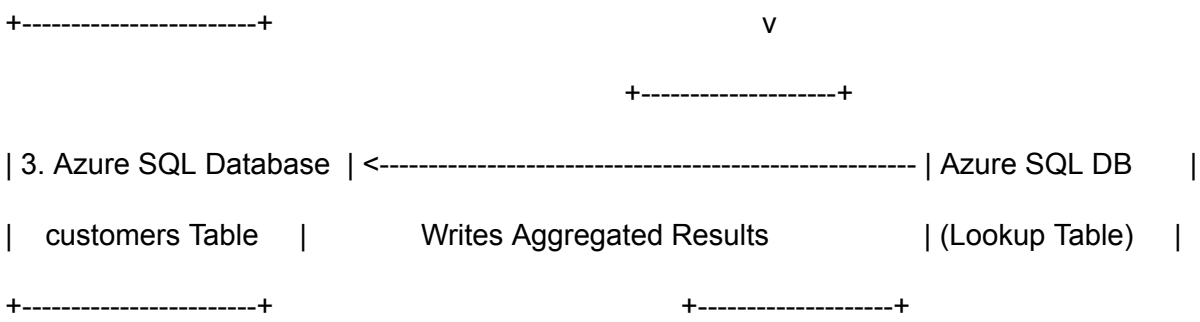
---

1. Enterprise Architecture & Project Requirements

This week synthesizes the entire curriculum—**Azure Storage, Azure Data Factory, Azure Databricks, Azure SQL Database, and Azure Key Vault**—into an end-to-end, event-driven data pipeline.

End-to-End Data Pipeline Architecture





Core Business Objectives

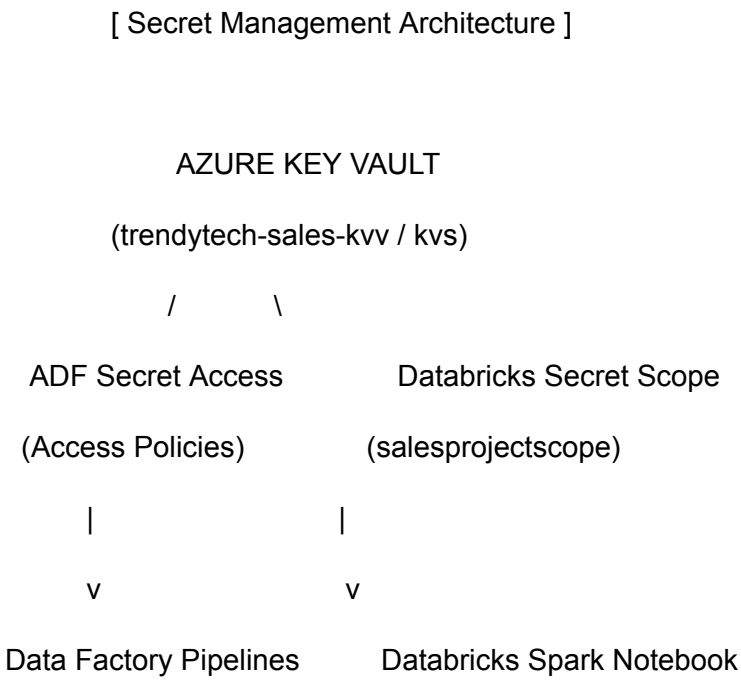
1. **Event-Driven Landing Detection**
   Automatically trigger ingestion as soon as a third-party vendor drops raw `orders.csv` files into the ADLS Gen2 `landing/` path.
2. **Multi-Stage Data Validation**
   Validation 1 — Uniqueness:
   Ensure zero duplicate `order_id` values exist inside incoming batches.
   Validation 2 — Lookup Integrity:
   Verify that all `order_status` values match a dynamically updated lookup list stored inside an Azure SQL Database table.
3. **Dynamic Routing — Staging vs. Discarded**
   - **If Valid:** Process and move raw files to the `staging/` directory.
   - **If Invalid:** Terminate pipeline processing and route files to the `discarded/` directory.
4. **Hybrid Multi-Cloud Joins**
   Combine landed `orders` from ADLS Gen2 with `order_items` from AWS S3 and `customers` from Azure SQL Database to produce aggregated reporting metrics.

---

2. Infrastructure Setup & Azure Key Vault Security

All administrative credentials, storage keys, and database passwords are managed inside **Azure Key Vault**.

Secret Management Architecture



Provisioned Azure Resources

Resource Name	Type	Purpose / Description
`trendytechsa`	Storage Account (ADLS Gen2)	Primary Data Lake containing `sales` container (`landing/`, `staging/`, `discarded/`).
`trendytech-sales-df`	Azure Data Factory	Pipeline orchestration engine with Storage Event triggers.
`trendytech-sales-databricks-ws`	Azure Databricks Workspace	Compute workspace running validation and aggregation Spark code.

`trendytechsqlserver` / `db`	Azure SQL Database	Stores dynamic lookup lists (`valid_order_status`) and customer entities.
`trendytech-sales-kvs`	Azure Key Vault	Centralized secret store for SQL passwords, DB tokens, and storage keys.

---

3. Azure SQL Database Configuration & Dynamic Lookup Setup

Instead of hardcoding valid order statuses inside notebook scripts, statuses are maintained in a **dynamic SQL lookup table**.

```
-- Step 1: Create Validation Lookup Table inside Azure SQL Database

CREATE TABLE valid_order_status (

    status_name VARCHAR(50) NOT NULL PRIMARY KEY

);



-- Step 2: Populate Initial Permitted Statuses

INSERT INTO valid_order_status (status_name) VALUES

    ('ON_HOLD'),

    ('PAYMENT_REVIEW'),

    ('PROCESSING'),

    ('CLOSED'),

    ('SUSPECTED_FRAUD'),

    ('COMPLETE'),
```

```
    ('PENDING'),
    ('CANCELED'),
    ('PENDING_PAYMENT');

-- Step 3: Create Customers Table for Cross-Source Enrichment
CREATE TABLE customers (
    customer_id INT NOT NULL,
    customer_fname VARCHAR(45) NOT NULL,
    customer_lname VARCHAR(45) NOT NULL,
    customer_email VARCHAR(45) NOT NULL,
    customer_password VARCHAR(45) NOT NULL,
    customer_street VARCHAR(255) NOT NULL,
    customer_city VARCHAR(45) NOT NULL,
    customer_state VARCHAR(45) NOT NULL,
    customer_zipcode VARCHAR(45) NOT NULL,
    PRIMARY KEY (customer_id)
);
```

---

4. Production-Grade Databricks Validation & Processing Notebook

The following PySpark validation script is executed by the pipeline. It features:

- Idempotent mounting logic
- Key Vault secret integration

- Duplicate detection
- SQL JDBC status checks
- Dynamic routing of valid and invalid files
- Multi-source enrichment
- Aggregated reporting output

```
#
================================================================================
======

# STEP 1: DYNAMIC PARAMETER RETRIEVAL & IDEMPOTENT MOUNTING

#
================================================================================
======



dbutils.widgets.text("filename", "", "Incoming Filename")

filename = dbutils.widgets.get("filename")



storage_account_name = "trendytechsa"

container_name = "sales"

mount_point = "/mnt/sales"



# Securely fetch Storage Account Key via Databricks Secret Scope

storage_key = dbutils.secrets.get(

    scope="salesprojectscope",

    key="storage-account-key"

)



# Generic Idempotent Mount Check

if not any(m.mountPoint == mount_point for m in dbutils.fs.mounts()):
```

```
    dbutils.fs.mount(
        source=f"wasbs://{container_name}@{storage_account_name}.blob.core.windows.net",
        mount_point=mount_point,
        extra_configs={
            f"fs.azure.account.key.{storage_account_name}.blob.core.windows.net": storage_key
        }
    )
    print("Mounting completed successfully.")
else:
    print("Storage path already mounted.")



# ============================================================================
# STEP 2: READ INCOMING DATA & VALIDATION 1 (DUPLICATE CHECK)
# ============================================================================

file_path = f"{mount_point}/landing/{filename}"

orders_df = spark.read.format("csv") \
    .option("header", "true") \
    .option("inferSchema", "true") \
    .load(file_path)
```

```
total_records = orders_df.count()


distinct_orders = orders_df \
    .select("order_id") \
    .distinct() \
    .count()


# Check for duplicate order_ids
if total_records != distinct_orders:


    # Route corrupted file to discarded folder and exit
    dbutils.fs.mv(
        file_path,
        f"{mount_point}/discarded/{filename}"
    )


    dbutils.notebook.exit(
        '{"errorFlg": "true", "errorMsg": "Duplicate order_id detected"}'
    )



#
==========================================================================
======
```

```
# STEP 3: VALIDATION 2 (DYNAMIC AZURE SQL LOOKUP CHECK)
#
================================================================================
======


# JDBC Connection Parameters
db_server = "trendytechsqlserver"
db_name = "trendytechsqldb"
db_user = "tt-sql-user"


db_password = dbutils.secrets.get(
    scope="salesprojectscope",
    key="sql-password"
)


jdbc_url = (
    f"jdbc:sqlserver://{db_server}.database.windows.net:1433;"
    f"database={db_name};"
)


connection_properties = {
    "user": db_user,
    "password": db_password,
    "driver": "com.microsoft.sqlserver.jdbc.SQLServerDriver"
}
```

```
# Fetch valid order statuses dynamically from Azure SQL DB
valid_status_df = spark.read.jdbc(
    url=jdbc_url,
    table="dbo.valid_order_status",
    properties=connection_properties
)

# Identify invalid statuses using LEFT ANTI JOIN
invalid_records = orders_df.join(
    valid_status_df,
    orders_df.order_status == valid_status_df.status_name,
    "left_anti"
)

if invalid_records.count() > 0:

    # Route file with illegal statuses to discarded folder and exit
    dbutils.fs.mv(
        file_path,
        f"{mount_point}/discarded/{filename}"
    )

    dbutils.notebook.exit(
```

```
        '{"errorFlg": "true", "errorMsg": "Invalid order_status values present"}'
    )



#
==========================================================================
======
# STEP 4: SUCCESSFUL STAGING & MULTI-SOURCE METRIC ENRICHMENT
#
==========================================================================
======


# Move validated file to staging directory
dbutils.fs.mv(
    file_path,
    f"{mount_point}/staging/{filename}"
)


# Read Amazon S3 Order Items
# Ingested to ADLS by ADF Copy Activity
order_items_df = spark.read.json(
    f"{mount_point}/order_items/order_items.json"
)


# Read Azure SQL Database Customers Table
customers_df = spark.read.jdbc(
```

```
    url=jdbc_url,
    table="dbo.customers",
    properties=connection_properties
)

# Perform Multi-Source Analytical Join Across Ingested Streams
enriched_summary = orders_df \
    .join(
        order_items_df,
        orders_df.order_id == order_items_df.order_item_order_id
    ) \
    .join(
        customers_df,
        orders_df.customer_id == customers_df.customer_id
    ) \
    .groupBy(
        customers_df.customer_id,
        customers_df.customer_fname,
        customers_df.customer_lname
    ) \
    .agg(
        {"order_id": "count", "order_item_subtotal": "sum"}
    ) \
    .withColumnRenamed(
```

```
        "count(order_id)",
        "total_orders_placed"
    ) \
    .withColumnRenamed(
        "sum(order_item_subtotal)",
        "total_amount_spent"
    )

# Write Aggregated Results Back to Azure SQL Database for BI Reporting
enriched_summary.write.jdbc(
    url=jdbc_url,
    table="dbo.customer_sales_summary",
    mode="append",
    properties=connection_properties
)

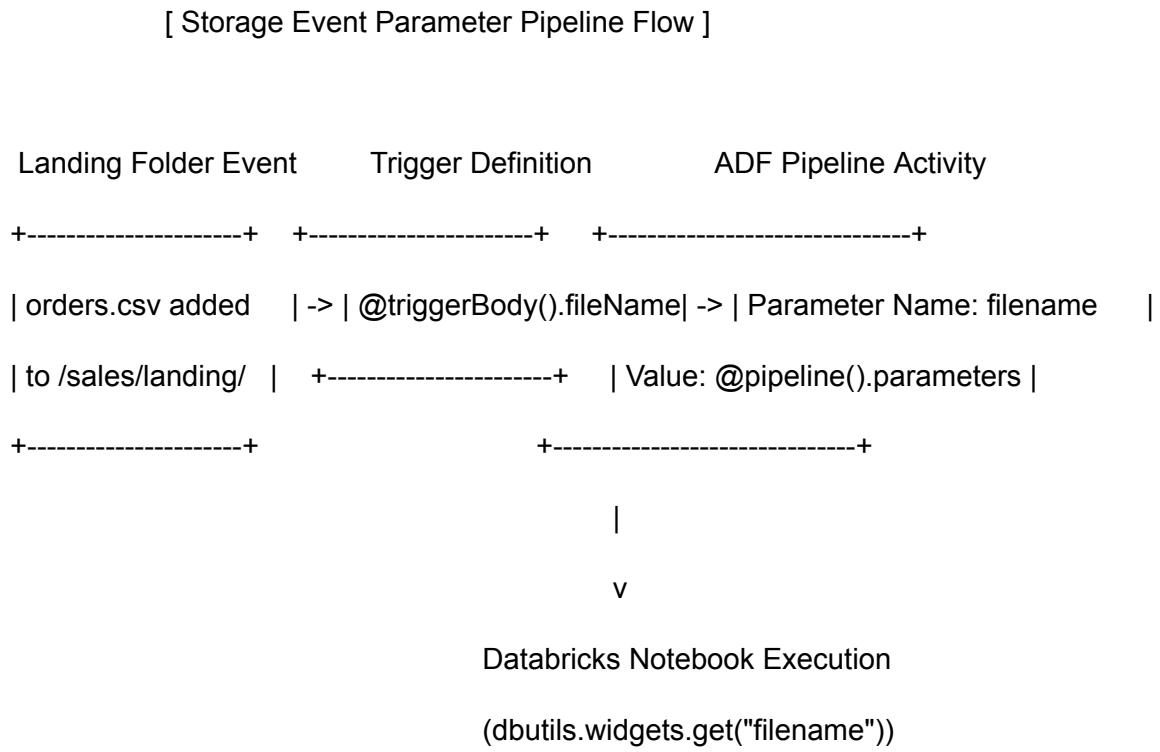
dbutils.notebook.exit(
    '{"errorFlg": "false", "errorMsg": "Pipeline Executed Successfully"}'
)
```

---

5. ADF Parameterized Orchestration & Storage Event Triggers

To automate pipeline execution, **Azure Data Factory** utilizes **Storage Event Triggers** and dynamic **Trigger Body Parameter Passing**.

Storage Event Parameter Pipeline Flow



Event Trigger Parameter Mapping

1. Trigger Configuration

- **Trigger Type:** Storage Event
- **Storage Account / Container:** `trendytechsa` / `sales`
- **Blob Path Begins With:** `landing/`
- **Event:** Blob Created

2. Passing Dynamic File Names to Databricks

Inside the Data Factory Pipeline:

1. Create a string parameter named `filename`.
2. Bind the parameter to the Storage Event Trigger body expression:

filename = @triggerBody().fileName

3. In the Databricks Notebook Activity properties, pass `filename` into the Notebook Base Parameter map.

---

6. FULL DATA ENGINEERING CURRICULUM MASTER SUMMARY

==========

MASTER BIG DATA & AZURE DATA ENGINEERING CURRICULUM

==========

[WEEKS 1 - 7: Distributed Storage & Compute Core]

• HDFS Architecture, Block Distribution, Replication, High Availability & YARN Schedulers

• MapReduce Concepts & Apache Spark Fundamentals (RDDs, Transformations vs. Actions)

[WEEKS 8 - 16: PySpark Core, Internals & Optimization]

• DataFrames API, Spark SQL, Catalyst Optimizer & Tungsten Execution Engine

• Memory Management (Storage vs. Execution Pools), Partitioning vs. Bucketing

• Join Strategies (Broadcast, Shuffle Hash, Sort-Merge) & Partition Skew Handling

• Performance Tuning (AQE, Z-Ordering, Salting) & Driver/Executor Resource Estimation

• CI/CD Pipelines (Git, Pytest) & Interview Question Preparation

[WEEK 17: Azure Cloud Fundamentals & Core Services]

• Cloud Economics (CapEx vs. OpEx), Service Models (IaaS, PaaS, SaaS)

• Azure Infrastructure: Geographies, Region Pairs, Availability Zones, Fault/Update Domains

[WEEKS 18 - 20: Databricks, Storage & Delta Lake Architecture]

• Azure Storage (Blob, ADLS Gen2, Access Tiers, LRS/ZRS/GRS Redundancy)

• Azure Databricks (Control vs. Data Plane, All-Purpose/Job Clusters, DBFS, dbutils)

• Delta Lake (ACID Transactions, Commit Logs, Time Travel, DML Operations)

• Medallion Architecture (Bronze -> Silver -> Gold Design Pattern) & Change Data Feed (CDF)

• Delta Engine Optimizations (Data Skipping, Compaction, Z-Ordering, Vacuuming, Photon)

[WEEK 21: Enterprise Data Governance (Unity Catalog)]

• Centralized Metastore Layer, 3-Level Namespace (catalog.schema.table)

• Managed vs. External Tables, SCIM User Provisioning & ANSI SQL Access Policies

[WEEKS 22 - 23: Azure Data Factory (ADF) Orchestration]

• Copy Activity, Mapping Data Flows, Parameterization, Lookup & ForEach Looping

• Production Control Flow (Validation, Get Metadata, Fail, If Condition, Key Vault Integration)

• Pipeline Triggers (Schedule, Tumbling Window, Storage Event)

[WEEK 24: Azure Synapse Analytics & Enterprise Data Warehousing]

• OLTP vs. OLAP Architecture, Serverless SQL Pools (OPENROWSET, External Tables)

• Dedicated SQL Pools (MPP Architecture, 60 Distributions, DWU Allocation)

• Distribution Strategies (Round-Robin, Hash, Replicated) & High-Speed Ingestion (PolyBase, COPY)

[WEEK 25: End-to-End Enterprise Data Pipeline Project]

• Event-Driven Azure Solution (ADFS, Databricks, Azure SQL, Key Vault, AWS S3)

• Multi-Stage Data Validation, Dynamic Routing, Multi-Cloud Ingestion & Reporting Aggregations

==========

WEEK 25 — KEY TAKEAWAYS

Architecture

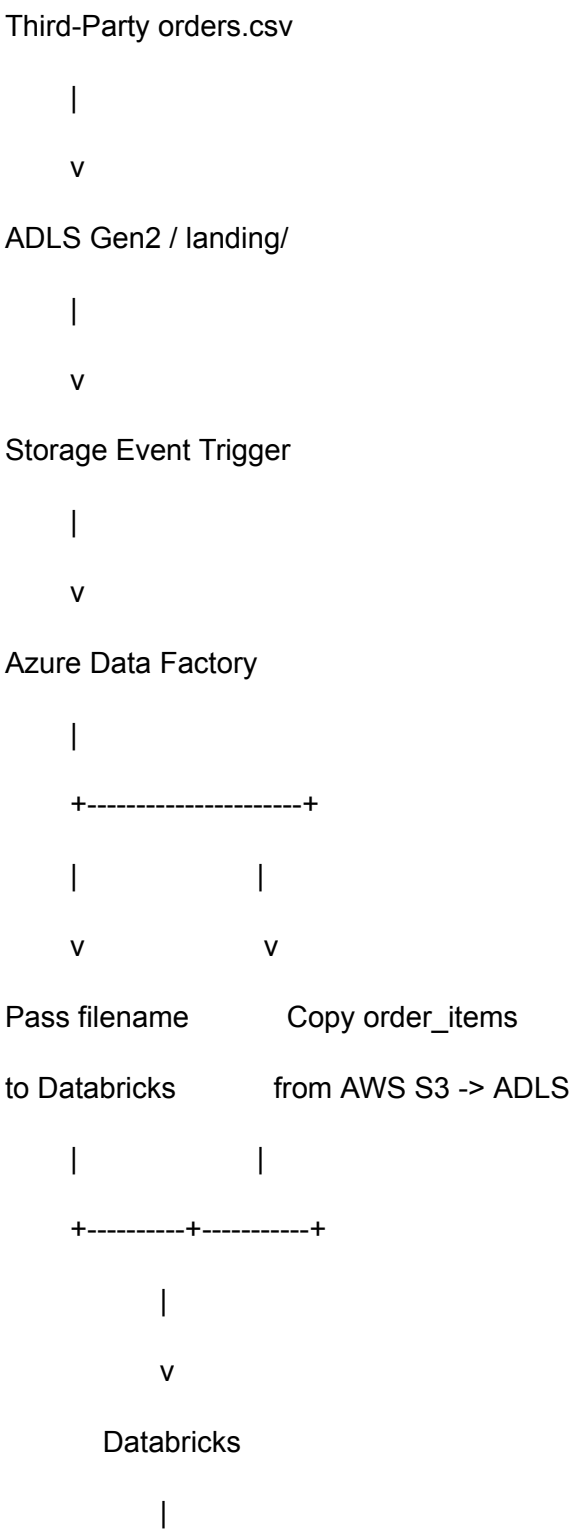
ADLS Gen2 → ADF → Databricks → Azure SQL Database

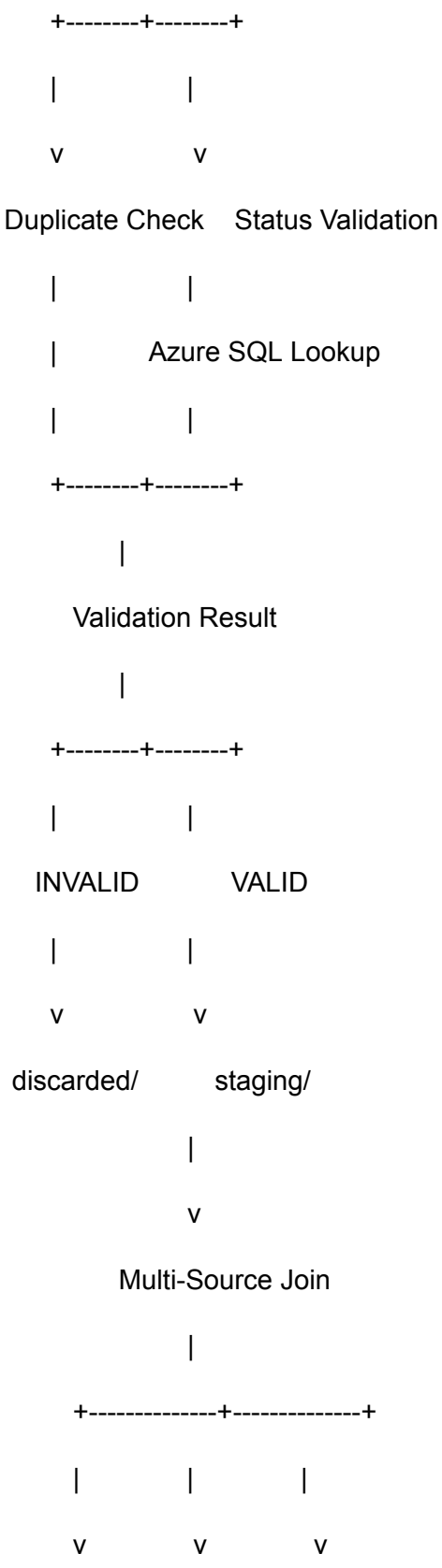
with **AWS S3** as an additional external data source and **Azure Key Vault** for centralized secret management.

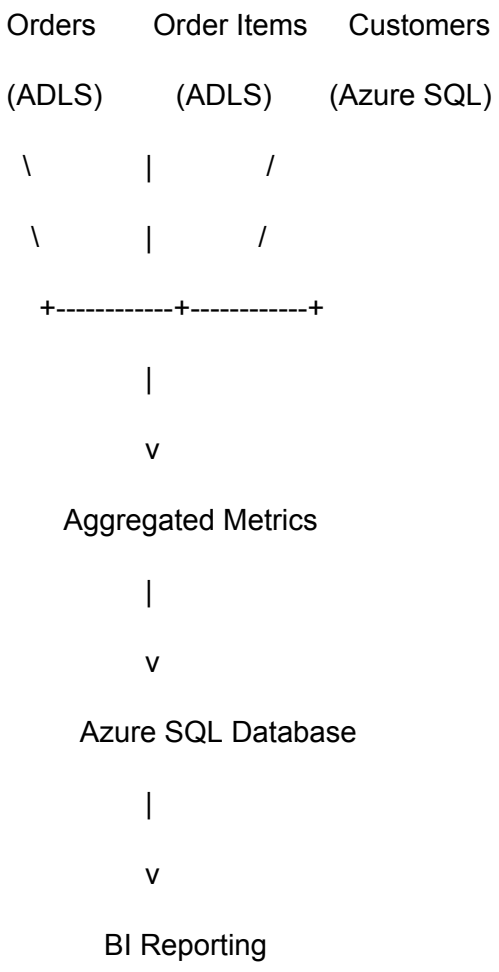
Core Concepts to Remember

1. **Storage Event Trigger** detects new files automatically.
2. **ADF** orchestrates the end-to-end workflow.
3. **Databricks** performs validation, transformation, joins, and aggregation.
4. **Azure Key Vault** protects sensitive credentials and secrets.
5. **Duplicate validation** ensures unique `order_id` values.
6. **Dynamic SQL lookup** validates permitted `order_status` values.
7. **Invalid files** are routed to `discarded/`.
8. **Valid files** are moved to `staging/`.
9. **AWS S3 + ADLS + Azure SQL** demonstrate multi-cloud data integration.
10. **Aggregated customer metrics** are written back to Azure SQL for reporting.
11. **ADF parameters** allow dynamic file names to flow from the event trigger into Databricks.
12. The project demonstrates how individual Azure data engineering services combine into a **production-oriented enterprise data pipeline**.

WEEK 25 — END-TO-END FLOW TO MEMORIZE




Duplicate Check
Status Validation
Azure SQL Lookup
Validation Result
INVALID
VALID
discarded/
staging/
Multi-Source Join



Final Concept

The key objective of Week 25 is to understand **how individual data engineering technologies work together as a complete enterprise solution**:

Azure Data Factory = Orchestration

ADLS Gen2 = Data Lake Storage

Azure Databricks = Processing & Analytics

Azure SQL Database = Relational Serving / Lookup Layer

AWS S3 = External Data Source

Azure Key Vault = Secret Management

Storage Event Trigger = Event-Driven Automation

Together, these components form an **end-to-end, event-driven, multi-cloud enterprise data pipeline**.

This version is formatted to paste directly into **Word/Google Docs** while keeping the technical material, SQL, PySpark, architecture, and revision sections intact.

WEEK 26: ENTERPRISE DATA MODELING, DIMENSIONAL DESIGN & PYSPARK SCD TYPE-2 IMPLEMENTATION

---

1. Fundamentals of Data Modeling & Database Paradigms

Data Modeling is the structural process of defining, organizing, and establishing relationships between data entities, integrity rules, and constraints to serve specific application access patterns.

OLTP vs. OLAP System Requirements

OLTP (Online Transactional Processing)	OLAP (Online Analytical Processing)
System Goal: High-speed row writes	**System Goal:** High-throughput reads
Structure: Normalized (1NF to 3NF)	**Structure:** Denormalized (Star Schema)
Data Scope: Current operational rows	**Data Scope:** Multi-year historical data
Query Profile: Low-latency CRUD operations	**Query Profile:** Aggregations & large scans

Consumer: End users & web applications **Consumer:** BI, Analysts, Data Engineers

Modeling for System Type & Data Consumers

1. **OLTP Systems (Database Design):** Optimized for operational consistency, rapid single-row inserts/updates, and strict transactional integrity. Data is normalized to remove redundancy.
2. **OLAP Systems (Data Warehouse Design):** Optimized for reading and aggregating massive historical datasets. Data is denormalized into **Fact** and **Dimension** structures to eliminate costly multi-table joins.
3. **Consumer Profiles:**
    - **Data Analysts:** Prefer flattened, low-complexity denormalized tables such as Star Schema or One Big Table for straightforward SQL querying without complex joins.
    - **Data Engineers:** Build ETL/ELT pipelines to extract normalized operational data, transform it, and load it into analytical structures.

---

2. Relational Normalization (OLTP Design)

Normalization breaks down large tables into smaller, related tables to minimize duplicate storage and eliminate data update anomalies.

Progressive Normal Forms

1NF (Atomicity) → 2NF (No Partial Dependencies) → 3NF (No Transitive Dependencies)

Normal Form Standards

Normal Form	Structural Criterion	Violated Example	Corrected Structure
1NF	• Values in every cell must be **atomic**.• Must have a **Primary Key**.•	`StudentID: 1Courses: Math, English, Music`	Split multi-valued cells into distinct rows per course: `(1, Math)`, `(1, English)`, `(1, Music)`.

	No repeating column groups.		
2NF	• Must be in **1NF**.• Every non-primary attribute must depend on the **entire composite candidate key**.• No partial dependencies.	Composite Key: `(CAND_ID, SUBJECT_NO)SUBJECT_FEE` depends only on `SUBJECT_NO`.	Separate into `Candidate_Subject(CAND_ID, SUBJECT_NO)` and `Subject_Fee(SUBJECT_NO, SUBJECT_FEE)`.
3NF	• Must be in **2NF**.• No **transitive dependencies** .• Non-key attributes must not depend on other non-key attributes.	`STUDENT_ID → COURSE → INSTRUCTOR → INSTRUCTOR_OFFICEINSTRUCTOR_OFFICE` depends on `INSTRUCTOR`, not `STUDENT_ID`.	Separate into `Course_Registration(STUDENT_ID, COURSE, INSTRUCTOR)` and `Instructor_Office(INSTRUCTOR, INSTRUCTOR_OFFICE)`.

Drawbacks of Redundancy in Operational Systems

- **Excess Storage Costs:** Repeating string attributes across millions of rows inflates storage footprints.
- **Update Anomalies:** Updating a customer's address requires modifying hundreds of duplicated rows, creating severe data inconsistencies if any updates fail mid-flight.

---

3. Dimensional Modeling (OLAP / Data Warehouse Design)

Formalized by **Ralph Kimball**, **Dimensional Modeling** organizes business processes into numeric measurements (**Facts**) and descriptive attributes (**Dimensions**).

Dimensional Modeling Architecture

Time Dimension

- `time_key (PK)`
- `day`
- `month`
- `year`

Customer Dimension

- `customer_key (PK)`
- `customer_name`
- `city`

Sales Fact

- `time_key (FK)`
- `customer_key (FK)`
- `product_key (FK)`
- `sales_amount (FACT)`
- `quantity (FACT)`

Product Dimension

- `product_key (PK)`
- `product_name`
- `brand`

The **Sales Fact** sits at the center and connects to the surrounding **Customer**, **Product**, and **Time** dimensions.

Core Concepts

Facts vs. Dimensions

Facts: Quantitative, numeric metrics representing actual operational transactions.

Examples:

- `sale_amount`
- `order_quantity`
- `unit_price`

Decimal/numeric data types usually indicate potential facts.

Dimensions: Descriptive context surrounding business events that answers questions such as:

- Who?
- What?
- Where?
- When?
- Why?

Examples:

- `customer_name`
- `store_location`
- `product_category`

String data types usually indicate potential dimensions.

Key Attributes of Fact Tables

1. Fact tables consist primarily of **Foreign Keys** linking to surrounding Dimension tables alongside numeric measurement columns.
2. Fact tables are usually the largest tables in a Data Warehouse, accumulating hundreds of millions to billions of transaction records over time.

---

4. Surrogate Keys vs. Natural / Business Keys

A **Surrogate Key** is an artificially generated integer assigned by the Data Warehouse to uniquely identify a record in a dimension table.

It is completely independent of the operational source system's primary key, also called the **Natural Key** or **Business Key**.

Example

Operational Source System

- `CustomerID: "CUST-998"`
- `Name: John Doe`

↓

Data Warehouse Dimension

- `customer_skey: 100012` — Integer Primary Key
- `customer_id: "CUST-998"` — Natural Key

Why Surrogate Keys Are Critical

1. **Enables Historical Tracking (SCD Type 2):** Primary keys cannot repeat. When a record changes, a surrogate key generates a new unique row for the updated state while retaining the original Natural Key across multiple rows.
2. **Insulates Against Source Schema Changes:** Prevents changes in operational database key formats, such as string IDs to UUIDs, from breaking analytical pipelines and join logic.
3. **Resolves Multi-Source Conflicts:** Prevents primary key collisions when ingesting data from multiple operational source systems that reuse identical numeric IDs.
4. **Optimizes Join Performance:** Joining tables on low-footprint `INTEGER` surrogate keys is significantly faster than joining on wide `VARCHAR` operational keys.

---

5. Schema Design Strategies

Star Schema vs. Snowflake Schema vs. One Big Table

Star Schema	Snowflake Schema	One Big Table (OBT)
Denormalized	Normalized	Completely flattened
Dimensions connect strictly to Fact	Dimensions link to other Dimensions	Single massive table
Fast queries, fewer joins	Slower queries due to complex joins	Zero joins required
Easy for business users	More normalized dimensional structure	High storage overhead

Simple analytical navigation	More complex analytical navigation	Columnar compression can reduce physical impact

5.1 Star Schema

A central Fact table surrounded by fully denormalized Dimension tables.

Dimensions do not connect to other dimensions.

Advantages:

- Highly performant.
- Requires fewer joins.
- Easy for business users to query.

5.2 Snowflake Schema

A variation of the Star Schema where Dimension tables are further normalized into sub-dimension tables.

Example:

`Product_Dimension → Category_Dimension → Department_Dimension`

Disadvantage:

Requires complex multi-level joins that can degrade query performance in analytical processing.

5.3 One Big Table (OBT)

Stores all factual measurements and descriptive dimensions inside a single, massive, denormalized table.

Advantages:

- Completely eliminates join operations.
- Can maximize query throughput on modern columnar storage engines such as Parquet/Delta.
- Simplifies pipeline maintenance.

Disadvantages:

- High data redundancy.

- Difficult row-level updates.

---

6. Four-Step Process for Dimensional Design

According to Ralph Kimball, every dimensional model should follow four sequential design steps:

1. Select Business Process → 2. Declare the Grain → 3. Identify Dimensions → 4. Identify Facts

Step 1: Select the Business Process

Identify the operational activity to model.

Example:

`Retail Point-of-Sale Transactions`

Step 2: Declare the Grain

Define what an individual row in the Fact table represents.

Example:

> One row per individual item line on a customer receipt.

This provides detailed product analytics.

If the grain is instead:

> One row per overall receipt.

Then visibility is limited to total order-level values.

Step 3: Identify the Dimensions

Define descriptive context attached to each event at the declared grain.

Examples:

- Customer
- Store
- Product

- Promotion
- Time

Step 4: Identify the Facts

Identify the numeric metrics generated by the event.

Examples:

- `Quantity_Sold`
- `Extended_Price`
- `Discount_Amount`
- `Net_Tax`

---

7. Slowly Changing Dimensions (SCD) Types

Dimensional attributes change over time, such as when a customer changes their residential city.

Slowly Changing Dimensions (SCD) define strategies for handling historical changes.

SCD Types Overview

SCD Type	Management Mechanics	Historical Track Record	Ideal Use Case
SCD Type 0	Attribute values are static and never modified after initial insert.	Immutable baseline.	Date of Birth, Place of Birth.
SCD Type 1	Overwrites the old attribute value with the newly incoming value.	Historical data is lost.	Correcting spelling mistakes, non-critical attributes.

SCD Type 2	Inserts a **new row** for the updated entity with a new Surrogate Key, setting `effective_date`, `end_date`, and an `active_flag`.	**Full history retained.**	**Industry Standard** for critical dimensions such as Customer address and Product pricing.
SCD Type 3	Adds a dedicated `previous_column` alongside `current_column` within the existing row.	Partial history — only tracks the immediate previous state.	Tracking immediate past job titles or previous sales territories.

SCD Type 0 — Fixed

Attributes never change.

Example:

- Date of Birth
- Place of Birth

SCD Type 1 — Overwrite

The current value overwrites the old value.

No history is retained.

SCD Type 2 — Full History

A new row is inserted whenever a tracked attribute changes.

The new row receives:

- A new surrogate key
- `effective_date`
- `end_date`
- `active_flag`

The previous row is expired.

Full historical tracking is retained.

SCD Type 3 — Partial History

The current and previous values are maintained as separate columns in the same row.

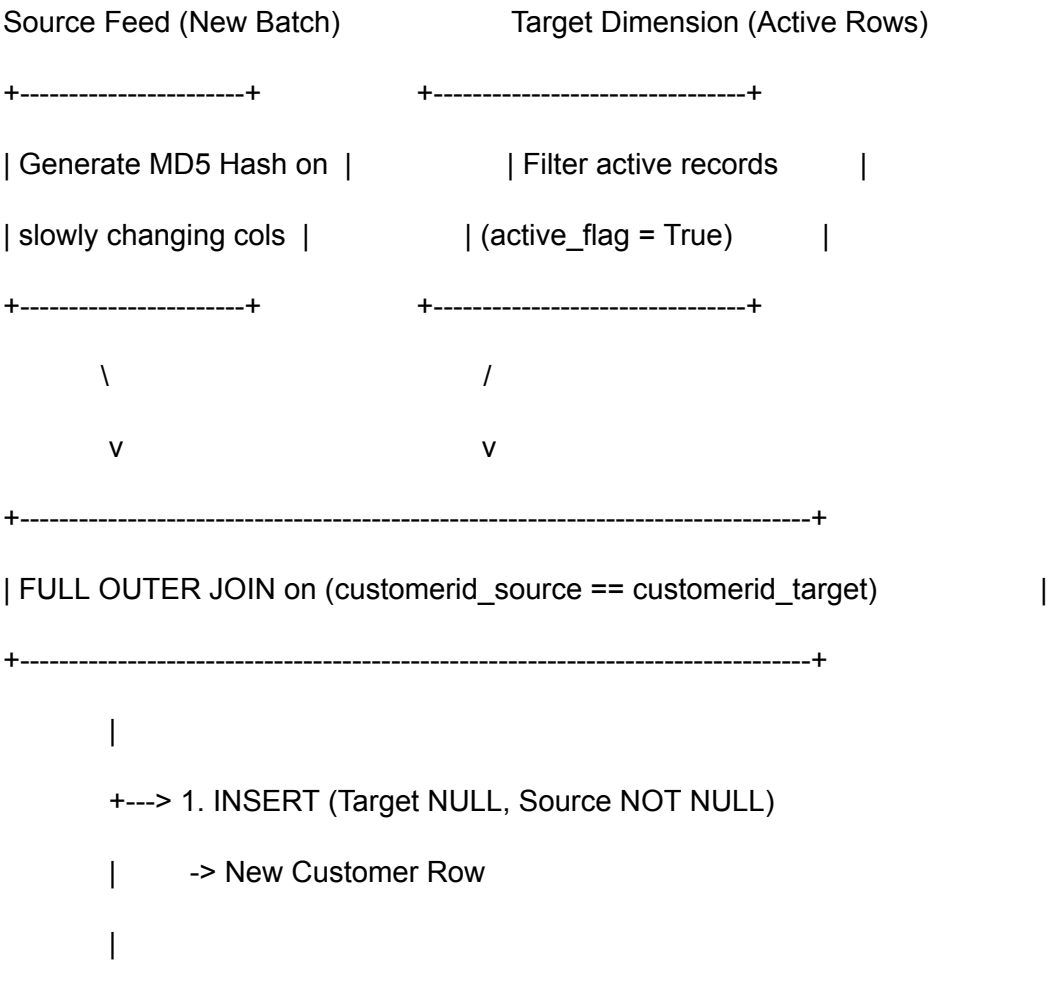
Only the immediate previous state is retained.

---

8. End-to-End PySpark Implementation of SCD Type 2

The standard pattern for building an SCD Type-2 pipeline in PySpark uses **MD5 hashes** to detect changes between source records and target dimensions.

PySpark SCD Type-2 Execution Flow



+---> 2. DELETE (Source NULL, Target NOT NULL)

| -> Invalidate Active Flag

|

+---> 3. UPDATE (Hashes Mismatch)

| -> Expire Old Row + Insert New Row

|

+---> 4. NO CHANGE (Hashes Match)

  -> Retain Active Record

---

9. PySpark Implementation Script

```
from pyspark.sql import SparkSession

from pyspark.sql.functions import (

    col, md5, concat_ws, lit, current_date, date_format,

    when, row_number, coalesce

)

from pyspark.sql.window import Window


spark = SparkSession.builder.appName("SCD_Type_2_Pipeline").getOrCreate()


#
==========================================================================
======

# 1. DEFINE SCHEMAS & LOAD SOURCE / TARGET DATASETS
```

```
#
================================================================================
======


source_path = "/user/itv005857/scd_demo/source"

target_path = "/user/itv005857/scd_demo/target"


source_schema = """
    customerid LONG, firstname STRING, lastname STRING, email STRING,
    phone STRING, address STRING, city STRING, state STRING, zipcode LONG
"""


target_schema = """
    customerid LONG, firstname STRING, lastname STRING, email STRING,
    phone STRING, address STRING, city STRING, state STRING, zipcode LONG,
    customer_skey LONG, effective_date DATE, end_date DATE, active_flag BOOLEAN
"""


# Read incoming operational source batch
source_df = spark.read.format("csv") \
    .option("header", "true") \
    .schema(source_schema) \
    .load(source_path)


# Read existing Data Warehouse Target Dimension
```

```
target_df = spark.read.format("csv") \
    .option("header", "true") \
    .schema(target_schema) \
    .load(target_path)


# ================================================================================
# 2. GENERATE MD5 HASHES FOR CHANGE DETECTION
# ================================================================================


slowly_changing_cols = [
    "firstname",
    "lastname",
    "email",
    "phone",
    "address",
    "city",
    "state",
    "zipcode"
]


def add_hash(df, cols, suffix):
    """Generates MD5 hash over key attributes and renames columns with a suffix."""
```

```
    hash_expr = md5(
        concat_ws(
            "||",
            *[
                coalesce(col(c).cast("string"), lit(""))
                for c in cols
            ]
        )
    )

    df_hashed = df.withColumn(
        f"hash_md5_{suffix}",
        hash_expr
    )

    for c in df.columns:
        df_hashed = df_hashed.withColumnRenamed(
            c,
            f"{c}_{suffix}"
        )

    return df_hashed

# Isolate active target records for comparison
```

```
active_target_df = target_df.filter(
    col("active_flag") == True
)

# Apply MD5 hashing and column suffixing
src_hashed = add_hash(
    source_df,
    slowly_changing_cols,
    "source"
)

tgt_hashed = add_hash(
    active_target_df,
    slowly_changing_cols +
    [
        "customer_skey",
        "effective_date",
        "end_date",
        "active_flag"
    ],
    "target"
)
```

```
#
================================================================================
======

# 3. FULL OUTER JOIN FOR DELTA IDENTIFICATION

#
================================================================================
======


joined_df = src_hashed.join(

    tgt_hashed,

    col("customerid_source") == col("customerid_target"),

    "full_outer"

)


# Categorize record state

delta_df = joined_df.withColumn(

    "action_flag",

    when(

        col("customerid_target").isNull(),

        lit("INSERT")

    )

    .when(

        col("customerid_source").isNull(),

        lit("DELETE")

    )

    .when(
```

```
        col("hash_md5_source") != col("hash_md5_target"),
        lit("UPDATE")
    )
    .otherwise(
        lit("NO_CHANGE")
    )
)

#
================================================================================
======
# 4. PROCESS RECORD ACTIONS
#
================================================================================
======

# Action 1: Unchanged Records
no_change_records = target_df.join(
    delta_df.filter(
        col("action_flag") == "NO_CHANGE"
    ),
    target_df.customer_skey ==
    delta_df.customer_skey_target
).select(
    target_df["*"]
)
```

```
# Action 2: Inactivated/Deleted Records
deleted_records = target_df.join(
    delta_df.filter(
        col("action_flag") == "DELETE"
    ),
    target_df.customer_skey ==
    delta_df.customer_skey_target
).select(
    target_df["*"]
).withColumn(
    "end_date",
    current_date()
).withColumn(
    "active_flag",
    lit(False)
)

# Action 3: Expired Historical Records from UPDATES
expired_history_records = target_df.join(
    delta_df.filter(
        col("action_flag") == "UPDATE"
    ),
    target_df.customer_skey ==
```

```
    delta_df.customer_skey_target
).select(
    target_df["*"]
).withColumn(
    "end_date",
    current_date()
).withColumn(
    "active_flag",
    lit(False)
)

# Action 4: New Insertions
# Includes both brand-new customers and updated rows
new_inserts = delta_df.filter(
    col("action_flag").isin(
        "INSERT",
        "UPDATE"
    )
).select(
    col("customerid_source").alias("customerid"),
    col("firstname_source").alias("firstname"),
    col("lastname_source").alias("lastname"),
    col("email_source").alias("email"),
    col("phone_source").alias("phone"),
```

```
    col("address_source").alias("address"),
    col("city_source").alias("city"),
    col("state_source").alias("state"),
    col("zipcode_source").alias("zipcode")
)

# Assign new surrogate keys and validity attributes
max_skey = target_df.agg(
    {"customer_skey": "max"}
).collect()[0][0] or 0

window_spec = Window.orderBy("customerid")

formatted_new_inserts = new_inserts \
    .withColumn(
        "customer_skey",
        max_skey + row_number().over(window_spec)
    ) \
    .withColumn(
        "effective_date",
        current_date()
    ) \
    .withColumn(
        "end_date",
```

```
        lit("9999-12-31").cast("date")
    ) \
    .withColumn(
        "active_flag",
        lit(True)
    )

#
================================================================================
======
# 5. UNION ALL STREAMS & WRITE BACK TO DIMENSION
#
================================================================================
======

inactive_target_history = target_df.filter(
    col("active_flag") == False
)

final_dimension_df = inactive_target_history \
    .union(no_change_records) \
    .union(deleted_records) \
    .union(expired_history_records) \
    .union(formatted_new_inserts)

final_dimension_df.write \
```

```
.mode("overwrite") \
.format("parquet") \
.save(target_path)
```

---

10. SCD Type-2 Processing Logic — Interview View

For an incoming source record, compare it against the currently active dimension record using the **Natural/Business Key**.

Source vs. Target Condition	Action
Source exists, Target does not	**INSERT** new dimension record
Source does not exist, Target exists	**DELETE / INACTIVATE** existing record
Both exist + Hashes differ	**UPDATE** → Expire old row + insert new row
Both exist + Hashes match	**NO CHANGE** → Retain active record

Key SCD Type-2 Columns

Column	Purpose

`customerid`	Natural / Business Key
`customer_skey`	Surrogate Key
`effective_date`	Date from which this version becomes valid
`end_date`	Date on which this version expires
`active_flag`	Identifies the currently active version
`hash_md5`	Detects changes in tracked attributes

Example

Suppose the source initially contains:

customerid	city	customer_skey	effective_date	end_date	active_flag
101	Bangalore	1001	2026-01-01	9999-12-31	TRUE

Later, the customer moves to Delhi.

The SCD Type-2 process:

Step 1 — Expire old record

customerid	city	customer_skey	effective_date	end_date	active_flag
101	Bangalore	1001	2026-01-01	2026-07-26	FALSE

Step 2 — Insert new version

customerid	city	customer_skey	effective_date	end_date	active_flag
101	Delhi	1002	2026-07-26	9999-12-31	TRUE

This preserves the complete customer history while ensuring only one version is active.

---

11. Week 26 Key Takeaways

Data Modeling

- **OLTP → Normalized → Transaction-focused**
- **OLAP → Denormalized → Analytics-focused**

Normalization

- **1NF:** Atomic values
- **2NF:** No partial dependency
- **3NF:** No transitive dependency

Dimensional Modeling

- **Fact:** Numeric measurements
- **Dimension:** Descriptive context
- **Surrogate Key:** Warehouse-generated identifier
- **Natural Key:** Source/business identifier

Schema Types

- **Star Schema:** Denormalized dimensions + central fact
- **Snowflake Schema:** Normalized dimensions
- **OBT:** Everything flattened into one large table

Kimball Design Process

Business Process → Grain → Dimensions → Facts

SCD

- **Type 0:** Never change
- **Type 1:** Overwrite
- **Type 2:** Full historical tracking
- **Type 3:** Current + previous value

SCD Type 2 Implementation

Source → Hash → Compare Active Target → Full Outer Join → Identify Action → Expire / Insert → Write Dimension

---

MASTER CURRICULUM ROADMAP — WEEKS 1–26

==========

MASTER BIG DATA & AZURE DATA ENGINEERING CURRICULUM

==========

[WEEKS 1 - 16: DISTRIBUTED STORAGE & COMPUTE CORE]

• HDFS Architecture, Block Distribution, Replication, High Availability & YARN Schedulers

• MapReduce Concepts & Apache Spark Fundamentals

• RDDs, Transformations vs. Actions

• DataFrames API, Spark SQL, Catalyst Optimizer & Tungsten Execution Engine

• Memory Management, Partitioning vs. Bucketing

• Join Optimization & Partition Skew

• Performance Tuning: AQE, Z-Ordering, Salting

• Driver/Executor Resource Estimation

• CI/CD Pipelines, Git, Pytest & Production Incident Debugging

[WEEKS 17 - 21: AZURE CLOUD ARCHITECTURE & DATA GOVERNANCE]

• Cloud Infrastructure: Regions, Availability Zones, Fault/Update Domains

• CapEx vs. OpEx & Cloud Service Models

• Azure Storage: Blob, ADLS Gen2, Access Tiers

• LRS / ZRS / GRS Redundancy Models

• Azure Databricks Architecture: Control Plane vs. Data Plane

• Compute Clusters, DBFS & dbutils

• Delta Lake: ACID Logs, Time Travel, Schema Enforcement & Evolution

• Lakehouse Architecture

• Delta Engine Optimization

• Medallion Architecture: Bronze → Silver → Gold

• Change Data Feed (CDF)

• Unity Catalog: Centralized Metastore

• Three-Level Namespace

• ANSI SQL Access Policies

• SCIM User Provisioning

[WEEKS 22 - 25: PIPELINE ORCHESTRATION & CLOUD INTEGRATION]

• Azure Data Factory

• Copy Activity

• Pipeline Parameterization

• Mapping Data Flows

• Triggers: Schedule, Tumbling Window & Storage Event

• Lookup & ForEach Looping

• Production Control Flow

• Validation, Get Metadata, Fail & If Condition

• Azure Key Vault Integration

• Azure Synapse Analytics

• Serverless SQL Pools

• Dedicated SQL Pools & MPP Architecture

• 60 Data Distributions

• Round-Robin, Hash & Replicated Distribution

• PolyBase & COPY Ingestion

• Spark Pool Integration

• Enterprise Capstone Project

• Event-Driven Pipeline

• Azure SQL Integration

• Key Vault Security

• AWS S3 Integration

[WEEK 26: ENTERPRISE DATA MODELING & DATA WAREHOUSING]

• Relational Normalization: 1NF, 2NF & 3NF

• Data Anomalies & Redundancy Prevention

• Dimensional Modeling

• Kimball Methodology

• Business Process, Grain, Dimensions & Facts

• Star Schema vs. Snowflake Schema vs. One Big Table

• Surrogate Keys vs. Natural / Business Keys

• Slowly Changing Dimensions: SCD 0, 1, 2 & 3

• Full Historical Tracking with SCD Type 2

• MD5 Hash-Based Change Detection

• PySpark SCD Type-2 Implementation

• Full Outer Join for Delta Identification

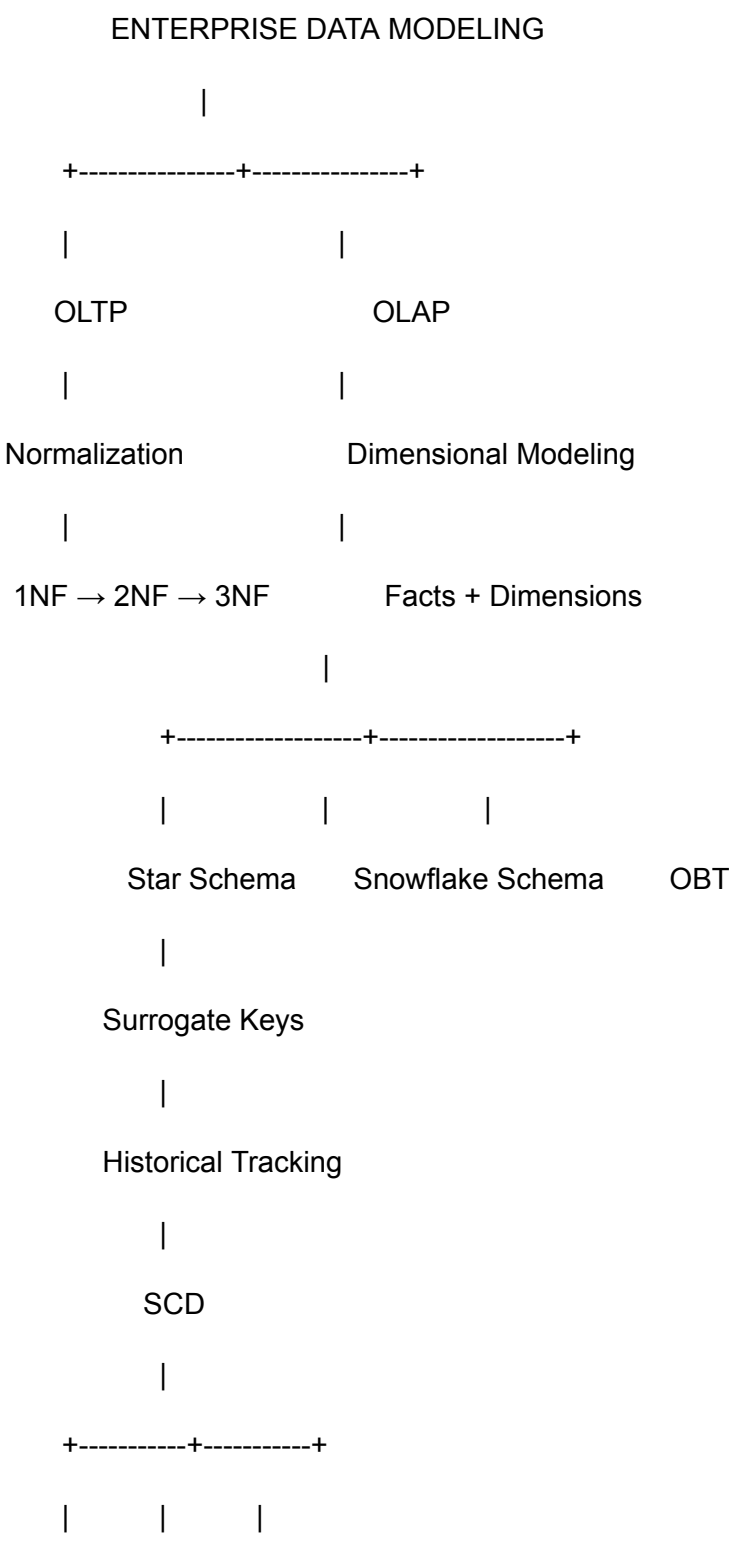
• Record Action Classification

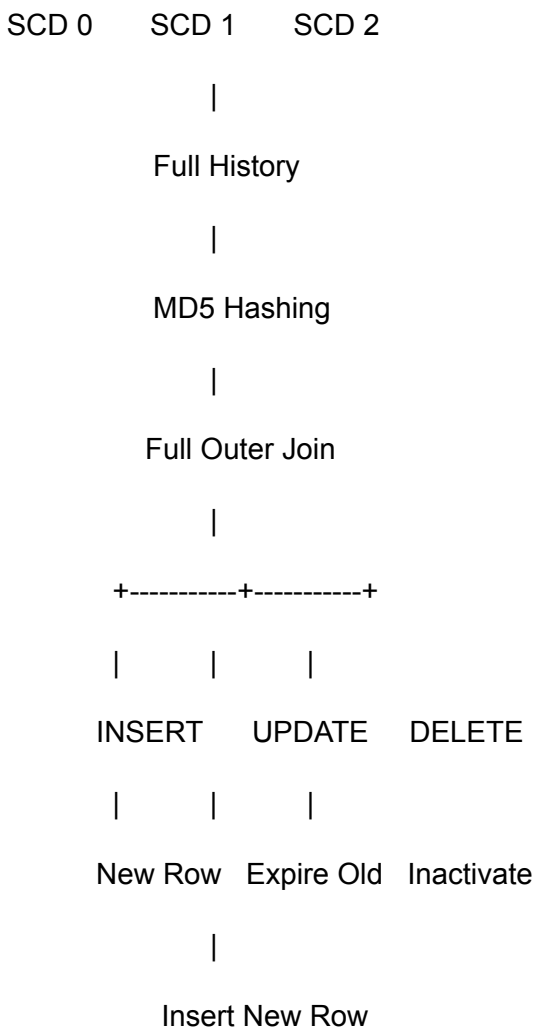
• Surrogate Key Generation

• Effective / End Dates & Active Flags

==========
==========

WEEK 26 — FINAL REVISION FLOW





Week 26 Interview Formula

OLTP = Normalize for transactions
OLAP = Denormalize for analytics
Fact = Measures
Dimension = Context
Surrogate Key = Warehouse identity
SCD Type 2 = Preserve full history
MD5 Hash = Detect attribute changes
Full Outer Join = Identify inserts, deletes, updates and unchanged records

---

WEEK 27: COMPREHENSIVE INTERVIEW GUIDE — AZURE STORAGE & AZURE DATA FACTORY

---

1. AZURE STORAGE SERVICES: TECHNICAL Q&A & INTERVIEW FRAMEWORK

Q1. What are the primary storage services offered by Microsoft Azure, and how do you choose between Azure Blob Storage and ADLS Gen2?

Azure offers four core storage abstractions:

1. **Azure Blob / ADLS Gen2:** Unstructured and semi-structured object storage.
2. **Azure Files:** Fully managed cloud file shares using SMB/NFS protocols.
3. **Azure Queues:** Asynchronous message queuing for workflow decoupling.
4. **Azure Tables:** NoSQL key-value store for semi-structured datasets.

Azure Blob Storage vs. Azure Data Lake Storage Gen2

Azure Blob Storage	Azure Data Lake Storage Gen2
Object storage engine	Enterprise analytics engine
Flat Namespace	Hierarchical Namespace (HNS)
Ideal for raw media and logs	Ideal for enterprise analytics
Slashes are part of blob names	Real directory structure

Limited directory-level operations	Atomic directory-level renames and moves
Container/blob-level access controls	POSIX-style file and directory ACLs

Core Differences & Architectural Selection

Metric / Feature	Azure Blob Storage	Azure Data Lake Storage Gen2 (ADLS Gen2)
Namespace	**Flat Namespace** — no physical subdirectories; slashes are part of the blob name.	**Hierarchical Namespace (HNS)** — real physical directory trees.
Performance	Pseudo-folder operations can require scanning nested blobs.	Directory-level rename, move, and metadata operations are optimized through the hierarchical namespace.
Security	Container and blob-level access controls.	POSIX-style file and folder-level **Access Control Lists (ACLs)**.
Primary Use Cases	Images, documents, media streaming, logs, backups, and disaster recovery.	Big Data analytics platforms such as Spark, Databricks, and Synapse processing large analytical datasets.

Interview Shortcut

Blob Storage = General-purpose object storage
ADLS Gen2 = Blob Storage + Hierarchical Namespace + Analytics-oriented filesystem capabilities

---

Q2. What key technical and architectural factors influence Azure Storage costs?

Storage costs depend on six primary dimensions:

1. Region / Geographic Location

Data-center infrastructure costs vary globally. Choosing a region closer to the workload generally reduces latency, while certain high-demand regions may have higher pricing.

2. Redundancy Level

Increasing replication generally increases storage cost:

LRS → ZRS → GRS → RA-GZRS

3. Account Performance Tier

Standard Tier

- Backed by HDD-based storage.
- Cost-effective for general-purpose storage.

Premium Tier

- Backed by SSD-based storage.
- Provides lower latency and higher IOPS/throughput for demanding workloads.

4. Access Tiers

Azure Storage provides different access tiers to balance storage and retrieval costs:

Hot → Cool → Cold → Archive

Generally:

More frequently accessed data → warmer tier
Less frequently accessed data → colder tier

5. Data Transfer / Egress

Data transferred into Azure is generally free, while data transferred out of Azure or across certain boundaries can incur bandwidth charges.

6. Transaction Density

Storage costs are also affected by the number of operations performed:

- Read
- Write
- List
- Delete

Q3. How do you transition data across Access Tiers automatically, and how do you monitor expenditure?

Automated Data Lifecycle Management

Azure Storage Lifecycle Management can automatically transition blobs to colder access tiers or delete them based on rules such as:

- Days since last modification
- Days since last access
- Blob age
- Blob prefix/path
- Blob type

Cost Tracking

Use:

Azure Cost Management + Billing → Cost Analysis

Costs can be filtered by:

- Storage account
- Resource
- Resource group
- Tags
- Subscription

Interview Answer

> "I would use Storage Lifecycle Management to automatically move infrequently accessed data from Hot to Cool/Cold/Archive tiers and use Azure Cost Management's Cost Analysis to monitor storage and transaction expenditure."

Q4. Compare the different authentication and authorization mechanisms for Azure Storage.

Azure Storage supports several security models:

Authentication / Authorization Model	Description	Typical Use
Account Access Keys	Full access to the storage account using symmetric keys.	Administrative or legacy applications
SAS Tokens	Time-limited and permission-scoped access.	Temporary third-party access
Microsoft Entra ID / RBAC	Identity-based, granular authorization.	Enterprise applications and users

1. Account Access Keys

Azure Storage provides two account access keys:

- Key 1
- Key 2

This allows key rotation without requiring service downtime.

Because access keys provide broad access, they should be protected using **Azure Key Vault** rather than hardcoded into applications.

2. Shared Access Signature (SAS)

SAS provides temporary, scoped access to storage resources.

Permissions can include:

- Read
- Write
- Delete

SAS is particularly useful when a third-party application needs temporary access without receiving the storage account's master access keys.

3. Microsoft Entra ID / RBAC

Identity-based access can be granted to:

- Users
- Groups
- Applications
- Service Principals

Example roles include:

- **Storage Blob Data Reader**
- **Storage Blob Data Contributor**

Interview Shortcut

> **Access Key = Broad account access**
> **SAS = Temporary scoped access**
> **RBAC = Identity-based granular access**

Q5. How do you guarantee Data Protection and Data Security at Rest in Azure Storage?

Data Protection — Accidental Deletion

Use **Soft Delete** for blobs and containers.

Soft Delete retains deleted data for a configurable retention period, allowing accidental deletions to be recovered.

Encryption at Rest

Azure Storage encrypts data at rest using **256-bit AES encryption**.

Two major key-management approaches are:

Microsoft-Managed Keys (MMK)

- Azure manages the encryption keys.
- Default encryption model.
- Minimal operational overhead.

Customer-Managed Keys (CMK)

- Enterprise manages encryption keys.
- Keys can be stored in **Azure Key Vault**.
- Provides greater control over key lifecycle and rotation.

---

2. AZURE DATA FACTORY (ADF): CORE CONCEPTS & ORCHESTRATION Q&A

Q1. What is Azure Data Factory, and what core problems does it solve?

Azure Data Factory (ADF) is a managed, serverless cloud data integration and orchestration service used to build automated ETL/ELT pipelines.

Core Capabilities

Hybrid Data Ingestion

Copies data across:

- On-premises systems
- Cloud storage
- SaaS platforms
- Databases

using a large set of native connectors.

Code-Free Transformations

Mapping Data Flows provide visual transformation capabilities executed on managed scale-out Spark infrastructure.

Workflow Orchestration

ADF manages:

- Pipeline dependencies
- Event triggers
- Scheduling
- Retries
- Conditional execution
- Failure handling
- Alerts

Interview Answer

> "Azure Data Factory is primarily an integration and orchestration service. I would use it to ingest data from different sources, coordinate transformations, manage dependencies and triggers, and monitor production workflows."

Q2. What are the fundamental components required to move data from Azure SQL DB to ADLS Gen2?

A standard ADF data movement pipeline consists of:

Source → Linked Service → Dataset → Copy Activity → Dataset → Linked Service → Sink

1. Linked Services

Typically two Linked Services are required:

- Azure SQL Database Linked Service
- ADLS Gen2 Linked Service

A Linked Service stores connection information and authentication configuration.

2. Datasets

Typically two datasets are configured:

- Source dataset representing the SQL table.
- Sink dataset representing the ADLS storage path and file format.

3. Pipeline

A Pipeline is the logical workflow containing activities.

4. Copy Data Activity

The **Copy Data Activity** performs high-throughput data movement from the source to the sink.

Simplified Flow

Azure SQL DB

|

v

Source Linked Service

|

v

Source Dataset

|

v

Copy Data Activity

|

v

Sink Dataset

|

v

ADLS Gen2 Linked Service

|

v

ADLS Gen2

Q3. List key ADF control flow activities used in production pipelines.

Copy Data

Transfers data between source and sink data stores.

Mapping Data Flow

Provides visual, code-free transformation logic executed on Spark-based infrastructure.

Databricks Notebook

Triggers Spark/PySpark processing inside an Azure Databricks workspace.

Get Metadata

Retrieves metadata such as:

- File existence
- Directory contents
- Child items
- Last-modified attributes
- File size

Lookup

Retrieves configuration or lookup records from sources such as:

- SQL tables
- JSON files
- Configuration datasets

ForEach

Iterates through an array returned by activities such as:

- Lookup
- Get Metadata

If Condition

Evaluates expressions and branches pipeline execution based on the result.

Validation

Waits for required conditions to become true before allowing the pipeline to continue.

Q4. Compare ADF Trigger Models: Schedule Triggers vs. Tumbling Window Triggers

Capability	Schedule Trigger	Tumbling Window Trigger
Execution Pattern	Clock/calendar-based execution at designated times.	Fixed, non-overlapping, contiguous time windows.
Example	Every day at 05:00 UTC.	Every 30 minutes.
Pipeline Relationship	Many-to-Many.	One-to-One.
Historical Backfill	Requires manual rerun/parameter handling.	Native historical backfill support.
Dependencies	Cannot natively wait for another schedule trigger.	Supports trigger dependencies and window chaining.
Best Use Case	Regular calendar-based jobs.	Time-windowed processing and dependency-aware workloads.

Schedule Trigger

Use when execution is based on a calendar:

Every day

↓

05:00 UTC

↓

Run Pipeline

Tumbling Window Trigger

Use when processing is logically divided into fixed time windows:

00:00 ——— 00:30 ——— 01:00 ——— 01:30

Window 1 Window 2 Window 3

Tumbling Window Triggers are particularly useful when historical windows need to be backfilled or when execution dependencies exist between windows.

Debug Mode vs. Trigger Execution

Debug execution manually runs a pipeline from the ADF interface for development and testing.

It does **not** require invoking the production trigger.

Interview Shortcut

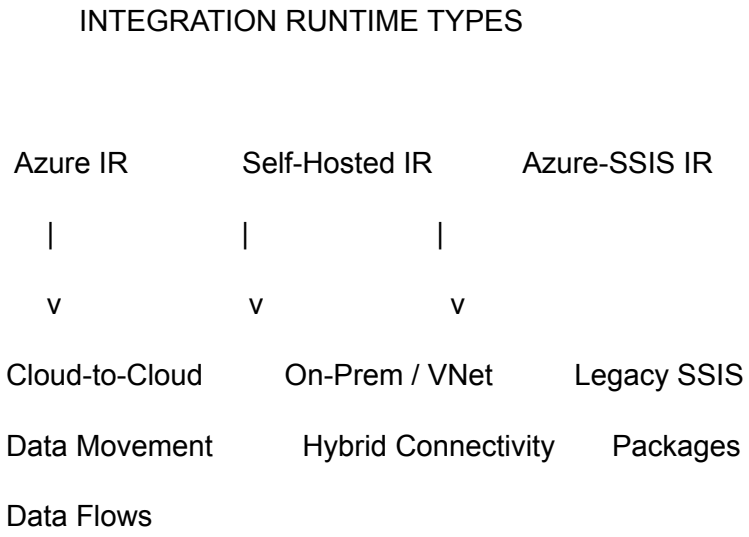
> **Schedule = "Run at this time."**
> **Tumbling Window = "Process this specific time window."**

---

3. INTEGRATION RUNTIMES (IR) & PERFORMANCE OPTIMIZATION

An **Integration Runtime (IR)** provides the compute infrastructure and connectivity used by Azure Data Factory to execute activities and move/process data.

ADF commonly uses three Integration Runtime types:



3.1 Azure Integration Runtime

Scope

Fully managed cloud compute used for:

- Cloud-to-cloud data movement
- Mapping Data Flows
- Other supported ADF execution workloads

Region Resolution

Azure IR can automatically select an appropriate region based on the connected data stores and workload requirements.

Explicit Region Selection

An Azure IR can also be created in a specific region where required for:

- Data residency

- Compliance
- Data sovereignty
- Network architecture

3.2 Self-Hosted Integration Runtime (SHIR)

A **Self-Hosted Integration Runtime** is installed inside a controlled environment such as:

- On-premises network
- Private network
- Virtual Network environment

It is useful when ADF needs secure connectivity to systems behind firewalls.

Typical Architecture

On-Premises Database

|

v

Self-Hosted IR

|

v

Azure Data Factory

|

v

Azure Services

Sharing SHIR

A Self-Hosted IR can be shared across multiple Data Factory environments when appropriately configured.

3.3 Azure-SSIS Integration Runtime

Azure-SSIS IR provides managed infrastructure for executing existing **SQL Server Integration Services (SSIS)** packages in Azure.

Primary Use Case

> **Legacy SSIS migration to Azure**

---

4. TUNING SLOW ADF COPY ACTIVITIES

When a Copy Activity performs poorly, investigate both network placement and parallelism.

1. Colocate Integration Runtime

Where appropriate, deploy the Azure IR in a region close to the relevant data store, especially the sink, to minimize network latency and cross-region transfer overhead.

---

2. Adjust Data Integration Units (DIUs)

Data Integration Units (DIUs) represent a measure of compute capability available to ADF Copy Activity.

Increasing DIUs can provide more resources and parallelism for eligible copy workloads.

Examples include:

- 16 DIUs
- 32 DIUs
- 64 DIUs

The correct value depends on the source, sink, data volume, and workload characteristics.

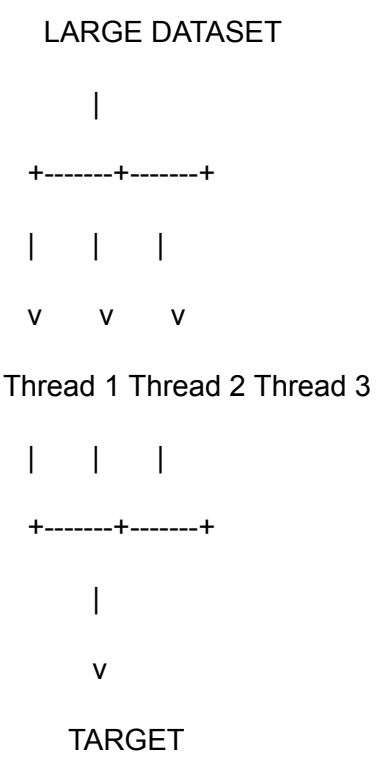
---

3. Enable Parallel Copies

Increase copy parallelism when the workload supports it.

The `degreeOfCopyParallelism` setting can be used to control parallel copy execution.

Conceptually:



Performance-Tuning Checklist

When Copy Activity is slow, check:

1. **Source throughput**
2. **Sink throughput**
3. **Integration Runtime location**
4. **DIU allocation**
5. **Copy parallelism**
6. **File sizes and number of files**
7. **Network connectivity**
8. **Source/sink throttling**

---

5. WEEK 27 INTERVIEW RAPID REVISION

Topic	One-Line Interview Answer
Blob Storage	General-purpose object storage using a flat namespace.
ADLS Gen2	Analytics-oriented storage built on Blob Storage with Hierarchical Namespace and ACL support.
Access Keys	Broad storage-account-level credentials with two keys available for rotation.
SAS	Time-bound, permission-scoped access to storage resources.
RBAC	Identity-based authorization using Microsoft Entra ID roles.
Soft Delete	Protects against accidental deletion by retaining deleted data for recovery.
MMK	Microsoft-managed encryption keys.
CMK	Customer-managed encryption keys, typically managed through Key Vault.
ADF	Cloud data integration and orchestration service.
Linked Service	Stores connection/authentication information for a data store.
Dataset	Represents the structure/location of data used by an activity.

Pipeline	Logical grouping of activities implementing a workflow.
Copy Activity	Moves data from a source to a sink.
Lookup	Retrieves configuration or lookup data.
ForEach	Iterates through an array of items.
Get Metadata	Retrieves metadata about files/folders/data sources.
Schedule Trigger	Executes pipelines according to calendar/time schedules.
Tumbling Window Trigger	Executes pipelines for fixed, non-overlapping time windows with dependency/backfill support.
Azure IR	Managed cloud execution environment for ADF workloads.
SHIR	Integration Runtime for secure hybrid/on-premises connectivity.
Azure-SSIS IR	Managed runtime for running SSIS packages in Azure.
DIU	Unit representing compute capability allocated to Copy Activity.
Parallel Copy	Uses multiple concurrent threads/operations to improve data movement throughput.

---

6. HIGH-VALUE SCENARIO QUESTIONS

Scenario 1: You have an on-premises SQL Server and need to load data into ADLS Gen2. Which IR would you use?

Answer: Self-Hosted Integration Runtime (SHIR).

Reason:

The source is inside an on-premises/private environment and may not be directly accessible from Azure-managed infrastructure.

On-Prem SQL Server

|

v

Self-Hosted IR

|

v

ADF

|

v

ADLS Gen2

Scenario 2: You need to allow an external vendor to upload files temporarily to Azure Storage. Would you provide the account access key?

Answer: No.

Use a **SAS token** with:

- Limited permissions
- Limited scope
- Limited validity period

This follows the principle of least privilege.

Scenario 3: Your organization needs Spark/Databricks/Synapse to process petabytes of analytical data. Blob Storage or ADLS Gen2?

Answer: ADLS Gen2.

The hierarchical namespace and filesystem-oriented access controls make it better suited for large-scale analytical workloads.

Scenario 4: You need a pipeline to execute every day at 2 AM.

Answer: Schedule Trigger.

The requirement is based on a fixed calendar schedule.

Scenario 5: You need to process every 30-minute interval and backfill missing historical intervals.

Answer: Tumbling Window Trigger.

It is designed around fixed time windows and supports historical backfill and dependency management.

Scenario 6: Your ADF Copy Activity is slow. What would you investigate?

A strong interview response:

> "I would first identify whether the bottleneck is the source, sink, network, or Integration Runtime. Then I would check IR region placement, increase DIUs where appropriate, enable copy parallelism, and verify whether source or sink throttling is limiting throughput."

---

7. 27-WEEK MASTER CURRICULUM ARCHITECTURE ROADMAP

==========

ULTIMATE BIG DATA & AZURE DATA ENGINEERING MASTER GUIDE

==========

[WEEKS 1 - 7: DISTRIBUTED STORAGE & COMPUTE CORE]

• HDFS Architecture, NameNode/DataNode Internals, YARN Schedulers & MapReduce Mechanics

• Apache Spark Foundations: RDD Abstractions, Transformations vs. Actions, DAG Execution

[WEEKS 8 - 16: PYSPARK OPTIMIZATION, MEMORY TUNING & RESOURCE SIZING]

• DataFrames API, Catalyst Optimizer, Tungsten Engine & Memory Pool Architecture

• Partitioning vs. Bucketing

• Join Strategies: Broadcast, Shuffle Hash, Sort-Merge

• Resource Sizing Frameworks

• Partition Skew Strategies

• Adaptive Query Execution (AQE)

• Salting

• CI/CD Testing

[WEEKS 17 - 21: AZURE CLOUD INFRASTRUCTURE & DATABRICKS GOVERNANCE]

• Azure Global Infrastructure

• Availability Zones

• Fault/Update Domains

• CapEx vs. OpEx

• Azure Storage Services

• Access Tiers

• LRS/ZRS/GRS Redundancy Models

• ADLS Gen2 Hierarchical Namespace

• Azure Databricks Architecture

• Control Plane vs. Data Plane

• DBFS

• Secret Scopes

• Delta Lake ACID Transactions

• Databricks Lakehouse Architecture

• Medallion Pattern: Bronze → Silver → Gold

• Delta Engine Optimization

• Unity Catalog

• Three-Level Namespace

• Centralized Metastore

• SCIM Provisioning

• ANSI SQL ACLs

[WEEKS 22 - 25: PIPELINE ORCHESTRATION & END-TO-END ENTERPRISE PROJECT]

• Azure Data Factory

- Copy Activity
- Parameterization
- Mapping Data Flows
- Lookup
- ForEach
- Pipeline Triggers

• Azure Synapse Analytics

- Serverless SQL Pools
- Dedicated SQL Pool / MPP Architecture
- Spark Pools

• Production Capstone Project

- Event-Driven Pipeline
- Azure Key Vault
- Azure SQL
- AWS S3 Integration
- Databricks Processing

[WEEK 26: ENTERPRISE DATA MODELING & DATA WAREHOUSING]

• Relational Normalization
- 1NF
- 2NF
- 3NF

• Kimball Dimensional Design
- Business Process
- Grain
- Dimensions
- Facts
- Star Schema

• Surrogate Keys
• Natural / Business Keys
• Slowly Changing Dimensions

- SCD Type 0
- SCD Type 1
- SCD Type 2
- SCD Type 3

• PySpark SCD Type-2 Implementation

• MD5 Hash-Based Change Detection

[WEEK 27: PRODUCTION TECHNICAL Q&A — AZURE STORAGE & ADF]

• Azure Blob Storage vs. ADLS Gen2

• Hierarchical Namespace (HNS)

• Storage Lifecycle Management

• Access Keys

• SAS Tokens

• Microsoft Entra ID / RBAC

• Soft Delete

• Encryption at Rest

• Microsoft-Managed Keys (MMK)

• Customer-Managed Keys (CMK)

• Azure Key Vault

• ADF Ingestion Architecture

• Linked Services

• Datasets

• Pipelines

• Copy Activity

• Mapping Data Flows

• Lookup

• ForEach

• Get Metadata

• If Condition

• Validation

• Schedule Triggers

• Tumbling Window Triggers

• Azure Integration Runtime

• Self-Hosted Integration Runtime (SHIR)

• Azure-SSIS Integration Runtime

• Data Integration Units (DIUs)

• Copy Activity Performance Tuning

• Parallel Copy Execution

==========

MASTER GUIDE — WEEKS 1 TO 27

==========

WEEK 27 — FINAL INTERVIEW TAKEAWAYS

Remember These 10 Points

1. Blob vs ADLS Gen2

Blob = object storage
ADLS Gen2 = analytics-focused storage with HNS + ACLs

2. Access Key vs SAS vs RBAC

Access Key = broad access
SAS = temporary scoped access
RBAC = identity-based access

3. Soft Delete

Protects against accidental deletion.

4. Key Vault

Centralized secure management of sensitive credentials and encryption keys.

5. ADF Linked Service

Connection/authentication information.

6. ADF Dataset

Defines/points to the data structure and location used by activities.

7. ADF Pipeline

Groups activities into a workflow.

8. Schedule vs Tumbling Window

Schedule = calendar-based execution
Tumbling Window = fixed time-window processing + backfill/dependencies

9. IR Types

Azure IR = cloud
SHIR = on-prem/private network
Azure-SSIS IR = SSIS workloads

10. Copy Activity Performance

Check source/sink → IR location → DIUs → parallelism → throttling

---

MASTER STUDY GUIDE STATUS

Weeks 1–27 completed.

The curriculum now spans:

Distributed Systems → Hadoop → Spark → PySpark → Performance Tuning → Azure Fundamentals → Azure Storage → Databricks → Delta Lake → Unity Catalog → ADF → Synapse → Enterprise Pipelines → Data Modeling → SCD Type 2 → Azure Storage & ADF Interview Preparation

The next stage should focus on **scenario-based interview questions, architecture design, hands-on coding, and production troubleshooting** rather than only theoretical revision.

Weeks 28 & 29: Spark Structured Streaming — Micro-Batch Engine, Triggers, Watermarking & Stateful Processing

---

1. Batch vs. Stream Processing Paradigms

Data processing engines handle data using two fundamental paradigms depending on **volume, arrival frequency, and latency requirements**.

Processing Paradigm Trade-Offs

Batch Processing	Stream Processing

Processing on bounded, historical datasets at rest	Continuous processing on unbounded streams in motion
Latency: Minutes, Hours, Days	Latency: Seconds, Sub-seconds
High throughput, bulk groups	Immediate insights, incremental processing
Ideal for BI and standard ETL	Ideal for fraud detection and monitoring

Comparative Analysis

Feature	Batch Processing	Stream / Real-Time Processing
Data Scope	Bounded historical files collected over a specific time window	Unbounded continuous data streams arriving dynamically
Data Latency	High tolerance for delays — minutes to days	Strict low-latency demands — seconds to sub-seconds
Computation Model	Scheduled jobs process entire datasets at rest as a single block	Continuous processing on incremental micro-batches or individual events
Primary Use Cases	Financial reporting, end-of-day reconciliation, historical ML training	Real-time credit card fraud detection, website log monitoring, IoT sensor alerts

Hadoop MapReduce vs. Apache Spark

- **Hadoop MapReduce:** Exclusively designed for disk-bound batch processing; cannot handle real-time streaming data natively.

- **Apache Spark:** A unified general-purpose engine supporting both batch workflows and real-time streaming through its **Structured Streaming** framework.

---

2. Spark Structured Streaming Fundamentals & Architecture

Spark Structured Streaming is a scalable, fault-tolerant stream processing engine built on top of the Spark SQL Catalyst engine.

It processes continuous streams as an **unbounded table** that continuously appends new incoming records.

Unbounded Table Conceptual Model

Stream Input

|

v

[Record 1 | Record 2 | Record 3 | Record N...]

| (Appended continuously)

v

[Continuous Execution Engine]

|

v

[Result Table (Incremental)]

|

v

Append Mode / Update Mode / Complete Mode

|

v

Target Sink

Architectural APIs: Legacy DStreams vs. Structured Streaming

1. DStreams API — Spark Streaming

- Low-level RDD-based streaming abstraction.
- Does not provide Catalyst SQL optimizations.
- Does not provide the typed DataFrame-based programming model available in Structured Streaming.

2. Structured Streaming API

- High-level DataFrame/SQL API.
- Provides a unified programming model.
- Batch and streaming applications can share the same transformation logic.

Key Engineering Challenges in Streaming

When operating micro-batch processing over small intervals, standard frameworks encounter four core obstacles:

1. **Backpressure:** System performance degrades if incoming batch frequencies outpace the job's processing throughput.
2. **Late-Arriving Records:** Network delays can cause events generated at 10:08 AM to physically arrive at 10:20 AM.
3. **State Management:** Running aggregations, such as historical user totals, must be tracked across continuous micro-batches.
4. **Fault Tolerance:** Intermediate state must be recovered without losing data or creating duplicates after application crashes.

> Spark Structured Streaming automates state persistence, micro-batch looping, and checkpoint recovery so developers can focus primarily on business logic.

---

3. Streaming Data Pipeline Lifecycle: Read, Process, Write

A Spark Structured Streaming application follows a three-stage pipeline:

DataStreamReader (.readStream)

|

v

DataFrame Transformations

|

v

DataStreamWriter (.writeStream)

Complete Example

```
# 1. READ: DataStreamReader
from pyspark.sql import SparkSession
from pyspark.sql.functions import explode, split, col

spark = SparkSession.builder \
    .appName("StreamingWordCount") \
    .config("spark.sql.shuffle.partitions", 3) \
    .getOrCreate()

lines = spark.readStream \
    .format("socket") \
    .option("host", "localhost") \
    .option("port", 9999) \
    .load()
```

```
# 2. PROCESS: Standard DataFrame Transformations
words = lines.select(explode(split(col("value"), " ")).alias("word"))

word_counts = words.groupBy("word").count()

# 3. WRITE: DataStreamWriter with Checkpointing
query = word_counts.writeStream \
    .outputMode("complete") \
    .format("console") \
    .option("checkpointLocation", "/path/to/checkpoint/dir") \
    .start()

query.awaitTermination()
```

Streaming Input Sources

File Source

Reads new files landing in a target directory as a continuous stream.

Supported formats include:

- CSV
- JSON
- ORC
- Parquet
- Text

Files are processed according to modification time, or in reverse order when `latestFirst=true`.

Kafka Source

Reads continuous streaming records directly from **Apache Kafka topics**.

Socket Source

Reads UTF-8 text lines from a network socket using `host:port`.

Primarily designed for **testing and prototyping** because it lacks end-to-end replayability.

---

4. Output Modes

Spark Structured Streaming provides three output modes controlling how result data is written to external sinks.

Output Mode	Behavior	Aggregation Constraint
Append Mode	Only new rows from the latest micro-batch are written to the sink	Does not support streaming aggregations unless a Watermark is defined
Update Mode	Only rows modified or newly inserted in the latest micro-batch are output	Supported with or without aggregations
Complete Mode	Entire updated result table is rewritten on every micro-batch	Requires streaming aggregations such as `groupBy` or `window`

1. Append Mode

Only new rows added since the previous batch are written.

APPEND MODE

+----------------------+

| Only NEW rows added |

| since last batch are |

| written. |

+----------------------+

2. Update Mode

Only newly inserted or updated rows are written.

This is useful for **upsert-style processing**.

UPDATE MODE

+----------------------+

| Only UPDATED / NEW |

| rows are written. |

+----------------------+

3. Complete Mode

The entire updated result table is written on every micro-batch.

COMPLETE MODE

+----------------------+

| ENTIRE updated result |

| table is rewritten. |

+----------------------+

---

5. Trigger Strategies & Micro-Batch Execution Models

Triggers determine the timing and execution frequency of streaming micro-batches.

Trigger Strategy Types

Trigger	Behavior
Unspecified / Default	Executes the next batch immediately after the current batch completes if new data exists
Fixed Interval	Executes micro-batches at defined time intervals
Available Now	Processes all available data across sequential micro-batches and then terminates

---

5.1 Unspecified / Default Trigger

With the default trigger, execution continues in a continuous back-to-back fashion.

As soon as one micro-batch finishes, the next batch evaluates immediately if new data is available.

```
# Default trigger

query = df.writeStream \

    .outputMode("append") \
```

```
    .format("parquet") \
    .start()
```

5.2 Fixed Interval Trigger

A fixed processing interval defines how frequently Spark attempts to execute micro-batches.

```
# Fixed interval trigger — every 10 seconds
query = df.writeStream \
    .trigger(processingTime="10 seconds") \
    .outputMode("append") \
    .start()
```

Execution Behavior

Scenario A:

If a batch is configured for a 5-minute interval but completes in 2 minutes, the engine waits for the remaining 3 minutes before starting the next micro-batch.

Scenario B:

If a batch takes 8 minutes and exceeds the configured 5-minute interval, the next batch begins immediately after the previous batch completes.

5.3 Available Now Trigger

`availableNow=True` processes all currently available incoming data through sequential micro-batches and then automatically terminates the streaming query.

```
query = df.writeStream \
```

```
    .trigger(availableNow=True) \
    .outputMode("append") \
    .option("checkpointLocation", "/mnt/telemetry/checkpoints") \
    .start()
```

Key Advantage

Available Now combines:

- Incremental state tracking
- Streaming checkpointing
- On-demand batch-style execution
- Automatic cluster/resource release after processing

This makes it useful when continuous streaming is unnecessary but incremental processing is still desired.

---

6. Fault Tolerance & Exactly-Once Processing Semantics

To maintain fault tolerance during infrastructure failures or cluster restarts, streaming applications need to support **Exactly-Once Semantics**.

The objective is to ensure that input records are neither skipped nor duplicated in the final output.

Exactly-Once Execution Architecture

```
CHECKPOINT LOCATION
        |
        | Tracks offsets and state
        v
REPLAYABLE SOURCE
```

|

| Re-read data after failure

v

IDEMPOTENT SINK

|

v

Final Consistent Output

Four Requirements for Exactly-Once Processing

1. Checkpointing & Write-Ahead Logs (WAL)

The streaming engine stores:

- Read offsets
- Metadata
- State updates

inside a persistent `checkpointLocation`.

After failure, the application uses the checkpoint to resume execution from the last committed offset.

2. Replayable Source

The source must support re-reading historical uncommitted data from explicit offsets.

Examples:

- Kafka
- ADLS / Data Lake files

A socket source is **non-replayable**.

3. Deterministic Computation

Processing the same input repeatedly should generate the same output.

Example:

sqrt(4) → 2

is deterministic.

Dynamic timestamps and currency conversions may be non-deterministic.

4. Idempotent Sink

The sink must handle re-executed micro-batches safely.

Common approaches include:

- Overwriting previous records
- Upserting using `MERGE`
- Delta Lake `MERGE INTO`
- `foreachBatch`

---

7. Stateful Processing & State Store Management

Streaming operations can be classified as **Stateless** or **Stateful**.

Stateless Transformations	Stateful Transformations
`select`	`groupBy`
`filter`	Streaming joins
`map`	Windowing

`explode`	Stateful aggregations
Each record can be evaluated independently	Requires historical information across micro-batches

State Store

A **State Store** maintains intermediate values across micro-batches for stateful operations.

The study material describes the State Store as an in-memory key-value store on worker nodes backed by technologies such as **RocksDB or HDFS**.

Memory Management Risk

Consider an unbounded aggregation:

groupBy(customer_id)

If the stream contains hundreds of millions of active customer IDs without time limits, state can continuously grow and eventually result in executor or driver **Out-Of-Memory (OOM)** failures.

Mitigation

Use:

- Time-bounded windows
- Watermarking
- State eviction

These allow Spark to remove old state from the State Store.

---

8. Windowing Aggregations & Late-Data Handling

Windowed aggregations calculate metrics over time-bound buckets based on **Event Time** rather than Processing/Trigger Time.

Event Time vs. Processing Time

Event Time:
The timestamp representing when the event actually occurred.

Processing Time:
The timestamp representing when Spark receives/processes the event.

This distinction is critical because events can arrive late.

8.1 Tumbling Window

A tumbling window consists of fixed-size, contiguous, non-overlapping intervals.

Example:

10:00 ———— 10:15

10:15 ———— 10:30

10:30 ———— 10:45

A 15-minute tumbling window creates independent 15-minute buckets.

8.2 Sliding Window

A sliding window consists of overlapping intervals.

Example:

Window Duration = 15 minutes

Slide Duration = 5 minutes

10:00 ———————— 10:15

10:05 ———————— 10:20

10:10 ———————— 10:25

Comparison

Window	Description
Tumbling	Fixed-size, contiguous, non-overlapping intervals
Sliding	Overlapping intervals defined using window duration and slide duration

---

9. Watermarking

A **Watermark** defines an allowed arrival-delay threshold relative to the maximum event time observed by Spark.

Formula

Watermark Boundary

=

Maximum Event Time Observed

-

Allowed Late Threshold

Example

Suppose:

Maximum Event Time Observed = 11:45 AM

Allowed Late Threshold = 30 minutes

Then:

Watermark Boundary = 11:15 AM

Therefore:

Event Time 11:20 AM → ACCEPTED

Event Time 11:05 AM → DROPPED

Why Watermarking Is Important

Watermarking allows Spark to:

1. Handle late-arriving records within an acceptable threshold.
2. Drop records that arrive later than the defined boundary.
3. Evict expired aggregation state.
4. Prevent indefinite growth of the State Store.
5. Reduce the risk of OOM failures.

Watermarked Tumbling Window Aggregation

```
from pyspark.sql.functions import window, col, sum


orders_stream = spark.readStream \

    .format("json") \

    .schema("order_id LONG, order_date TIMESTAMP, amount DOUBLE") \

    .load("data/inputdir")
```

```
windowed_sales = orders_stream \
    .withWatermark("order_date", "30 minutes") \
    .groupBy(
        window(col("order_date"), "15 minutes")
    ) \
    .agg(
        sum("amount").alias("total_sales")
    )

query = windowed_sales.writeStream \
    .outputMode("update") \
    .format("console") \
    .start()
```

Sliding Window Aggregation

A sliding window can be defined using both duration and slide interval.

Example:

- Window duration = 15 minutes
- Slide = 5 minutes

```
sliding_sales = orders_stream \
    .withWatermark("order_date", "30 minutes") \
    .groupBy(
```

```
        window(
            col("order_date"),
            "15 minutes",
            "5 minutes"
        )
    ) \
    .agg(
        sum("amount").alias("total_sales")
    )
```

---

10. Streaming Joins

Spark Structured Streaming supports joining:

1. A streaming DataFrame with a static DataFrame.
2. Two streaming DataFrames.

10.1 Stream-to-Static Join

A **Stream-to-Static Join** enriches live streaming events using a static reference dataset.

Example

Streaming Data

(Live ATM Swipes)

```
        |
        | card_id
```

v

Static Customer Table

(Customer Database)

|

v

Enriched Streaming Data

Example:

Streaming Transaction

---

transaction_id: TX-991

card_id: 557242753831

amount: $200.00

Static Customer Table

---

card_id: 557242753831

member_id: M-88120

country: United States

The resulting stream contains both transaction and customer information.

Stream-to-Static Left Outer Join

```
streaming_transactions = spark.readStream \
    .format("kafka") \
```

```
    .load()


static_customers = spark.read.table(
    "prod.retail.customers"
)


enriched_stream = streaming_transactions.join(
    static_customers,
    streaming_transactions.card_id == static_customers.card_id,
    "left_outer"
)
```

Outer Join Restrictions

According to the study material:

- **Left Outer Join:** The left side must be Streaming and the right side Static.
- **Right Outer Join:** The right side must be Streaming and the left side Static.

Reversing these directional relationships can result in an `AnalysisException`.

10.2 Stream-to-Stream Join

A Stream-to-Stream Join joins two continuous streaming datasets.

Because Spark must maintain state for both streams, the query requires:

- Watermarks on both streams.
- A time-range constraint to limit how long state must be retained.

Example

```
impressions = spark.readStream \
    .format("kafka") \
    .load() \
    .withWatermark("impression_time", "2 hours")

clicks = spark.readStream \
    .format("kafka") \
    .load() \
    .withWatermark("click_time", "3 hours")

matched_ad_conversions = impressions.join(
    clicks,
    expr("""
        click_ad_id = impression_ad_id AND
        click_time >= impression_time AND
        click_time <= impression_time + interval 1 hour
    """),
    "inner"
)
```

Key Interview Point

A Stream-to-Stream Join requires bounded state.

Therefore:

Watermark + Time Range Constraint

↓

Bounded State

↓

Controlled Memory Consumption

---

11. Upserts in Streaming: Delta Lake + `foreachBatch`

When streaming data needs to perform micro-batch **upserts** using Delta Lake, `foreachBatch` can be used to execute a `MERGE INTO` operation for every micro-batch.

Custom Micro-Batch Function

```
def merge_micro_batch(micro_batch_df, batch_id):
    """Executes atomic Delta MERGE INTO for each micro-batch."""

    micro_batch_df.createOrReplaceTempView("updates")

    merge_sql = """
        MERGE INTO orders_final_result t
        USING updates s
        ON t.customer_id = s.customer_id

        WHEN MATCHED THEN
          UPDATE SET
```

```
        t.products_purchased = s.products_purchased,
        t.orders_placed = s.orders_placed,
        t.amount_spent = s.amount_spent

    WHEN NOT MATCHED THEN
      INSERT (
        customer_id,
        orders_placed,
        products_purchased,
        amount_spent
      )
      VALUES (
        s.customer_id,
        s.orders_placed,
        s.products_purchased,
        s.amount_spent
      )
  """

  micro_batch_df._jdf.sparkSession().sql(merge_sql)
```

Writing the Streaming Query

```
query = aggregated_orders.writeStream \
  .format("delta") \
```

```
    .outputMode("update") \
    .option(
        "checkpointLocation",
        "/mnt/delta/checkpoints/orders_merge"
    ) \
    .foreachBatch(merge_micro_batch) \
    .start()
```

`foreachBatch` Execution Pattern

```
Streaming Data
      |
      v
Micro-Batch
      |
      v
foreachBatch()
      |
      v
Create Temporary View
      |
      v
Delta MERGE INTO
      |
      +---- MATCHED ------> UPDATE
```

|

+---- NOT MATCHED -> INSERT

---

12. Key Interview Revision Points

Batch vs. Streaming

Remember:

Batch

→ Bounded Data

→ Higher Latency

→ Scheduled Processing

Streaming

→ Unbounded Data

→ Low Latency

→ Continuous / Incremental Processing

Structured Streaming

readStream

↓

DataFrame Transformations

↓

writeStream

Output Modes

Append → New rows

Update → Changed/new rows

Complete → Entire result table

Trigger Types

Default

→ Process next batch immediately

Processing Time

→ Process according to fixed interval

Available Now

→ Process all available data and terminate

Exactly-Once Processing

Remember the four major requirements:

Checkpoint

+

Replayable Source

+

Deterministic Processing

+

Idempotent Sink

=

Exactly-Once Processing

Stateful Processing

Stateful Operation

↓

State Store

↓

Historical State

↓

Watermark + Window

↓

State Eviction

Watermarking

Maximum Event Time

-

Allowed Late Arrival

=

Watermark Boundary

Streaming Joins

Stream + Static

→ Stream Enrichment

Stream + Stream

→ Watermark Both Sides

→ Time-Bounded Join

→ Controlled State

Delta Upserts

Streaming Data

↓

Micro-Batch

↓

foreachBatch

↓

MERGE INTO

↓

UPDATE / INSERT

---

13. Complete Master Curriculum Progress Roadmap

==========

ULTIMATE BIG DATA & AZURE DATA ENGINEERING MASTER GUIDE

==========

[WEEKS 1 - 16: Distributed Computing, PySpark Core & Tuning]

• HDFS, YARN, MapReduce, Spark RDDs, DataFrames, Catalyst Optimizer, Memory Management

• Join Optimizations, Partitioning vs. Bucketing, Partition Skew, AQE, Salting, Resource Sizing

[WEEKS 17 - 21: Azure Cloud Infrastructure & Governance]

• Cloud Economics (CapEx vs OpEx), Regions, Availability Zones, Fault/Update Domains

• Azure Storage (Blob, ADLS Gen2, HNS, Access Tiers, LRS/ZRS/GRS Redundancy)

• Databricks Architecture, DBFS, Secret Scopes, Delta Lake ACID Logs, Time Travel, Medallion Design

• Unity Catalog: 3-Level Namespace, Centralized Metastore, SCIM Provisioning & Access Control

[WEEKS 22 - 25: Pipeline Orchestration & End-to-End Cloud Solution]

• Azure Data Factory: Copy Activity, Mapping Data Flows, Dynamic Parameterization, Triggers

• Azure Synapse Analytics: Serverless SQL, Dedicated MPP Architecture (60 Distributions), Spark

• Production Capstone Project: Event-Driven Architecture, Key Vault, Azure SQL, AWS S3 Integration

[WEEKS 26 - 27: Data Modeling & Production Technical Q&A]

• Normalization (1NF to 3NF), Kimball Dimensional Design (Grain, Facts, Dimensions, Star/Snowflake)

• Surrogate Keys, Slowly Changing Dimensions (SCD 0/1/2/3), PySpark SCD-2 Pipeline Implementation

• Production Q&A: Azure Storage Security, Lifecycle Rules, ADF Integration Runtimes & DIU Tuning

[WEEKS 28 - 29: Spark Structured Streaming Architecture]

• Batch vs. Stream Processing, Micro-Batch Engine Internals, DataStreamReader / DataStreamWriter

• Output Modes (Append, Update, Complete), Triggers (Default, Fixed Interval, Available Now)

• Checkpoint Storage, Write-Ahead Logs (WAL), Exactly-Once Processing, Replayable Sources

• Stateful vs. Stateless Transformations, In-Memory State Store, RocksDB

• Event Time vs. Processing Time, Windowing (Tumbling vs. Sliding), Watermarking Boundaries

• Streaming Joins (Stream-to-Static Outer Join Rules, Stream-to-Stream), Micro-batch Delta Upserts

==========

Master Study Guide Complete — Weeks 1–29

You have now completed the **29-week Big Data & Azure Data Engineering curriculum**, covering the progression from distributed systems and Spark fundamentals through Azure cloud architecture, Databricks, data engineering pipelines, dimensional modeling, and production-grade Structured Streaming.

Weeks 28–29: What You Should Be Able to Explain in an Interview

By the end of these two weeks, you should be comfortable explaining:

- Batch vs. streaming processing.
- Spark Structured Streaming architecture.
- DStreams vs. Structured Streaming.
- `readStream` → transformations → `writeStream`.
- Streaming sources and sinks.
- Append vs. Update vs. Complete output modes.
- Default, Processing Time, and Available Now triggers.
- Checkpointing and fault tolerance.
- Exactly-once processing requirements.
- Stateless vs. stateful operations.
- State Store and state-growth problems.
- Event Time vs. Processing Time.
- Tumbling vs. Sliding windows.
- Watermarking and late-arriving data.
- Stream-to-Static joins.
- Stream-to-Stream joins.
- Watermarks and time-range constraints for streaming joins.
- Delta Lake upserts using `foreachBatch` and `MERGE INTO`.

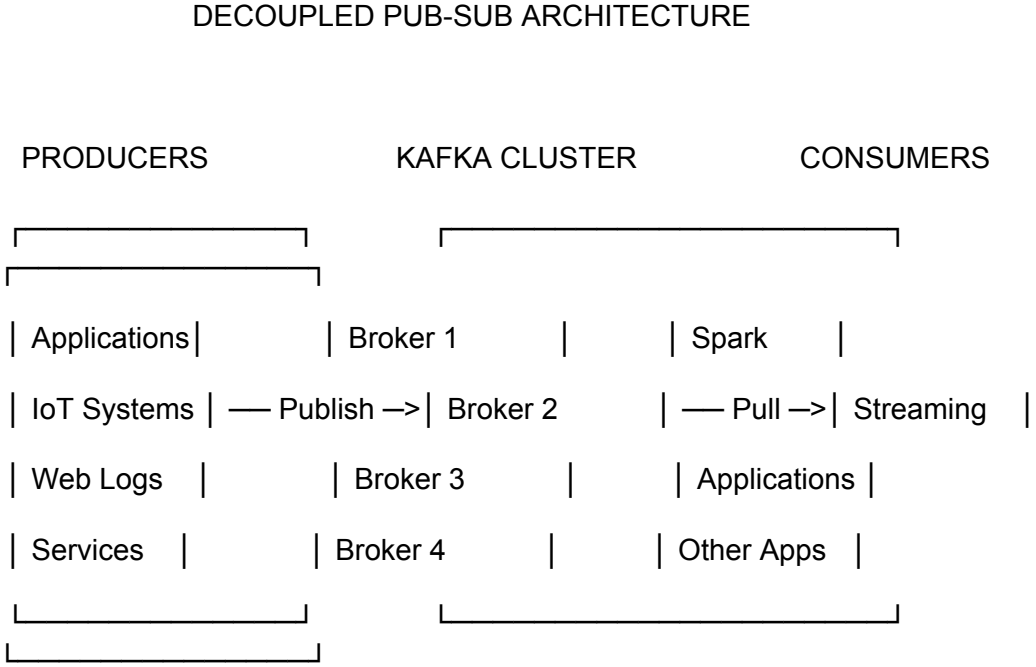
This gives you a strong **interview-focused Structured Streaming foundation** to build upon in the next weeks.

Week 30: Apache Kafka Architecture, PySpark Structured Streaming Integration & Production Messaging

1. Apache Kafka Architectural Overview & Distributed Ecosystem

Apache Kafka is an open-source, distributed event-streaming platform designed for **high-throughput, scalable, fault-tolerant, real-time messaging and event processing**.

Kafka follows a **decoupled publish-subscribe architecture**:



Why Kafka Instead of Network Sockets?

Simple network sockets such as `nc -lk` can demonstrate streaming concepts, but they are unsuitable for production-grade messaging.

Capability	Network Socket	Apache Kafka
Data persistence	No	Yes
Replayability	No	Yes
Buffering	Limited	Built-in

Multiple consumers	Difficult	Native
Fault tolerance	Very limited	High
Scalability	Limited	High
Consumer offset tracking	No	Yes
Production suitability	Low	High

Kafka's Key Advantages

1. Replayability

Kafka persists messages on disk, allowing consumers to re-read historical messages.

2. Buffering

Kafka decouples producers and consumers. If producers temporarily generate data faster than consumers can process it, Kafka retains the messages until consumers catch up.

3. Multiple Independent Consumers

Different applications can consume the same topic independently through separate consumer groups.

4. Horizontal Scalability

Topics can be divided into multiple partitions and distributed across brokers.

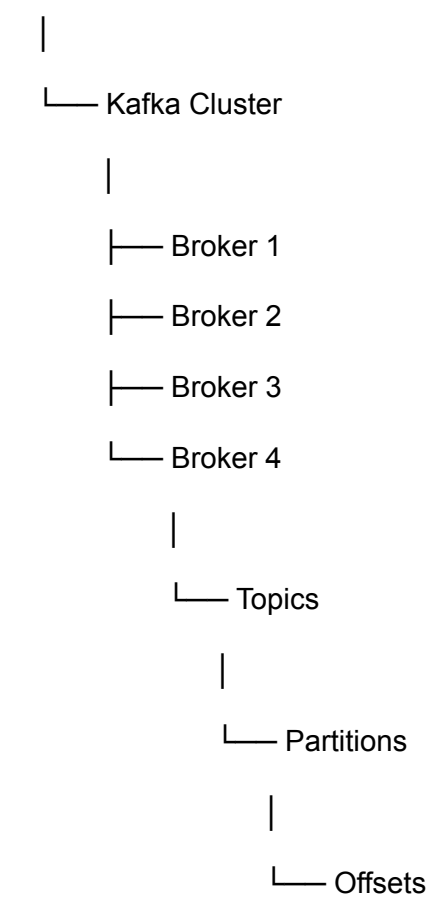
5. Fault Tolerance

Partitions can be replicated across brokers so that data remains available if a broker fails.

---

2. Kafka Namespace Hierarchy

Kafka's logical organization can be visualized as:



Core Kafka Components

1. Producer

A **producer** is an application that publishes messages/events to Kafka topics.

Examples:

- Web applications
- IoT devices
- Microservices
- Application logs
- Transaction systems

2. Consumer

A **consumer** is an application that reads messages from Kafka.

Examples:

- Spark Structured Streaming
- Databricks
- Flink
- Custom Python applications
- Monitoring systems

3. Broker

A **broker** is an individual Kafka server responsible for storing and serving partitions.

A Kafka cluster can contain multiple brokers.

4. Topic

A **topic** is a logical stream of events.

For example:

retail-orders

customer-events

payment-transactions

website-logs

A topic is conceptually similar to a table in a database, but it represents a continuously growing event stream rather than a static dataset.

5. Partition

A topic is divided into **partitions** for scalability and parallel processing.

Example:

Topic: retail-orders

Partition 0 → [0, 1, 2, 3, 4, ...]

Partition 1 → [0, 1, 2, 3, 4, ...]

Partition 2 → [0, 1, 2, 3, 4, ...]

Partition 3 → [0, 1, 2, 3, 4, ...]

Each partition is an **ordered, append-only log**.

6. Offset

An **offset** is the sequential identifier assigned to a record within a partition.

For example:

Partition 0

Offset 0 → Event A

Offset 1 → Event B

Offset 2 → Event C

Offset 3 → Event D

Offsets are unique **within a partition**, not across the entire Kafka topic.

Locating a Kafka Record

A specific record can be identified using:

(Topic Name, Partition ID, Partition Offset)

For example:

("retail-orders", 2, 1456)

means:

> Record at offset 1456 in partition 2 of the `retail-orders` topic.

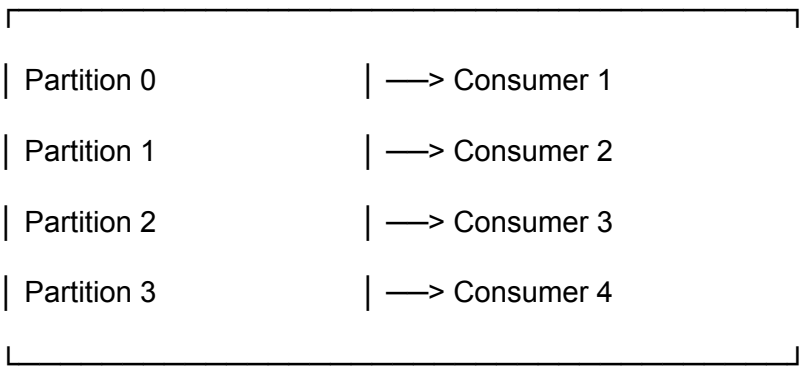
---

3. Kafka Partitioning & Consumer Groups

Partitioning is one of Kafka's most important mechanisms for achieving **parallelism and scalability**.

Suppose a topic contains four partitions:

> TOPIC: retail-orders



With four consumers in the same consumer group, each consumer can process one partition.

Consumer Group

A **consumer group** is a logical collection of consumers working together to consume a Kafka topic.

The key rule is:

> Within a consumer group, a partition can be assigned to only one consumer at a time.

Consumer Group Scaling Rules

Rule 1 — One Partition → One Consumer

A single partition cannot simultaneously be assigned to two consumers within the same consumer group.

This preserves ordering.

Rule 2 — One Consumer Can Read Multiple Partitions

If there are fewer consumers than partitions:

4 Partitions

2 Consumers

Consumer 1 → Partition 0 + Partition 1

Consumer 2 → Partition 2 + Partition 3

Rule 3 — Consumers Cannot Exceed Active Partitions

If there are:

4 partitions

6 consumers

then:

Consumer 1 → Partition 0

Consumer 2 → Partition 1

Consumer 3 → Partition 2

Consumer 4 → Partition 3

Consumer 5 → IDLE

Consumer 6 → IDLE

Therefore:

Maximum active consumers in a group

≈ Number of partitions

---

4. Kafka Message Keys & Partition Routing

Kafka records can contain:

(Key, Value)

For example:

Key = customer_id = 256

Value = {

"order_id": 1001,

"customer_id": 256,

"amount": 499.99

}

When a key is supplied, Kafka's partitioning strategy generally uses the key to consistently route records to the same partition.

Why Is This Important?

Suppose:

customer_id = 256

has three orders:

Order 1 → customer 256

Order 2 → customer 256

Order 3 → customer 256

Using `customer_id` as the Kafka key helps ensure that these records are routed to the same partition.

This is useful when downstream processing requires ordering by customer.

Interview Insight

A good Kafka key should generally:

- Distribute records reasonably evenly.
- Preserve ordering where required.
- Avoid creating a hot partition.

For example:

customer_id

may be a good key for customer-centric event processing.

---

5. Python Kafka Producer Using Confluent-Kafka

The `confluent-kafka` Python library provides a high-performance Kafka client.

from confluent_kafka import Producer

import json

conf = {

```
    "bootstrap.servers": "your-kafka-bootstrap-server:9092",
    "security.protocol": "SASL_SSL",
    "sasl.mechanism": "PLAIN",
    "sasl.username": "YOUR_API_KEY",
    "sasl.password": "YOUR_API_SECRET",
    "client.id": "python-producer-app"
}

producer = Producer(conf)
```

Sample Payload

```
customer_id = "256"

payload = {
    "order_id": 2,
    "customer_id": 256,
    "customer_fname": "David",
    "customer_lname": "Rodriguez",
    "city": "Chicago",
    "state": "IL",
    "pincode": 60625,
    "line_items": [
        {
            "order_item_id": 2,
```

```
            "order_item_product_id": 1073,
            "order_item_quantity": 1,
            "order_item_product_price": 199.99,
            "order_item_subtotal": 199.99
        }
    ]
}
```

Delivery Callback

```
def acked(err, msg):
    if err is not None:
        print(f"Failed to deliver message: {err}")
    else:
        print(
            f"Message produced to Topic: {msg.topic()} "
            f"| Partition: {msg.partition()}"
        )
```

Producing a Message

```
producer.produce(
    topic="retail-orders",
    key=customer_id,
    value=json.dumps(payload),
    callback=acked
```

)

`poll()` vs `flush()`

This distinction is frequently asked in interviews.

`poll(0)`

producer.poll(0)

Processes delivery callbacks and other producer events.

`flush()`

producer.flush()

Waits for outstanding messages to be delivered before the application exits.

Interview Question

Why does `produce()` not immediately guarantee that the message has reached Kafka?

Because `produce()` is asynchronous. The record is initially placed into the producer's local buffer.

The producer then communicates with Kafka brokers in the background.

produce()

↓

Local Producer Buffer

↓

Kafka Broker

↓

Acknowledgement

↓

Delivery Callback

---

6. PySpark Structured Streaming + Kafka

PySpark provides native Kafka integration through:

.format("kafka")

The major distinction is:

spark.read

versus:

spark.readStream

6.1 Batch Reading from Kafka

Batch mode reads a finite snapshot of Kafka records and then terminates.

```
bootstrap_servers = "your-kafka-bootstrap-server:9092"
topic_name = "retail-orders"

kafka_batch_df = spark.read \
    .format("kafka") \
    .option("kafka.bootstrap.servers", bootstrap_servers) \
```

```
    .option("subscribe", topic_name) \
    .load()
```

Kafka's `key` and `value` columns are binary.

Therefore, they are commonly converted into strings:

```
formatted_batch_df = kafka_batch_df.selectExpr(
    "CAST(key AS STRING) AS key",
    "CAST(value AS STRING) AS value",
    "topic",
    "partition",
    "offset",
    "timestamp"
)
```

Important Kafka Metadata

Spark's Kafka DataFrame includes useful metadata:

key

value

topic

partition

offset

timestamp

timestampType

---

7. Streaming Kafka → PySpark → Delta Lake

For continuous processing:

spark.readStream

is used.

The architecture becomes:

Kafka Topic

readStream



JSON Parsing



DataFrame Transformations

Flatten Nested Structures

Delta Lake

Reading Kafka as a Stream

```
kafka_stream_df = spark.readStream \
    .format("kafka") \
    .option("kafka.bootstrap.servers", bootstrap_servers) \
    .option("subscribe", topic_name) \
    .option("startingOffsets", "earliest") \
    .option("maxOffsetsPerTrigger", 50) \
    .load()
```

Important Options

`startingOffsets`

Controls where the streaming query starts reading.

Examples:

earliest

latest

`earliest` is useful when initially processing historical Kafka data.

`maxOffsetsPerTrigger`

Controls the maximum number of Kafka records processed in each micro-batch.

Example:

```
.option("maxOffsetsPerTrigger", 50)
```

This can help control workload and prevent large Kafka backlogs from overwhelming the Spark cluster.

---

8. Parsing JSON from Kafka

Kafka generally provides the message value as binary data.

The first step is converting it to a string:

```
kafka_stream_df = kafka_stream_df.selectExpr(
    "CAST(value AS STRING) AS raw_json"
)
```

Then parse JSON using `from_json()`.

```
from pyspark.sql.functions import from_json, col

orders_schema = """
    order_id LONG,
    customer_id LONG,
    customer_fname STRING,
    customer_lname STRING,
    city STRING,
    state STRING,
    pincode LONG
"""

parsed_stream_df = kafka_stream_df \
    .withColumn(
```

```
        "data",
        from_json(col("raw_json"), orders_schema)
    ) \
    .select("data.*")
```

This converts:

Raw JSON string

↓

Structured Spark columns

---

9. Flattening Nested Arrays Using `explode()`

Real-world Kafka messages often contain nested arrays.

Example:

```
{
  "order_id": 1001,
  "customer_id": 256,
  "line_items": [
    {
      "product_id": 101,
      "quantity": 2
    },
```

```
    {
      "product_id": 202,
      "quantity": 1
    }
  ]
}
```

Spark represents `line_items` as:

ARRAY<STRUCT<...>>

Use:

```
from pyspark.sql.functions import explode

flattened_df = parsed_stream_df \
    .select(
        "order_id",
        "customer_id",
        explode("line_items").alias("item")
    )
```

Then extract individual fields:

```
flattened_df = flattened_df.select(
    "order_id",
    "customer_id",
```

```
    col("item.product_id").alias("product_id"),
    col("item.quantity").alias("quantity")
)
```

The result becomes:

```
order_id | customer_id | product_id | quantity
-----------------------------------------------
1001     | 256         | 101        | 2
1001     | 256         | 202        | 1
```

Interview Question

What does `explode()` do?

It converts each element of an array or map into a separate row.

---

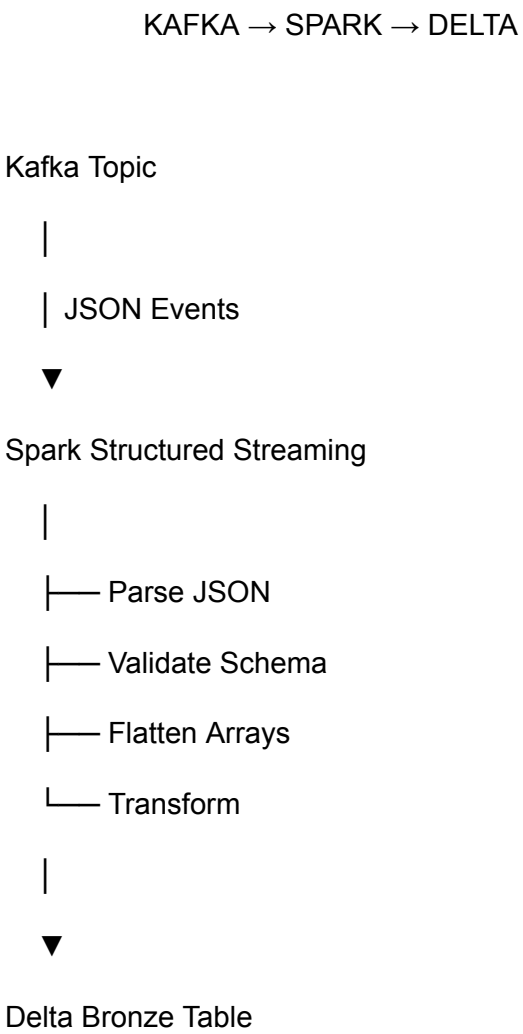
10. Writing Kafka Streams to Delta Lake

After transformation, the stream can be persisted into Delta Lake.

```
query = flattened_df.writeStream \
    .queryName("KafkaToDeltaIngestion") \
    .format("delta") \
    .outputMode("append") \
    .option(
        "checkpointLocation",
        "/mnt/delta/checkpoints/kafka_orders"
```

```
    ) \
    .toTable("retail_db.orders_flattened_bronze")
```

Architecture

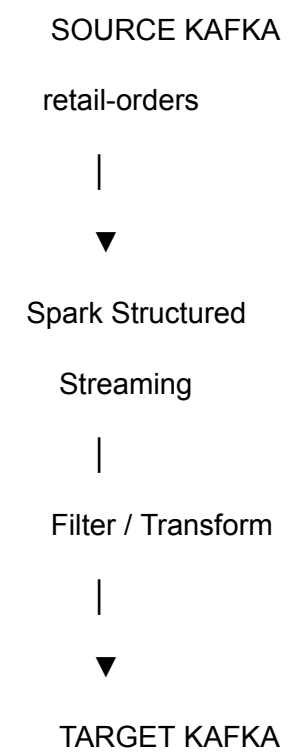


This is a common **Bronze-layer ingestion pattern** in a Lakehouse architecture.

---

11. Kafka → PySpark → Kafka Streaming Pipeline

Kafka can also act as both the source and sink.



Example:

```
from pyspark.sql.functions import col, struct, to_json


chicago_orders_df = parsed_stream_df \
    .filter(col("city") == "Chicago") \
    .selectExpr(
        "CAST(customer_id AS STRING) AS key",
        """
```

```
        to_json(
            struct(
                order_id,
                customer_id,
                customer_fname,
                customer_lname,
                city,
                state,
                pincode
            )
        ) AS value
        """
    )
```

Then write to Kafka:

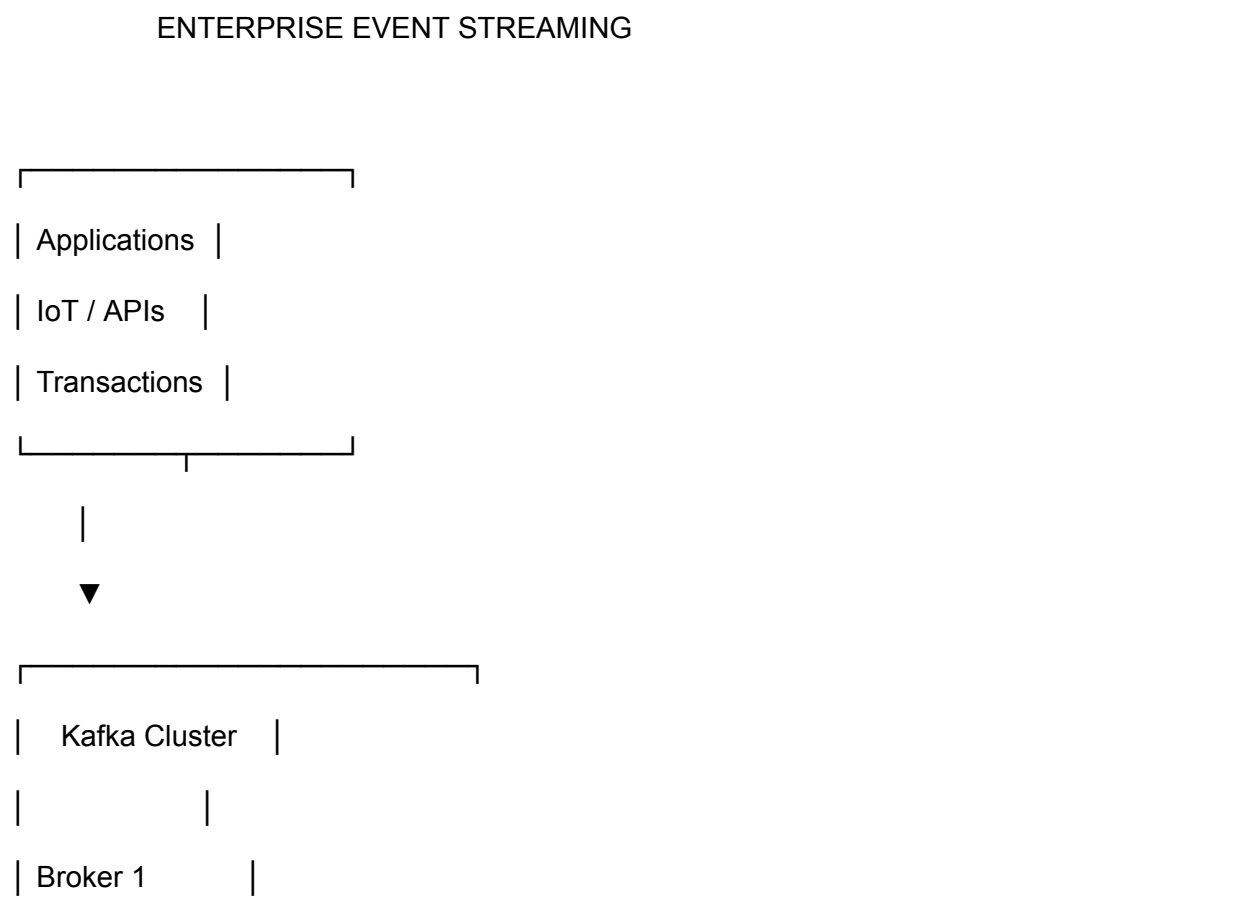
```
sink_query = chicago_orders_df.writeStream \
    .format("kafka") \
    .outputMode("append") \
    .option(
        "kafka.bootstrap.servers",
        bootstrap_servers
    ) \
    .option(
        "topic",
```

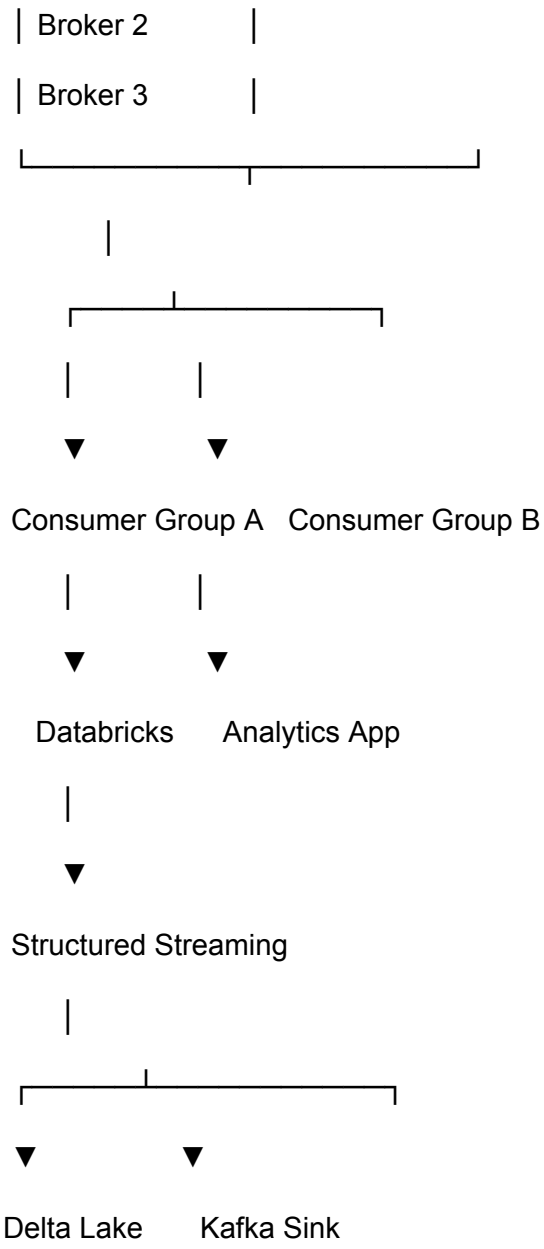
```
    "chicago-orders-sink"
  ) \
  .option(
    "checkpointLocation",
    "/mnt/delta/checkpoints/chicago_orders"
  ) \
  .start()
```

---

12. Production Kafka Architecture

A typical enterprise architecture may look like:





The key architectural benefit is **decoupling**.

Producers do not need to know:

- Who consumes their data.
- How many consumers exist.
- How quickly consumers process records.
- Where downstream data is stored.

---

13. Important Kafka Interview Concepts

Kafka vs Traditional Message Queues

Kafka is optimized for **distributed, durable event streaming and high-throughput workloads**.

Traditional queues often focus more on:

Message → Consumer → Message Removed

Kafka follows an append-only log model:

Producer

↓

Kafka Log

↓

Consumer A → Offset 100

Consumer B → Offset 75

Consumer C → Offset 20

Different consumers can independently maintain their positions.

Partition vs Consumer

Remember:

Partitions = Parallelism

Consumers = Processing Workers

If:

8 partitions

4 consumers

then consumers can process multiple partitions.

If:

4 partitions

8 consumers

only four consumers can actively process partitions at a time.

Kafka Topic vs Partition

Topic

Logical stream of related events.

Partition

Physical/logical subdivision of a topic that enables:

- Parallelism
- Scalability
- Ordering within the partition

Kafka Offset vs Spark Checkpoint

These concepts should not be confused.

Kafka Offset

Tracks:

Which Kafka record has been consumed.

Example:

Partition 2 → Offset 1456

Spark Checkpoint

Tracks streaming query progress and state so Spark can recover after failures.

Kafka Offset

↓

Spark Streaming

↓

Checkpoint

↓

Delta Sink

---

14. Key Production Considerations

When designing Kafka + Spark Structured Streaming systems, consider:

1. Partition Count

Choose enough partitions to provide the required processing parallelism.

2. Partition Key

Choose a key that:

- Preserves required ordering.
- Avoids data hotspots.
- Distributes traffic evenly.

3. Consumer Group Design

Separate applications requiring independent consumption should normally use separate consumer groups.

4. Checkpointing

Always use a persistent checkpoint location for production Spark streaming jobs.

5. Schema Management

For enterprise pipelines, establish schema governance and compatibility rules for evolving event payloads.

6. Security

Production Kafka deployments commonly require:

TLS / SSL

SASL authentication

ACL authorization

Secret management

7. Monitoring

Monitor:

- Consumer lag
- Broker health
- Partition distribution
- Throughput
- Error rate
- Failed records
- Spark processing latency
- Checkpoint health

---

15. High-Value Interview Questions

Beginner

1. What is Apache Kafka?
2. What is a Kafka topic?
3. What is a partition?
4. What is an offset?
5. What is a Kafka broker?
6. What is a producer?
7. What is a consumer?
8. What is a consumer group?
9. Why is Kafka better than a socket for production streaming?
10. What is the difference between `spark.read` and `spark.readStream`?

Intermediate

11. Why does Kafka use partitions?
12. How does Kafka maintain ordering?
13. What happens when consumers outnumber partitions?
14. What happens when partitions outnumber consumers?
15. How does a Kafka message key influence partition assignment?
16. What is consumer lag?
17. What is `startingOffsets` in Spark Kafka integration?
18. What is `maxOffsetsPerTrigger`?
19. Why do we use `from_json()`?
20. Why do we use `explode()`?
21. How do you write Kafka data to Delta Lake?
22. How do you write Spark streaming results back to Kafka?

Advanced

23. How would you design Kafka partitions for a high-volume application?
24. How would you avoid a hot partition?
25. How would you design Kafka consumer groups for multiple downstream applications?
26. How does Spark recover from a streaming failure?
27. What is the relationship between Kafka offsets and Spark checkpoints?
28. How would you implement exactly-once processing across Kafka and Delta?
29. How would you handle schema evolution in Kafka events?
30. How would you troubleshoot increasing Kafka consumer lag?
31. How would you design a Kafka → Spark → Delta architecture for millions of events per minute?
32. What happens if a Spark executor fails while consuming Kafka data?

---

16. Week 30 Quick Revision Sheet

Concept	Remember
Kafka	Distributed event-streaming platform
Broker	Kafka server
Topic	Logical event stream
Partition	Unit of parallelism
Offset	Record position within partition
Producer	Publishes events
Consumer	Reads events
Consumer Group	Consumers working together
Partition Key	Controls routing and ordering
`produce()`	Asynchronously queues message
`poll()`	Processes producer callbacks/events

`flush()`	Waits for outstanding messages
`spark.read`	Batch Kafka consumption
`spark.readStream`	Continuous Kafka consumption
`startingOffsets`	Starting Kafka position
`maxOffsetsPerTrigger`	Controls micro-batch ingestion volume
`from_json()`	Converts JSON into structured columns
`explode()`	Converts array elements into rows
Checkpoint	Stores Spark streaming progress/state
Delta Lake	Persistent analytical streaming sink
Kafka → Spark → Kafka	Streaming transformation pipeline

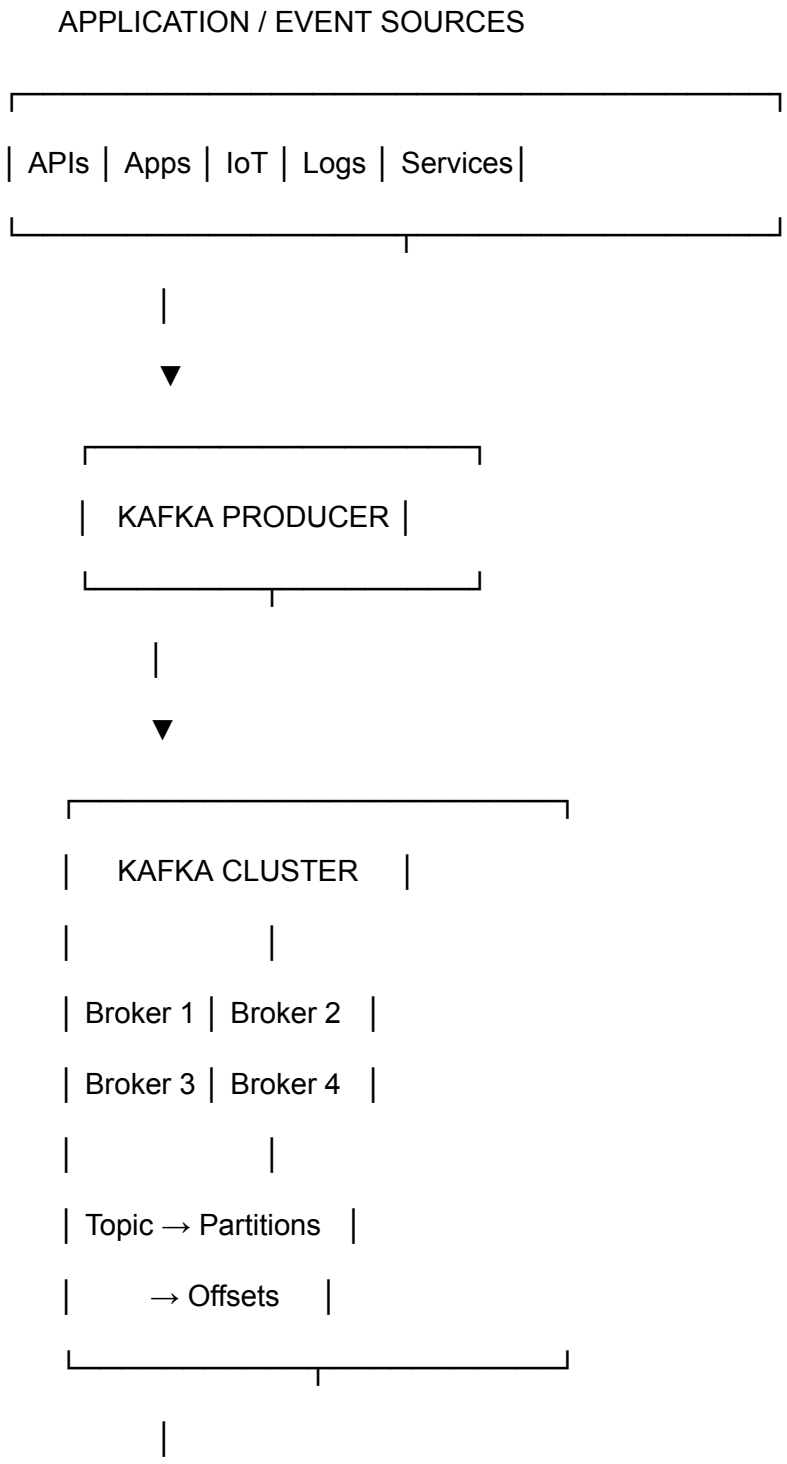
---

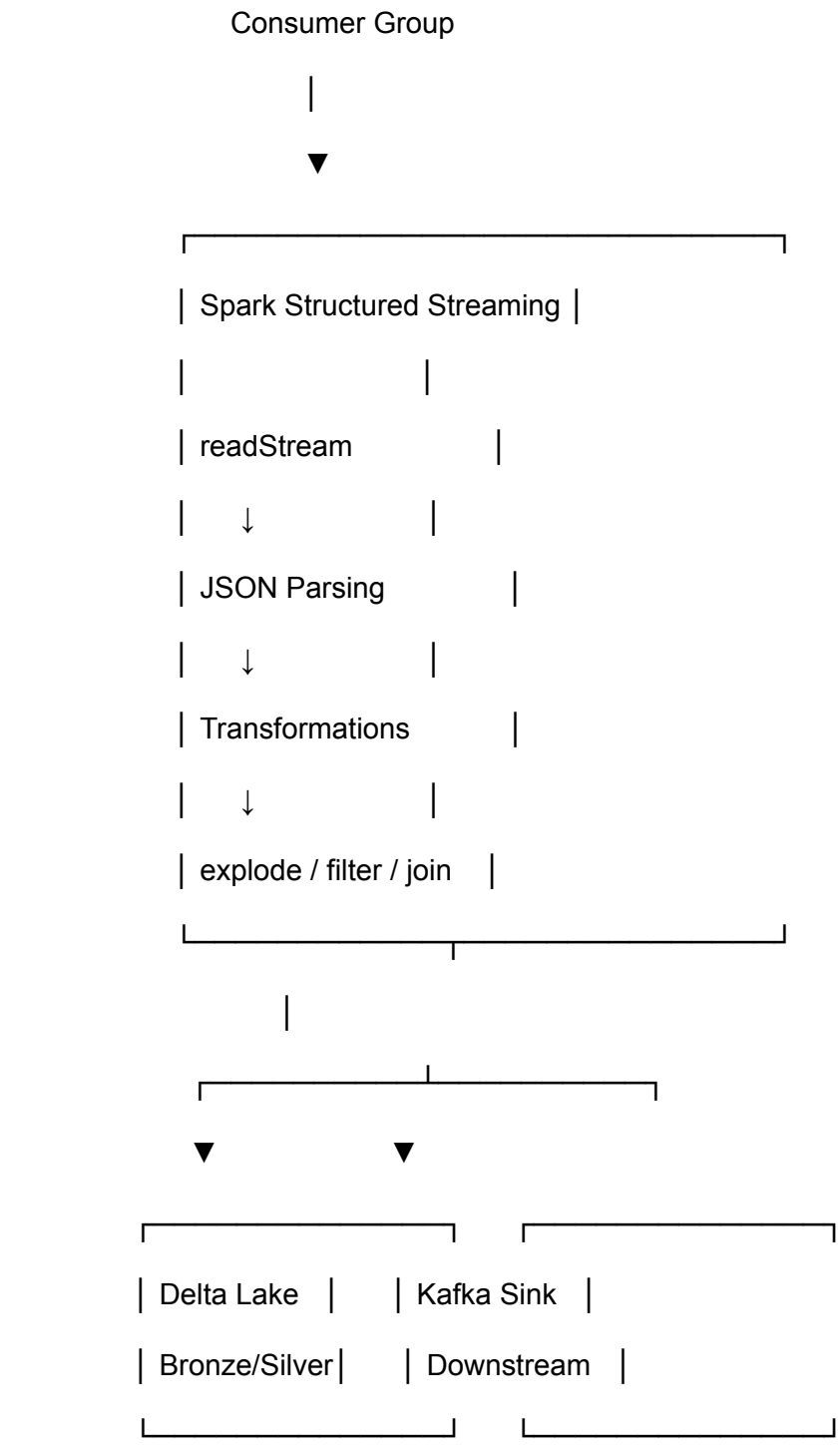
17. Week 30 Master Architecture

==========

WEEK 30 — APACHE KAFKA & REAL-TIME DATA ENGINEERING

==========




Consumer Group
Spark Structured Streaming
readStream
JSON Parsing
Transformations
explode / filter / join
Delta Lake
Bronze/Silver
Kafka Sink
Downstream

==========

Week 30 Core Takeaway

> **Kafka provides durable, scalable event transport; Spark Structured Streaming provides distributed real-time processing; Delta Lake provides reliable analytical storage.**

The complete production pattern to remember is:

SOURCE

↓

KAFKA PRODUCER

↓

KAFKA TOPIC

↓

PARTITIONS

↓

CONSUMER GROUP

↓

SPARK STRUCTURED STREAMING

↓

PARSE → TRANSFORM → ENRICH → FILTER

↓

DELTA LAKE / KAFKA / DOWNSTREAM SYSTEM

This gives you the complete **Week 30 interview + revision content** in a cleaner form, without the `[cite:...]` artifacts from the original Gemini output.

Week 31: AWS Big Data Architecture, Compute Families & Serverless Analytics

---

1. AWS Cloud Ecosystem & Cross-Cloud Service Mapping

The AWS Big Data ecosystem provides services that map closely to the cloud analytics architecture established in Azure and GCP.

[Cross-Cloud Service Equivalency Matrix]

AWS SERVICE	AZURE SERVICE	FUNCTIONAL ROLE
Amazon S3	ADLS Gen2 / Blob	Object Storage
Amazon EC2	Azure VMs	IaaS Compute
AWS Glue	Azure Data Factory	ETL & Catalog
Amazon Athena	Synapse Serverless	Pay-Per-Scan Query
Amazon Redshift	Synapse Dedicated	Enterprise MPP DWH
Amazon EMR	Azure HDInsight	Managed Hadoop/Spark
AWS Lambda	Azure Functions	Serverless Compute

Core AWS Data Architecture Components

- **Amazon S3 (Simple Storage Service):** Scalable cloud object storage that forms the foundation of many AWS data lake architectures.
- **AWS Glue & Glue Catalog:** Managed serverless ETL service combined with a Hive-compatible metadata catalog.
- **Amazon Athena:** Serverless interactive query engine that executes SQL directly against data stored in Amazon S3 without requiring the data to be moved into a separate database.
- **Amazon Redshift:** Enterprise Data Warehouse using Massively Parallel Processing (MPP) for high-performance analytical and reporting workloads.
- **Amazon EMR (Elastic MapReduce):** Managed cluster platform for running open-source Big Data frameworks such as Apache Spark, Hive, and Presto/Trino.

---

2. Amazon EC2 Compute Architecture & Provisioning Options

Amazon Elastic Compute Cloud (Amazon EC2) provides scalable Virtual Machines (VMs) running on physical infrastructure partitioned through virtualization.

[Physical Server Virtualization]

Physical Server (e.g., 128 GB RAM / 32 CPU Cores / 4 TB Storage)

├── VM 1 (m4.large): 32 GB RAM / 8 Cores / Guest OS

├── VM 2 (m4.large): 32 GB RAM / 8 Cores / Guest OS

├── VM 3 (m4.large): 32 GB RAM / 8 Cores / Guest OS

└── VM 4 (m4.large): 32 GB RAM / 8 Cores / Guest OS

EC2 Hardware Configuration Components

1. Amazon Machine Image (AMI)

An **AMI** is a pre-configured template containing the Guest Operating System, such as Linux or Windows, along with pre-installed application software and configuration.

It is used as the starting point when launching an EC2 instance.

2. Key Pair Authentication

EC2 uses public-key cryptography for secure SSH authentication.

- The **public key** is stored on the EC2 instance.
- The **private key** is retained by the client.
- The private key is commonly provided as a `.pem` file.

3. Storage Types

Instance Store Volumes

- Ephemeral storage physically attached to the host server.
- Very high performance.
- Data is lost when the instance is stopped or terminated.

Amazon EBS

Amazon Elastic Block Store (EBS) provides persistent, network-attached block storage.

Unlike ephemeral instance storage, EBS volumes are designed to persist independently of the EC2 instance lifecycle.

EC2 Instance Families

AWS categorizes EC2 instances into families optimized for different resource requirements.

Instance Family	Prefix	Resource Profile	Example	Primary Use Case
General Purpose	**M**	Balanced compute, memory and networking	`m4.large` — 2 vCPUs / 8 GB RAM	Web servers, development, build servers
Compute Optimized	**C**	High compute-to-memory ratio	`c5.xlarge` — 4 vCPUs / 8 GB RAM	Batch processing, compute-heavy workloads
Memory Optimized	**R**	High memory-to-compute ratio	`r5.xlarge` — 2 vCPUs / 16 GB RAM	In-memory workloads, Spark drivers, distributed databases

Interview Shortcut

Think of the families as:

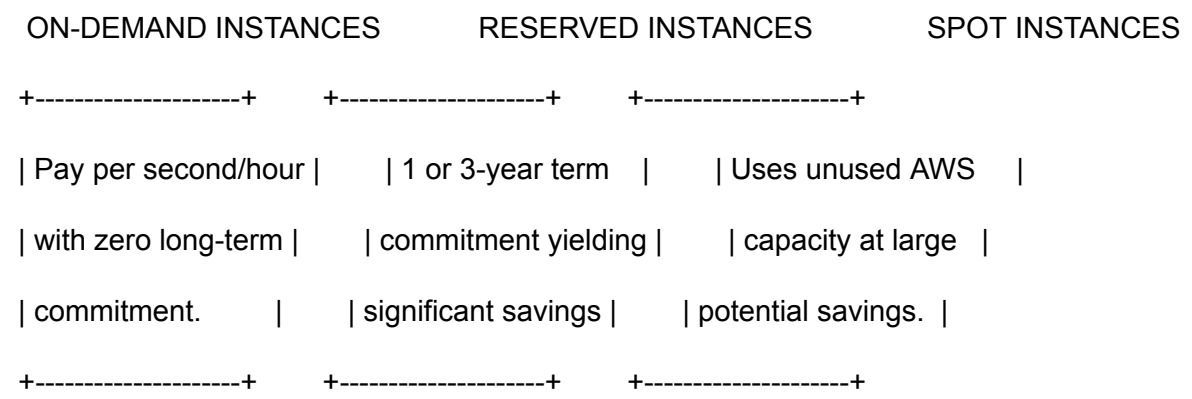
M → General Purpose → Balanced

C → Compute Optimized → CPU Heavy

R → Memory Optimized → RAM Heavy

EC2 Pricing Models

[EC2 Cost Optimization Options]



On-Demand

- Pay for compute without long-term commitment.
- Useful for unpredictable workloads.
- Highest flexibility.

Reserved Instances

- Commitment-based purchasing model.
- Suitable for predictable, continuously running workloads.
- Provides significant discounts compared with On-Demand pricing.

Spot Instances

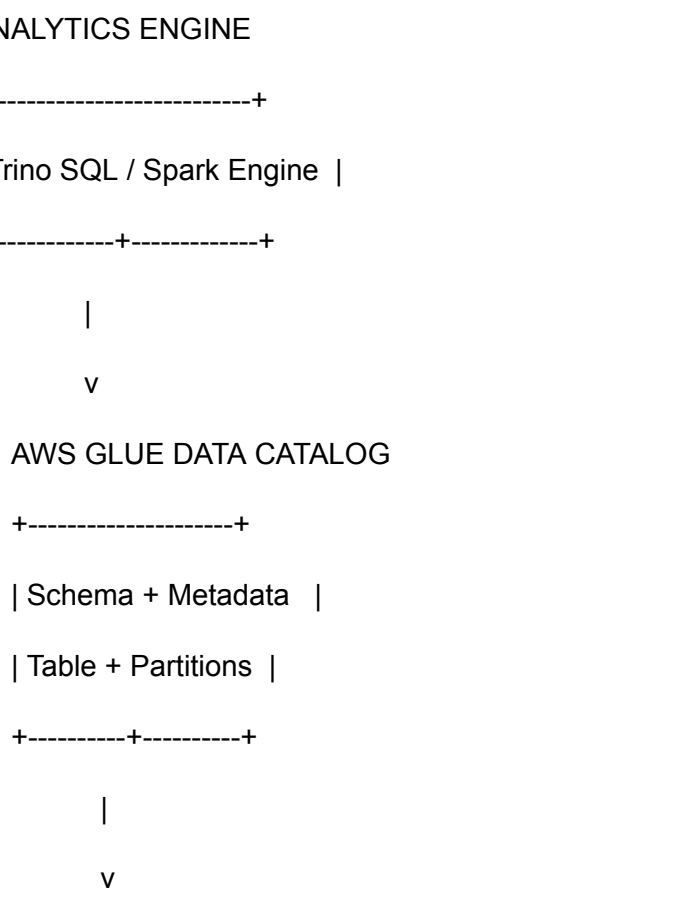
- Uses spare AWS capacity.
- Can provide substantial cost savings.
- Instances can be interrupted when AWS needs the capacity.
- Best suited to fault-tolerant workloads such as certain batch processing and distributed Spark workloads.

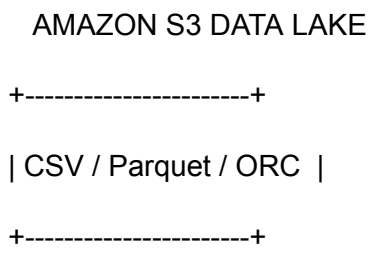
---

3. Amazon Athena Architecture & Analytics Engines

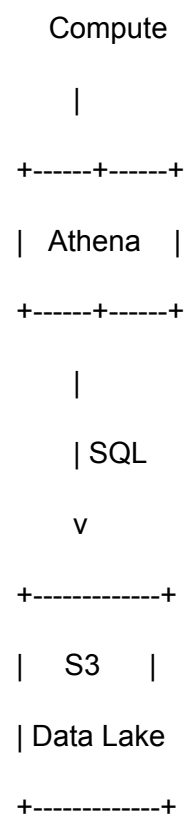
Amazon Athena is a serverless interactive query service used to analyze data directly from Amazon S3 using SQL.

[Amazon Athena Decoupled Architecture]





The key architectural idea is **separation of storage and compute**:



Data remains in S3 rather than being loaded into Athena.

Supported Execution Engines

1. Trino SQL Engine

A distributed SQL engine designed for interactive analytical queries over large datasets.

For SQL queries, the source material describes pricing as being based on the volume of data scanned from S3.

2. Apache Spark Engine

Athena can also provide Spark-based analytics for workloads requiring Python/PySpark processing.

The source material describes Spark billing using **Data Processing Units (DPUs)**.

Athena Billing Model

Trino SQL Engine

The provided study material uses the following pricing model:

- Standard rate: **$5 per TB scanned**
- Minimum billable amount: **10 MB**
- Failed queries: no charge
- Cancelled queries: charged for the data scanned before cancellation

The learning formula is:

{Query Scan Cost}

{Data Scanned in TB} times $5

For example, if a query scans 0.25 TB:

[0.25 times $5 = $1.25]

Spark Engine — DPU Framework

According to the supplied material:

- **1 DPU = 4 vCPUs + 16 GB RAM**
- DPU usage is charged based on execution resources consumed.
- The notebook server itself is described as free.

- Driver/coordinator resources consume DPU capacity while the session is active.
- Worker DPUs are allocated during execution.

Example

Suppose:

- Driver = 1 DPU for 1 hour
- Workers = 20 DPUs
- Six calculations each execute for 1 minute

Worker consumption:

$$
6 \times 20 \times \frac{1}{60}
= 2\text{ DPU-Hours}
$$

Total:

$$
1 + 2 = 3\text{ DPU-Hours}
$$

At the supplied rate of ₹/$0.35 per DPU-hour in the source:

$$
3 \times 0.35 = 1.05
$$

Interview takeaway: Athena SQL is primarily associated with **data scanned**, whereas Athena Spark is associated with **compute resources / DPUs**.

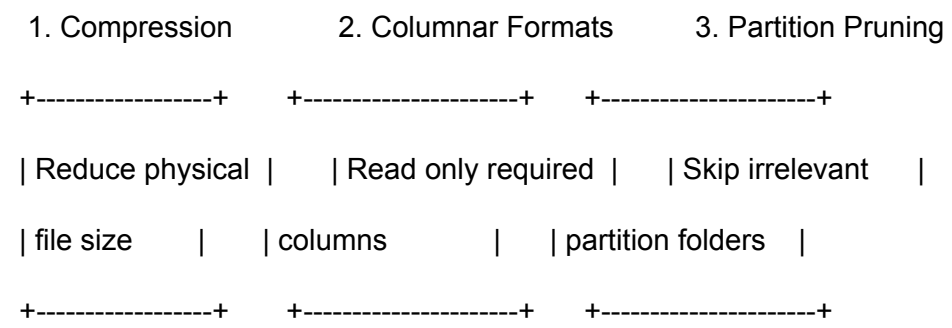
---

4. Query Optimization & Cost Reduction in S3/Athena

Athena costs can be reduced by minimizing the amount of physical data read from S3.

The three major techniques covered are:

[Athena Scan Reduction Strategies]



1. Compression

Compression reduces the physical amount of data stored and scanned.

Examples:

- Snappy
- Gzip

Smaller files generally mean fewer bytes scanned.

---

2. Columnar Formats

Formats such as **Parquet** and **ORC** store data by columns rather than rows.

Suppose a dataset contains:

customer_id

name

city

salary

If the query only requires:

SELECT salary

FROM employees;

a columnar format can avoid reading unnecessary columns.

This is known as **column pruning**.

---

3. Partition Pruning

Partitioning organizes data into separate directory structures based on partition columns.

For example:

s3://data/orders/

year=2025/

month=01/

month=02/

year=2026/

month=01/

month=02/

A query such as:

SELECT *

FROM orders

WHERE year = 2026

AND month = 7;

can avoid scanning irrelevant partitions.

Scan Cost Comparison

Consider a dataset containing **3 TB** of raw data with four columns, where the query needs only one column.

Storage Strategy	Format	Approx. Stored Size	Approx. Data Read	Example Scan Cost
Raw uncompressed	CSV	3 TB	3 TB	₹/$15
Compressed	Gzip CSV	1 TB	1 TB	₹/$5
Compressed columnar	Parquet	0.25 TB	~0.0625 TB	₹/$1.25

Core principle:

> Less data scanned → lower Athena query cost.

---

5. AWS Glue Catalog, Crawlers & Partition Management

AWS Glue Data Catalog serves as a centralized metadata repository containing information such as:

- Table definitions
- Column schemas
- Data types
- Partition metadata
- S3 locations

> [AWS Glue Metadata Pipeline]

Amazon S3

|

v

Glue Crawler

|

| Schema / Partition Discovery

v

Glue Data Catalog

|

v

Amazon Athena

|

v

SQL Queries

AWS Glue Crawler

A **Glue Crawler** can:

1. Scan S3 locations.
2. Discover file formats.
3. Infer schemas.
4. Detect partitions.
5. Register metadata in the Glue Data Catalog.

This avoids manually defining every schema and partition in many ingestion scenarios.

Manual Partition Registration

When partitioned data exists in S3 but the catalog does not yet contain the corresponding partition metadata, partitions can be registered using:

```
MSCK REPAIR TABLE table_name;
```

Example

```
CREATE EXTERNAL TABLE IF NOT EXISTS trendytech_db.students_partitioned (
    student_id INT,
    exam_center_id INT,
    year INT,
    quarter INT,
    score INT,
    grade STRING
)
PARTITIONED BY (subject STRING)
STORED AS PARQUET
LOCATION 's3://trendytech-athena-demo/athena-input-parquet/';
```

Then:

```
MSCK REPAIR TABLE trendytech_db.students_partitioned;
```

What happens?

S3 Partition Folders

|

v

MSCK REPAIR TABLE

|

v

Glue/Hive Metadata

|

v

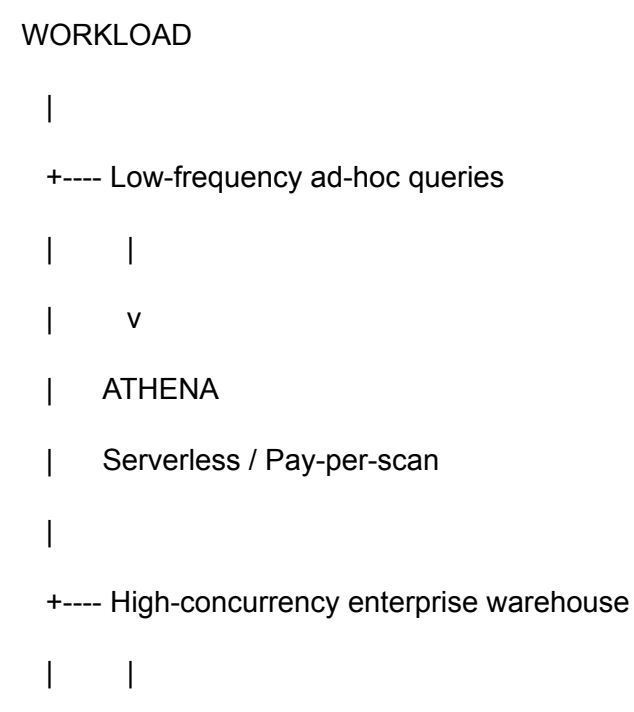
Athena recognizes partitions

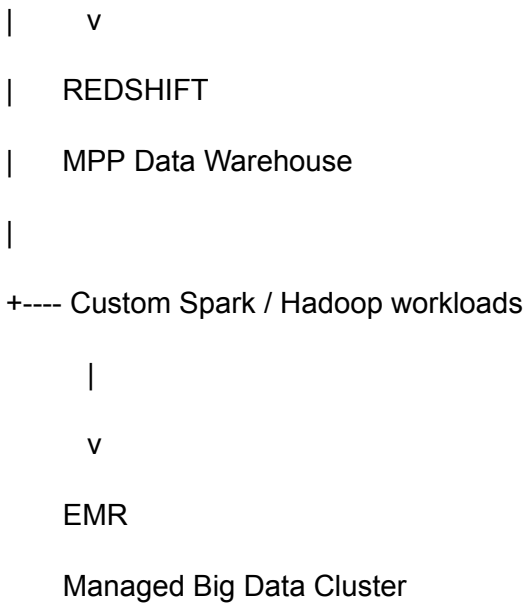
---

6. Athena vs. Redshift vs. EMR

The correct AWS analytics service depends on workload characteristics.

[AWS Analytical Engine Decision Framework]





Requirement	Athena	Redshift	EMR
Infrastructure management	Minimal	Managed warehouse	Cluster management required
Primary purpose	Ad-hoc S3 analytics	Enterprise DWH	Custom Big Data processing
Query style	SQL	SQL / analytics	Spark, Hive and related frameworks
Storage	S3	Redshift storage	Typically S3 + cluster storage
Pricing model	Query/data-scan oriented	Warehouse compute/storage	Cluster/compute resources

Best suited for	Occasional analytical queries	High-concurrency BI/DWH	Custom Spark/Hadoop workloads

Interview Rule of Thumb

Athena → "I already have data in S3 and want to query it."

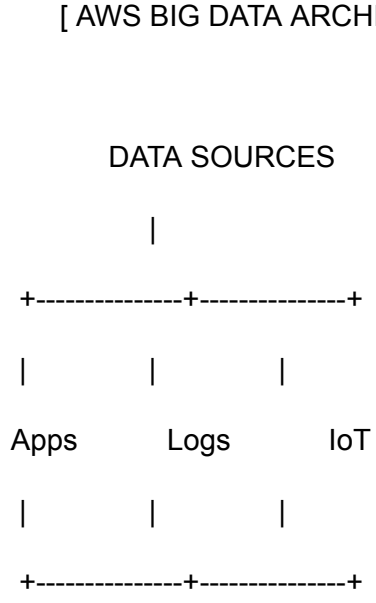
Redshift → "I need an enterprise analytical warehouse."

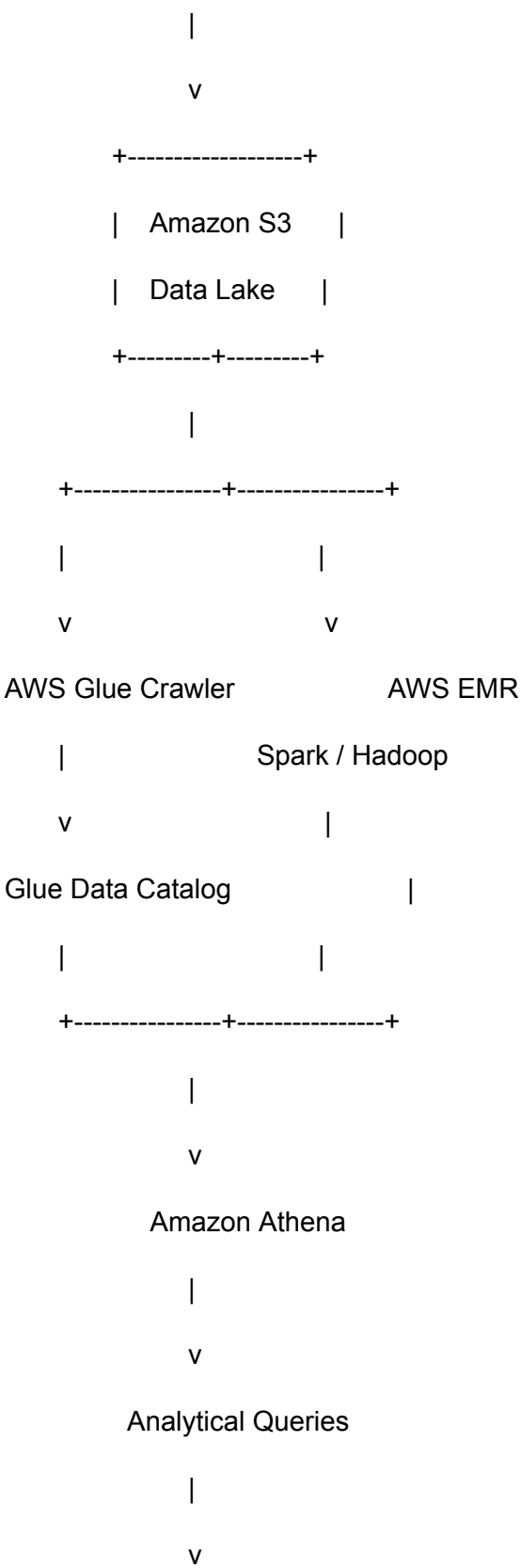
EMR → "I need control over Spark/Hadoop processing."

---

7. AWS Big Data Architecture — End-to-End View

The concepts from this week can be assembled into a typical AWS data platform:




Amazon S3
Data Lake
AWS Glue Crawler
AWS EMR
Spark / Hadoop
Glue Data Catalog
Amazon Athena
Analytical Queries

BI / Reporting

For enterprise warehouse workloads:

S3 / ETL / Applications

|

v

Amazon Redshift

|

v

BI / Enterprise DWH

---

8. Key Interview Questions — Week 31

Q1. What is Amazon S3's role in a Big Data architecture?

Answer:
Amazon S3 provides highly scalable object storage and is commonly used as the foundation of AWS data lakes. Analytical services such as Athena and EMR can process data directly from S3, separating storage from compute.

Q2. What is the difference between EC2 and EMR?

EC2 provides virtual machines where you control the operating environment and installed software.

EMR is a managed Big Data platform designed to simplify running distributed frameworks such as Spark and Hadoop.

Interview line:

EC2 gives me infrastructure-level compute; EMR gives me managed Big Data cluster capabilities.

Q3. What is the difference between Athena and Redshift?

Athena:

- Serverless
- Queries data directly in S3
- Excellent for ad-hoc analysis
- Data-scan-oriented pricing model

Redshift:

- Enterprise analytical warehouse
- MPP architecture
- Better suited to persistent, high-concurrency warehouse workloads

---

Q4. Why is Parquet preferred over CSV for Athena workloads?

Because Parquet is a **columnar format**.

If the query needs only a subset of columns, Athena can potentially read only those columns instead of scanning the entire row-based file.

It also works well with compression.

Q5. What is partition pruning?

Partition pruning allows the query engine to skip partitions that cannot satisfy the query predicate.

For example:

SELECT *

FROM orders

WHERE year = 2026

AND month = 7;

If the table is partitioned by `year` and `month`, Athena can avoid scanning unrelated partitions.

Q6. What is the AWS Glue Data Catalog?

It is a centralized metadata repository that stores information about datasets, including:

- Tables
- Columns
- Data types
- S3 locations
- Partitions

Athena can use this metadata to query S3 data.

Q7. What does a Glue Crawler do?

A crawler scans data sources, detects schemas and partitions, and registers the resulting metadata in the Glue Data Catalog.

Q8. What does `MSCK REPAIR TABLE` do?

It discovers partition directories in the underlying storage location and registers those partitions in the metastore/catalog so that Athena can query them.

Q9. Explain EC2 M, C and R instance families.

M → General Purpose

Balanced CPU + Memory

C → Compute Optimized

CPU intensive

R → Memory Optimized

Memory intensive

For example, a memory-heavy Spark workload may benefit from an **R-family** instance.

Q10. How can you reduce Athena costs?

The key techniques are:

1. Store data in compressed formats.
2. Prefer columnar formats such as Parquet.
3. Partition data appropriately.
4. Use partition filters in SQL queries.
5. Select only the required columns.
6. Avoid unnecessarily scanning large datasets.

---

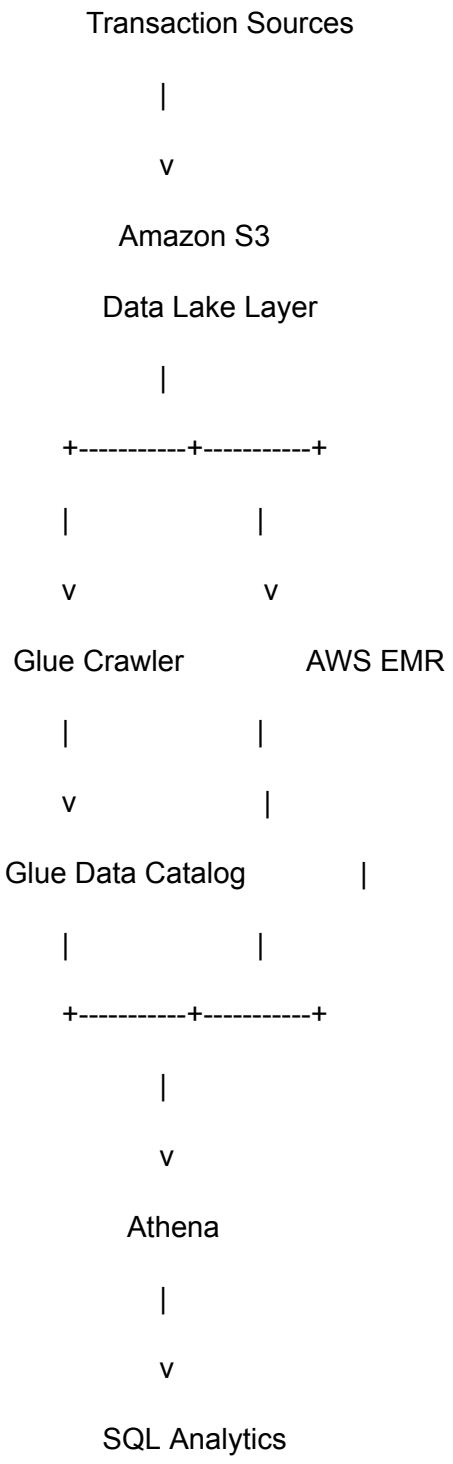
9. Production Scenario — Designing an AWS Data Lake

Scenario

An organization receives several TB of transaction data every day and needs:

- Low-cost storage
- Ad-hoc SQL analysis
- Spark processing
- Metadata discovery
- Minimal infrastructure management

Recommended Architecture



Why?

- **S3** → scalable data lake storage
- **Glue** → metadata/catalog management
- **EMR** → custom Spark processing
- **Athena** → serverless SQL analytics

This architecture keeps **storage and compute decoupled**.

---

10. Week 31 — Interview Revision Sheet

AWS Core Services

S3 → Object Storage / Data Lake

EC2 → Virtual Machines

Glue → ETL + Metadata Catalog

Athena → Serverless SQL on S3

Redshift → Enterprise MPP Data Warehouse

EMR → Managed Spark/Hadoop

Lambda → Serverless Functions

EC2 Families

M → General Purpose

C → Compute Optimized

R → Memory Optimized

EC2 Storage

Instance Store → Ephemeral

EBS → Persistent Block Storage

Athena Optimization

Compression

+

Parquet / ORC

+

Partition Pruning

=

Lower Data Scanned

=

Lower Query Cost

Glue

S3

↓

Crawler

↓

Glue Data Catalog

↓

Athena

↓

SQL Analytics

Analytics Engine Selection

Ad-hoc S3 SQL → Athena

Enterprise DWH → Redshift

Custom Spark/Hadoop → EMR

---

11. Complete Master Curriculum Progress Roadmap

==========

ULTIMATE BIG DATA & AZURE/AWS DATA ENGINEERING MASTER GUIDE

==========

[WEEKS 1 - 16: Distributed Computing, PySpark Core & Optimization]

• HDFS Architecture, YARN Schedulers, MapReduce, Spark RDDs, DataFrames API

• Catalyst Optimizer, Tungsten Execution Engine, Memory Pools, Partitioning vs Bucketing

• Join Optimizations, Partition Skew, AQE, Salting, Resource Sizing & CI/CD Testing

[WEEKS 17 - 21: Azure Infrastructure, Databricks & Unity Catalog]

• Azure Global Infrastructure, Availability Zones, Fault/Update Domains, CapEx vs OpEx

• Storage Services (Blob, ADLS Gen2, Access Tiers, LRS/ZRS/GRS Redundancy Models)

• Databricks Architecture, Control/Data Planes, DBFS, Secret Scopes, Delta Lake ACID Logs

• Lakehouse Architecture, Medallion Design (Bronze/Silver/Gold), Delta Engine & Photon

• Unity Catalog: 3-Level Namespace, Centralized Metastore, SCIM Provisioning & Access Control

[WEEKS 22 - 25: Pipeline Orchestration & End-to-End Enterprise Project]

• Azure Data Factory: Copy Activity, Mapping Data Flows, Dynamic Parameterization & Triggers

• Azure Synapse Analytics: Serverless SQL Pools, Dedicated SQL MPP Engine & Spark Pools

• Production Capstone Project: Event-Driven Pipeline, Key Vault, Azure SQL & AWS S3 Integration

[WEEKS 26 - 27: Enterprise Data Modeling & Technical Interview Prep]

• Normalization (1NF to 3NF) & Kimball Dimensional Design (Star vs Snowflake vs OBT)

• Surrogate Keys, Slowly Changing Dimensions (SCD 0/1/2/3) & PySpark SCD-2 Implementations

• Azure Storage & ADF Production Interview Q&A, Integration Runtimes & Performance Tuning

[WEEKS 28 - 29: Spark Structured Streaming Architecture]

• Micro-Batch Engine, DataStreamReader / DataStreamWriter, Output Modes & Triggers

• Checkpoint Storage, WAL Logs, Exactly-Once Guarantees, Watermarking & Streaming Joins

[WEEK 30: Apache Kafka Architecture & Real-Time Event Streaming]

• Distributed Event Platform Internals, Brokers, Topics, Partitions, Offsets & Consumer Groups

• Python Confluent-Kafka Producer Implementation & PySpark Kafka Stream Consumption

[WEEK 31: AWS Big Data Architecture & Serverless Analytics]

• AWS Ecosystem Mapping: S3, EC2, Glue, Athena, Redshift, EMR & Lambda

• EC2 Compute Families: M, C & R; AMIs, EBS, Instance Store & Purchasing Models

• Athena: Serverless SQL, S3 Query Architecture, Scan-Based Pricing & Spark/DPU Execution

• Athena Optimization: Compression, Parquet/ORC, Column Pruning & Partition Pruning

• AWS Glue Data Catalog, Crawlers, Partition Metadata & MSCK REPAIR TABLE

• AWS Analytics Selection: Athena vs Redshift vs EMR

==========

Master Study Guide Progress Status

Weeks 1–31 Complete.

The curriculum has now progressed from:

Distributed Computing → Spark → PySpark Optimization → Azure → Databricks → Data Modeling → Structured Streaming → Kafka → AWS Big Data Architecture.

Week 31 core interview chain

S3 → Glue Catalog → Athena

for serverless data-lake analytics,

S3 → EMR

for custom Spark/Hadoop processing, and

S3/ETL → Redshift

for enterprise warehouse workloads.

This is the key architecture to remember from Week 31.

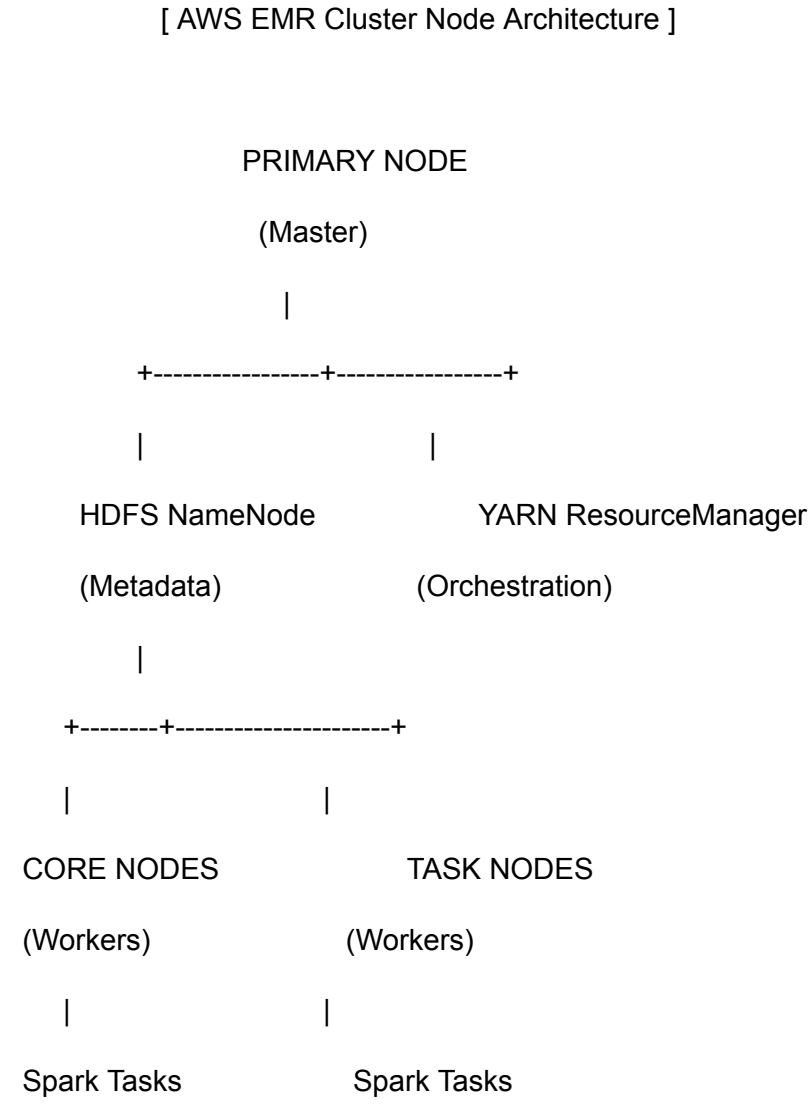
Week 32: Enterprise Big Data Compute & Governance — Amazon EMR & AWS Glue

---

1. Amazon EMR Architecture & Node Types

Amazon EMR (Elastic MapReduce) is AWS's managed Big Data platform that simplifies provisioning and running distributed frameworks such as **Apache Spark, Apache Hive, HBase, Presto, and Trino** over EC2 infrastructure.

EMR Cluster Architecture



YARN NodeManager YARN NodeManager

| |

HDFS DataNodes NO HDFS Storage

Stores Data Processing Only

EMR Node Types

1. Primary Node

The **Primary Node** is responsible for coordinating the EMR cluster.

It runs important cluster-management services such as:

- **HDFS NameNode** — maintains HDFS metadata.
- **YARN ResourceManager** — manages cluster resources and schedules applications.
- Cluster coordination and management processes.

Think of the Primary Node as the **control/coordination layer** of the EMR cluster.

---

2. Core Nodes

Core Nodes are worker nodes responsible for both:

- **Computation**
- **HDFS storage**

They typically run:

- Spark executors/tasks
- YARN NodeManager
- HDFS DataNode

Because Core Nodes contain HDFS data, removing them can result in data loss if the data has not been replicated elsewhere.

---

3. Task Nodes

Task Nodes are **compute-only worker nodes**.

They:

- Execute Spark/YARN tasks.
- Run YARN NodeManager.
- **Do not store HDFS data.**

This makes Task Nodes ideal for:

- Temporary workloads
- Elastic scaling
- Spot Instances
- Cost optimization

Because they do not contain HDFS data, they can be added or removed more freely than Core Nodes.

Core vs Task Nodes

Feature	Core Node	Task Node
Compute	Yes	Yes
Spark tasks	Yes	Yes
YARN NodeManager	Yes	Yes
HDFS DataNode	Yes	No
Stores HDFS data	Yes	No
Safe to scale down aggressively	No	Yes

Good candidate for Spot	Possible	Excellent
Primary purpose	Compute + Storage	Compute only

Interview Question

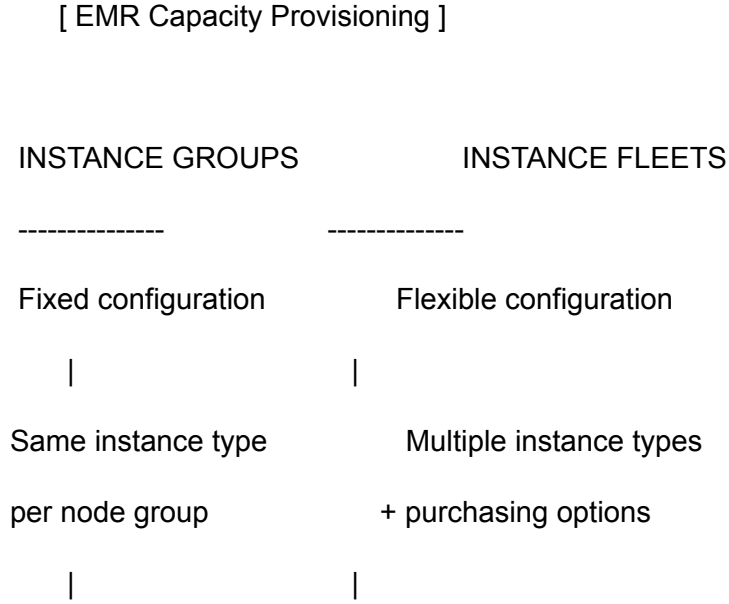
Q: Why would you use Task Nodes in an EMR cluster?

Answer:
Task Nodes provide additional compute capacity without adding HDFS storage. They are particularly useful when Spark workloads require more processing power but do not require additional distributed storage. Since they are stateless from the HDFS perspective, they can also be scaled down or provisioned with Spot capacity for cost optimization.

---

2. EMR Provisioning — Instance Groups vs Instance Fleets

EMR provides two major approaches for configuring EC2 capacity.



Simple / predictable capacity	Cost + availability optimization

Instance Groups

Instance Groups provide a relatively simple and uniform configuration.

For example:

Core Group

├── m5.xlarge

├── m5.xlarge

├── m5.xlarge

└── m5.xlarge

Instances in the same group use the configured instance type and purchasing model.

Advantages

- Simple configuration
- Predictable capacity
- Easy to understand
- Suitable for stable workloads

Disadvantage

Less flexibility when a particular EC2 instance type is unavailable or expensive.

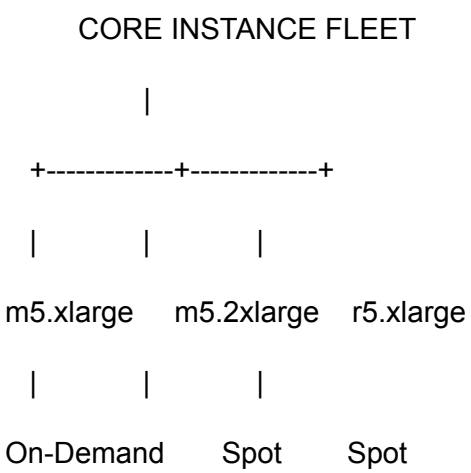
Instance Fleets

Instance Fleets provide greater flexibility.

A fleet can use:

- Multiple EC2 instance types
- Multiple sizes
- On-Demand capacity
- Spot capacity

Example:



EMR can use the available capacity options to satisfy the desired compute capacity.

Instance Groups vs Fleets

Feature	Instance Groups	Instance Fleets
Instance types	More uniform	Multiple types
Purchasing options	Simpler	Flexible
Configuration	Simple	More complex
Availability flexibility	Lower	Higher

Cost optimization	Moderate	Strong
Best use	Predictable workloads	Dynamic/cost-sensitive workloads

Interview Question

Q: Why would an enterprise choose Instance Fleets instead of Instance Groups?

Answer:
Instance Fleets allow the cluster to use multiple EC2 instance types and purchasing options, increasing the probability of obtaining capacity while also providing greater opportunities for Spot-based cost optimization.

---

3. EMR Cluster Resource Sizing

Consider a cluster with:

- 1 Primary Node
- 2 Core Nodes
- 1 Task Node

Each instance is:

m4.large

2 vCPUs

8 GB RAM

32 GB EBS

The processing worker pool consists of:

2 Core + 1 Task = 3 Worker Nodes

Compute Capacity

If each worker provides 2 vCPUs:

Total vCPUs

= 2 vCPUs × 3 workers

= 6 vCPUs

The source material's example applies an additional multiplier and arrives at **12 vCPUs**; retain that figure when following the source's specific calculation framework.

YARN Memory Allocation

Assuming 75% of memory is available to YARN:

Memory per worker

= 8 GB × 75%

= 6 GB

Total YARN memory

= 6 GB × 3 workers

= 18 GB

Therefore:

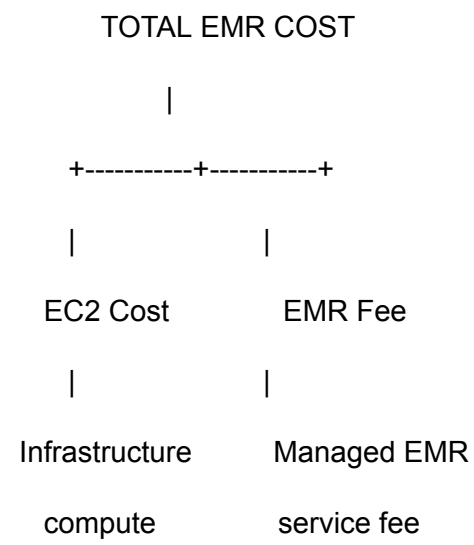
Usable YARN memory = 18 GB

> **Interview Tip:** Always distinguish between the physical memory of an EC2 instance and the memory actually available to YARN/Spark after reserving resources for the operating system and EMR services.

---

4. EMR Cost Model

EMR cluster costs involve two major components:



The source example uses:

EC2 m4.large

= $0.105 / instance-hour

EMR management fee

= $0.030 / instance-hour

Total

= $0.135 / instance-hour

For 4 instances:

4 × $0.135

= $0.54 per hour

Interview Question

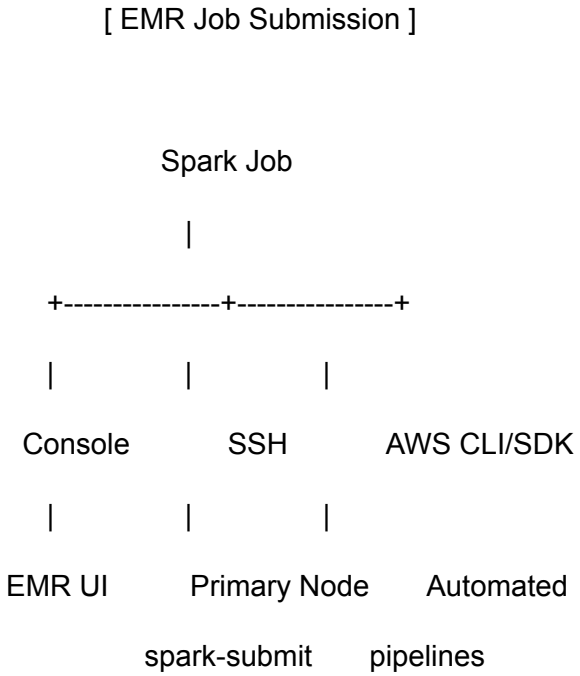
Q: What contributes to the cost of an EMR cluster?

Answer:
The major components include the underlying EC2 instance cost and the EMR service charge. Depending on the architecture, additional costs can come from storage, networking, S3, CloudWatch, and other AWS services used by the workload.

---

5. EMR Job Execution Models

EMR Spark applications can be submitted using several approaches.



1. EMR Management Console

Spark steps can be submitted through the AWS Management Console.

This is useful for:

- Manual testing
- Learning

- Interactive administration
- Small operational tasks

---

2. SSH into Primary Node

An engineer can connect to the Primary Node and execute:

```
spark-submit \

  --deploy-mode cluster \

  s3://bucket/path/job.py
```

This is useful for:

- Debugging
- Manual execution
- Interactive investigation

---

3. AWS CLI / SDK

For production automation, steps can be submitted programmatically.

Example:

```
aws emr add-steps \

  --cluster-id j-TSDACODLWSO6 \

  --region ap-south-1 \

  --steps
Type=CUSTOM_JAR,Name="AggregatedOrdersJob",ActionOnFailure=CONTINUE,Jar=command-runner.jar,Args=["spark-submit","--deploy-mode","cluster","s3://sample-emr-project2/spark-script/emr_aggregated_orders_data.py"]
```

This approach is particularly useful for:

- Airflow
- CI/CD
- Automated ETL
- Event-driven workflows
- Production orchestration

---

6. Transient vs Long-Running EMR Clusters

One of the most important EMR architecture decisions is whether a cluster should be temporary or continuously running.

Feature	Transient Cluster	Long-Running Cluster
Lifecycle	Created for a workload	Runs continuously
Termination	Automatically after completion	Manually/operationally terminated
Cost	Lower for intermittent workloads	Higher when idle
Best for	Scheduled ETL/batch	Shared/ad-hoc workloads
Resource utilization	High	Can be low during idle periods
Automation	Excellent	Moderate

Transient Cluster

Typical architecture:

10:00 PM

↓

Create EMR Cluster

↓

Run Spark ETL

↓

Write output to S3

↓

Terminate Cluster

This avoids paying for idle compute.

Long-Running Cluster

Typical architecture:

EMR Cluster

|

+-- Job A

+-- Job B

+-- Job C

+-- Interactive Analysis

+-- Job D

Useful when workloads arrive frequently and cluster startup overhead needs to be avoided.

Interview Question

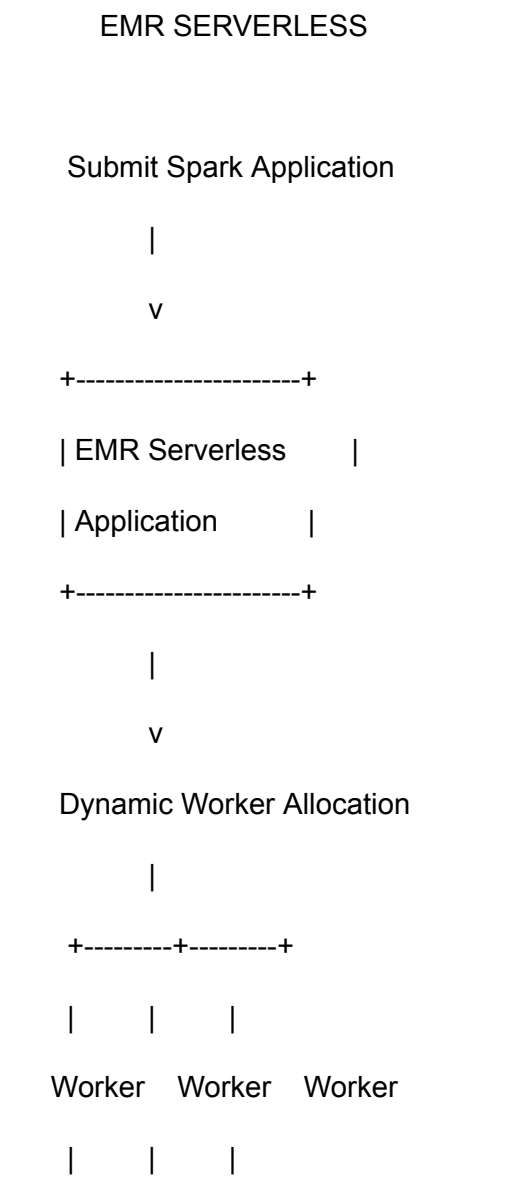
Q: Which EMR architecture would you choose for a daily 30-minute ETL job?

Answer:
A transient EMR cluster is generally preferable. The cluster can be created immediately before the ETL job, execute the Spark steps, write results to S3, and automatically terminate after successful completion. This minimizes idle infrastructure cost.

---

7. EMR Serverless

EMR Serverless allows users to run Apache Spark and Hive workloads without provisioning or managing EC2-based EMR clusters.



Spark Spark Spark

AWS manages:

- Infrastructure provisioning
- Worker allocation
- Scaling
- Capacity management

The user focuses primarily on the Spark application.

---

8. EMR Serverless Cost Model

The source material models EMR Serverless pricing using:

- **vCPU-hours**
- **GB-hours of memory**

The source rates are:

vCPU:

$0.052624 / vCPU-hour

Memory:

$0.0057785 / GB-hour

Example

Suppose each worker has:

4 vCPUs

30 GB RAM

Phase 1:

25 workers

4 vCPUs × 25

= 100 vCPUs

100 × 0.5 hours

= 50 vCPU-hours

vCPU cost:

50 × $0.052624

= $2.6312

Memory:

25 × 30 GB

= 750 GB

750 × 0.5 hours

= 375 GB-hours

Memory cost:

375 × $0.0057785

= $2.1669

The same calculation can be performed for subsequent autoscaling phases.

Key Interview Concept

EMR Serverless separates application logic from infrastructure management.

Instead of deciding:

> "How many EC2 instances should I provision?"

you primarily configure application-level resources and allow EMR Serverless to provision workers dynamically.

---

9. EMR vs EMR Serverless

Feature	EMR on EC2	EMR Serverless
Infrastructure	EC2-based cluster	AWS-managed
Cluster management	Required	Minimal
Node selection	Full control	Abstracted
Scaling	Configure/manage	Automatic
Customization	Very high	Lower
Good for	Complex/custom workloads	Variable serverless workloads
Idle cost	Possible	Reduced

Operational effort	Higher	Lower

Interview Question

Q: When would you choose EMR on EC2 over EMR Serverless?

Answer:
Choose EMR on EC2 when you need detailed control over instance types, networking, node configuration, storage, cluster topology, or specialized workload tuning. Choose EMR Serverless when reducing infrastructure-management overhead and dynamically scaling variable Spark workloads are more important.

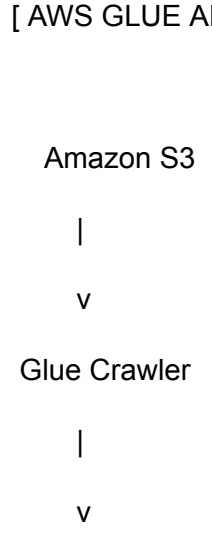
---

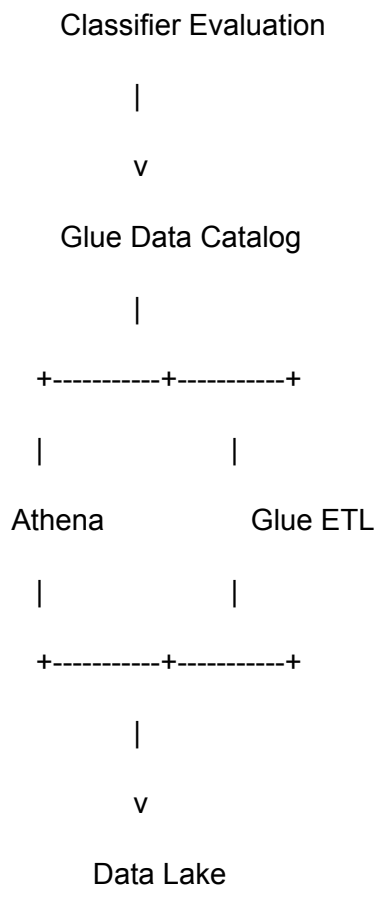
10. AWS Glue — Serverless Data Integration

AWS Glue is a serverless data integration service providing:

- Data discovery
- Metadata management
- ETL
- Schema discovery
- Cataloging
- Data preparation

A simplified architecture:





---

11. AWS Glue Data Catalog

The **Glue Data Catalog** is a centralized metadata repository.

It stores information such as:

- Table names
- Column names
- Data types
- S3 locations
- Partition information
- Table properties

It is compatible with the Hive metastore model and can be used by services such as Athena and Glue.

Important Distinction

The Glue Catalog **does not contain the actual data**.

Instead:

S3

↓

Actual Data

Glue Catalog

↓

Metadata describing the data

For example:

S3:

s3://company-data/orders/

Glue Catalog:

Table: orders

Columns:

order_id BIGINT

customer_id BIGINT

amount DOUBLE

order_date DATE

Location:

s3://company-data/orders/

---

12. AWS Glue Crawlers

A **Glue Crawler** automatically examines data sources and populates or updates metadata in the Glue Data Catalog.

Typical workflow:

S3 Files

|

v

Glue Crawler

|

+-- Detect Format

+-- Infer Schema

+-- Detect Partitions

+-- Apply Classifiers

|

v

Glue Data Catalog

Crawlers can work against sources such as:

- Amazon S3
- Amazon Redshift
- Relational databases
- Other supported data stores

---

13. Glue Schema Evolution

Suppose an S3 dataset initially contains:

order_id

customer_id

order_date

amount

The Glue Catalog initially stores four columns.

Later, a new file arrives containing:

order_id

customer_id

order_date

amount

discount

After the crawler runs again, the catalog can evolve to:

order_id

customer_id

order_date

amount

discount

Historical records that don't contain the new field can appear as:

discount = NULL

Interview Question

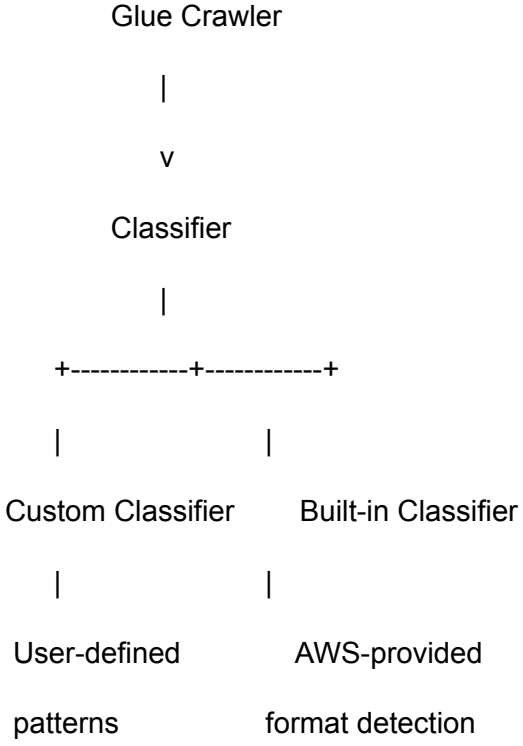
Q: What happens when a new column is added to files in an S3 data lake?

Answer:
A Glue crawler can detect the changed schema and update the Data Catalog metadata. Existing records that don't contain the new attribute can be represented with `NULL` when queried using a compatible schema interpretation.

---

14. AWS Glue Classifiers

A classifier determines the structure and format of data encountered by a crawler.



Built-in Classifiers

AWS provides classifiers for common formats such as:

- CSV
- JSON
- Parquet
- Avro
- ORC

Custom Classifiers

Custom classifiers are useful when data doesn't conform to standard formats.

For example:

[2026-07-26 10:30:15] [ERROR] jacky@example.com Connection failed

A custom Grok pattern could extract:

timestamp

severity

email

message

Conceptually:

Raw Log

|

v

Custom Grok Classifier

|

+--> timestamp

+--> severity

+--> email

+--> message

Classifier Priority

The source material describes:

1. Custom classifiers evaluated first
2. Built-in classifiers used as fallback

If a custom classifier successfully identifies the structure, the crawler can use that classification rather than continuing through the built-in classification list.

---

15. Handling Unstructured Logs with Grok

Consider:

[2026-07-26T10:30:00Z] [ERROR] jacky@example.com Connection failed

A conceptual Grok expression could identify:

TIMESTAMP_ISO8601 → timestamp

WORD → severity

EMAILADDRESS → email

remaining text → message

Result:

timestamp	severity	email	message

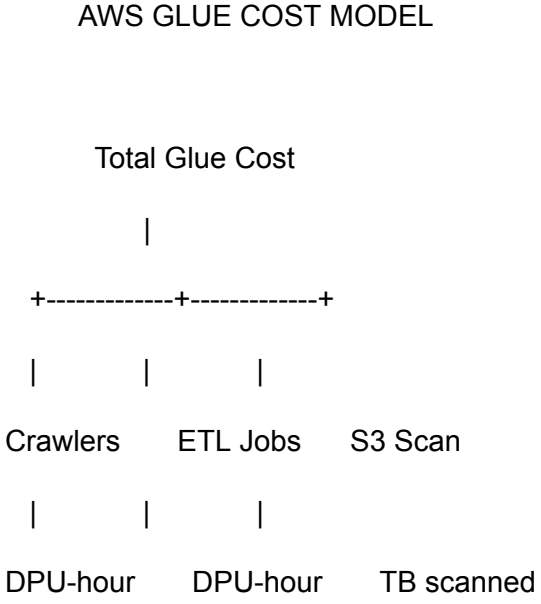
2026-07-26 10:30 ERROR jacky@example.com Connection failed

This transforms otherwise unstructured logs into queryable structured data.

---

16. Glue Pricing Components

The source material identifies the following pricing components:



Source-example rates:

Glue Crawler:

$0.44 / DPU-hour

Glue ETL:

$0.44 / DPU-hour

S3 scanning:

$5.00 / TB scanned

> **Important:** These are the rates provided in the study material and should be treated as the course's example pricing model; actual AWS pricing varies by region, service configuration, and pricing model.

---

17. EMR vs Glue — Interview Comparison

This is a very common architecture interview question.

Feature	Amazon EMR	AWS Glue
Primary purpose	Big Data compute platform	Serverless data integration
Spark	Yes	Yes
Infrastructure	EC2 or Serverless	Serverless
Cluster control	High with EMR on EC2	Abstracted
ETL	Yes	Strong focus
Metadata catalog	Not primary purpose	Core capability
Crawlers	No	Yes

Custom Spark configuration	High	More abstracted
Best for	Custom distributed processing	Managed ETL/cataloging

Simple Decision Rule

Need highly customized Spark/Hadoop compute?

|

YES

↓

EMR

Need serverless ETL + catalog + schema discovery?

|

YES

↓

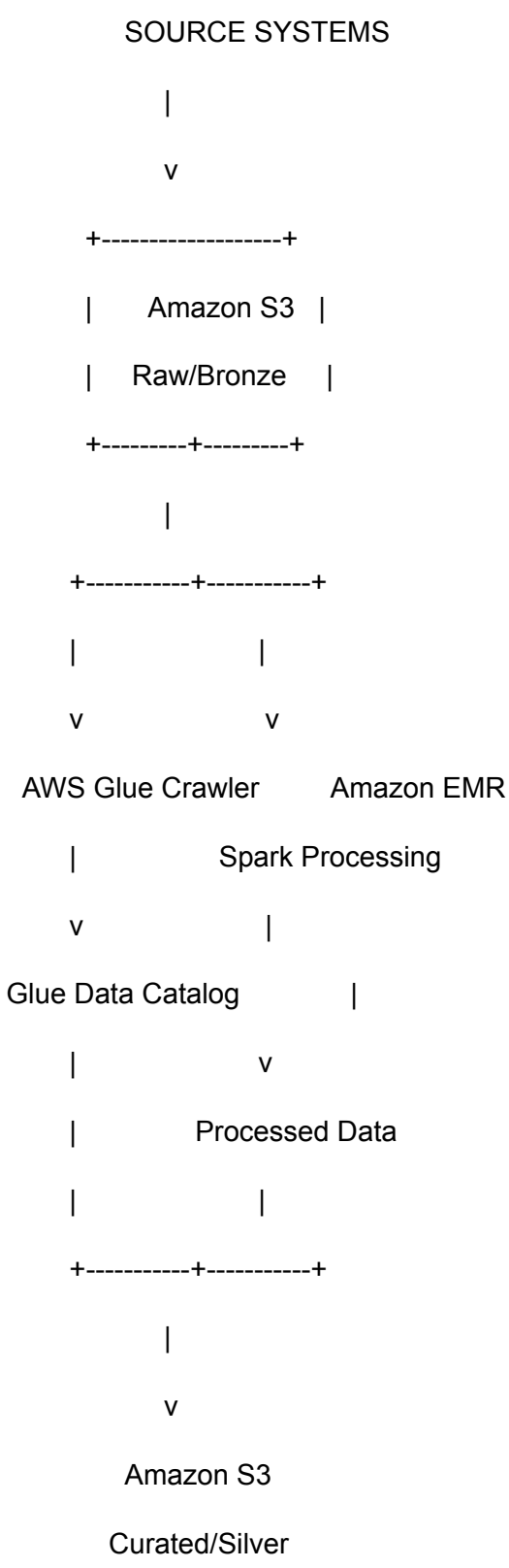
Glue

---

18. Production Architecture — EMR + Glue + S3 + Athena

A common AWS data-lake architecture combines these services:

AWS DATA LAKE ARCHITECTURE


SOURCE SYSTEMS
Amazon S3
Raw/Bronze
AWS Glue Crawler
Amazon EMR
Spark Processing
Glue Data Catalog
Processed Data
Amazon S3
Curated/Silver

|

v

Glue Catalog

|

v

Amazon Athena

|

v

BI / Analytics Users

Example Enterprise Flow

1. Raw files land in **Amazon S3**.
2. **Glue Crawler** discovers their schema.
3. Metadata is registered in **Glue Data Catalog**.
4. **EMR Spark** performs large-scale transformations.
5. Processed data is written back to S3 in an analytics-friendly format.
6. Glue updates the metadata.
7. **Athena** queries the curated datasets.
8. BI tools consume analytical results.

---

19. Key Interview Questions — Week 32

Q1. What are the three major EMR node types?

Answer:
Primary, Core, and Task Nodes.

- Primary → cluster coordination
- Core → compute + HDFS storage
- Task → compute only

Q2. Why are Task Nodes suitable for Spot Instances?

Answer:
Task Nodes don't store HDFS data, so their interruption does not directly remove persistent HDFS data. They can therefore be used as elastic compute capacity and are good candidates for cost-efficient Spot capacity.

Q3. Core Node vs Task Node?

Answer:
Core Nodes provide both compute and HDFS storage, while Task Nodes provide compute only.

Q4. Instance Groups vs Instance Fleets?

Answer:
Instance Groups use a simpler, more uniform configuration. Instance Fleets provide greater flexibility by allowing multiple EC2 instance types and purchasing options.

Q5. When would you use a transient EMR cluster?

Answer:
For intermittent workloads such as scheduled ETL jobs. The cluster can be created for the workload and automatically terminated afterward, reducing idle infrastructure cost.

Q6. What is EMR Serverless?

Answer:
A serverless execution environment for frameworks such as Spark and Hive where AWS manages the underlying compute infrastructure and dynamically allocates workers.

Q7. What is the Glue Data Catalog?

Answer:
A centralized metadata repository containing information about datasets, including schemas, tables, locations, and partitions. It stores metadata rather than the underlying data.

Q8. What does a Glue Crawler do?

Answer:
It scans supported data sources, determines schemas and formats, detects partitions, applies classifiers, and registers or updates table metadata in the Glue Data Catalog.

Q9. What is schema evolution in Glue?

Answer:
Schema evolution occurs when the structure of a dataset changes over time—for example, when a new column is added. A crawler can detect the change and update the Data Catalog schema.

Q10. When would you use a custom Glue classifier?

Answer:
When the source data doesn't conform to standard formats recognized by built-in classifiers, such as application logs containing custom patterns.

Q11. EMR vs Glue?

Answer:
EMR is primarily a distributed compute platform for running Spark, Hadoop and related workloads with significant infrastructure control. Glue is a serverless data integration and metadata platform focused on ETL, crawling, cataloging and schema discovery.

---

20. Week 32 Interview Cheat Sheet

╠═══╣

EMR

• Managed Big Data platform

• Primary = Coordination

• Core = Compute + HDFS

• Task = Compute only

PROVISIONING

• Instance Groups = Simple / Uniform

• Instance Fleets = Flexible / Multiple instance types

CLUSTER LIFECYCLE

• Transient = Create → Process → Terminate

• Long-running = Persistent shared cluster

EMR SERVERLESS

• No EC2 cluster management

• Dynamic worker allocation

• vCPU + Memory based billing

GLUE

• Serverless data integration

• Crawler = Discover schema

• Catalog = Store metadata

╠ • Classifier = Identify structure ╣

╠ • Custom Grok = Unstructured/custom logs ╣

╠ • Schema evolution = Catalog adapts to new structure ╣

╠ ╣

╠ ARCHITECTURE ╣

╠ S3 → Glue Crawler → Glue Catalog → EMR → S3 → Athena ╣

╚═══╝

---

21. Complete Master Curriculum Progress Roadmap

==========

ULTIMATE BIG DATA & AZURE/AWS DATA ENGINEERING MASTER GUIDE

==========

[WEEKS 1 - 16: Distributed Computing, PySpark Core & Optimization]

• HDFS Architecture, YARN Schedulers, MapReduce, Spark RDDs, DataFrames API

• Catalyst Optimizer, Tungsten Execution Engine, Memory Pools, Partitioning vs Bucketing

• Join Optimizations, Partition Skew, AQE, Salting, Resource Sizing & CI/CD Pytest Frameworks

[WEEKS 17 - 21: Azure Infrastructure, Databricks & Unity Catalog]

• Azure Global Infrastructure, Availability Zones, Fault/Update Domains, CapEx vs OpEx

• Storage Services (Blob, ADLS Gen2, Access Tiers, LRS/ZRS/GRS Redundancy Models)

• Databricks Architecture, Control/Data Planes, DBFS, Secret Scopes, Delta Lake ACID Logs

• Lakehouse Architecture, Medallion Design (Bronze/Silver/Gold), Delta Engine & Photon

• Unity Catalog: 3-Level Namespace, Centralized Metastore, SCIM Provisioning & Access Control

[WEEKS 22 - 25: Pipeline Orchestration & End-to-End Cloud Project]

• Azure Data Factory: Copy Activity, Mapping Data Flows, Dynamic Parameterization & Triggers

• Azure Synapse Analytics: Serverless SQL Pools, Dedicated SQL MPP Engine & Spark Pools

• Production Capstone: Event-Driven Pipeline, Key Vault, Azure SQL & AWS S3 Integration

[WEEKS 26 - 27: Enterprise Data Modeling & Technical Interview Prep]

• Normalization (1NF to 3NF) & Kimball Dimensional Design

• Surrogate Keys, Slowly Changing Dimensions (SCD 0/1/2/3)

• Azure Storage & ADF Production Q&A, Integration Runtimes & Performance Tuning

[WEEKS 28 - 29: Spark Structured Streaming Architecture]

• Micro-Batch Engine, DataStreamReader / DataStreamWriter, Output Modes & Triggers

• Checkpointing, WAL Logs, Exactly-Once Guarantees, Watermarking & Streaming Joins

[WEEK 30: Apache Kafka Architecture & Real-Time Event Streaming]

• Distributed Event Platform Internals, Brokers, Topics, Partitions, Offsets & Consumer Groups

• Python Confluent-Kafka Producer & PySpark Kafka Stream Consumption

[WEEK 31: AWS Big Data Architecture & Serverless Analytics]

- AWS Service Mapping: S3, EC2, Glue, Athena, Redshift, EMR & Lambda
- EC2 Instance Families, AMI, EBS & Purchasing Models
- Athena Query Engine, Scan Optimization & Serverless Analytics

[WEEK 32: Enterprise Big Data Compute & Governance]

- Amazon EMR Architecture: Primary, Core & Task Nodes
- Instance Groups vs Instance Fleets & EMR Cluster Resource Sizing
- EMR Step Execution: Console, SSH, AWS CLI & SDK
- Transient vs Long-Running EMR Clusters
- EMR Serverless Architecture, Autoscaling & Resource-Based Billing
- AWS Glue Data Catalog, Crawlers & Metadata Management
- Glue Schema Evolution & Custom Grok Classifiers
- EMR vs Glue Architecture Selection & Production Design Patterns

==========

Week 32 Status

Master Study Guide: Weeks 1–32 complete.

The progression now moves from **distributed computing → Spark/PySpark → Azure → streaming/Kafka → AWS analytics → enterprise AWS compute and governance**, with Week 32 adding the important distinction between **EMR as a compute platform** and **Glue as a serverless integration/catalog platform**.

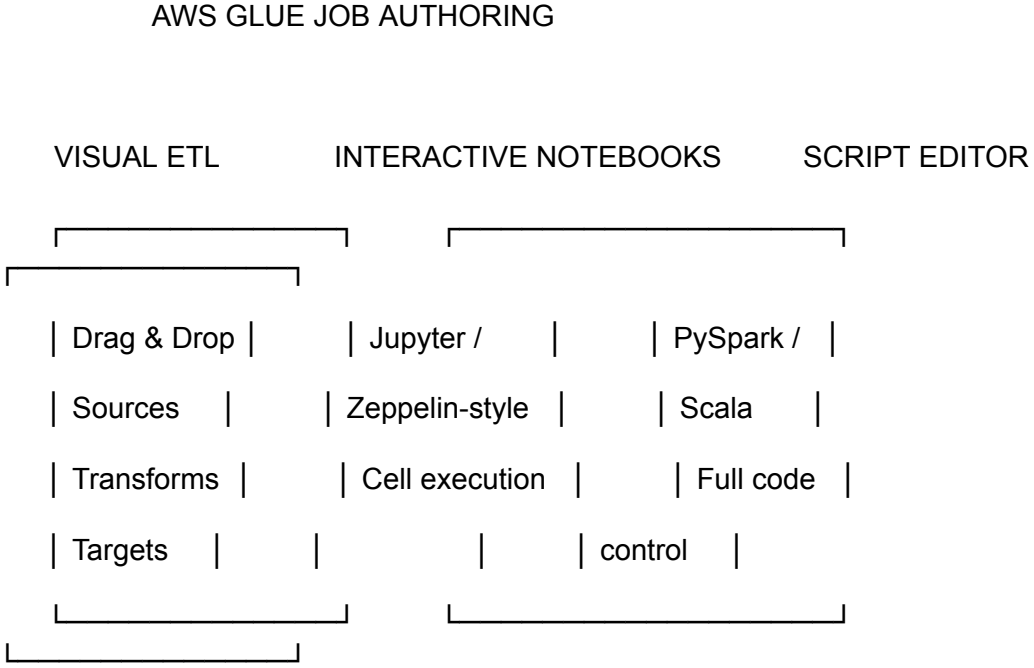
Week 33: AWS Glue Visual ETL, Data Governance, Sensitive Data Protection & Custom Transforms

---

1. AWS Glue ETL Execution Models & Infrastructure

AWS Glue is a fully managed, serverless data integration service used for ETL processing across services such as **Amazon S3, Amazon RDS/MySQL, and Amazon Redshift**.

AWS Glue Job Authoring Paradigms



Three Ways to Build Glue Jobs

Method	Description	Best For

Visual ETL	Drag-and-drop graphical pipeline	Low-code ETL development
Interactive Notebooks	Cell-by-cell execution	Exploration and development
Script Editor	Direct PySpark/Scala coding	Complex/custom transformations

---

Glue Worker Types

AWS Glue provisions serverless compute using **Data Processing Units (DPUs)**.

Worker Type	Resources	Typical Use
G.1X	4 vCPUs / 16 GB RAM	Standard ETL workloads
G.2X	8 vCPUs / 32 GB RAM	Heavy compute/memory workloads
Flex	Spare AWS capacity	Non-urgent, cost-sensitive jobs

Important Defaults from the Course

- Spark ETL jobs allocate **10 workers by default**.
- Minimum worker configuration: **2 workers**.
- Default job timeout: **2,880 minutes / 48 hours**.
- **Flex** is useful when execution start time can vary.

Key Interview Point

> **G.1X vs G.2X:** G.2X provides approximately twice the vCPU and memory resources per worker compared with G.1X.

---

2. AWS Glue Visual ETL & Built-In Transformations

Glue Visual ETL represents an ETL pipeline as a graph consisting of:

SOURCE

  |

  ▼

TRANSFORM

  |

  ▼

TARGET

Example:

Orders ──────────────┐

                ├──> JOIN ──> CHANGE SCHEMA ──> S3 Parquet

Customers ──────────┘

    customer_id

Three Major Node Categories

1. Sources

Examples:

- Amazon S3
- Amazon RDS
- Amazon Redshift

- Glue Data Catalog

2. Transforms

Transform and manipulate the incoming data.

3. Targets

Write the processed data to a destination such as S3, RDS, or Redshift.

Core Visual Transformations

Change Schema

Equivalent to **ApplyMapping**.

Used to:

- Rename columns
- Change data types
- Drop unwanted fields
- Perform schema coercion

Example:

order_id STRING

↓

order_id BIGINT

Join

Combines multiple datasets using matching fields.

Example:

Orders.customer_id

=

Customers.customer_id

SQL Query

Allows SQL expressions to be executed against temporary views directly inside the visual pipeline.

Useful when standard visual transformations aren't sufficient.

Aggregate

Performs grouping and mathematical operations.

Example:

GROUP BY customer_id

COUNT(order_status)

SUM(order_amount)

AVG(order_amount)

Filter

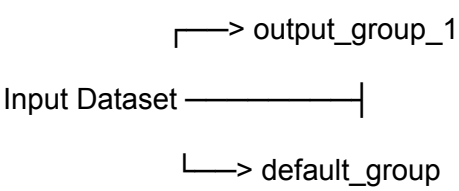
Removes records that don't satisfy specified conditions.

Supports:

- Global AND
- Global OR
- Regex-based conditions
- String comparisons
- Numeric comparisons

Conditional Router

Splits one dataset into multiple branches.



Useful when different records need different downstream processing.

Utility Transformations

Glue also provides:

- Select Fields
- Drop Fields
- Rename Fields
- Drop Null Fields
- Drop Duplicates

Drop Duplicates can operate on:

- Entire records
- A selected subset of key fields

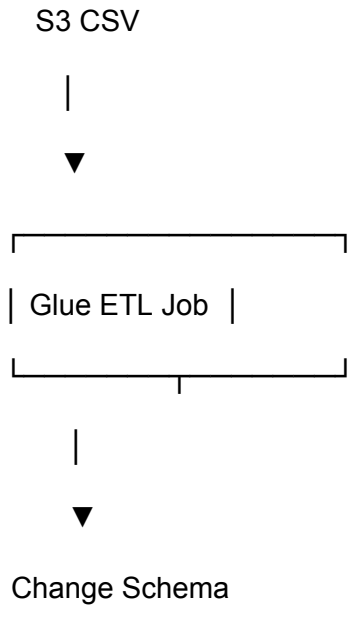
---

3. S3 → Amazon RDS Relational Ingestion

AWS Glue can transfer data from **Amazon S3 to Amazon RDS** using:

- Glue Visual ETL
- JDBC connections
- Glue Data Catalog
- Glue Crawlers

Architecture



Step 1 — Provision RDS

Create an RDS MySQL instance.

The course example uses:

Port: 3306

The VPC security group must permit the required inbound connection.

Step 2 — Create Database and Table

Example:

```
CREATE DATABASE sales_db;


USE sales_db;


CREATE TABLE orders (
    orderid_id BIGINT,
    order_date VARCHAR(255),
    customer_id BIGINT,
    order_status VARCHAR(255)
);
```

Step 3 — Configure Glue JDBC Connection

Glue connects to RDS using a JDBC endpoint such as:

jdbc:mysql://<rds-endpoint>:3306/sales_db

A Glue Crawler can then inspect the database and register its tables in the **Glue Data Catalog**.

Step 4 — Execute ETL

S3 CSV

↓

Glue Visual ETL

↓

Change Schema

↓

Glue Catalog

↓

JDBC

↓

RDS Table

Key Concept

JDBC Connection handles connectivity between Glue and the relational database, while the **Glue Data Catalog** provides metadata about the target/source structures.

---

4. Sensitive Data Protection & PII Governance

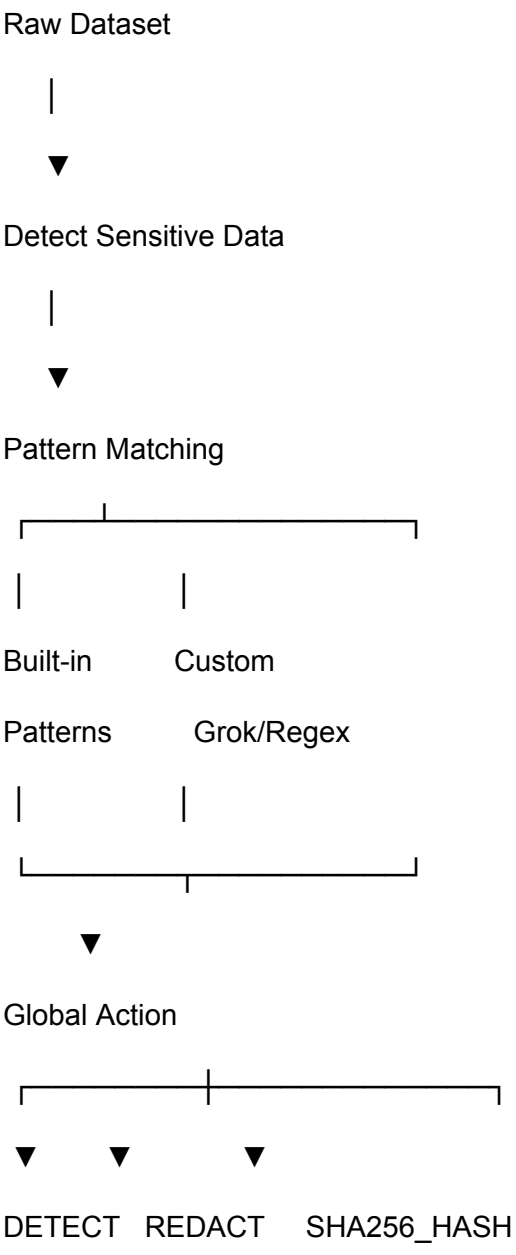
Data lakes frequently contain **Personally Identifiable Information (PII)**.

Examples include:

- Email addresses
- Phone numbers
- Social Security Numbers
- Birth dates

AWS Glue provides sensitive-data detection and protection mechanisms.

PII Governance Pipeline



Detection Models

1. Identify Sensitive Data in Each Row

Performs a deeper scan across individual cells.

The purpose is to identify individual PII occurrences.

---

2. Identify Columns with Sensitive Data

Uses a sampled set of rows and a **Detection Threshold**.

For example:

Rows scanned = 100%

Detection threshold = 20%

If PII occurrences exceed the threshold, the entire column is classified as sensitive.

---

5. Custom PII Detection Using Regex

Built-in detection patterns may not recognize organization-specific fields.

For example, a custom pattern can identify dates of birth.

Course example:

\d{4}-(0[1-9]|1[0-2])-(0[1-9]|[12][0-9]|3[01])

This targets a date format similar to:

YYYY-MM-DD

Why Custom Patterns?

Use custom regex patterns when:

- The data is organization-specific.
- Built-in classifiers don't recognize the field.
- You need precise control over sensitive-data identification.

---

6. PII Protection Actions

After sensitive information is detected, Glue can apply a global action.

Action	What Happens	Example Result
DETECT	Identifies sensitive data without modifying original values	Metadata identifying `EMAIL`, `PHONE_NUMBER`, etc.
REDACT	Masks sensitive values	`john@example.com → ****`
SHA256_HASH	Applies one-way SHA-256 hashing	Original value → irreversible hash

DETECT

Preserves original values and adds information describing the detected sensitive entities.

REDACT

Masks the original sensitive value.

Useful when the downstream system does not need the actual PII.

SHA256_HASH

Converts the original value into a one-way cryptographic hash.

Useful when you need to:

- Protect the original value.
- Maintain consistency.
- Perform secure matching/joins using the transformed value.

Important Distinction

DETECT → Identify

REDACT → Hide

SHA256_HASH → Transform irreversibly

---

7. AWS Glue Data Quality & DeeQu

AWS Glue supports data-quality evaluation using the open-source **DeeQu** framework.

Rules are expressed using **Data Quality Definition Language (DQDL)**.

Two major operating modes are covered:

AWS GLUE DATA QUALITY

DATA QUALITY AT REST

│

├── Static cataloged tables

├── DQDL rulesets

└── Results → S3 / CloudWatch

VS

DATA QUALITY IN MOVE

├── Active Visual ETL pipeline

├── Dynamic evaluation

└── originalData + ruleOutcomes

Data Quality at Rest

Used when evaluating data that already exists.

Example:

S3 Dataset

↓

Glue Catalog Table

↓

DQDL Rules

↓

Quality Evaluation

↓

Results

Data Quality in Move

Quality checks occur **inside an active ETL pipeline**.

The pipeline can separate:

Input

  |

  ▼

Data Quality Evaluation

  |

  ├──> originalData

  |

  └──> ruleOutcomes

This makes it possible to evaluate quality while data is being transformed.

---

8. DQDL Rules

Example ruleset:

```
Rules = [
    IsPrimaryKey "customerid",
    IsComplete "firstname",
    Uniqueness "firstname" > 0.95,
    IsComplete "email",
    Uniqueness "email" = 1.0,
    IsComplete "phonenumber",
```

```
    ColumnValues "phonenumber" between 1111111110 and 9999999999,
    ColumnValues "loyaltypoints" >= 0,
    ColumnCount = 8,
    CustomSql "select count(*) from primary where joindate > lastpurchasedate" = 0
]
```

Important Rules

`IsPrimaryKey`

Checks whether the column satisfies primary-key expectations.

`IsComplete`

Checks whether values are populated.

IsComplete "email"

`Uniqueness`

Checks uniqueness percentage.

Uniqueness "email" = 1.0

means the email values must be completely unique according to the rule.

`ColumnValues`

Checks whether values satisfy a specified condition.

Example:

ColumnValues "loyaltypoints" >= 0

`ColumnCount`

Validates the number of columns.

ColumnCount = 8

`CustomSql`

Allows custom SQL logic when built-in rules aren't sufficient.

---

9. Custom Transformations in AWS Glue

Visual ETL is powerful, but complex business logic may require custom PySpark code.

AWS Glue supports custom transformations using:

- Python
- PySpark
- DynamicFrames
- DataFrames

DynamicFrameCollection

Glue custom transformation nodes receive datasets as a:

DynamicFrameCollection

To manipulate the data with standard PySpark DataFrame APIs:

DynamicFrameCollection

↓

Select DynamicFrame

↓

.toDF()

↓

PySpark DataFrame

↓

Transform

↓

DynamicFrame

↓

DynamicFrameCollection

---

10. Custom PySpark Transformation Example

```
from awsglue.dynamicframe import DynamicFrame, DynamicFrameCollection
from pyspark.sql.functions import when


def MyTransform(glueContext, dfc) -> DynamicFrameCollection:


    # Extract DynamicFrame from collection
    df = dfc.select(list(dfc.keys())[0]).toDF()


    # Apply transformation
    result_df = df.withColumn(
        'CustomerSegment',
        when(df['LoyalPoints'] < 1000, 'Bronze')
```

```
        .when(
            (df['LoyalPoints'] >= 1000) &
            (df['LoyalPoints'] < 2000),
            'Silver'
        )
        .otherwise('Gold')
    )

    # Convert back to DynamicFrame
    dyf_segmented = DynamicFrame.fromDF(
        result_df,
        glueContext,
        "segmented_loyalty_points"
    )

    # Return collection
    return DynamicFrameCollection(
        {"CustomTransform0": dyf_segmented},
        glueContext
    )
```

Transformation Logic

LoyalPoints < 1000

↓

Bronze

1000 ≤ LoyalPoints < 2000

↓

Silver

LoyalPoints ≥ 2000

↓

Gold

---

11. Why Select From Collection Is Important

A custom transform can return multiple DynamicFrames inside a:

DynamicFrameCollection

However, downstream Glue nodes generally need a specific **DynamicFrame**.

Therefore:

DynamicFrameCollection

↓

Select From Collection

↓

Specific DynamicFrame

↓

S3 / RDS / Other Target

Interview Question

Why do we need Select From Collection after a custom transform?

Answer: Because a custom transform returns a `DynamicFrameCollection`, while downstream processing generally needs a specific `DynamicFrame`. The Select From Collection node extracts the required DynamicFrame.

---

12. Reusable Custom Visual Transforms

Custom transformation logic can be converted into reusable Visual ETL nodes that can be shared across development teams.

Storage Location

The course specifies the Glue assets bucket path:

s3://aws-glue-assets-<account_number>-<region>/transforms/

Two files are required.

---

1. Configuration File

Example:

MyTransform.json

Defines:

- Parameter inputs
- UI labels
- Node name
- Description
- Configuration exposed to users

---

2. Transformation Function

Example:

MyTransform.py

Contains:

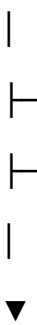
- PySpark logic
- DynamicFrame processing
- Business transformation implementation

Reusable Transform Architecture

MyTransform.json



MyTransform.py



Reusable Visual ETL Node

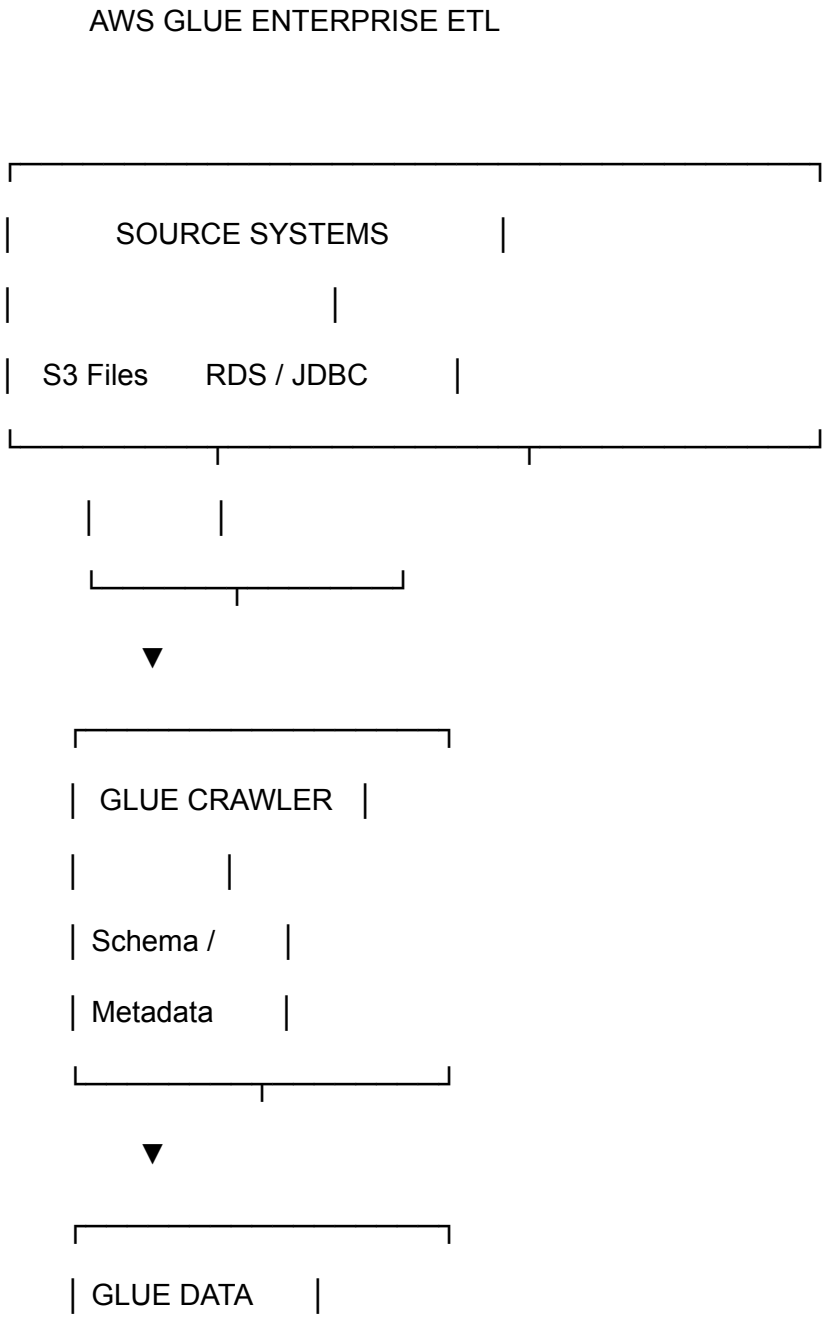


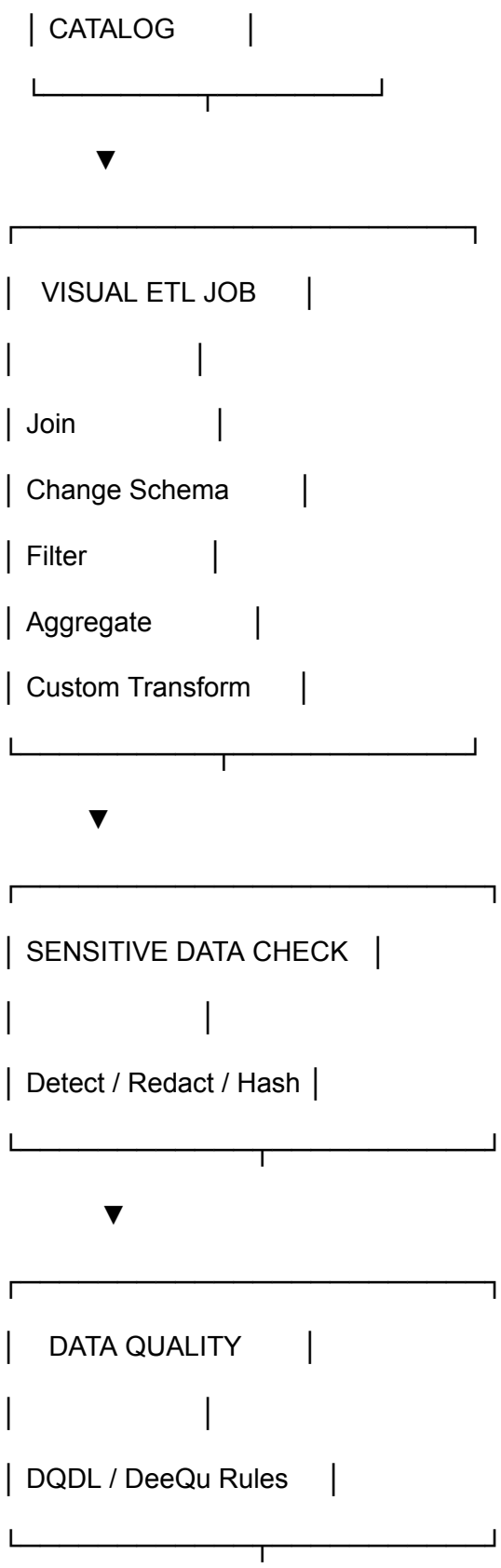
└── Team C

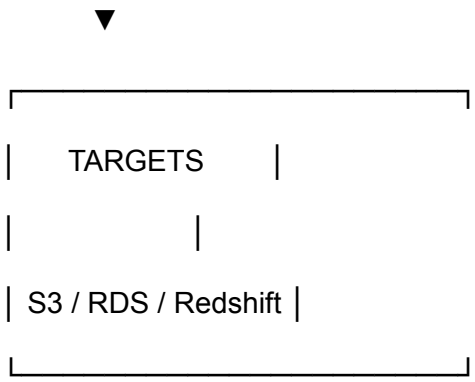
---

13. Week 33 — End-to-End Architecture

The major concepts from this week can be connected into one enterprise pipeline:




CATALOG
VISUAL ETL JOB
Join
Change Schema
Filter
Aggregate
Custom Transform
SENSITIVE DATA CHECK
Detect / Redact / Hash
DATA QUALITY
DQDL / DeeQu Rules



---

14. Week 33 — Key Concepts to Remember

AWS Glue

> **Serverless data integration + ETL + metadata + data quality + governance**

Visual ETL

> **Source → Transform → Target**

DynamicFrame vs DataFrame

> **Glue-native processing uses DynamicFrames; custom PySpark logic can convert them to DataFrames.**

PII Protection

DETECT → Identify

REDACT → Mask

SHA256_HASH → One-way hash

Data Quality

At Rest → Evaluate existing/cataloged data

In Move → Evaluate data during ETL processing

Custom Transform

DynamicFrameCollection

↓

Select DynamicFrame

↓

DataFrame

↓

PySpark Logic

↓

DynamicFrame

↓

DynamicFrameCollection

Reusable Visual Transform

JSON Configuration

+

Python Transformation

↓

Reusable Glue Visual Node

---

15. Interview Revision — Week 33

Q1. What are the three ways to author an AWS Glue job?

Answer: Visual ETL, Interactive Notebooks, and Script Editor.

Q2. What is the difference between G.1X and G.2X?

Answer: G.1X provides 4 vCPUs and 16 GB RAM, while G.2X provides 8 vCPUs and 32 GB RAM per worker.

Q3. What does Change Schema do?

Answer: It can rename columns, change data types, and remove unwanted fields.

Q4. Why is Glue Data Catalog important?

Answer: It stores metadata describing datasets, schemas, partitions, and table structures used by Glue and other AWS analytics services.

Q5. What is the purpose of a Glue Crawler?

Answer: It discovers data structures, infers schemas, detects partitions, and registers metadata in the Glue Data Catalog.

Q6. What are the three PII global actions?

Answer:

DETECT

REDACT

SHA256_HASH

Q7. What is the difference between row-level and column-level sensitive-data detection?

Answer: Row-level detection scans individual cells/rows to identify specific PII instances, whereas column-level detection determines whether an entire column should be treated as sensitive based on sampled detection and a threshold.

Q8. What is DQDL?

Answer: Data Quality Definition Language, used to define AWS Glue Data Quality rules.

Q9. What framework powers Glue Data Quality?

Answer: The course identifies the open-source **DeeQu** framework.

Q10. Why convert DynamicFrames to DataFrames?

Answer: To use standard PySpark DataFrame APIs and functions when implementing custom transformation logic.

Q11. Why does a custom transform return a DynamicFrameCollection?

Answer: It allows a custom transformation to produce one or multiple DynamicFrames as outputs.

Q12. Why use Select From Collection?

Answer: To extract a specific DynamicFrame from the returned DynamicFrameCollection for downstream processing.

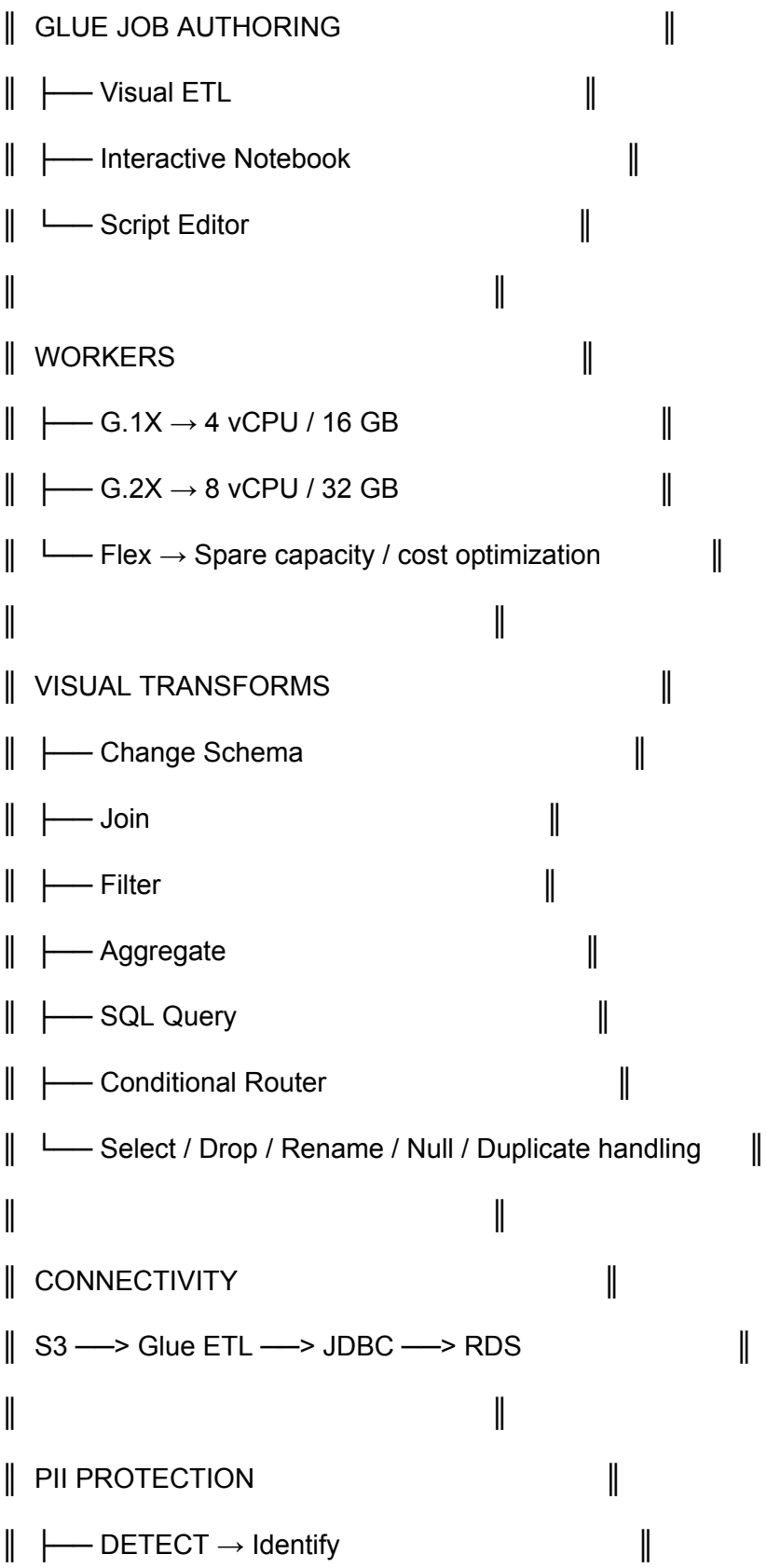
Q13. How do you make a custom transformation reusable?

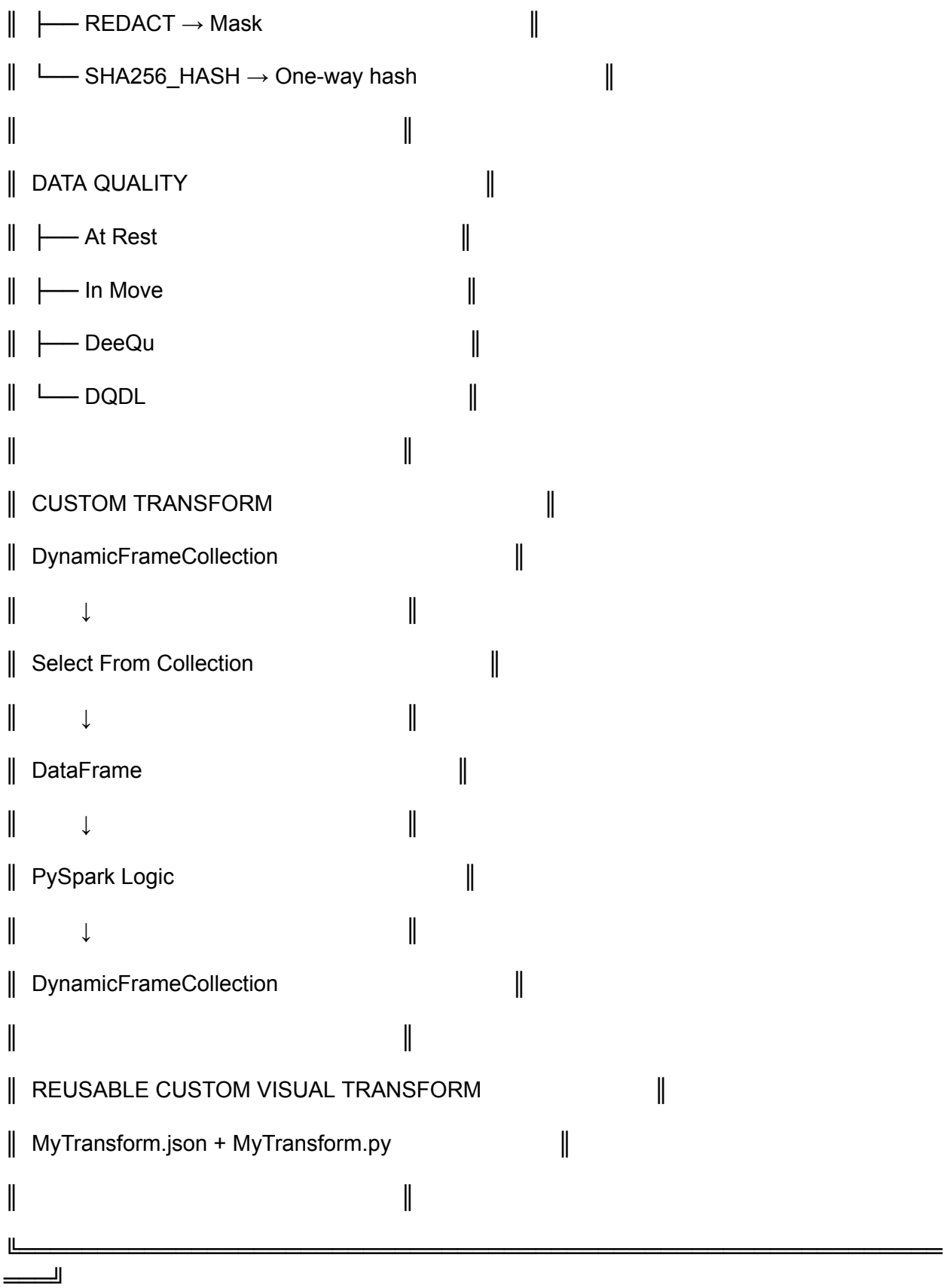
Answer: Package the transformation configuration and implementation into the required JSON and Python files and place them in the Glue assets `transforms/` location so they can be exposed as reusable visual nodes.

---

16. Week 33 — One-Page Revision Sheet




GLUE JOB AUTHORING
Visual ETL
Interactive Notebook
Script Editor
WORKERS
G.1X → 4 vCPU / 16 GB
G.2X → 8 vCPU / 32 GB
Flex → Spare capacity / cost optimization
VISUAL TRANSFORMS
Change Schema
Join
Filter
Aggregate
SQL Query
Conditional Router
Select / Drop / Rename / Null / Duplicate handling
CONNECTIVITY
S3 —> Glue ETL —> JDBC —> RDS
PII PROTECTION
DETECT → Identify


├── REDACT → Mask
└── SHA256_HASH → One-way hash
DATA QUALITY
├── At Rest
├── In Move
├── DeeQu
└── DQDL
CUSTOM TRANSFORM
DynamicFrameCollection
↓
Select From Collection
↓
DataFrame
↓
PySpark Logic
↓
DynamicFrameCollection
REUSABLE CUSTOM VISUAL TRANSFORM
MyTransform.json + MyTransform.py

---

17. Master Curriculum Progress — Updated Through Week 33

==========

ULTIMATE BIG DATA & AZURE/AWS DATA ENGINEERING MASTER GUIDE

==========

[WEEKS 1 - 16: Distributed Computing, PySpark Core & Optimization]

- HDFS Architecture, YARN Schedulers, MapReduce, Spark RDDs, DataFrames API
- Catalyst Optimizer, Tungsten Execution Engine, Memory Pools
- Partitioning vs Bucketing, Join Optimization, Partition Skew
- AQE, Salting, Resource Sizing & CI/CD Pytest Frameworks

[WEEKS 17 - 21: Azure Infrastructure, Databricks & Unity Catalog]

- Azure Global Infrastructure, Availability Zones, Fault/Update Domains
- Blob, ADLS Gen2, Access Tiers, LRS/ZRS/GRS
- Databricks Architecture, DBFS, Secret Scopes, Delta Lake ACID
- Lakehouse Architecture, Medallion Design & Delta Engine
- Unity Catalog, SCIM Provisioning & Access Control

[WEEKS 22 - 25: Pipeline Orchestration & Enterprise Project]

- Azure Data Factory: Copy Activity, Mapping Data Flows,

Dynamic Parameterization & Triggers

- Azure Synapse: Serverless SQL, Dedicated SQL & Spark Pools
- Event-Driven Pipeline, Key Vault, Azure SQL & AWS S3 Integration

[WEEKS 26 - 27: Enterprise Data Modeling & Interview Preparation]

- Normalization & Kimball Dimensional Design
- Star vs Snowflake vs OBT
- Surrogate Keys & SCD 0/1/2/3
- PySpark SCD-2, ADF & Azure Storage Interview Preparation

[WEEKS 28 - 29: Spark Structured Streaming]

- Micro-Batch Engine
- DataStreamReader / DataStreamWriter
- Output Modes & Triggers
- Checkpointing, WAL, Exactly-Once
- Watermarking & Streaming Joins

[WEEK 30: Apache Kafka & Real-Time Event Streaming]

- Brokers, Topics, Partitions, Offsets & Consumer Groups
- Confluent-Kafka Producer
- PySpark Kafka Stream Consumption
- Kafka → Spark → Delta / Kafka Sink Pipelines

[WEEKS 31 - 32: AWS Big Data Ecosystem & Compute]

• S3, EC2, Glue, Athena, Redshift, EMR & Lambda

• EC2 M/C/R Families, AMI, EBS & Purchasing Models

• Athena Trino SQL vs Spark DPU

• Query Scan Reduction & Partition Pruning

• EMR Primary/Core/Task Nodes

• Instance Groups vs Instance Fleets

• EMR Serverless & Cost Calculations

• Glue Catalog, Crawlers, Schema Evolution & Classifiers

[WEEK 33: AWS Glue Visual ETL, Governance & Security]

• Visual ETL, G.1X/G.2X/Flex Workers & Transformation Nodes

• S3 → RDS Ingestion using JDBC

• PII Detection & Custom Regex Patterns

• DETECT, REDACT & SHA256_HASH

• Glue Data Quality, DeeQu & DQDL

• Data Quality at Rest vs In Move

• DynamicFrameCollection & Custom PySpark Transforms

• Select From Collection

• Reusable Custom Visual Transforms

==========

Week 33 takeaway

Week 31 established the AWS analytics ecosystem → **Week 32** went deeper into EMR and Glue infrastructure → **Week 33** moves into the **enterprise Glue layer: visual ETL, connectivity, PII governance, data quality, and reusable custom transformations**.

The key mental model for this week is:

Discover → Catalog → Transform → Protect → Validate → Deliver → Reuse

Week 34: AWS Object Storage, Event-Driven Compute & Redshift Enterprise Data Warehousing

1. Amazon S3 — Object Storage & Lifecycle Management

Amazon S3 (Simple Storage Service) is AWS object storage designed for storing and retrieving virtually unlimited amounts of unstructured and semi-structured data.

S3 Object Hierarchy

GLOBAL AWS NAMESPACE

|

▼

REGION

(us-east-1)

|

▼

BUCKET

(tt-s3-demo-1)

|

▼

OBJECT KEY

(input/orders.csv)

|

├── Object Data / Payload

└── Metadata + Access Policies

Key S3 Concepts

Concept	Meaning
Object Storage	Data is stored as objects identified by keys
Bucket	Logical container for objects
Object Key	Complete identifier/path of an object
Region	AWS geographical location hosting the bucket
Object Payload	Actual stored data
Metadata	Information associated with the object

Important Limits / Characteristics

- Designed durability: **99.999999999% (11 9s)**
- A single object can be up to **5 TB**
- Bucket names have a **global namespace**
- S3 is fundamentally **object storage**, not a traditional hierarchical filesystem.

---

2. S3 Storage Classes

S3 provides multiple storage classes based on **access frequency, retrieval requirements, and cost**.

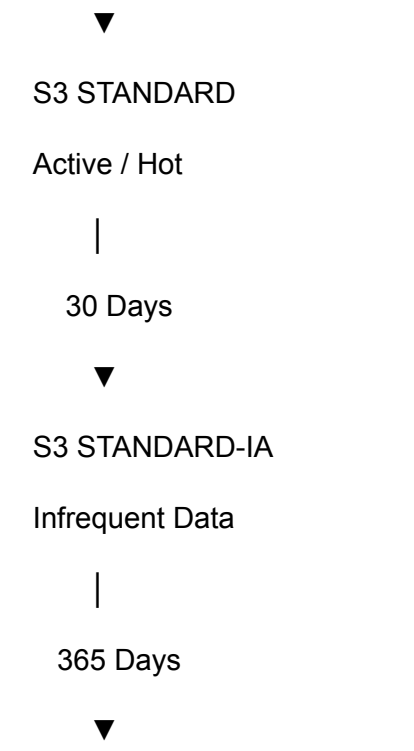
Storage Class	Access Pattern	Typical Usage
S3 Standard	Frequent	Active data lakes and frequently accessed data
S3 Intelligent-Tiering	Unknown/changing	Automatically optimizes storage cost
S3 Standard-IA	Infrequent	Backups and DR data
S3 Express One Zone	High-performance, low latency	Latency-sensitive workloads
S3 Glacier Instant Retrieval	Rare but immediate	Compliance/archive data needing fast retrieval
S3 Glacier Flexible Retrieval	Rare	Historical archives
S3 Glacier Deep Archive	Extremely rare	Long-term preservation

Lifecycle Management

Instead of manually moving files between storage tiers, S3 **Lifecycle Rules** can automate transitions.

DATA AGE

|



Core Idea

> **Store hot data in expensive/high-performance tiers and automatically transition older data to cheaper archival tiers.**

This reduces long-term storage cost without requiring manual movement of objects.

---

3. AWS Lambda — Event-Driven Serverless Compute

AWS Lambda is a serverless compute service that executes code in response to events.

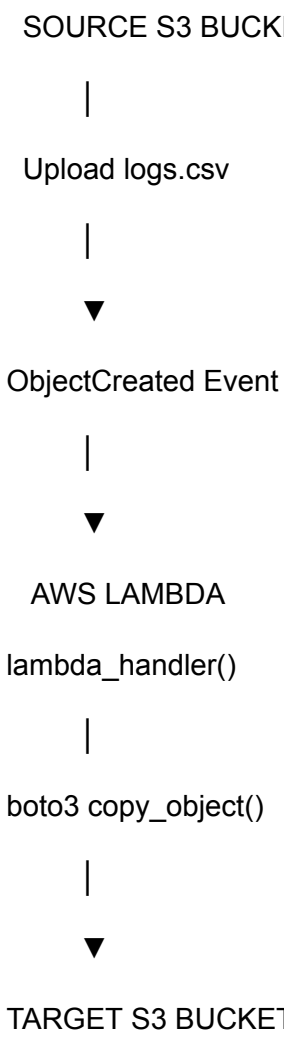
Examples:

- S3 object upload

- API Gateway request
- Other AWS service events

The key advantage is that **you don't provision or manage servers**.

S3 → Lambda Architecture



Lambda Execution Characteristics

- Maximum execution time: **15 minutes / 900 seconds**
- Automatically scales horizontally with incoming requests.
- Suitable for **event-driven, short-lived workloads**.
- Not intended for long-running processing jobs.

---

4. Lambda S3 Event Handler

The Lambda function receives information about the newly created S3 object through the event payload.

Event Information

event

```
└── Records[]
    └── s3
        ├── bucket
        │   └── name
        └── object
            └── key
```

The application extracts:

source_bucket = event['Records'][0]['s3']['bucket']['name']

source_key = event['Records'][0]['s3']['object']['key']

A destination bucket can be retrieved from an environment variable:

destination_bucket = os.environ.get("target_bucket")

Then the object can be copied using the AWS SDK:

```
s3_client.copy_object(
    Bucket=destination_bucket,
    CopySource={'Bucket': source_bucket, 'Key': source_key},
    Key=destination_key
```

)

Production Design Principle

The S3 client is initialized **outside the Lambda handler**:

s3_client = boto3.client('s3')

This allows the execution environment to potentially reuse the client across invocations.

---

5. Lambda Layers

A **Lambda Layer** is a reusable `.zip` package containing dependencies or shared code that can be attached to Lambda functions.

Useful for:

- Custom Python modules
- `pandas`
- `requests`
- Other reusable libraries

Layer Structure

lambda_layer.zip

└── python/

  └── lib/

    └── python3.12/

      └── site-packages/

        └── custom_module/

Why Use Layers?

Without layers:

Lambda Function A ──> pandas

Lambda Function B ──> pandas

Lambda Function C ──> pandas

With a shared layer:

```
            ┌── Lambda A
            │
Shared Layer ──────┼── Lambda B
            │
            └── Lambda C
```

Key benefit: reusable dependencies can be maintained independently from individual Lambda deployment packages.

---

6. Amazon Redshift — Enterprise Data Warehouse

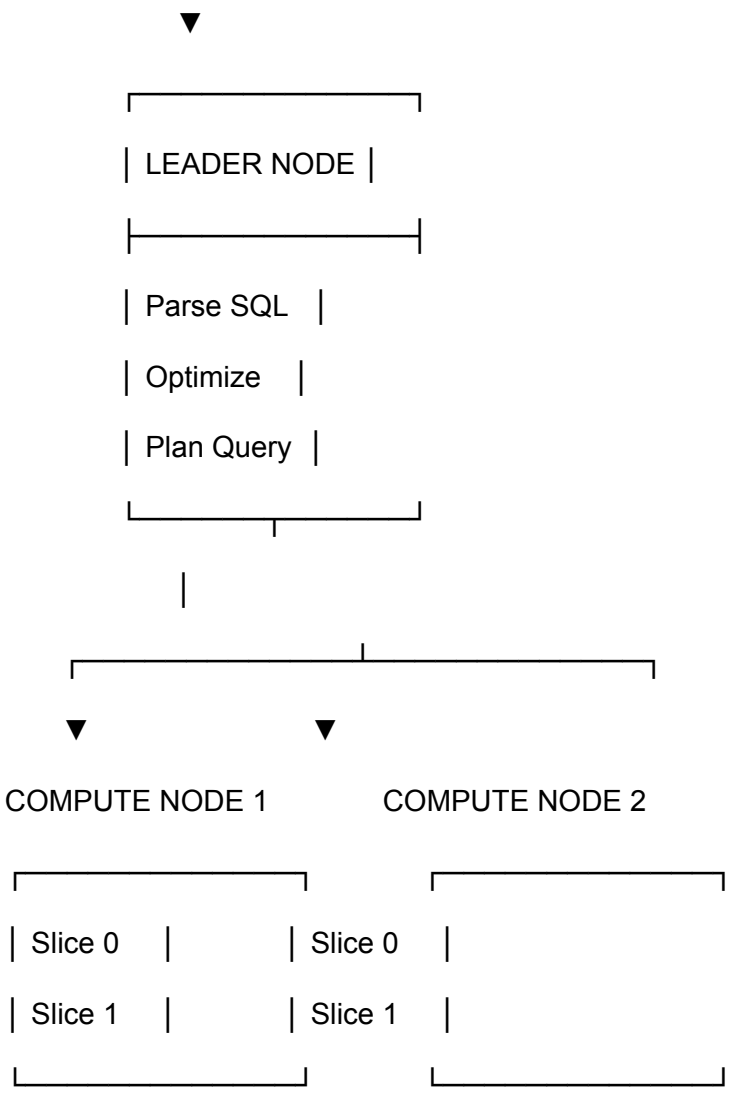
Amazon Redshift is a managed, cloud-based **OLAP data warehouse** designed for large-scale analytical workloads.

Its architecture is based on **Massively Parallel Processing (MPP)**.

CLIENT APPLICATIONS

BI Tools / SQL / ETL

|



Redshift Components

1. Leader Node

Responsible for:

- Receiving client queries
- Parsing SQL
- Creating optimized execution plans
- Compiling query code
- Distributing work
- Aggregating results

2. Compute Nodes

Execute the actual query-processing workload against distributed data.

3. Slices

Each compute node is divided into **virtual slices**.

Each slice manages a portion of:

- CPU
- Memory
- Disk

This enables parallel execution.

---

7. Redshift Performance Optimization

Redshift's analytical performance comes from several architectural features.

Columnar Storage

Instead of storing:

Row 1 → A B C D

Row 2 → A B C D

Row 3 → A B C D

Redshift stores values by column:

Column A → A A A

Column B → B B B

Column C → C C C

Column D → D D D

If a query needs only two columns, unnecessary data does not need to be read.

MPP

Large queries are distributed across multiple compute slices.

Column Compression

Compression reduces:

- Storage footprint
- Disk I/O
- Amount of data processed

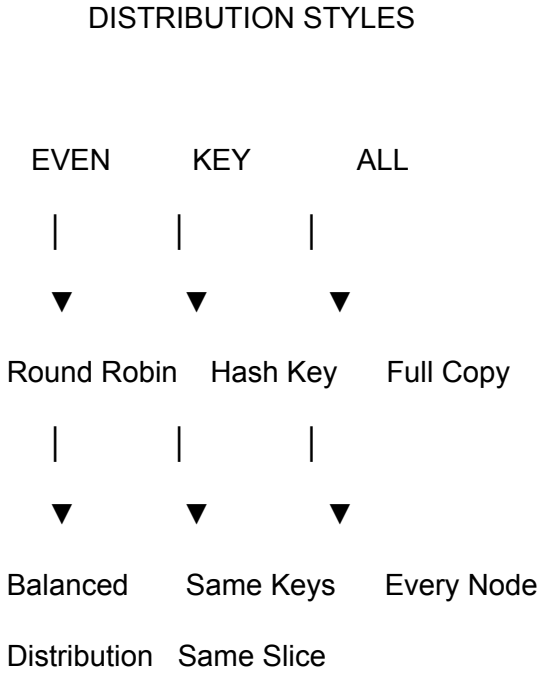
Result Set Caching

Previously generated query results can be cached, allowing identical subsequent queries to return much faster.

---

8. Redshift Distribution Styles

Distribution determines **where rows are physically stored across compute-node slices**.



Distribution Style Comparison

Style	Mechanism	Best Use
AUTO	Redshift chooses distribution strategy	Default/general workloads
EVEN	Round-robin distribution	Tables without strong join relationships
KEY	Hashes a selected column	Frequently joined large tables
ALL	Copies table to every node	Small dimension tables

KEY Distribution

Suppose:

DISTKEY = customer_id

Rows with the same customer ID are directed to the same slice.

This can reduce network data movement during joins.

---

9. Redshift Sort Keys

A **SORTKEY** determines the physical ordering of data within storage blocks.

Zone Map Pruning

Redshift maintains metadata about values contained in blocks.

For example:

Block 1 → order_id: 1–1000

Block 2 → order_id: 1001–2000

Block 3 → order_id: 2001–3000

If the query asks:

WHERE order_id = 2500

Redshift can skip Blocks 1 and 2 and scan only Block 3.

Sort Merge Bucket (SMB) Joins

Using compatible distribution and sorting strategies can allow highly efficient local merge joins without unnecessary:

- Network shuffling
- Sorting

---

10. DISTKEY + SORTKEY Example

```
CREATE TABLE retail_db.public.orders (
    order_id INTEGER,
    order_date VARCHAR(100),
    order_customer_id INTEGER DISTKEY,
    order_status VARCHAR(30)
)
SORTKEY(order_id);
```

Here:

order_customer_id → DISTKEY

order_id → SORTKEY

Remember

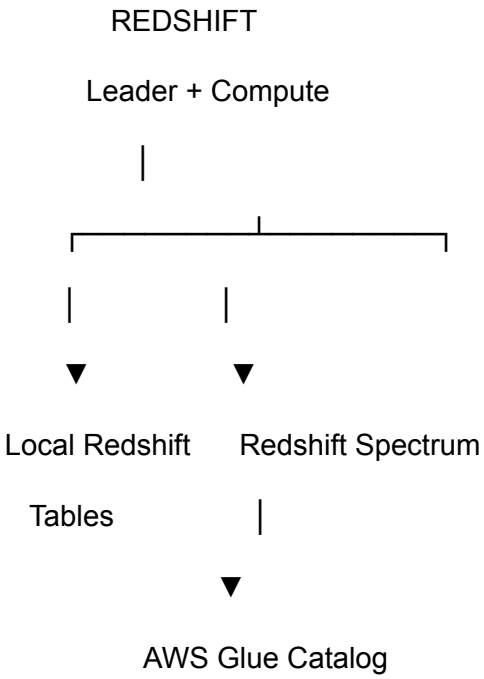
> **DISTKEY controls WHERE data goes.**
>
> **SORTKEY controls HOW data is physically ordered.**

This distinction is extremely important in Redshift interviews.

---

11. Redshift Spectrum — Query S3 Without Loading Data

Redshift Spectrum allows Redshift to query data directly in Amazon S3.





Why Spectrum?

Traditional approach:

S3 → COPY → Redshift → Query

Spectrum:

S3 ——————————————→ Query directly

↑

Redshift Spectrum

This is particularly useful when data is too large or unnecessary to fully ingest into Redshift.

---

12. Spectrum External Schema

An external schema connects Redshift to the AWS Glue Data Catalog.

```
CREATE EXTERNAL SCHEMA spectrum
FROM DATA CATALOG
DATABASE 'retaildemo'
IAM_ROLE 'arn:aws:iam::123456789012:role/RedshiftSpectrumRole'
CREATE EXTERNAL DATABASE IF NOT EXISTS;
```

Then an external table can point to S3:

```
CREATE EXTERNAL TABLE spectrum.orders (
    order_id INTEGER,
    order_date VARCHAR(100),
    order_customer_id INTEGER,
    order_status VARCHAR(30)
)
ROW FORMAT DELIMITED FIELDS TERMINATED BY ','
STORED AS TEXTFILE
LOCATION 's3://tt-redshift-demo/retail-data/orders';
```

The table definition is in the catalog, while the actual data remains in S3.

---

13. Federated Query — S3 + Redshift

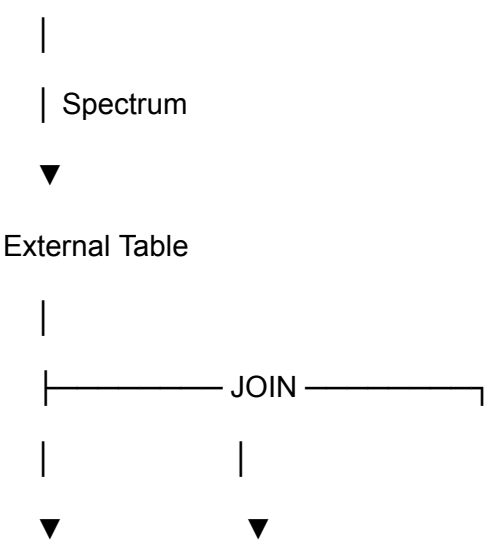
One powerful use case is joining:

- External S3 data
- Internal Redshift tables

```
SELECT
    o.order_id,
    o.order_date,
    c.customer_fname,
    c.customer_lname
FROM spectrum.orders o
```

```
JOIN retail_db.public.customers c
  ON o.order_customer_id = c.customer_id;
```

Conceptually:



---

14. Redshift COPY Command

The **COPY command** is designed for high-throughput bulk loading of data from S3 into Redshift.

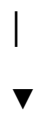
```
COPY retail_db.sales.orders
FROM 's3://tt-redshift-demo/retail-data/orders/'
IAM_ROLE 'arn:aws:iam::123456789012:role/RedshiftAccessRole'
FORMAT AS CSV
DELIMITER ','
```

IGNOREHEADER 1

REGION AS 'us-east-1';

Basic Flow

S3 Files



COPY Command



Parallel Loading



Redshift Managed Storage

COPY vs Spectrum

Feature	COPY	Spectrum
Data location	Moves into Redshift	Remains in S3
Primary purpose	Data ingestion	Query external data
Processing	Redshift-managed storage	S3 + Spectrum

Best for	Frequently queried warehouse data	Large/external lake data

---

15. Redshift Auto Copy

Auto Copy can automate ingestion of new S3 files.

Conceptually:

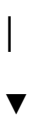
New S3 File



S3 Detection



Auto Copy Job



Redshift Table

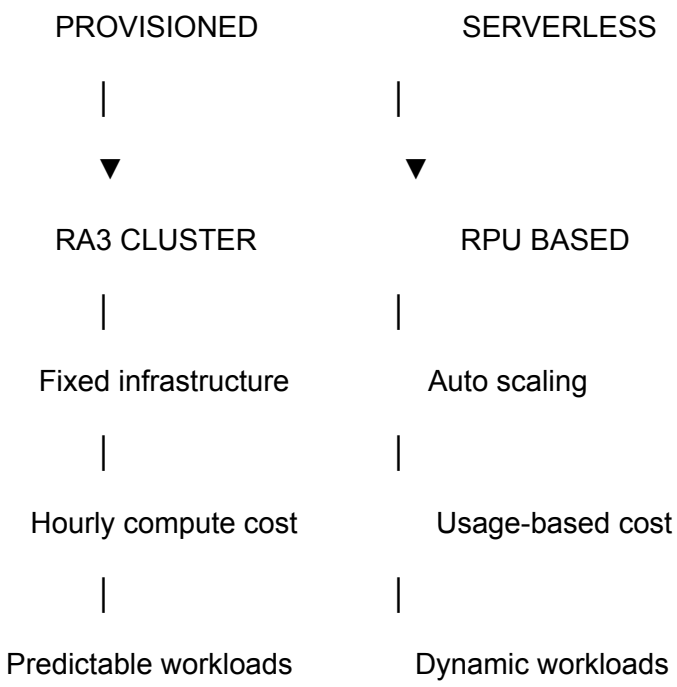
This reduces the need to manually run COPY commands whenever new files arrive.

---

16. Redshift Provisioned vs Serverless

Redshift supports two major deployment models.

REDSHIFT DEPLOYMENT OPTIONS



Provisioned RA3

Characteristics:

- Fixed cluster configuration
- Hourly compute billing
- Managed storage billed separately
- Appropriate for predictable workloads

Redshift Serverless

Characteristics:

- Automatically scales processing resources
- Uses **Redshift Processing Units (RPUs)**
- Usage-based billing
- Well suited for variable or unpredictable workloads

---

17. Redshift Cost Calculation

Provisioned Cluster

General model:

Compute Cost

=

Number of Nodes

×

Hourly Node Price

×

Hours Running

Then:

Total Cost

=

Compute Cost

+

Managed Storage Cost

+

Spectrum Cost

Example from the Week 34 Material

4 × ra3.xlarge

730 hours/month

$3.26/hour

Compute:

4 × $3.26 × 730

= $9,519.20

Storage:

40 TB × 1,024 GB × $0.024

= $983.04

Spectrum:

20 TB × $5/TB

= $100

Total:

$9,519.20

+ $983.04

+ $100

= $10,602.24/month

---

18. Redshift Serverless RPU Calculation

For serverless workloads:

Cost

=

RPU

×

Runtime

×

RPU Hourly Price

The Week 34 example uses:

128 RPUs

630 seconds/run

13 runs/day

$0.375/RPU-hour

Total runtime:

13 × 630

= 8,190 seconds

The resulting daily cost from the provided material is:

$109.20/day

---

19. Spectrum File-Format Optimization

The amount of data scanned directly affects Spectrum cost.

Consider:

100-column dataset

4 TB total data

Query requires only 1 column

Comparison

Storage Format	Data Scanned	Cost
Uncompressed CSV	4 TB	**$20.00**
GZIP CSV	1 TB	**$5.00**
Compressed Parquet	~10 GB	**$0.05**

Why Parquet Wins

Parquet is **columnar**.

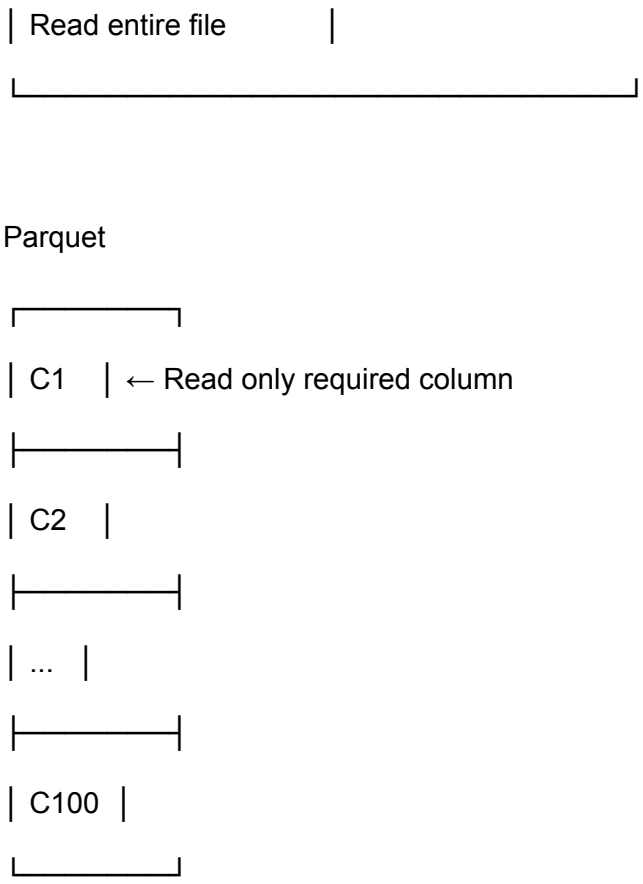
If the query needs:

SELECT customer_id

it doesn't need to read every column.

CSV

```
┌──────────────────────────────────────┐
│ C1 C2 C3 C4 ... C100         │
```

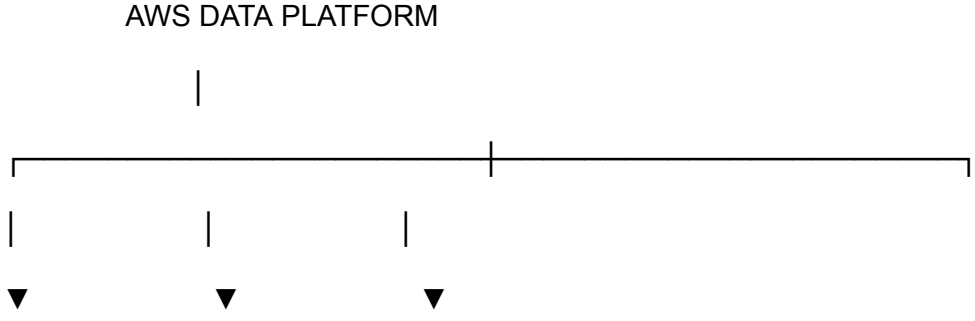


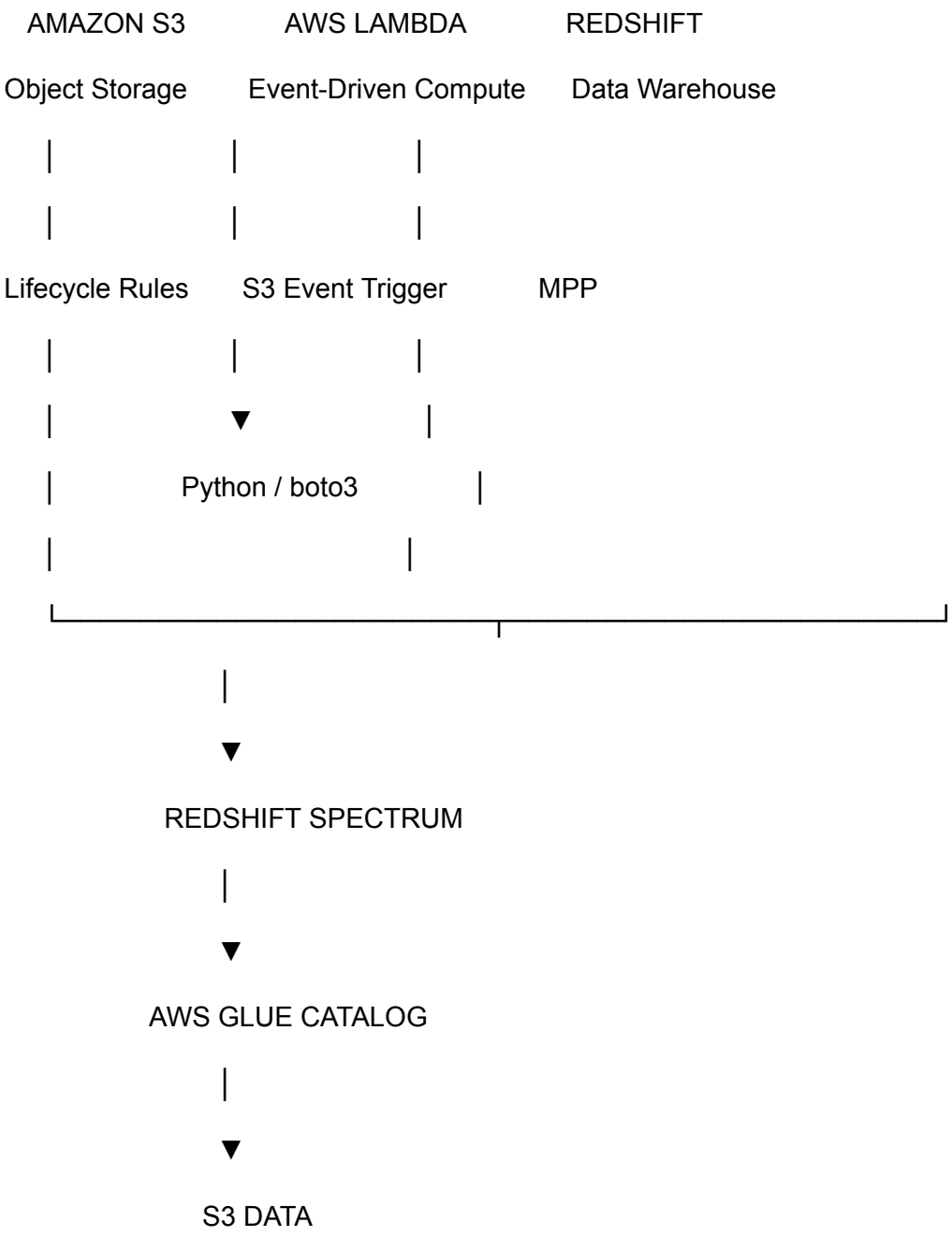
Key Optimization Principle

> **Compression reduces the physical size of data; columnar formats reduce the amount of data that must be read for selective queries.**

---

20. Week 34 — Architecture at a Glance





---

21. Interview Cheat Sheet

S3

Q: What is S3?
Object storage service for scalable storage and retrieval of unstructured/semi-structured data.

Q: What is S3 Lifecycle Management?
Automatic transition of objects between storage classes based on age/access patterns.

Q: Why use Glacier?
For low-cost long-term archival data.

Lambda

Q: What is AWS Lambda?
Serverless, event-driven compute that runs code without server management.

Q: Maximum Lambda execution duration?
15 minutes / 900 seconds.

Q: What is a Lambda Layer?
Reusable package containing dependencies or shared code.

Redshift

Q: What is Redshift?
Managed cloud data warehouse optimized for analytical OLAP workloads.

Q: What is MPP?
Massively Parallel Processing — distributing query execution across multiple compute nodes/slices.

Q: Leader Node vs Compute Node?

Leader Node → Parse + Optimize + Coordinate

Compute Node → Execute

Slice → Parallel processing unit within compute node

Distribution vs Sort Key

Q: What does DISTKEY do?

Determines **where rows are distributed** across slices.

Q: What does SORTKEY do?

Determines **how data is physically ordered** for efficient block pruning.

Spectrum

Q: What is Redshift Spectrum?

Query data directly in S3 without first loading it into Redshift.

Q: What catalog does Spectrum use?

AWS Glue Data Catalog.

COPY

Q: What is COPY used for?

High-throughput ingestion of data from S3 into Redshift-managed tables.

Serverless vs Provisioned

	Provisioned RA3	**Serverless**
Infrastructure	Fixed	Automatically managed
Billing	Node/hour	RPU usage
Best for	Predictable workload	Variable workload

Scaling	Cluster configuration	Automatic

---

22. Week 34 — Key Takeaways

S3

│

├── Object storage

├── 11 9s durability

├── Storage classes

└── Lifecycle automation

Lambda

│

├── Serverless

├── Event-driven

├── S3 triggers

├── 15-minute maximum

└── Layers for reusable dependencies

Redshift

│

├── OLAP Data Warehouse

├── MPP architecture

├── Leader + Compute + Slices

├── Columnar storage

├── Compression

├── DISTSTYLE

├── SORTKEY

└── Result caching

Redshift Spectrum

│

├── Query S3 directly

├── Uses Glue Catalog

├── External tables

└── Federated S3 + Redshift queries

Data Loading

│

└── COPY → S3 → Redshift

Cost Optimization

│

├── Compression

├── Parquet

├── Column pruning

├── Spectrum scan reduction

└── Serverless for variable workloads

Master Curriculum — Week 34 Added

[WEEK 34: AWS Object Storage, Event-Driven Compute & Redshift]

• Amazon S3 object storage, durability, buckets, object keys and storage classes

• S3 Lifecycle Management and automated storage-tier transitions

• AWS Lambda serverless event-driven architecture and S3 event triggers

• Lambda Python event handlers, boto3 S3 operations and Lambda Layers

• Amazon Redshift MPP architecture: Leader Nodes, Compute Nodes and Slices

• Columnar storage, compression, result caching and parallel query execution

• Redshift DISTSTYLE: AUTO, EVEN, KEY and ALL

• Redshift SORTKEY, Zone Map pruning and Sort Merge Bucket joins

• Redshift Spectrum for querying S3 through the Glue Data Catalog

• External schemas, external tables and federated S3 + Redshift queries

• COPY command and Auto Copy for S3-to-Redshift ingestion

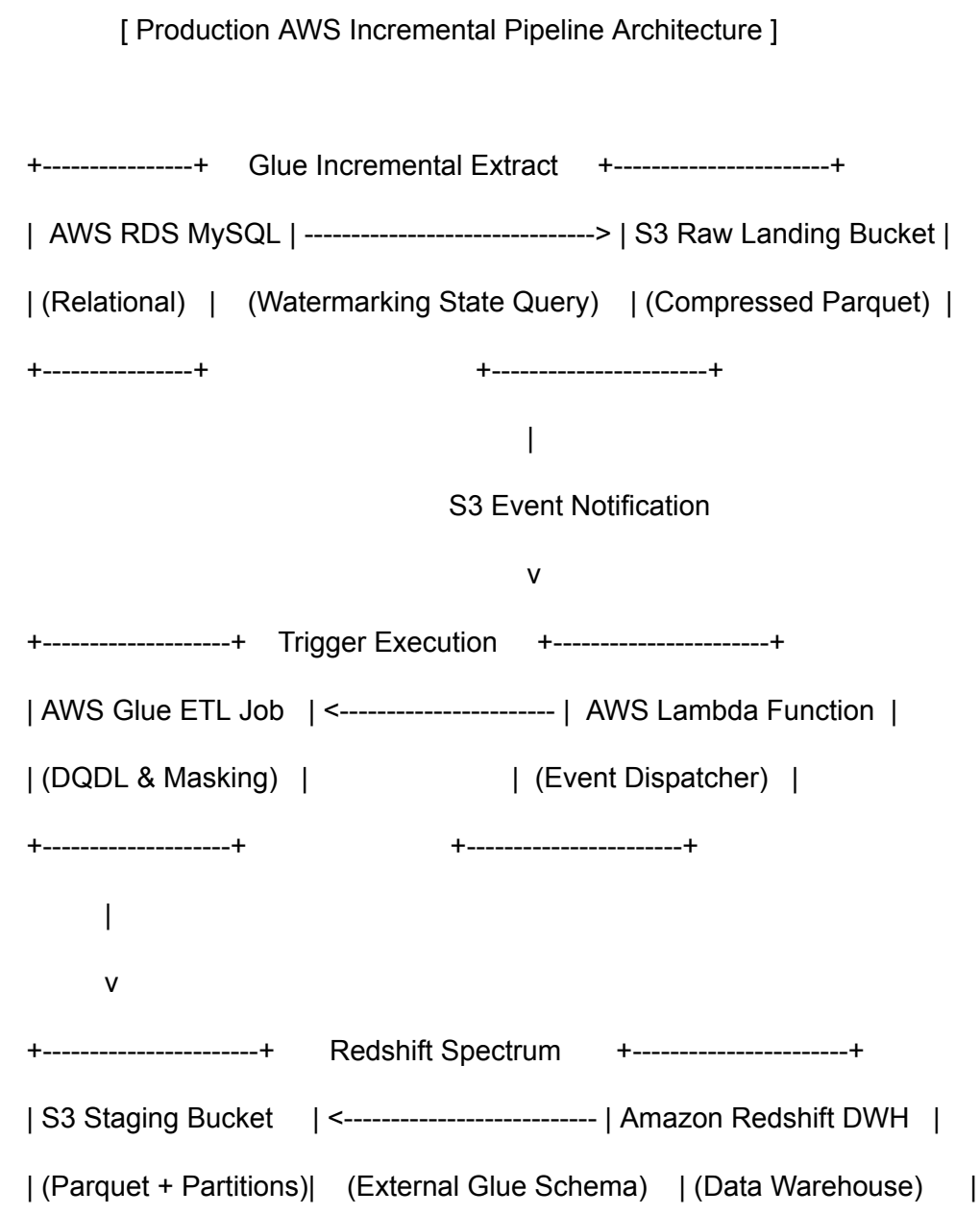
• Provisioned RA3 vs Redshift Serverless and RPU-based billing

• Spectrum scan-cost optimization using compression and Parquet

Week 34 is now organized as a complete revision/reference chapter, with the important interview distinctions—`DISTKEY vs SORTKEY`, `COPY vs Spectrum`, and `Provisioned vs Serverless`—made explicit.

Week 35: Enterprise AWS Incremental Data Engineering Pipeline

Week 35 focuses on building a resilient, event-driven, production-grade incremental ETL pipeline on AWS. This architecture handles incremental extraction from relational stores (AWS RDS MySQL), landing raw data into Amazon S3, triggering serverless validation workflows (AWS Lambda + Glue), and staging optimized data into Amazon Redshift for enterprise analytics.



+----------------------+ +----------------------+

---

1. Incremental Extraction & Watermarking Pattern

To avoid expensive full-table scans, an **incremental extraction pattern** isolates only newly inserted or modified records based on a tracking column (`updated_at` or `last_modified` timestamp).

Watermark Tracking Strategy

1. **State Store:** A dedicated table (`watermark_tracker`) in RDS or DynamoDB tracks the latest execution timestamp for each source table.
2. **Extraction Query:** The extraction job selects records where `updated_at > MAX(last_success_timestamp)`.
3. **Commit Phase:** Once S3 writes complete successfully, the state store is updated with the max timestamp from the extracted batch.

Unabridged Glue Extraction Job (`glue_incremental_extract.py`)

```
import sys

from awsglue.utils import getResolvedOptions

from pyspark.context import SparkContext

from awsglue.context import GlueContext

from awsglue.job import Job

from awsglue import DynamicFrame

from pyspark.sql import functions as F


# Initialize AWS Glue and Spark Contexts

args = getResolvedOptions(sys.argv, ['JOB_NAME'])

sc = SparkContext()
```

```
glueContext = GlueContext(sc)

spark = glueContext.spark_session

job = Job(glueContext)

job.init(args['JOB_NAME'], args)


# 1. Read source data and watermark tracking state table from AWS RDS MySQL

orders_dyf = glueContext.create_dynamic_frame.from_options(

    connection_type="mysql",

    connection_options={"useConnectionProperties": "true", "dbtable": "orders",
"connectionName": "RDS_MySQL_Conn"}

)


watermark_dyf = glueContext.create_dynamic_frame.from_options(

    connection_type="mysql",

    connection_options={"useConnectionProperties": "true", "dbtable": "watermark_tracker",
"connectionName": "RDS_MySQL_Conn"}

)


orders_dyf.toDF().createOrReplaceTempView("src_orders")

watermark_dyf.toDF().createOrReplaceTempView("src_watermark")


# 2. Extract only incremental records modified since the last watermark execution

incremental_df = spark.sql("""

    SELECT * FROM src_orders

    WHERE updated_at > (
```

```
        SELECT COALESCE(MAX(last_success_timestamp), '1900-01-01 00:00:00')
        FROM src_watermark WHERE table_name = 'orders'
    )
""")

# 3. Save extracted incremental batch to S3 Landing Zone in compressed Parquet format
if incremental_df.count() > 0:
    output_dyf = DynamicFrame.fromDF(incremental_df.coalesce(1), glueContext, "output_dyf")

    glueContext.write_dynamic_frame.from_options(
        frame=output_dyf,
        connection_type="s3",
        connection_options={"path": "s3://production-data-lake-raw/landing/orders/"},
        format="glueparquet",
        format_options={"compression": "snappy"}
    )

    # 4. Update the Watermark State Table with the new maximum batch timestamp
    new_max_ts = incremental_df.agg(F.max("updated_at")).collect()[0][0]
    update_watermark_df = spark.createDataFrame([(new_max_ts, 'orders')],
['last_success_timestamp', 'table_name'])

    glueContext.write_dynamic_frame.from_options(
        frame=DynamicFrame.fromDF(update_watermark_df, glueContext,
"update_watermark_df"),
```

```
        connection_type="mysql",
        connection_options={"useConnectionProperties": "true", "dbtable": "watermark_tracker", "connectionName": "RDS_MySQL_Conn"}
    )

job.commit()
```

---

2. Event-Driven Lambda Dispatcher Strategy

When new files hit the S3 Landing Zone, an S3 `ObjectCreated` event triggers a serverless **AWS Lambda Function**. The function parses the event payload and starts the downstream AWS Glue ETL processing job.

```
import boto3
import os

glue_client = boto3.client('glue')

def lambda_handler(event, context):
    # Parse event metadata from S3 trigger notification
    source_bucket = event['Records'][0]['s3']['bucket']['name']
    source_key = event['Records'][0]['s3']['object']['key']

    target_job_name = os.environ.get("GLUE_JOB_NAME", "pyspark_data_cleansing_job")

    # Pass input file details directly to Glue Job arguments
```

```
    response = glue_client.start_job_run(
        JobName=target_job_name,
        Arguments={
            '--SOURCE_BUCKET': source_bucket,
            '--SOURCE_KEY': source_key
        }
    )

    return {
        'statusCode': 200,
        'body': f"Triggered Glue Job {target_job_name} with RunId {response['JobRunId']}"

    }
```

---

3. Amazon EventBridge Automation & Cron Scheduling

For batch workloads that execute on fixed schedules rather than event notifications, **Amazon EventBridge** schedules triggers using standard cron syntax.

$$\text{Cron Expression Syntax: } \texttt{cron(Minutes Hours Day-of-Month Month Day-of-Week Year)}$$

Schedule Requirement	EventBridge Cron Expression
Every 5 Minutes	`cron(*/5 * * * ? *)`

Daily at 02:30 AM UTC	`cron(30 2 * * ? *)`
Mon–Fri 09:00 AM to 05:00 PM UTC	`cron(0 9-17 ? * MON-FRI *)`
1st of Every Month at Midnight UTC	`cron(0 0 1 * ? *)`

---

4. Querying S3 Staging Data via Amazon Redshift Spectrum

Once transformed data lands in the S3 Processed Zone, **Amazon Redshift Spectrum** enables querying Parquet files directly using standard Redshift SQL queries without loading data onto Redshift disks.

```
-- 1. Create External Schema pointing to Glue Data Catalog Database

CREATE EXTERNAL SCHEMA spectrum_sales_schema

FROM DATA CATALOG DATABASE 'analytics_catalog_db'

IAM_ROLE 'arn:aws:iam::123456789012:role/RedshiftSpectrumRole'

CREATE EXTERNAL DATABASE IF NOT EXISTS;


-- 2. Define External Table matching Parquet structure in S3 Staging Bucket

CREATE EXTERNAL TABLE spectrum_sales_schema.ext_orders (

    order_id BIGINT,

    customer_id BIGINT,

    amount DOUBLE PRECISION,

    order_status VARCHAR(50)

)
```

```
PARTITIONED BY (order_date VARCHAR(10))
STORED AS PARQUET
LOCATION 's3://production-data-lake-processed/orders/';

-- 3. Synchronize Partitions
ALTER TABLE spectrum_sales_schema.ext_orders RECOVER PARTITIONS;

-- 4. Execute Federated Query joining Redshift internal tables with S3 external tables
SELECT
    c.customer_name,
    COUNT(o.order_id) AS total_orders,
    SUM(o.amount) AS total_spend
FROM public.dim_customers c
JOIN spectrum_sales_schema.ext_orders o
  ON c.customer_id = o.customer_id
WHERE o.order_date = '2026-07-25'
GROUP BY c.customer_name;
```

---

5. Technical Interview Q&A — AWS Incremental Pipelines

Q1: How do you handle schema evolution when performing incremental loads from S3 into Amazon Redshift?

Answer: Schema changes (like adding new columns) can break static copy jobs.

1. **AWS Glue Data Catalog Integration:** AWS Glue Crawlers automatically detect new columns in Parquet files and update table definitions in the Glue Data Catalog.
2. **Redshift Spectrum:** Spectrum dynamically queries updated Parquet schemas using the Glue Data Catalog, returning `NULL` values for older Parquet files that lack newly added columns.
3. **Redshift `COPY` Command:** When loading directly into internal Redshift tables, use the `FORMAT AS PARQUET` option along with explicit column mapping to align file attributes with database definitions.

Q2: What is the difference between AWS Lambda event triggers and Amazon EventBridge rules for pipeline orchestration?

Answer:

- **AWS Lambda S3 Event Triggers:** Best for direct 1-to-1 event notifications (e.g., executing a single function immediately when a specific S3 file arrives).
- **Amazon EventBridge:** An enterprise event bus used for complex routing, event filtering, batching, and multi-target fan-out workflows (e.g., routing S3 object events to multiple target services like Glue, SQS, SNS, and Step Functions simultaneously).

Week 35 Summary

Core Architecture:

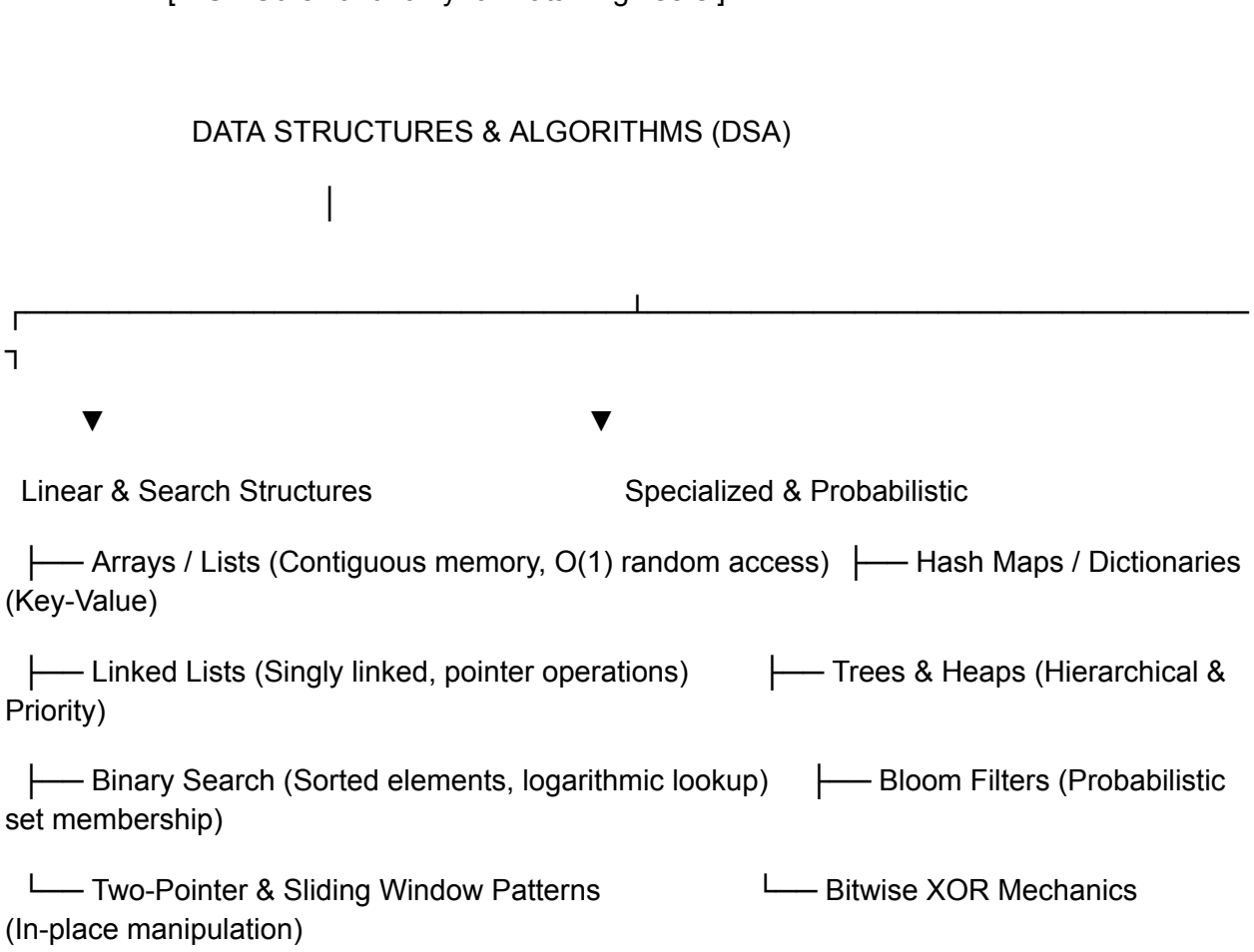
`RDS MySQL → AWS Glue → S3 Landing → S3 Event → Lambda → Glue ETL → S3 Processed → Redshift Spectrum`

Core Concepts:

- Incremental data extraction
- Watermarking
- AWS Glue PySpark
- Amazon S3 data lake
- Event-driven Lambda orchestration
- Amazon EventBridge scheduling
- AWS Glue Data Catalog
- Redshift Spectrum
- Schema evolution
- Enterprise ETL architecture
- Technical interview preparation

Week 36: Data Structures & Algorithms (DSA) for Data Engineers

Data engineering interviews at top product-based companies evaluate core computer science fundamentals—specifically how effectively you manage memory, choose efficient data structures, and optimize time and space complexity for large-scale data manipulation.



---

1. Time & Space Complexity Analysis (O-Notation)

Time complexity measures how the number of system operations grows relative to input size n. Space complexity evaluates auxiliary memory allocated during execution.

Complexity Order:

`O(1) < O(log n) < O(n) < O(n log n) < O(n²) < O(n³) < O(2ⁿ) < O(n!)`

Time Complexity	Classification	Operations (n = 1024)	Common Examples
O(1)	Constant	1	Direct Array Indexing (`arr[i]`), Hash Map Lookup
O(log n)	Logarithmic	10	Binary Search on a sorted array
O(n)	Linear	1,024	Linear Search, Single Array Sweep
O(n log n)	Linearithmic	10,240	Merge Sort, Quick Sort
O(n²)	Quadratic	1,048,576	Nested Loop Comparison (Brute-Force)

---

2. Fundamental Array Algorithms & Coding Implementations

2.1 Two-Sum Problem (LeetCode #1)

Find indices of two numbers in an array `nums` that add up to a specified `target`.

Approaches:

- **Brute-Force — O(n²) Time, O(1) Space:** Uses two nested loops to test every combination.
- **Sorting & Two-Pointers — O(n log n) Time, O(n) Space:** Sorts an array copy and adjusts left/right pointers based on sum.
- **Hash Map Lookup — O(n) Time, O(n) Space:** Computes `complement = target - num` and checks dictionary membership in a single pass.

```
from typing import List


class Solution:
    def twoSum(self, nums: List[int], target: int) -> List[int]:
        seen = {}  # Map value to index
        for i, num in enumerate(nums):
            complement = target - num
            if complement in seen:
                return [seen[complement], i]
            seen[num] = i
        return []
```

2.2 Single Number (LeetCode #136) — Bitwise XOR Optimization

Find the single element in an array where every other element appears twice.

Using the properties of Bitwise XOR (⊕):

- $x \oplus x = 0$
- $x \oplus 0 = x$

Duplicate numbers evaluate to zero, leaving only the unique value.

```
class Solution:
    def singleNumber(self, nums: List[int]) -> int:
        result = 0
        for num in nums:
            result ^= num  # XOR bitwise operation
```

```
        return result
```

Time Complexity: O(n)
Space Complexity: O(1)

2.3 Majority Element (LeetCode #169) — Boyer-Moore Voting Algorithm

Find the element appearing more than $\lfloor n / 2 \rfloor$ times.

```
class Solution:
    def majorityElement(self, nums: List[int]) -> int:
        candidate = None
        count = 0
        for num in nums:
            if count == 0:
                candidate = num
            count += (1 if num == candidate else -1)
        return candidate
```

Time Complexity: O(n)
Space Complexity: O(1)

2.4 Maximum Subarray (LeetCode #53) — Kadane's Algorithm

Find the contiguous subarray with the largest sum. Resets the running current sum to zero whenever it drops below zero.

```
class Solution:
    def maxSubArray(self, nums: List[int]) -> int:
```

```
        max_sum = nums[0]
        current_sum = 0
        for num in nums:
            if current_sum < 0:
                current_sum = 0
            current_sum += num
            if current_sum > max_sum:
                max_sum = current_sum
        return max_sum
```

Time Complexity: O(n)
Space Complexity: O(1)

2.5 Array Rotation in Place (LeetCode #189)

Rotate an array to the right by k steps using three array reversals.

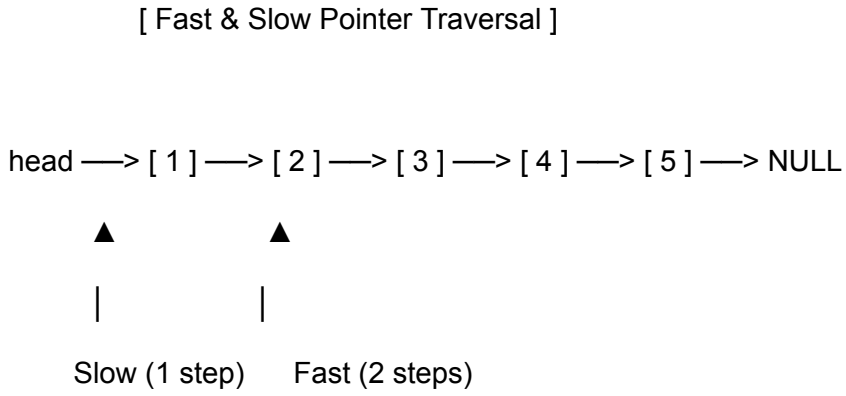
```
class Solution:
    def reverse(self, nums: List[int], start: int, end: int) -> None:
        while start < end:
            nums[start], nums[end] = nums[end], nums[start]
            start += 1
            end -= 1

    def rotate(self, nums: List[int], k: int) -> None:
        n = len(nums)
        k %= n
```

```
        self.reverse(nums, 0, n - 1)  # Reverse entire array
        self.reverse(nums, 0, k - 1)  # Reverse first k elements
        self.reverse(nums, k, n - 1)  # Reverse remaining elements
```

---

3. Linked List Algorithms & Fast/Slow Pointer Mechanics



3.1 Reverse a Singly Linked List — Iterative Three-Pointer Approach

Reverses pointers in place using `prev`, `curr`, and `temp_next` pointers.

```
class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next


class Solution:
    def reverseList(self, head: ListNode) -> ListNode:
        prev = None
        curr = head
```

```
        while curr is not None:
            temp_next = curr.next
            curr.next = prev
            prev = curr
            curr = temp_next
        return prev
```

Time Complexity: O(n)
Space Complexity: O(1)

3.2 Find Middle of Linked List — Floyd's Fast & Slow Pointers

The fast pointer moves two steps while the slow pointer moves one step. When the fast pointer reaches the end, the slow pointer rests at the exact middle node.

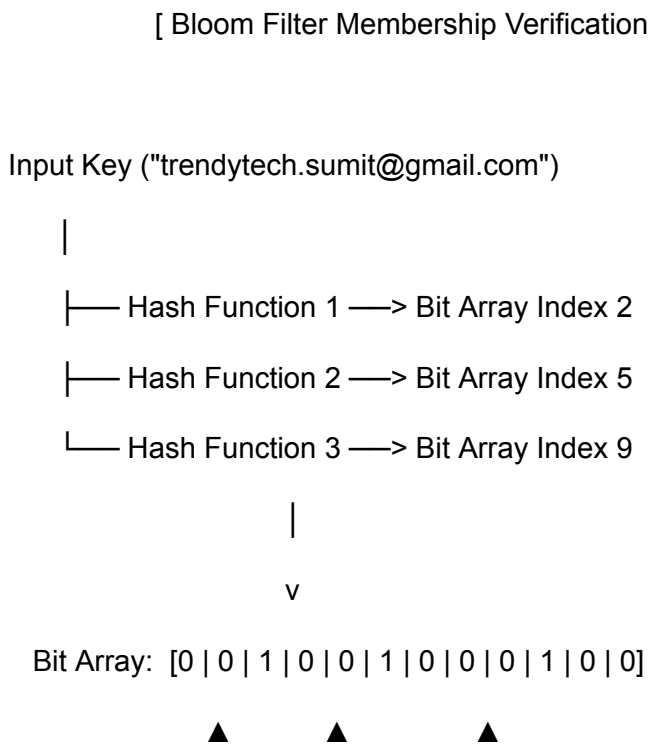
```
class Solution:
    def middleNode(self, head: ListNode) -> ListNode:
        slow = head
        fast = head
        while fast is not None and fast.next is not None:
            slow = slow.next
            fast = fast.next.next
        return slow
```

---

4. Specialized Data Engineering Structure: Bloom Filters

A **Bloom Filter** is a space-efficient, probabilistic data structure designed to quickly verify whether an element is guaranteed **not** in a set or **possibly** in a set.

Example use case: checking whether a user ID or email exists in disk storage before executing expensive I/O operations.



Bloom Filter Properties

- **False Positives ARE Possible:** Multiple keys can set the same bit positions, causing the system to indicate an item exists when it does not.
- **False Negatives ARE IMPOSSIBLE:** If any hash bit index returns `0`, the element is guaranteed to **not** exist in the underlying dataset.

Week 37: Interview Preparation, ATS Resume Engineering & Job Search Strategy

---

1. ATS-Compliant Resume Architecture & Formatting Rules

An **Applicant Tracking System (ATS)** parses resume text into standardized database categories prior to recruiter review.

[High-Impact ATS Resume Structure]

==========

FULL NAME | AWS/Azure Certified Senior Data Engineer

Phone Number | Professional Email | LinkedIn URL | Portfolio/GitHub

==========

PROFESSIONAL SUMMARY

4-5 lines summarizing years of experience, core cloud stack, and business impact.

==========

TECHNICAL SKILLS

Cloud Platforms, Big Data Frameworks, Databases, Languages, Orchestration

==========

PROFESSIONAL EXPERIENCE

Reverse Chronological: Action Verb + Task / Scope + Quantified Result

==========

PROJECT HIGHLIGHTS

Problem Statement ⟶ Architecture / Pipeline ⟶ Business Metric

==========

EDUCATION & CERTIFICATIONS

Degree, College/University, Graduation Year | Official Cloud Certifications

==========

Core Resume Writing Standards

1. Length Threshold

- Maximum **1 page** for professionals with under 2 years of experience.
- Maximum **2 pages** for experienced candidates.

2. Bullet Construction Formula

Bullet Formula:

`Action Verb + Technical Scope/Task + Quantified Impact Metric`

Weak Example:

> Worked on building Data Pipelines in Azure Data Factory.

Strong Example:

> Engineered 12+ automated Azure Data Factory ETL pipelines, ingesting 500 GB daily data into Delta Lake and reducing data turnaround time by 35%.

3. Action Verb Upgrades

Replace passive phrases such as:

- Involved in
- Worked on
- Responsible for

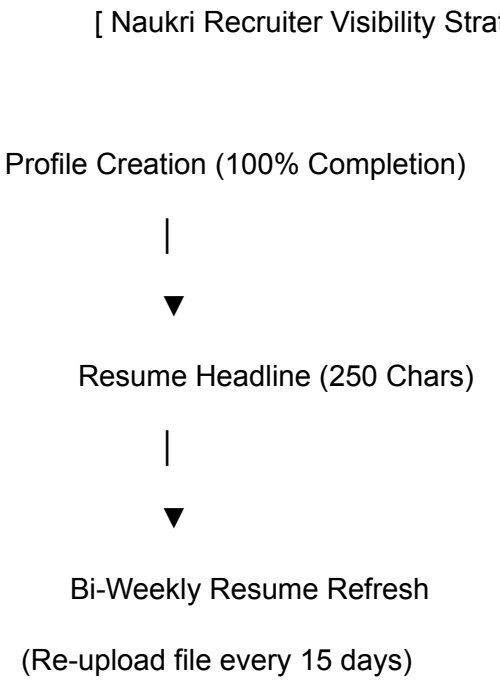
With high-impact active verbs:

- Pioneered
- Engineered
- Optimized
- Architected
- Automated
- Streamlined

---

2. Naukri.com Platform Profile Optimization

To maximize visibility in recruiter queries, configure your Naukri profile to align with search algorithms.



Naukri Key Configuration Settings

Resume Headline

The resume headline should highlight:

- Designation
- Total experience

- Primary technology stack
- Key achievements
- Certifications

Template:

```
Skilled [Designation] with [X] Years experience specializing in [Key Skills] | [Education] from [Institute] with [Certification Name].
```

Example:

```
Skilled Data Engineer with 4 Years experience in PySpark, Databricks & Azure Data Factory | Built pipelines processing 100+ TB data | B.Tech NIT with AWS Certified Developer Associate.
```

Algorithm Refresh Trigger

Re-uploading your resume document every **14–15 days** updates your "Last Active" timestamp, placing your profile at the top of recruiter searches.

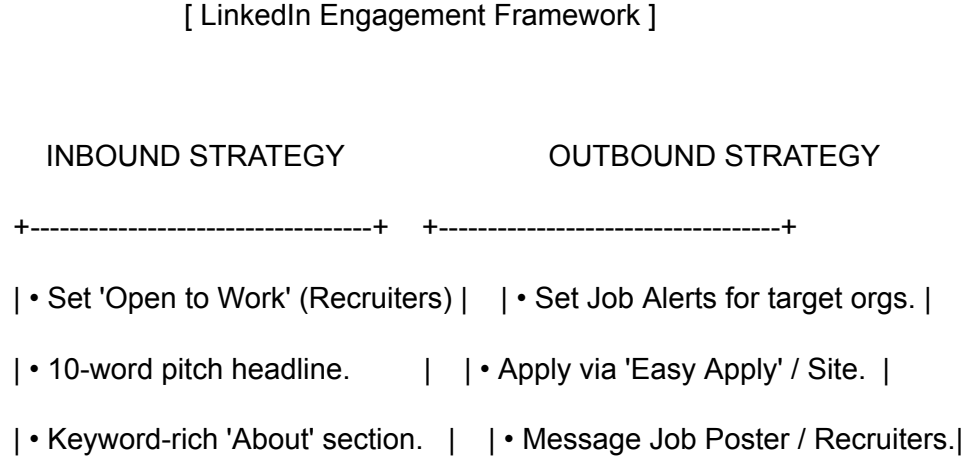
Notice Period & Salary Range

Setting notice periods to under 30 days can increase recruiter call rates. Set realistic preferred salary expectations based on market averages.

---

3. LinkedIn Strategic Networking & Inbound/Outbound Growth

LinkedIn enables both inbound recruiter reach-outs and targeted job applications.



+----------------------------------+   +----------------------------------+

1. Profile Optimization — Inbound

- Write a clear headline summarizing your core stack.
- Populate the **About** section with relevant domain keywords.
- Enable **Open to Work** targeting recruiter visibility.

2. Targeted Applications — Outbound

- Search specific target companies.
- Filter roles by location and experience.
- Turn on **Job Alerts**.
- Apply promptly after a relevant job is posted.

3. Direct Recruiter Outreach

Reach out directly to the **Job Poster** listed on LinkedIn job advertisements.

Send a concise, professional message summarizing:

- Your experience
- Your technical stack
- Your fit for the role

Master 37-Week Data Engineering Study Guide Roadmap

==========

ULTIMATE BIG DATA, MULTI-CLOUD & DATA ENGINEERING MASTER STUDY GUIDE

==========

[WEEKS 1 - 16: Distributed Computing, PySpark Core & Optimization]

- HDFS Architecture, YARN Schedulers, MapReduce, Spark RDDs, DataFrames API
- Catalyst Optimizer, Tungsten Execution Engine, Memory Pools, Partitioning vs Bucketing
- Join Optimizations, Partition Skew, AQE, Salting, Resource Sizing & Pytest Frameworks

[WEEKS 17 - 21: Azure Infrastructure, Databricks & Unity Catalog]

- Azure Global Infrastructure, Availability Zones, Fault/Update Domains, CapEx vs OpEx
- Storage Services (Blob, ADLS Gen2, Access Tiers, LRS/ZRS/GRS Redundancy Models)
- Databricks Architecture, Control/Data Planes, DBFS, Secret Scopes, Delta Lake ACID Logs
- Lakehouse Architecture, Medallion Design (Bronze/Silver/Gold), Delta Engine & Photon
- Unity Catalog: 3-Level Namespace, Centralized Metastore, SCIM Provisioning & Access Control

[WEEKS 22 - 25: Pipeline Orchestration & End-to-End Enterprise Project]

- Azure Data Factory: Copy Activity, Mapping Data Flows, Dynamic Parameterization & Triggers
- Azure Synapse Analytics: Serverless SQL Pools, Dedicated SQL MPP Engine & Spark Pools
- Production Capstone Project: Event-Driven Pipeline, Key Vault, Azure SQL & AWS S3 Integration

[WEEKS 26 - 27: Enterprise Data Modeling & Technical Interview Prep]

- Normalization (1NF to 3NF) & Kimball Dimensional Design (Star vs Snowflake vs OBT)
- Surrogate Keys, Slowly Changing Dimensions (SCD 0/1/2/3) & PySpark SCD-2 Implementations
- Azure Storage & ADF Production Interview Q&A, Integration Runtimes & Performance Tuning

[WEEKS 28 - 29: Spark Structured Streaming Architecture]

• Micro-Batch Engine, DataStreamReader / DataStreamWriter, Output Modes & Triggers

• Checkpoint Storage, WAL Logs, Exactly-Once Guarantees, Watermarking & Streaming Joins

[WEEK 30: Apache Kafka Architecture & Real-Time Event Streaming]

• Distributed Event Platform Internals, Brokers, Topics, Partitions, Offsets & Consumer Groups

• Python Confluent-Kafka Producer Implementation & PySpark Kafka Stream Consumption

[WEEKS 31 - 33: AWS Cloud Big Data Ecosystem, EMR & Glue Integration]

• AWS Service Mapping, EC2 Instance Families (M, C, R Types), AMI Images, EBS Storage

• Athena Query Engine (Trino SQL vs Spark DPU), Pricing Calculator & Scan Reduction

• Amazon EMR Architecture (Primary, Core, Task Nodes), Instance Groups vs Fleets, Steps

• AWS Glue Visual ETL, Worker Types (G.1X, G.2X), Relational Ingestion (S3 to RDS MySQL)

• PII Protection: Row/Column Detection, Grok Patterns, REDACT & SHA256_HASH Actions

• Data Governance: Data Quality at Rest vs In Move using DeeQu & DQDL Rulesets

[WEEKS 34 - 35: AWS Data Lakes, Redshift DWH & Production Pipeline]

• Amazon S3 Durability (11 9's), Storage Classes (Standard, IA, Glacier), Lifecycle Rules

• AWS Lambda Architecture: S3 Event Triggers, Execution Timeouts (15 mins), Lambda Layers

• Amazon Redshift Architecture: Leader Node, Compute Nodes, Slices, MPP, Spectrum, Serverless

• Production AWS Incremental Pipeline: RDS MySQL, Watermarking, S3 Zones, Lambda, Glue, EventBridge

[WEEK 36: Data Structures & Algorithms (DSA) for Data Engineers]

• Asymptotic Time & Space Complexity Analysis (Big-O Growth Hierarchy)

• Array Algorithms: Two-Sum, Single Number (XOR), Majority Element, Kadane's Algorithm

• Linked List Mechanics: Iterative Reversal, Floyd's Fast & Slow Pointer Middle Node Search

• Specialized Structures: Bloom Filters (Probabilistic Set Membership, False Positive Rules)

[WEEK 37: Interview Preparation & Job Search Engineering]

• ATS-Compliant Resume Architecture: Formatting, Active Action Verbs & Quantified Metrics

• Naukri.com Platform Optimization: Headline Formulation, Algorithm Refresh & Notice Period

• LinkedIn Outreach Framework: Profile Customization, Open To Work & Recruiter Direct Messaging

==========

With love in every byte — Jacky ⚡❤️👨‍💻